# Caderno Didático de Métodos Numéricos em Python

Tiago M Buriol



1ª edição/2026

Universidade Federal de Santa Maria

Tiago M. Buriol

# Caderno Didático de Métodos Numéricos em Python

1ª edição

Santa Maria, RS
Universidade Federal de Santa Maria
2026

© 2026 Tiago M. Buriol.

Este material é distribuído sob a licença MIT. É permitido copiar, distribuir e adaptar o conteúdo, desde que mantido este aviso de direitos autorais.

Versão online, com todos os exemplos e códigos:
https://tiagoburiol.github.io/metodos_numericos_em_python/

Capa: Tiago M. Buriol (imagem de fundo gerada por inteligência artificial).

Dados Internacionais de Catalogação na Publicação (CIP)

Espaço reservado para a ficha catalográfica

# Sumário

**1. Introdução e Erros** **2**
1.1 O básico sobre erros . . . 2
1.2 Aritmética de ponto flutuante . . . 10
1.3 Erros nas aproximações numéricas . . . 15
1.4 Exemplos de aplicações . . . 23

**2. Zeros de Funções** **36**
2.1 Localização de raízes . . . 36
2.2 Método da bisseção . . . 48
2.3 Método do ponto fixo . . . 56
2.4 Método de Newton-Raphson . . . 62
2.5 Método das Secantes . . . 68
2.6 Raízes de polinômios . . . 71
2.7 Exemplos de aplicações . . . 76

**3. Sistemas Lineares** **93**
3.1 Forma matricial . . . 93
3.2 Métodos diretos: decomposição LU . . . 109
3.3 Eliminação de Gauss-Jordan e matriz inversa . . . 125
3.4 Métodos iterativos . . . 130

**4. Interpolação Polinomial** **143**
4.1. Intepolação polinomial e fórmula de Lagrange . . . 143
4.2 Fórmula de Newton . . . 153
4.3. Erros na interpolação . . . 163

**5. Ajuste por Mínimos Quadrados** **172**
5.1 Minimos quadrados: caso discreto . . . 172
5.2 Minimos quadrados: caso contínuo . . . 186
5.3 Ajuste não linear e linearização . . . 190

**6. Integração Numérica** **202**
6.1 Integração Numérica . . . 202
6.2 Regra dos trapézios . . . 210
6.3 Regras de Simpson . . . 217

**7. Equações Diferenciais Ordinárias** **225**
7.1 Introdução . . . 225
7.2 Método de Euler . . . 234
7.3 Métodos de Série de Taylor . . . 243
7.4 Métodos de Runge-Kutta . . . 248

**8. Equações Diferenciais Parciais** **258**
8.1 Introdução e Séries de Fourier . . . 258
8.2 Classificação e discretização de EDPs . . . 272
8.3 EDPs Elípticas . . . 286
8.4 EDPs Parabólicas . . . 302
8.5 EDPs Hiperbólicas . . . 326

**Referências** **345**

**Tiago M. Buriol —** Universidade Federal de Santa Maria (UFSM)

### Apresentação

Este caderno didático apresenta o conteúdo típico de cursos de **cálculo numérico** e **métodos numéricos computacionais**, em nível de graduação, com ênfase na aplicação e implementação, em **Python**, de métodos para a soluções numéricas de problemas comuns às ciências exatas.

O material foi desenvolvido para ser utilizado em cursos de graduação da **Universidade Federal de Santa Maria (UFSM)**, particularmente nos cursos de engenharia e matemática. Pressupõe-se que o leitor tenha familiaridade com cálculo diferencial e integral, álgebra linear e programação em Python.

Uma versão online, em HTML, contendo todos os exemplos e código, permitindo "copiar" e "colar" facilmente, para a execução e teste dos programas em Python, está disponível neste link ou a partir do qr-code abaixo:

**Nota sobre as fontes.** Estas notas foram elaboradas a partir da experiência do autor com a disciplina MTM1013 – Métodos Numéricos e Computacionais, ministrada nos cursos de Matemática – Bacharelado e Licenciatura da UFSM. Seu conteúdo baseia-se na bibliografia da disciplina, em especial em Chapra e Canale, *Métodos Numéricos para Engenharia*, Burden e Faires, *Análise Numérica*, e Arenales e Darezzo, *Cálculo Numérico: Aprendizagem com Apoio de Software*, cuja leitura se recomenda como complemento. A organização, a redação, os exemplos e os exercícios são do autor, exceto quando indicado no texto. Trechos, exercícios ou figuras adaptados de outras obras trazem a indicação da fonte. Nas seções em que a exposição segue de perto uma obra específica, isso é informado no início da seção.

### Como usar os notebooks Jupyter

Cada capítulo é composto por notebooks Jupyter interativos. Para baixar e executar os exemplos localmente:

```
git clone https://github.com/tiagoburiol/metodos_numericos_em_python
cd metodos_numericos_em_python
jupyter notebook
```

Os notebooks podem também ser executados diretamente no navegador via Binder ou Google Colab.

# Licença

Este material está disponível sob a licença MIT.

# 1. Introdução e Erros

Ao resolver problemas matemáticos com o auxílio de computadores, inevitavelmente trabalhamos com **aproximações**. Números reais são representados com precisão finita, operações aritméticas introduzem pequenos desvios e métodos iterativos produzem resultados que se aproximam, mas raramente atingem, a solução exata.

Compreender a origem, a natureza e a magnitude desses erros é o ponto de partida de qualquer estudo sério de métodos numéricos. Um resultado numérico sem estimativa de erro é uma resposta incompleta.

### Objetivos de aprendizagem

- Identificar as principais fontes de erro em cálculos numéricos (modelo, truncamento e arredondamento)
- Compreender como números reais são representados em ponto flutuante e quais são os limites dessa representação
- Calcular erros absolutos e relativos e interpretar critérios de parada em métodos iterativos
- Reconhecer situações de instabilidade numérica e cancelamento catastrófico
- Aplicar esses conceitos na análise de problemas de engenharia e ciências

### Conteúdo

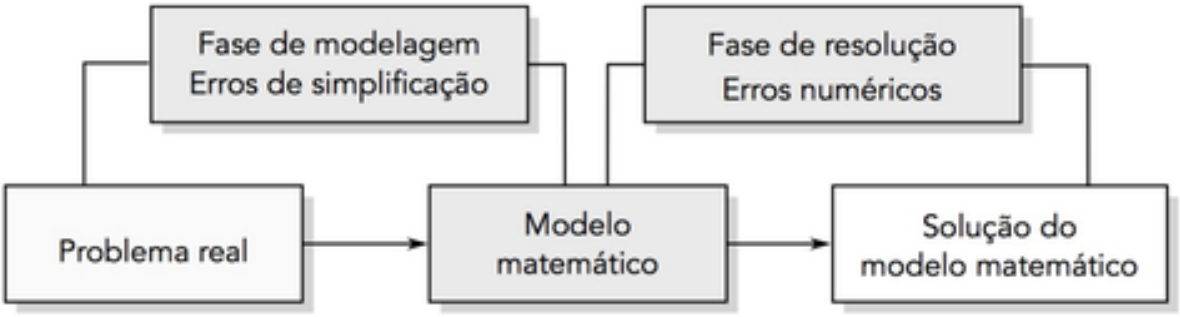

| Seção | Tema |
|---|---|
| 1.1 | O básico sobre erros |
| 1.2 | Aritmética de ponto flutuante |
| 1.3 | Erros nas aproximações numéricas |
| 1.4 | Exemplos de aplicações |


## 1.1 O básico sobre erros

De modo geral, a resolução de um problema, em ciências e engenharia, passa necessariamente por três etapas distintas:

1. A observação e entendimento do(s) fenômeno(s) envolvido(s);
2. A construção de um modelo matemático que represente o problema;
3. A resolução do modelo matemático por métodos numéricos ou analíticos.

As duas primeiras etapas consistem na **fase de modelagem** e a terceira é a **fase de resolução**. Tanto na fase de modelagem quanto na fase de resolução podem ocorrer erros, mesmo que esses erros não tornem a resposta obtida inválida ou inútil. Em geral busca-se uma solução aproximada, dentro de uma precisão pré-estabelecida.




Fonte: Arenalles e Darezzo (2016).

Os erros na fase de modelagem podem ser decorrentes de medições imprecisas, erros de anotação, simplificações e deduções equivocadas. Os erros na fase de resolução ocorrem devido a erros de cálculo, escolha de um método ineficaz, aproximações numéricas ou arredondamentos. Dizemos que métodos analíticos de resolução geram soluções exatas e métodos numéricos geram soluções aproximadas.

A maioria dos problemas envolvendo fenômenos reais produzem modelos matemáticos cuja solução analítica é difícil (ou impossível) de se obter, mesmo quando provamos que a solução existe. Nesses casos usamos **métodos numéricos**, os quais fornecem **soluções aproximadas**, que apesar de, em geral, serem diferentes da solução exata, podem ser suficientemente próximas para que sejam úteis em suas aplicações.

## Representação de números em ponto flutuante

Os métodos numéricos são especialmente úteis quando utilizamos computadores para realizar os cálculos, no entanto, existem limitações na capacidade de armazenar certos números internamente. A representação de números reais nas máquinas é feita, geralmente, utilizando o **sistema de ponto flutuante normalizado** e **base binária**. Ou seja, os números reais são armazenados em um conjunto fixo de bits. Cada bit pode assumir somente dois valores, 0 ou 1 (corte ou passagem de energia, respectivamente), dessa forma, nem todos os números reais podem ser representados de forma exata. Vamos entender isso um pouco melhor com os argumentos a seguir.

Sabemos que o número $0.125$, na base decimal, pode ser escrito como

$$0.125 = \frac{1}{10} + \frac{2}{100} + \frac{5}{1000}$$

ou, equivalentemente,

$$0.125 = 1 \times 10^{-1} + 2 \times 10^{-2} + 5 \times 10^{-3}.$$

De forma análoga, a fração binária de valor idêntico $0.001$ pode ser escrita como

$$0.001 = 0/2 + 0/4 + 1/8.$$

Nesse caso os denominadores das parcelas são potências de 2, ou seja, podemos escrever

$$0.001 = 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}.$$

No sistema de ponto flutuante normalizado (SPFN), um número é representado apenas pela sua forma fracionária, em que apenas um número finito de algarismos são armazenados (algarismos significativos), além do expoente da base e o sinal.

Por exemplo, o número $1.2345$ na base decimal será representado como

$$1.2345 = 0.12345 \times 10^1$$

em que $12345$ são os algarismos significativos e $1$ é o expoente da base $10$.

A maioria das máquinas que usamos atualmente operam na base 2 (binária) em que os algarismos disponíveis são 0 e 1. O número decimal $1.2$, pode ser representado na base binária pelo número $1.00110011001100110011\overline{0011}$ que tem infinitas casas após a vírgula, portanto nem todos algarismos significativos poderão ser armazenados. Supondo que apenas 10 algarismos possam ser armazenados, sua respresentação será

$$0.1001100110 \times 2^1.$$

No SPFN apenas os dígitos $1001100110$, o sinal (0 para positivo e 1 para negativo) e o expoente $1$ precisam ser armazenados para representar o número.

**Exemplo 1.1.1:** Para converter um inteiro decimal em binário, podemos usar a função `bin()` do Python. O resultado será uma string com o prefixo `'0b'` no começo. Veja alguns exemplos:

```
bin(22)
```

```
'0b10110'
```

Observe o que acontece fazendo a operação $1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$, como é mostrado a seguir

```
1*2**4 + 0*2**3 + 1*2**2 + 1*2**1 + 0*2**0
```

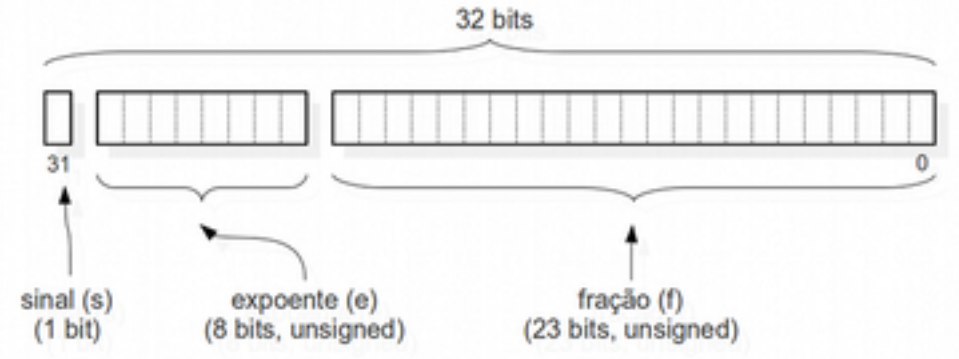
```
22
```

Use esse exemplo para testar outros números inteiros.

## Padrão IEEE

Em geral, uma máquina com 32 bit utiliza 23 bit para armazenar os algarismos da parte fracionária, 8 bit para os algarismos do expoente da base e 1 bit para armazenar o sinal. Para 64 bits, são 52 bits de digitos significativos, 11 bits para o expoente e 1 bit para o sinal.

O esquema para 32 bits é ilustrado na figura abaixo.




O padrão IEEE 754 é atualmente o mais utilizado para representar números reais. Na precisão `float` usa 23 bits,

para a parte fracionária, 8 bits para o expoente e 1 bit para o sinal. Na precisão `double`, usa 52 bits, para a parte fracionária, 11 bits para o expoente e 1 bit para o sinal, o que permite representar valores entre $2^{-1022}$ até $2^{1023}$. Alguns exemplos de representações do numero zero, infinito e "não número".

| IEEE 754 - Single Precision | | | Valor | |
|---|---|---|---|---|
| s | e | m | | |
| 0 | 0000 0000 | 000 0000 0000 0000 0000 0000 | +0 | Zero |
| 1 | 0000 0000 | 000 0000 0000 0000 0000 0000 | -0 | |
| 0 | 1111 1111 | 000 0000 0000 0000 0000 0000 | +Inf | Infinito Positivo |
| 1 | 1111 1111 | 000 0000 0000 0000 0000 0000 | -Inf | Infinito Negativo |
| 0 | 1111 1111 | 010 0000 0000 0000 0000 0000 | NaN | Not a Number |
| 1 | 1111 1111 | 010 0000 0000 0000 0000 0000 | -NaN | |

Alguns exemplos de tipos de dados disponíveis no módulo numérico **Numpy** (numpy.org) são mostrados abaixo para o caso de um

| | |
|---|---|
| `int8` | Byte (−128 to 127) |
| `int16` | Integer (−32768 to 32767) |
| `int32` | Integer (−2147483648 to 2147483647) |
| `int64` | Integer (−9223372036854775808 to 9223372036854775807) |
| `uint8` | Unsigned integer (0 to 255) |
| `uint16` | Unsigned integer (0 to 65535) |
| `uint32` | Unsigned integer (0 to 4294967295) |
| `uint64` | Unsigned integer (0 to 18446744073709551615) |
| `float_` | Shorthand for `float64` |
| `float16` | Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa |
| `float32` | Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa |
| `float64` | Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa |

Alguns números, como os irracionais ou a fração 1/3, por exemplo, têm representações fracionárias com infinitas casas decimais. Então, para serem computados precisam ser truncados ou arredondados. Além disso, alguns números com representação decimal finita, possuem representação binária infinita, e vice versa. Assim, erros de arredondamento podem ocorrer no processo de conversão entre bases e, também, nas operações aritméticas de ponto flutuante.

**Exemplo 1.1.2:** Observe o que acontece se fazemos $x = \frac{1}{3}$ e, então espera-se obter $x \times 10 \times 0.3 = \frac{1}{3} \times 3 = 1$. vejamos o que acontece computacionalmente.

```
x = 1/3
x
```

```
0.3333333333333333
```

```
x*10*0.3
```

```
0.9999999999999999
```

Tente fazer $x \times (10 \times 0.3)$ e tente explicar o que você observou nos resultados obtidos.

## Conversão entre bases

Usualmente, utilizamos o sistema de numeração decimal para representar números. Esse é um sistema de numeração posicional onde a posição do dígito indica a potência de 10 que o dígito representa. Por exemplo, o número 293 é decomposto como 293 = 2 centenas +9 dezenas + 3 unidades. O sistema de numeração posicional também pode ser usado com outras bases.

Dado um número real, $N$, é sempre possível representá-lo em qualquer base $b$, da seguinte forma

$$N_b = \sum_{i=n}^{m} a_i \times b^i$$

em que $a_i \in \{0, 1, 2, 3, ..., (b-1)\}$, com $n$ e $m$ inteiros.

### Base binária

$$N_2 = \sum_{i=n}^{m} a_i \times 2^i, \quad a_i \in \{0, 1\}$$

**Exemplos 1.1.3:** (Fonte: Arenalles e Darezzo, 2016)

a) $(1011)_2 = 1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3$

Neste caso, o binário só tem a parte inteira, com $i = 0, 1, 2, 3$, isto é, $n = 0$ e $m = 3$.

b) $(111.01)_2 = 1 \times 2^{-2} + 0 \times 2^{-1} + 1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2$

Neste caso, o binário tem parte inteira e parte fracionária, com $i = -2, -1, 0, 1, 2$, isto é, $n = -2$ e $m = 2$.

### Base decimal

$$N_{10} = \sum_{i=n}^{m} a_i \times 10^i, \quad a_i \in \{0, 1, 2, 3, ..., 9\}$$

**Exemplo 1.1.4:** (Fonte: Arenalles e Darezzo, 2016)

a) $(231)_{10} = 1 \times 10^0 + 3 \times 10^1 + 2 \times 10^2$

b) $(231.35)_{10} = 5 \times 10^{-2} + 3 \times 10^{-1} + 1 \times 10^0 + 3 \times 10^1 + 2 \times 10^2$

c) Podemos facilmente converter um número binário para a base decimal escrevendo o número em formato de somatória e realizando as 'contas' no sistema decimal, com é mostrado a seguir

$(1001.101)_2 =$

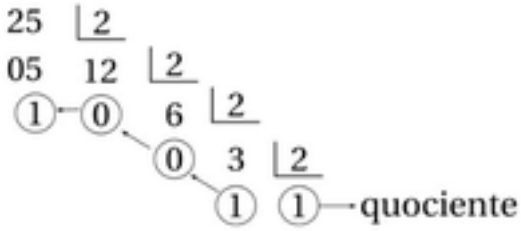
$1 \times 2^{-3} + 0 \times 2^{-2} + 1 \times 2^{-1} + 1 \times 2^0 + 0 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 =$

$1/8 + 0/4 + 1/2 + 1/1 + 0 \cdot 2 + 0 \cdot 4 + 1 \cdot 8 =$
$0.125 + 0.5 + 1.0 + 8.0 =$
$9.625$
Assim $(1001.101)_2 = (9,625)_{10}$.

A mudança de **base decimal para binária**, no caso de um número inteiro, é realizada dividindo o número sucessivamente por 2 até obter o quociente 1. Então, toma-se o quociente e os restos das divisões na ordem contrária em que foram obtidos para formar o número binário. Vejamos um exemplo.

**Exemplo 1.1.5:** (Fonte: Arenalles e Darezzo, 2016)

O número 25, na base 10, pode ser convertido para a base 2 procedendo sucessivas divisões por 2 como é mostrado abaixo.




Tomando-se o quociente 1 e os restos das divisões na ordem contrária em que foram obtidos, formamos o número binário 11001. Assim,

$$(25)_{10} = (11001)_2.$$

**Exemplo 1.1.6:** Faça um programa em Python para converter um inteiro decimal em um binário. Teste para o número $(11)_{10} = (1011)_2$. *Dica:* o operador `%` usado no lugar do operador `/` em uma divisão, retorna o resto dessa divisão.

```
# uma possível solução:
num = 11
while num>0:
    print(num%2)
    num = int(num/2)
```

```
1
1
0
1
```

Para converter uma fração decimal para a base binária procedemos sucessivas multiplicações por 2. A parte inteira do resultado obtido será o primeiro dígito binário. A parte fracionária será multiplicada novamente por 2 para obter-se os próximos dígitos no número binário, sucessivamente. vejamos um exemplo.

**Exemplo 1.1.7:**

O número $(0.1875)_{10}$ pode ser convertido para a base binária procedendo como é mostrado na figura abaixo

$$\begin{aligned}
(0.1875) \times (2) = 0.375 &\rightarrow \text{parte inteira } = 0 \text{ e parte fracionária } = 0.375 \\
(0.375) \times (2) = 0.75 &\rightarrow \text{parte inteira } = 0 \text{ e parte fracionária } = 0.75 \\
(0.75) \times (2) = 1.5 &\rightarrow \text{parte inteira } = 1 \text{ e parte fracionária } = 0.5 \\
(0.5) \times (2) = 1.0 &\rightarrow \text{parte inteira } = 1 \text{ e parte fracionária } = 0
\end{aligned}$$

Assim, obtemos que $(0.1875)_{10} = (0.0011)_2$.

**Exemplo 1.1.8:** Converta os números $(13.25)_{10}$ e $(0.2)_{10}$ para a base binária.

Commeçando com a parte inteira 13:

$$\begin{aligned} 13 \div 2 = 6 &\quad \rightarrow \text{resto} = 1 \\ 6 \div 2 = 3 &\quad \rightarrow \text{resto} = 0 \\ 3 \div 2 = 1 &\quad \rightarrow \text{resto} = 1 \\ 1 \div 2 = 0 &\quad \rightarrow \text{resto} = 1 \end{aligned}$$

Assim, $(13)_{10} = (1101)_2$.

Agora a parte fracionária $0,25$:

$$\begin{aligned} 0.25 \times 2 = 0.5 &\quad \rightarrow \text{ parte inteira } = 0 \text{ e parte fracionária } = 0.5 \\ 0.5 \times 2 = 1.0 &\quad \rightarrow \text{ parte inteira } = 1 \text{ e parte fracionária } = 0 \end{aligned}$$

Assim, $(0.25)_{10} = (0.01)_2$. Somando a parte inteira e a parte fracionária, obtemos que $(13.25)_2 = (1101.01)_2$.

Vejamos o que acontece ao convertermos o número decimal $(0.2)_{10}$ para a base binária:

$$\begin{aligned} 0.2 \times 2 = 0.4 &\quad \rightarrow \text{ parte inteira } = 0 \text{ e parte fracionária } = 0.4 \\ 0.4 \times 2 = 0.8 &\quad \rightarrow \text{ parte inteira } = 0 \text{ e parte fracionária } = 0.8 \\ 0.8 \times 2 = 1.6 &\quad \rightarrow \text{ parte inteira } = 1 \text{ e parte fracionária } = 0.6 \\ 0.6 \times 2 = 1.2 &\quad \rightarrow \text{ parte inteira } = 1 \text{ e parte fracionária } = 0.2 \\ 0.2 \times 2 = 0.4 &\quad \rightarrow \text{ o ciclo se repete infinitamente.} \end{aligned}$$

Assim, tem-se que $(0.2)_{10} = (0.0011\overline{0011})$. Concluimos com isso que o $(0.2)_{10}$ não pode ser representado de forma exata em um computador.

**Exemplo 1.1.9:** Um programa em Python para converter um número da base decimal para a base binária.

```
from math import floor
```

```
num_dec = 13.25
bin_int = ''

# Parte inteira
num = floor(num_dec)
while num>0:
    resto = num%2
    num = floor(num/2)
    bin_int = bin_int+str(resto)

bin_int = bin_int[::-1]
print ('Parte inteira:', bin_int)
```

```
Parte inteira: 1101
```

```
# Parte fracionária
num = num_dec - floor(num_dec)
bin_frac = '.'
while num>0:
    p_int = floor(num*2)
    num = num*2 - floor(num*2)
    bin_frac = bin_frac+str(p_int)

print ('Parte fracionaria:', bin_frac)
```

```
Parte fracionaria: .01
```

```
print ('numero binário:', bin_int+bin_frac)
```

```
numero binário: 1101.01
```

## Outras bases

**Exemplo 1.1.10:** Em Python existem funções para representar números em diferentes bases, incluindo os sistemas octogonal e hexadecimal. Para converter um inteiro em um binário, octal, ou hexadecimal, podemos proceder como é mostrado abaixo

```
x = 1234
print ('Binário:     x =', bin(x))
print ('Octal:       x =', oct(x))
print ('Hexadecimal: x =', hex(x))
```

```
Binário:     x = 0b10011010010
Octal:       x = 0o2322
Hexadecimal: x = 0x4d2
```

Ou, alternativamente

```
print ('Binário:     x =', format(x, 'b'))
print ('Octal:       x =', format(x, 'o'))
print ('Hexadecimal: x =', format(x, 'x'))
```

```
Binário:     x = 10011010010
Octal:       x = 2322
Hexadecimal: x = 4d2
```

### Exercícios

(Fontes: Ruggiero (2016), Chapra e Canale (2016))

---

**1.** Converta os seguintes números decimais para sua forma binária:

a) $(37)_{10}$

b) $(2345)_{10}$

c) $(0,1217)_{10}$

d) $(71,263)_{10}$

e) $(3,147)_{10}$

f) $(625,125)_{10}$

---

**2.** Converta os seguintes números na base 2 para a base 10:

a) $(101101)_2$

b) $(101,101)_2$

c) $(110101011)_2$

d) $(0,1101)_2$

e) $(0,111111101)_2$

f) $(0,01101)_2$

---

### Respostas

**1.** $a)$ $(100101)_2$ $b)$ $(100100101001)_2$ $c)$ $(0{,}000111110010)_2$ $d)$ $(1000111{,}010000110101\ldots)_2$ $e)$ $(11{,}0010010110100\ldots)_2$ $f)$ $(1001110001{,}001)_2$

**2.** $a)$ $(45)_{10}$ $b)$ $(5{,}625)_{10}$ $c)$ $(427)_{10}$ $d)$ $(0{,}8125)_{10}$ $e)$ $(0{,}994140625)_{10}$ $f)$ $(0{,}40625)_{10}$

## 1.2 Aritmética de ponto flutuante

Um número no Sistema de Ponto Flutuante (STF) é caracterizado por uma base $b$, um número de dígitos significativos $n$ e um expoente $exp$. Assim, dizemos que um número real $n_r$ está representado no SPF se estiver na forma

$$n_r = \pm 0.d_1 d_2 d_3 ... d_n \times b^{exp}$$

em que $m = d_1 d_2 ... d_n$ é a mantissa com $n$ dígitos significativos $d_1, d_2, ..., d_n$ satisfazendo $0 \leq d_i \leq (b-1)$, $i = 1, ..., n$ e $d_1 \neq 0$.

O expoente $exp$ da base $b$ varia da seguinte maneira

$$exp_{min} \leq exp \leq exp_{max}$$

sendo $exp_{min} \leq 0$ e $exp_{max} \geq 1$ com $exp_{min}$ e $exp_{max}$ inteiros.

Considerando o sistema de ponto flutuante normalizado, na forma genérica $SPF(b, n, exp_{min}, exp_{max})$, temos que apenas um conjunto finito de números reais podem ser exatamente representados, tal que

a) o número zero é representado como:

$$0.000...0 \times b^{exp_{min}}$$

b) o menor positivo exatamente representável é

$$0.100...0 \times b^{exp_{min}}$$

c) o maior positivo exatamente representável é

$$0.(b-1)(b-1)(b-1)...(b-1) \times b^{exp_{max}}$$

d) o número máximo de mantissas positivas possíveis é

$$mantissas_{+} = (b-1)b^{n-1}$$

e) o número máximo de expoentes possíveis é

$$exp_{possiveis} = exp_{max} - exp_{min} + 1$$

f) o número máximo de reais positivos é

$$NR_{+} = mantissas_{+} \times exp_{possiveis}$$

g) assim, considerando os números negativos e o zero, o número total de números reais exatamente representáveis é

$$NR_{t} = 2 \times NR_{+} + 1$$

**Exemplo 1.2.1:** Em Python também dispomos de algumas funções para acessar as informações do sistema de ponto flutuante:

```
import sys
print ("Máximo representável:", sys.float_info.max)
print ("Mínimo representável:", sys.float_info.min)
print ("Épsilon da máquina:", sys.float_info.epsilon)
print ("Todas informações:", sys.float_info)
```

```
Máximo representável: 1.7976931348623157e+308
Mínimo representável: 2.2250738585072014e-308
Épsilon da máquina: 2.220446049250313e-16
Todas informações: sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_
↪10_exp=308, min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-307, dig=15,␣
↪mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)
```

**Exemplo 1.2.2:** Considere o sistema de ponto flutuante normalizado **SPF (3, 2,−1, 2)**, de base 3, 2 dígitos na mantissa, menor expoente igual a −1 e maior expoente 2.

Para esse sistema tem-se que os números

$$x = \frac{1}{9} = (0.10)_3 \times 3^{-1}$$

e

$$y = 5 = (0.12)_3 \times 3^2$$

são exatamente representáveis, no entanto,

$$(x + y) = (0.00010)_3 \times 3^2 + (0.12)_3 \times 3^2 = (0.1201)_3 \times 3^2$$

não é exatamente representável, uma vez que a mantissa é de 2 dígitos. Nesse sistema $(x + y) = (0.12)_3 \times 3^2$. Ou seja, $(x + y) = y$.

Assim, pode ocorrer de propriedades bem conhecidas no conjunto dos números reais, como as propriedades comutativa e associativa na adição e as propriedades comutativa e distributiva na multiplicação, não serem verdadeiras no sistema de ponto flutuante

Por exemplo, em um sistema de ponto flutuante, base decimal, 3 dígitos e arredondamento, as seguintes operações geram **erros de arredondamento**

Sejam $x = 5.26$, $y = 9.34$ e $z = 5.04$. Então:

$$(x + y) + z = (5.26 + 9.34) + 5.04 = 14.6 + 5.04 = 19.6$$

e

$$x + (y + z) = 5.26 + (9.34 + 5.04) = 5.26 + 14.4 = 19.7$$

Ou seja,

$$x + (y + z) \neq (x + y) + z$$

**Exemplo 1.2.3:** Exemplos de somas que fornecem resultados diferentes de forem realizadas em uma ordem diferente. Tente explicar.

```
print ("0.2 + 0.4 - 0.5 =", 0.2 + 0.4 - 0.5)
print ("- 0.5 + 0.4 + 0.2 =", - 0.5 + 0.4 + 0.2)
print ("0.2 -0.1 + 0.2 - 0.1 =", 0.2 -0.1 + 0.2 - 0.1)
print ("0.2 - 0.1 + (0.2 - 0.1) =", 0.2 - 0.1 + (0.2 - 0.1))
print ("0.2 + 0.3 + 0.1 =", 0.2 + 0.3 + 0.1 )
print ("0.2 + 0.1 + 0.3 =", 0.2 + 0.1 + 0.3)
```

```
0.2 + 0.4 - 0.5 = 0.10000000000000009
- 0.5 + 0.4 + 0.2 = 0.10000000000000003
0.2 -0.1 + 0.2 - 0.1 = 0.20000000000000004
0.2 - 0.1 + (0.2 - 0.1) = 0.2
0.2 + 0.3 + 0.1 = 0.6
0.2 + 0.1 + 0.3 = 0.6000000000000001
```

**Exemplo 1.2.4:** Alguns exemplos de operações simples que ilustram erros devido à limitação do computador em representar certos números reais.

```
0.1 + 0.2 == 0.3
```

```
False
```

```
0.4 + 0.2
```

```
0.6000000000000001
```

```
from decimal import Decimal
Decimal(0.1)
```

```
Decimal('0.1000000000000000055511151231257827021181583404541015625')
```

**Exemplo 1.2.5:** Considere o seguinte processo iterativo

$$x^{(0)} = \frac{1}{3}$$

e

$$x^{(n+1)} = 4x^{(n)} - 1, \quad n = 0, 1, 2, ...$$

O resultado dessa sequência é

$$x^{(1)} = x^{(2)} = x^{(3)} = ... = x^{(n)} = \frac{1}{3}$$

Agora observe o programa abaixo. Você pode explicar os resultados?

```
x = 1/3
for i in range(20):
    x = 4*x -1
    print (x)
```

```
0.33333333333333326
0.33333333333333304
0.33333333333333215
0.3333333333333286
0.3333333333333144
0.33333333333325754
0.33333333333303017
0.3333333333321207
0.3333333333284827
0.3333333333139308
0.3333333332557231
0.3333333330228925
0.3333333320915699
0.3333333283662796
0.3333333134651184
```

(continua na próxima página)

(continuação da página anterior)

```
0.33333325386047363
0.33333301544189453
0.3333320617675781
0.3333282470703125
0.33331298828125
```

## Exercícios

(Fontes: Ruggiero (2016), Chapra e Canale (2016))

---

**1.** Seja um sistema de aritmética de ponto flutuante de quatro dígitos e base decimal. Dados os números:

$$x = 0.7237 \times 10^4 \quad y = 0.2145 \times 10^{-3} \quad \text{e} \quad z = 0.2585 \times 10^1$$

efetue as seguintes operações e obtenha o erro relativo no resultado, supondo que $x, y$ e z estão exatamente representados:
a) $\mathrm{x} + \mathrm{y} + \mathrm{z}$
d) $(\mathrm{xy})/\mathrm{z}$
b) $x - y - z$
e) $\mathrm{x}(\mathrm{y}/\mathrm{z})$
c) $\mathrm{x}/\mathrm{y}$

---

**2.** (a) Calcule o polinômio $y = x^3 - 5x^2 + 6x + 0,55$ em $x = 1,37$. Use aritmética com 3 algarismos significativos e truncamento. Estime o erro.
(b) Repita (a), mas expresse $y$ como $y = ((x–5)x + 6)x + 0,55$ Estime o erro e compare com a parte (a)

---

**3.** Use aritmética com 5 algarismos significativos para determinar as raízes da equação

$$x^2–5000,002x + 10 = 0$$

com as fórmulas

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

e

$$x = \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}.$$

Então, calcule os erros percentuais para o seu resultado.

---

**4.** Faça seu próprio programa para determinar o épsilon da máquina do seu computador.

---

### Respostas

**1.**

$$a)\ x+y+z = 0{,}7240 \times 10^4 \qquad \left|\text{ER}_{x+y+z}\right| < 10^{-3}$$

$$b)\ x-y-z = 0{,}7234 \times 10^4 \qquad \left|\text{ER}_{x-y-z}\right| < 1{,}0002 \times 10^{-3}$$

$$c)\ x/y = 0{,}3374 \times 10^8 \qquad \left|ER_{x/y}\right| < \tfrac{1}{2} \times 10^{-3}$$

$$d)\ (xy)/z = 0{,}6004 \qquad \left|\text{ER}_{(xy)/z}\right| < 10^{-3}$$

$$e)\ x(y/z) = 0{,}6005 \qquad \left|\text{ER}_{x(y/z)}\right| < 10^{-3}$$

## 1.3 Erros nas aproximações numéricas

Uma solução numérica é geralmente uma solução aproximada, não exata. A diferença entre a solução exata e a solução aproximada é chamada de erro e é muitas vezes inerente ao processo. Em problemas reais, erros numéricos raramente podem ser evitados.

Os erros podem ocorrer na fase de modelagem matemática, devido a simplificações e a imprecisão nos dados, quanto na fase de resolução, no processo de conversão de base e devido à limitações da máquina em armazenar um número muito grande de dígitos. Ou também, na aproximação de funções representadas por séries infinitas. Como não é possível realizar a soma infinita, então, as série são truncadas e considera-se apenas um número finito de termos, introduzindo o que chamamos de **erro de truncamento**.

Sabemos também que o computador realiza operações aritméticas retendo apenas um número fixo de algarismos significativos. Números como o $\pi$, $e$ ou $\sqrt{7}$ não podem ser expressos por um número fixo de algarismos, portanto não podem ser representados exatamente por um computador. Além disso, alguns números que têm representação fracionária finita no sistema decimal, não podem ser representados com exatidão no sistema binário. A discrepância introduzida pela omissão de algarismos significativos é chamada **erro de arredondamento**.

**Arredondamento:** Dizemos que um número $n_r$ na base decimal foi arredondado na posição $k$ se todos os dígitos de ordem maior que $k$ forem descartados segundo o seguinte critério:

- O dígito de ordem $k$ é acrescido de uma unidade se o de ordem $(k+1)$ for maior ou igual a metade da base
- do contrário, o número $nr$ é representado com os $k$ dígitos iniciais.

**Exemplo 1.3.1:**

Consideremos o sistema $SPF(b, n, exp_{min}, exp_{max}) = SPF(10, 4, -5, 5)$

a) Se $a = 0,5324 \times 10^3$ e $b = 0,4212 \times 10^{-2}$, então $a \times b = 0,22424688 \times 10^1$, que é arredondado e armazenado como $a \times b = 0,2242 \times 10^1 = 2,242$.

b) Se $a = 0,5324 \times 10^3$ e $b = 0,1237 \times 10^2$, então $a + b = 0,54477 \times 10^3$, que é arredondado e armazenado como $a + b = 0,5448 \times 10^3 = 544,8$.

Usando Python, podemos reproduzir o os resultados acima fazendo:

```
#a)
a = 0.5324*10**3
b = 0.4212*10**-2
print('Exato:',a*b)
```

```
Exato: 2.2424688
```

```
print('Arredondado:',round(0.22424688, 4)*10**1)
```

```
Arredondado: 2.242
```

```
#b)
a = 0.5324*10**3
b = 0.1237*10**2
print('Exato:',a+b)
```

```
Exato: 544.77
```

```
print('Arredondado:',round(0.54477, 4)*10**3)
```

```
Arredondado: 544.8
```

**Exemplo 1.3.2:** Neste exemplo vamos calcular o polinômio $y = x^3 - 5x^2 + 6x + 0,55$ em $x = 1,37$ usando aritmética de ponto flutuante com 3 algarismos significativos. Em seguida, vamos repetir os cálculos expressando $y$ como $y = ((x–5)x + 6)x + 0,55$ e comparar.

```
x = 1.37
print('x³=',x**3)
print('x²=',x**2)
```

```
x³= 2.5713530000000007
x²= 1.8769000000000002
```

```
print('5x²=',5*1.88)
```

```
5x²= 9.399999999999999
```

```
6*x
```

```
8.22
```

```
y = (0.257 - 0.94 + 0.822 + 0.055)*10
print('Letra a: y=', y)
```

```
Letra a: y= 1.94
```

```
ye = x**3 - 5*x**2 + 6*x+ 0.55
print('Valor exato: y=',ye)
```

```
Valor exato: y= 1.956853
```

```
print('Erro letra a:', abs(y-ye))
```

```
Erro letra a: 0.016853000000000007
```

```
y = round((round((x-5)*x,2)+6)*x,2)+0.55
print('Letra b: y=', y)
```

```
Letra b: y= 1.96
```

```
print('Erro letra b:', abs(y-ye))
```

```
Erro letra b: 0.003147000000000011
```

**Truncamento:** Quando representamos uma função utilizando uma série infinita mas considerarmos apenas um número finito de termos na série, dizemos que estamos cometendo um erro de truncamento.

**Exemplo 1.3.3:** A função exponencial $e^x$ pode ser representada pela série

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

Sabemos que $e$ é um número irracional e portanto sua representação fracionária têm infinitas casas decimais. Podemos obter uma aproximação para o número $e$ truncando a série

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{3!} + \frac{1}{4!} + ...$$

Por exemplo, com 5 termos da série obtemos:

```
e = 1+1+1/2+1/6+1/24+1/120
print ('Valor aproximado:',e)
```

```
Valor aproximado: 2.7166666666666663
```

```
import math
print ('Valor "exato":', math.e)
```

```
Valor "exato": 2.718281828459045
```

Use o exemplo 1.3.3 como modelo para resolver o Exercício 1 no final do desta seção.

## Erro absoluto

Definimos erro absoluto por

$$E_{abs} = |a_{ex} - a_{aprox}|$$

em que $a_{ex}$ é o valor exato da grandeza considerada e $a_{aprox}$ é o valor aproximado da mesma grandeza numérica.

## Erro relativo

Definimos erro relativo por

$$E_{rel} = \frac{E_{abs}}{|a_{ex}|}$$

em que $a_{ex}$ é o valor exato da grandeza considerada e $a_{aprox}$ é o valor aproximado da mesma grandeza.

Podemos observar que o erro relativo nos fornece mais informações sobre a qua- lidade do erro que estamos cometendo num determinado cálculo, uma vez que no erro absoluto não é levada em consideração a ordem de grandeza do valor calculado, enquanto no erro relativo esta ordem é contemplada.

Quando obtemos uma aproximação numérica por meio de uma sequência convergete $x_0, x_1, x_2, ..., x_k, ...$mas não conhecemos a solução exata, o erro relativo no termo $k$ pode ser expresso como *diferença relativa*, dada por

$$\frac{|x_{k-1} - x_k|}{|x_k|}$$

**Exemplo 1.3.4:** A equação $f(x) = x^2 - a = 0$, com $a > 0$, podemos utilizar o seguinte processo iterativo:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right), \quad n = 1, 2, ...$$

A partir de uma aproximação inicial $x_0$, usamos essa expressão para gerar a sequência de soluções aproximadas $x_1$, $x_2$, …

Assim, por exemplo, para calcular aproximações para $\sqrt{2}$, podemos fazer

$x_0 = 1.5$

$x_1 = 1/2(1.5 + 2/1.5) = 1.4166666666666665$

$x_2 = 1/2(1.4166666666666665 + 2/1.4166666666666665) = 1.4142156862745097$

E assim por diante.

Em Python podemos fazer:

```
x = 1.0
x_ant = 0.0

err = 10.0

while err>0.0001:
    x = 0.5*(x + 2/x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print (x, err)
```

```
1.5 1.0
1.4166666666666665 0.058823529411764816
1.4142156862745097 0.001733102253032923
1.4142135623746899 1.5018239651360762e-06
```

Usando o exemplo 1.3.4 acima, resolva o Exercício 2 no final da seção.

**Exemplo 1.3.5:**

A sequência $x_k = 1 + \frac{1}{x_{k-1}}$ converge para $\overline{x} = \frac{1+\sqrt{5}}{2}$. Fazendo $i = 0, 1, 2, ..., 10$ e $x_0 = 1$, mostre a sequência de aproximações juntamente com as diferenças relativas e os erros relativos para cada termo.

```
import math
x = 1.
x_ant = x
x_exa = (1+math.sqrt(5))/2

for k in range(10):
    x = 1.+1./x
    dif = abs(x-x_ant)/abs(x)
    err = abs(x-x_exa)/abs(x_exa)
    print (x, x_exa, err, dif)
    x_ant = x
```

```
2.0 1.618033988749895 0.23606797749978967 0.5
1.5 1.618033988749895 0.07294901687515776 0.3333333333333333
1.6666666666666665 1.618033988749895 0.030056647916491288 0.09999999999999992
1.6 1.618033988749895 0.01114561800016822 0.04166666666666652
1.625 1.618033988749895 0.004305231718579094 0.01538461538461533
1.6153846153846154 1.618033988749895 0.0016374027886314115 0.005952380952380931
1.619047619047619 1.618033988749895 0.0006264579760202099 0.002262443438914019
1.6176470588235294 1.618033988749895 0.0002391358457583512 0.0008658008658008628
1.6181818181818182 1.618033988749895 9.136361346616536e-05 0.00033046926635820254
1.6179775280898876 1.618033988749895 3.48946069117676e-05 0.00012626262626264404
```

**Exemplo 1.3.6:** Podemos obter uma aproximação para a derivada primeira de uma função $y = f(x)$ usando a fórmula:

$$\frac{dy}{dx} \cong \frac{\Delta y}{\Delta x} = \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

tomando um valor pequeno de $\Delta x$.

Sabemos que a derivada primeira de $f(x) = sen(x)$ é $f'(x) = cos(x)$. Vamos calcular as derivadas numericamente para deversos pontos no intervalo $[0, 2\pi]$ e fazer o gráfico.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,2]
```

```
xi = np.linspace(0, 2*np.pi, 21)
h=(xi[-1]-xi[0])/(len(xi)-1)

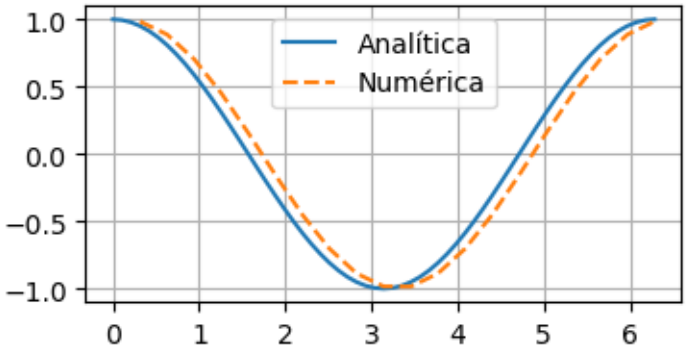
f = lambda x: np.sin(x)

dxi = (f(xi[1:])-f(xi[0:-1]))/h

xplot = np.linspace(0, 2*np.pi, 61)

plt.plot(xplot,np.cos(xplot),'-', label='Analítica')
plt.plot(xi[1:],dxi,'--',label='Numérica')

plt.grid()
plt.legend()
plt.show()
```




**Exemplo 1.3.7:** A **Série de Taylor** pode ser usada para aproximar funções em torno de um ponto $a$ truncando a somatória infinita

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n.$$

Nos códigos a seguir vamos usar a série de Taylor para calcular $\cos(x)$ e $e^x$ em um conjunto de pontos e, então, fazer o gráfico dessas funções.

```
# cos(x)
import numpy as np
import matplotlib.pyplot as plt
import math

X = np.linspace(-4., 3., 51)
print(X)


Y = []
#x=1.0
for x in X:
    valor = 1.0
```

(continua na próxima página)

(continuação da página anterior)

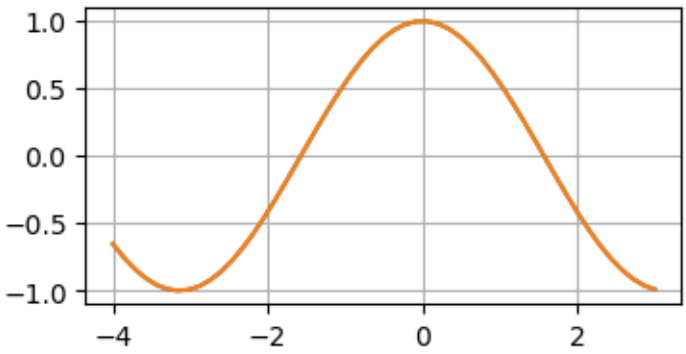
```
    for i in range(1,11,1):
        fat = math.factorial(2*i)
        valor = valor + (x**(2*i)*(-1)**i)/fat
        #fat = math.factorial(2*i+1)
        #valor = valor + (x**(2*i+1)*(-1)**i)/fat

        #print (x, valor)
    Y.append(valor)



plt.plot(X, np.cos(X),X, Y)
plt.grid()
plt.show()
```

```
[-4.   -3.86 -3.72 -3.58 -3.44 -3.3  -3.16 -3.02 -2.88 -2.74 -2.6  -2.46
 -2.32 -2.18 -2.04 -1.9  -1.76 -1.62 -1.48 -1.34 -1.2  -1.06 -0.92 -0.78
 -0.64 -0.5  -0.36 -0.22 -0.08  0.06  0.2   0.34  0.48  0.62  0.76  0.9
  1.04  1.18  1.32  1.46  1.6   1.74  1.88  2.02  2.16  2.3   2.44  2.58
  2.72  2.86  3.  ]
```




```
#e^x

X = np.linspace(-2., 2., 21)
#print(X)

Y = []
#x=1.0
for x in X:
    valor = 1.0
    fat_i = 1.0
    for i in range(1,11,1):
        fat_i = fat_i * i
        valor = valor + (x**i)/fat_i
```

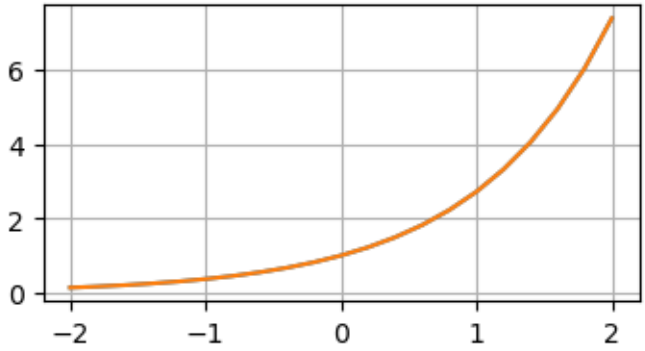
(continua na próxima página)

(continuação da página anterior)

```
        #print (x, valor)
    Y.append(valor)



plt.plot(X, np.exp(X), X, Y)
plt.grid()
plt.show()
```




## Exercícios

---

**1.** A função seno pode ser representada pela série de Maclaurin, fazendo

$$\operatorname{sen} x = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

Encontre aproximações para $\operatorname{sen}(\pi/4)$ com 3, 5 e 20 termos.

---

**2.** Usando o exemplo 1.3.4 como referência, encontre o valor de $\sqrt{3}$ com um erro relativo menor que $10^{-5}$. Quantas iterações são necessárias fazendo $x_0 = 1$? O número de iterações diminui se fizermos $x_0 = 1.5$?

---

**3.** A função cosseno pode ser representada pela série de Maclaurin, fazendo

$$\cos x = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}.$$

Descubra quantos termos da série são necessários para obter uma aproximação para $\cos(\pi/6)$ com erro relativo menor que $10^{-4}$.

---

**4.** Sabendo que a função $ln(1+x)$ pode ser representada pela série de Maclaurin, fazendo

$$\ln(1+x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{n+1} x^{n+1} \quad \text{para } |x| < 1$$

Descubra quantos termos da série são necessários para obter uma aproximação para $ln(0.2)$ com erro relativo menor que $10^{-4}$.

---

**5.** Usando a fórmula mostrada no Exemplo 1.3.6, encontre uma aproximação para a derivada de $f(x) = 2^x sen(x)$ em $x = 1,23$ e calcule o erro relativo.

---

## 1.4 Exemplos de aplicações

A seguir são apresentados alguns exemplos envolvendo a modelagem matemática de problemas físicos e sua respectiva resolução numéricas.

**Exemplo 1.4.1:** (Fórmula iterativa para obter raiz quadrada)

A equação $f(x) = x^2 - a = 0$, com $a > 0$, podemos utilizar o seguinte processo iterativo:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right), \quad n = 1, 2, ...$$

A partir de uma aproximação inicial $x_0$, usamos essa expressão para gerar a sequência de soluções aproximadas $x_1$, $x_2$, ...

Assim, por exemplo, para calcular aproximações para $\sqrt{2}$, podemos fazer

$x_0 = 1.5$

$x_1 = 1/2(1.5 + 2/1.5) = 1.4166666666666665$

$x_2 = 1/2(1.4166666666666665 + 2/1.4166666666666665) = 1.4142156862745097$

E assim por diante.

Em Python podemos fazer:

```
x = 1.0
x_ant = 0.0

err = 10.0

while err>0.0001:
    x = 0.5*(x + 2/x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print (x, err)
```

```
1.5 1.0
1.4166666666666665 0.058823529411764816
1.4142156862745097 0.001733102253032923
1.4142135623746899 1.5018239651360762e-06
```

Para obter $\sqrt{3}$ podemos fazer

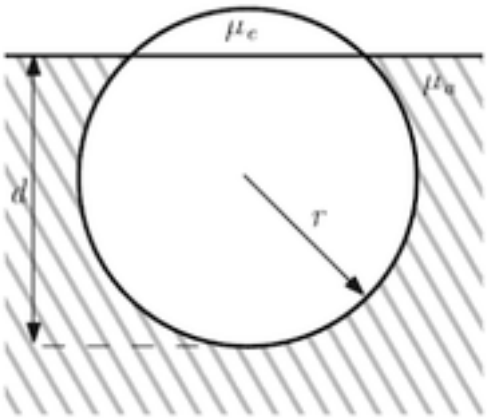
```
x = 2.0

for i in range(6):
    x = 0.5*(x + 3/x)
    print (x)
```

```
1.75
1.7321428571428572
1.7320508100147274
1.7320508075688772
1.7320508075688772
1.7320508075688772
```

**Exemplo 1.4.2:** (Obtendo a aproximação da raiz de uma equação de grau 3)

Uma esfera de madeira (densidade relativa $\mu_e = 0,638$ e raio $r = 10cm$) é colocada a flutuar sobre a água (densidade relativa $\mu = 1$), conforme figura. Qual é a altura $d$ da esfera que ficará submersa?




Inicialmente, observemos que, pelo *Princípio de Arquimedes* o peso da esfera deve ser equilibrado com o peso do volume de água deslocada

$$M_e g = M_a g$$

$$\mu_e \frac{4\pi r^3}{3} = \mu_a \int_0^d \pi \left[r^2 - (x-r)^2)\right] dx$$

$$\mu_e \frac{4\pi r^3}{3} = \mu_a \frac{\pi d^2(3r-d)}{3}$$

$$4\mu_e r^3 = \mu_a d^2(3r-d)$$

substituindo os valores obtemos

$$d^3 - 30d^2 + 2552 = 0.$$

A solução analítica para esse problema pode ser encontrada mas é um tanto complicada. No entanto, com uma rotina podemos encontrar uma aproximação numericamente.

*Solução:* Sabemos que a solução é um numero entre 0 e 20, então calculamos os valores da função $f(d) = d^3 - 30d^2 + 2552$ para valores entre 0 e 20 obtendo:

```
for d in range(20):
    print ('d=',d,', f(d)=', d**3-30*d**2+2552)
```

```
d= 0 , f(d)= 2552
d= 1 , f(d)= 2523
d= 2 , f(d)= 2440
d= 3 , f(d)= 2309
d= 4 , f(d)= 2136
d= 5 , f(d)= 1927
d= 6 , f(d)= 1688
d= 7 , f(d)= 1425
d= 8 , f(d)= 1144
d= 9 , f(d)= 851
d= 10 , f(d)= 552
d= 11 , f(d)= 253
d= 12 , f(d)= -40
d= 13 , f(d)= -321
d= 14 , f(d)= -584
d= 15 , f(d)= -823
d= 16 , f(d)= -1032
d= 17 , f(d)= -1205
d= 18 , f(d)= -1336
d= 19 , f(d)= -1419
```

Observemos que a função muda de sinal entre $d = 11$ e $d = 12$, então, podemos refinar a aproximação gerando alguns valores entre 11 e 12.

```
for x in range(10):
    d = 11+x*0.1
    print ('d=',d,', f(d)=', d**3-30*d**2+2552)
```

```
d= 11.0 , f(d)= 253.0
d= 11.1 , f(d)= 223.33100000000013
d= 11.2 , f(d)= 193.72800000000007
d= 11.3 , f(d)= 164.19700000000012
d= 11.4 , f(d)= 134.7439999999997
d= 11.5 , f(d)= 105.375
d= 11.6 , f(d)= 76.09599999999955
d= 11.7 , f(d)= 46.912999999999556
d= 11.8 , f(d)= 17.831999999999425
d= 11.9 , f(d)= -11.141000000000076
```

Observemos que a função muda de sinal entre $d = 11.8$ e $d = 11.9$, então podemos refinar a aproximação gerando alguns valores.

```
for x in range(10):
    d = 11.8+x*0.01
    print ('d=',d,', f(d)=', d**3-30*d**2+2552)
```

```
d= 11.8 , f(d)= 17.831999999999425
d= 11.81 , f(d)= 14.929740999999922
d= 11.82 , f(d)= 12.028567999999723
d= 11.83 , f(d)= 9.128486999999495
d= 11.84 , f(d)= 6.229503999999906
d= 11.850000000000001 , f(d)= 3.3316249999998035
d= 11.860000000000001 , f(d)= 0.43485599999894475
d= 11.870000000000001 , f(d)= -2.4607969999997295
d= 11.88 , f(d)= -5.355328000000554
d= 11.89 , f(d)= -8.248731000000134
```

Assim, podemos tomar como uma solução aproximada o ponto médio entre 11.86 e 11.87 que resolta em $d = 11.865cm$ com erro inferior a 0.005.

**Exemplo 1.4.3:** (Velocidade de um corpo em queda livre)

Um corpo em queda livre pode ser descrito pelo modelo matemático

$$\frac{dv}{dt} = \frac{F}{m}$$

derivado da 2ª Lei de Newton $F = ma$. A força $F$ atuante sobre o corpo, composta pela soma de duas forças de mesma direção e sentidos opostos, a força gravitacional para baixo ($F_D$) e a força da resistência do ar para cima ($F_U$). Então

$$\frac{dv}{dt} = \frac{(F_D + F_U)}{m}$$

como $F_D = mg$, com $m$ massa e $g$ aceleração da gravidade, e $F_U = -cv$, com $c$ sendo uma constante que depende das características do corpo em queda livre e $v$ a velocidade instantânea. Então

$$\frac{dv}{dt} = \frac{mg - cv}{m}$$

ou

$$\frac{dv}{dt} = g - \frac{c}{m}v$$

O modelo então consiste em uma equação diferencial cuja solução pode ser obtida analitivamente. Se o corpo em queda livre entrou em movimento a partir do repouso, ou seja, $v = 0$ em $t = 0$, a solução analítica é dada por

$$v(t) = \frac{mg}{c}\left(1 - e^{-(c/m)t}\right)$$

Se considerarmos $m = 68,1kg$ entrando em movimento de queda livre a partir do repouso $v(t) = 0$ em $t = 0$ e o coeficiente de arrasto $c = 12.5kg/s$, a solução fica

$$v(t) = \frac{9,81(68,1)}{12,5}(1 - e^{(-12,5/68,1)t} = 53,44(1 - e^{-0,18355t})$$

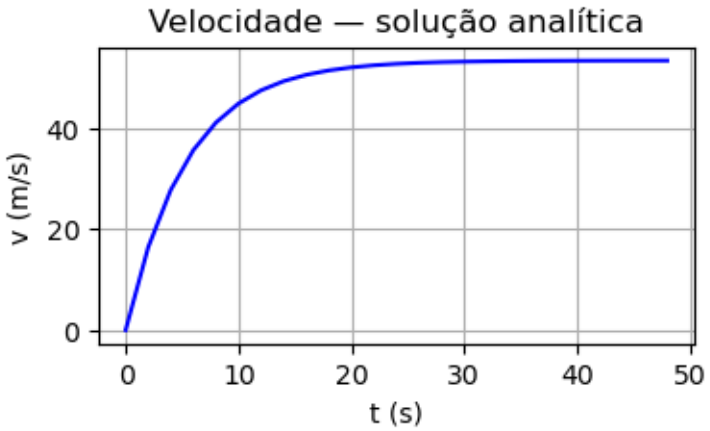
Assim, podemos obter o valor da velocidade para qualquer instante $t$. Tomando valores de $t$ de a cada 2 segundos, e

calculando a velocidade a partir de $t = 0$ obtemos

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,2]

t = np.arange(0, 50, 2)
v = lambda t: 53.44*(1 - np.exp(-0.18355*t))
print(t)
print(v(t))
plt.plot(t, v(t), "b-")
plt.xlabel("t (s)")
plt.ylabel("v (m/s)")
plt.title("Velocidade — solução analítica")
plt.grid()
plt.show()
```

```
[ 0  2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
 48]
[ 0.         16.41995476 27.79472025 35.67447948 41.13310679 44.91451827
 47.53405465 49.34871325 50.60580051 51.47663561 52.07989823 52.4978026
 52.78730183 52.98784963 53.12677718 53.22301791 53.28968775 53.33587264
 53.36786677 53.3900304  53.40538404 53.41602013 53.42338817 53.42849232
 53.43202816]
```




Por outro lado, supondo que a solução analítica não fosse conhecida (ou que fosse impossível obtê-la), poderíamos encontrar uma solução aproximada numericamente. Para isso podemos aproximar a derivada $\frac{dv}{dt}$ por

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1} - v(t_i))}{t_{i+1} - t_i}$$

então

$$\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} = g - \frac{c}{m} v(t_i)$$

que pode ser reescrita como

$$v(t_{i+1}) = v(t_i) + \left[g - \frac{c}{m}v(t_i)\right](t_{i+1} - t_i)$$

e ser usada para obter os valores de $v$ para cada instante $t_i$, $i = 0, 2, 4, 6, ...$, a partir do instante inicial $t_0 = 0$.

```
import numpy as np
import matplotlib.pyplot as plt

tempos = np.arange(0, 50, 2)

v_aprox = [0.]
v_ant = 0.

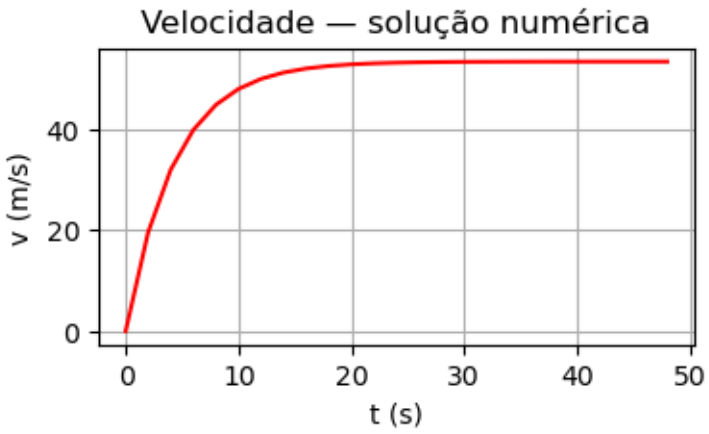
for t in tempos:
    result = v_ant + (9.81 - (12.5/68.1)*v_ant)*2
    v_aprox.append(result)
    v_ant = result

del v_aprox[-1]

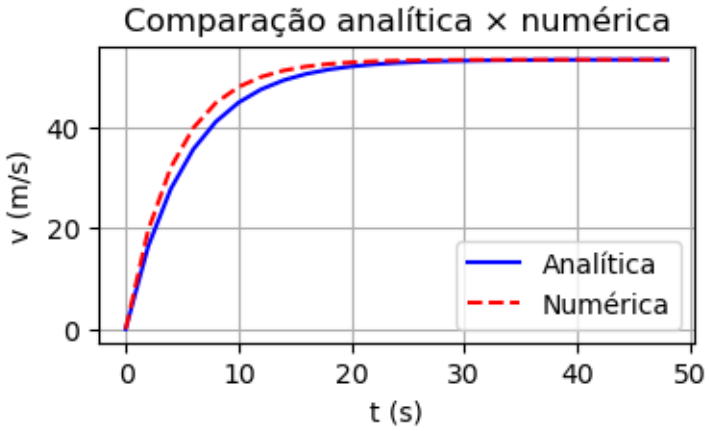
print(tempos)
print(v_aprox)

plt.plot(tempos, v_aprox, "r-")
plt.xlabel("t (s)")
plt.ylabel("v (m/s)")
plt.title("Velocidade - solução numérica")
plt.grid()
plt.show()
```

```
[ 0  2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
 48]
[0.0, 19.62, 32.037356828193836, 39.89621261813736, 44.87002590075947, 48.
↪01791653924718, 50.01019387432531, 51.2710918646611, 52.069105130204015, 52.
↪574161984020456, 52.89380883276478, 53.09611102337976, 53.22414662419483, 53.
↪305179434695994, 53.356464517406714, 53.38892247724272, 53.40946488647814, 53.
↪42246602947442, 53.43069435932962, 53.43590201008968, 53.439197894785096, 53.
↪441283836493945, 53.442604013992494, 53.44343954483225, 53.44396834628884]
```




```
# Comparando solução analítica (azul) e numérica (vermelho)
plt.plot(tempos, v(tempos), "b-", label="Analítica")
plt.plot(tempos, v_aprox, "r--", label="Numérica")
plt.xlabel("t (s)")
plt.ylabel("v (m/s)")
plt.title("Comparação analítica × numérica")
plt.legend()
plt.grid()
plt.show()
```




**Exemplo 1.4.4:**

Considere as expressões

$$\frac{e^{1/x}}{1+e^{1/x}} \quad e \quad \frac{1}{e^{-1/x}+1}$$

Verifique que, para $x > 0$, são funções idênticas, então, use um programa em Python para testar o valor de cada uma para alguns valores de $x$ entre $0.1$ e $0.001$. Qual dessas expressões é mais adequada quando $x$ é um número pequeno? Explique.

```
from math import exp
def f1(x): return exp(1./x)/(1+exp(1./x))
```

(continua na próxima página)

(continuação da página anterior)

```
def f2(x): return 1./(exp(-1./x)+1)

lista = [0.1, 0.075, 0.05, 0.01, 0.005, 0.002, 0.0015, 0.0014, 0.001]

for x in lista:
    try:
        print ("f1(x)=", f1(x))
    except ArithmeticError as erro:
        print("f1(x)=", erro)
    try:
        print ("f2(x)=", f2(x))
    except ArithmeticError as erro:
        print("f1(x)=", erro)
```

```
f1(x)= 0.9999546021312976
f2(x)= 0.9999546021312976
f1(x)= 0.9999983804058308
f2(x)= 0.9999983804058309
f1(x)= 0.9999999979388464
f2(x)= 0.9999999979388463
f1(x)= 1.0
f2(x)= 1.0
f1(x)= 1.0
f2(x)= 1.0
f1(x)= 1.0
f2(x)= 1.0
f1(x)= 1.0
f2(x)= 1.0
f1(x)= math range error
f2(x)= 1.0
f1(x)= math range error
f2(x)= 1.0
```

*Comentário:* Embora as duas expressões sejam idênticas, observando a saída dos códigos acima, percebe-se que para valores menores do que 0.0015 a primeira expressão resulta em um erro de Overflow (*ArithmeticError* = *math range error*) enquanto que a segunda expressão não ocorre o erro. Assim, podemos dizer que a segunda expressão é mais adequada para valores de x pequenos.

**Exemplo 1.4.5:** Apresente um programa em Python para obter aproximações para o valor da função $f(x) = ln(1+x)$ usando expansões em séries de Taylor em torno do ponto $x = 0$. Descubra quantos termos da série precisam ser retidos para calcular $ln(0.8)$ com erro absoluto inferior a 0.0001.

*Resolução:* A Série de Taylor em torno de x=0, também chamada de Série de McLaurin, para a função $ln(1 + x)$ é dada por $\sum_{n=0}^{\infty} \frac{(-1)^n x^{n+1}}{n+1}$

```
from math import log
soma = 0.0
```

(continua na próxima página)

(continuação da página anterior)

```
erro = 10.0
n = 0.0

while erro>0.0001:
    soma = soma + (-1.0)**n * (-0.2)**(n+1)/(n + 1)
    erro = abs(log(0.8)-soma)/abs(log(0.8))
    n = n + 1
    print ("Resultado:", soma, ", Erro:", erro)
```

```
Resultado: -0.2 , Erro: 0.1037159764550898
Resultado: -0.22000000000000003 , Erro: 0.014087574100598712
Resultado: -0.2226666666666667 , Erro: 0.002137120453333192
Resultado: -0.22306666666666672 , Erro: 0.00034455240624332016
Resultado: -0.22313066666666673 , Erro: 5.774151870891082e-05
```

*Comentário* Pelo resultado do código acima, os 5 primeiros termos da série são suficientes para a aproximar $ln(0.8)$, ou seja, $ln(1+x)$ com $x = -0.2$, com erro menor que 0.0001.

**Exemplo 1.4.6:**

De acordo com o *princípio de Arquimedes*, a força de *flutuação* é igual ao peso do fluido deslocado pela parte submersa de um objeto. Para o tronco de cone mostrado abaixo, use o método da bisseção e mais algum método de sua escolha para determinar a altura $h_1$ da parte que está acima da água. Use os seguintes valores para seus cálculos: $r_1 = 0,5m$, $r_2 = 1m$, $h = 1m$, $\rho_f =$ densidade do tronco $= 200kg/m^3$ e $\rho_w =$ densidade da água $= 1000kg/m^3$. O volume do trondo de cone é dado por: $$V = \frac{\pi h}{3}(r_1^2 + r_2^2 + r_1 r_2)$$

*Resolução:* A equação a ser resolvida é

$$200\frac{\pi 1.0}{3}(0.5^2 + 1.0^2 + 1.0 \times 0.5) = 1000\frac{\pi(1-h_1)}{3}((0.5 + h_1/2)^2 + 1.0^2 + 1.0 \times (0.5 + h_1/2))$$

```
import math
a = 0.1
b = 1.0
err = 10
x_ant = 0.0

f = lambda h: 1.75-5*(1-h)*(1.0+(0.5+h/2.)**2+(0.5+h/2.))

while err > 0.001:
    x = (a+b)/2.0
    if f(a)*f(x)<0:
        b = x
    else:
        a = x
    err = abs(x - x_ant)/abs(x)
    x_ant = x
    print ("| h1=%.10f"%x)
```

```
| h1=0.5500000000
| h1=0.7750000000
| h1=0.8875000000
| h1=0.8312500000
| h1=0.8593750000
| h1=0.8734375000
| h1=0.8804687500
| h1=0.8769531250
| h1=0.8751953125
| h1=0.8760742188
| h1=0.8756347656
```

```
import math

err = 1.0
x0 = 0.5
x1 = 1.0

f = lambda h: 1.75-5*(1-h)*(1.0+(0.5+h/2.)**2+(0.5+h/2.))

while err>0.001:
    x = (x0*f(x1)-x1*f(x0))/(f(x1)-f(x0))
    err = abs (x-x1)/abs(x)

    x0 = x1
    x1 = x

    print ("| h1=%.10f"%x)
```

```
| h1=0.8486486486
| h1=0.8740418082
| h1=0.8758028171
| h1=0.8757774319
```

**Exemplo 1.4.7:** O deslocamento de uma estrutura é definido pela seguinte equação para uma oscilação amortecida:

$$y = 8e^{-kt}\cos(wt)$$

onde $k = 0,5$ e $w = 3$. Use os métodos de Ponto Fixo e Newton-Raphson para determinar a raiz com $\varepsilon = 0.001$. Comente os resultados obtidos.

*Solução*: A raiz de

$$y = 8e^{-kt}\cos(wt)$$

é a mesma raiz de

$$\cos(wt) = 0.$$

```
x = 0.5
phi  = lambda x: x+(1/3)*math.cos(3*x)

for i in range(3):
    x = phi(x)
    print (x)
```

```
0.5235790672225676
0.5235987755982874
0.5235987755982989
```

```
x = 0.5
f  = lambda x: math.cos(3*x)
flinha = lambda x: -3*math.sin(3*x)

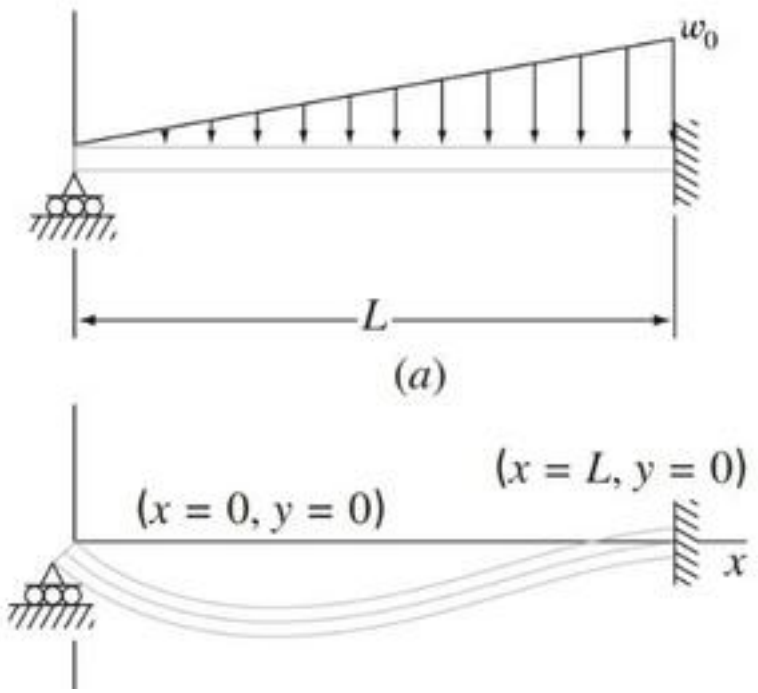
for i in range(3):
    x = x - f(x)/flinha(x)
    print (x)
```

```
0.5236382814342175
0.5235987755981139
0.5235987755982989
```

**Exemplo 1.4.8:** A figura abaixo mostra uma viga uniforme sujeita a uma carga distribuída de forma uniformemente crescente. A equação para a curva elástica resultante é

$$y = \frac{w_0}{120EIL}(-x^5 + 2L^2x^3 - L^4x)$$

Use um método de sua escolha para determinar o ponto de deflexão máxima (isto é, o valor de $x$ para o qual $dy/dx = 0$). A seguir, substitua esse valor na equação para determinar o valor da deflexão máxima. Use os seguintes valores dos parâmetros: $L = 450cm$, $E = 50.000kN/cm^2$, $I = 30.000cm^4$ e $w_0 = 1,75kN/cm$. Comente seus resultados e estime o erro.




```python
import numpy as np

L  = 450.0     # cm
E  = 50000.0   # kN/cm²
I  = 30000.0   # cm⁴
w0 = 1.75      # kN/cm

# dy/dx = (w0/(120*E*I*L)) * (-5x⁴ + 6L²x² - L⁴) = 0
# basta encontrar a raiz de g(x) = -5x⁴ + 6L²x² - L⁴ no intervalo (0, L)
g      = lambda x: -5*x**4 + 6*L**2*x**2 - L**4
glinha = lambda x: -20*x**3 + 12*L**2*x

# Newton-Raphson a partir de x0 = L/2
x = L / 2.0
for _ in range(50):
    dx = g(x) / glinha(x)
    x -= dx
    if abs(dx) < 1e-6:
        break

print(f"Ponto de deflexão máxima: x = {x:.4f} cm")

y = lambda x: (w0 / (120*E*I*L)) * (-x**5 + 2*L**2*x**3 - L**4*x)
print(f"Deflexão máxima:          y = {y(x):.6f} cm")

# Curva elástica completa
```

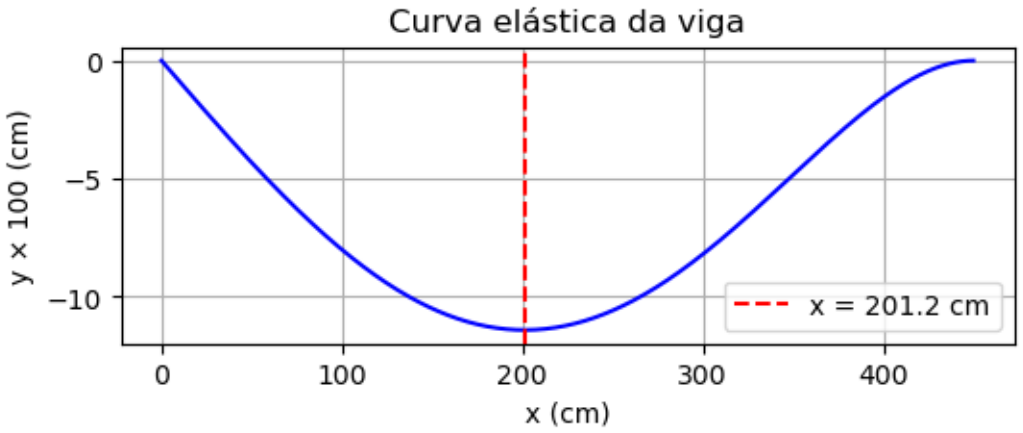
(continua na próxima página)

(continuação da página anterior)

```
xs = np.linspace(0, L, 500)
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 2))
plt.plot(xs, y(xs)*100, "b-")            # ×100 para melhor visualização
plt.axvline(x, color="r", linestyle="--", label=f"x = {x:.1f} cm")
plt.xlabel("x (cm)")
plt.ylabel("y × 100 (cm)")
plt.title("Curva elástica da viga")
plt.legend()
plt.grid()
plt.show()
```

```
Ponto de deflexão máxima: x = 201.2461 cm
Deflexão máxima:          y = -0.114107 cm
```



# 2. Zeros de Funções

Determinar os zeros de uma função.ou seja, os valores de $x$ para os quais $f(x) = 0$, é um dos problemas mais recorrentes em ciências e engenharia. Em geral, a solução analítica não existe ou é de difícil obtenção, tornando os métodos numéricos indispen sáveis.

### Objetivos de aprendizagem

- Localizar intervalos que contêm raízes de funções contínuas
- Aplicar métodos de refinamento iterativo: bisseção, ponto fixo, Newton-Raphson e secantes
- Analisar convergência, taxa de convergência e critérios de parada
- Calcular raízes de polinômios com métodos específicos
- Resolver problemas aplicados de engenharia e ciências

### Conteúdo

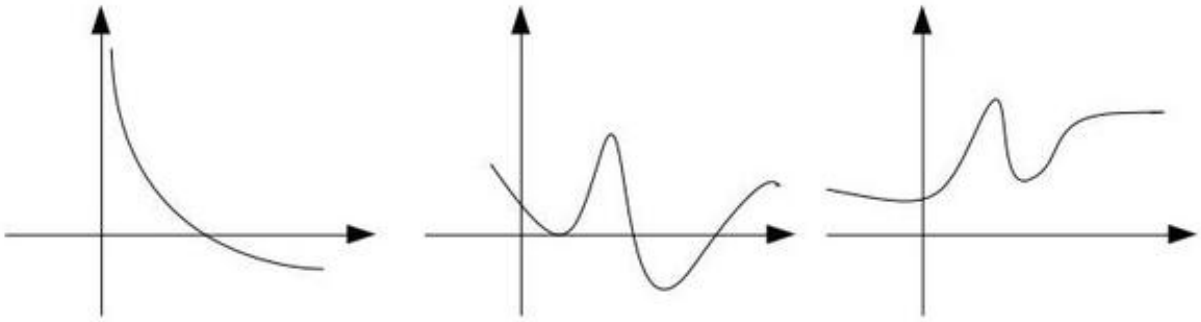

| Seção | Tema |
|---|---|
| 2.1 | Localização de raízes |
| 2.2 | Método da bisseção |
| 2.3 | Método do ponto fixo |
| 2.4 | Método de Newton-Raphson |
| 2.5 | Método das secantes |
| 2.6 | Raízes de polinômios |
| 2.7 | Exemplos de aplicações |


## 2.1 Localização de raízes

Em ciências e engenharias frequentemente a solução de um problema real é obtida resolvendo uma equação em uma variável real, a qual sempre pode ser escrita na forma

$$f(x) = 0$$

Dada uma função $f : R \to R$ dizemos que $\overline{x} \in R$, tal que $f(\overline{x}) = 0$ é um *zero* (ou *raiz*) da função $f$. Graficamente, uma raiz é o ponto onde a função $f(x)$ cruza ou toca o eixo $x$. Uma equação $f(x) = 0$ pode não ter solução (raiz) real, ter uma única raiz ou várias.



Quando a equação é simples, o valor de $x$ pode ser determinado analiticamente. Esse é o caso quando se escreve $x$ explicitamente após a aplicação de operações matemáticas, ou quando uma fórmula conhecida (como a fórmula de Bháskara para resolução de equações quadráticas) pode ser usada para determinar o valor exato de $x$. Em muitas situações, no entanto, é impossível determinar analiticamente a raiz de uma equação.

Por exemplo, é relativamente fácil obter analiticamente as raízes das equações

a) $2.1x - 47.3 = 0$

b) $x^2 + 2x - 3 = 0$

c) $senx - x = 0$

Já para as equações a seguir é preferível utilizar-se de métodos numéricos

a) $xe^x - 2 = 0$

b) $cos3\theta - \text{sen}\,\theta = 0$

c) $2.1^x + 5.7x = 0$

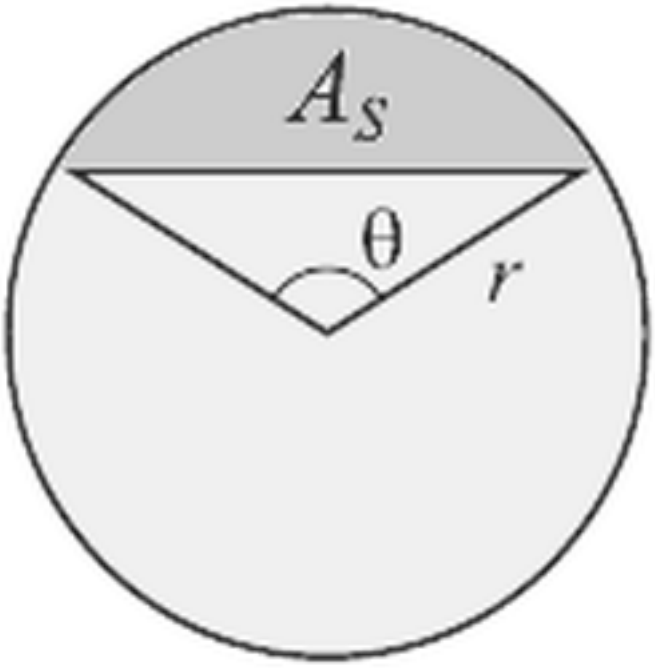
**Exemplo 2.1.1:** Em termodinâmica, sob determinadas condições, a relação entre o calor $Q$ fornecido a um gás e sua variação de temperatura $T_f - T_i$ é dada por

$$Q = nR\left[A(T_f - T_i) + \frac{B}{2}(T_f^2 - T_i^2) + \frac{C}{3}(T_f^3 - T_i^3)\right].$$

Para o gás metano tem-se $R = 8,314J/mol.K$, $A = 1,702$, $B = 9,081 \times 10^{-3}K^{-1}$, $C = -2,164 \times 10^{-6}K^{-2}$.

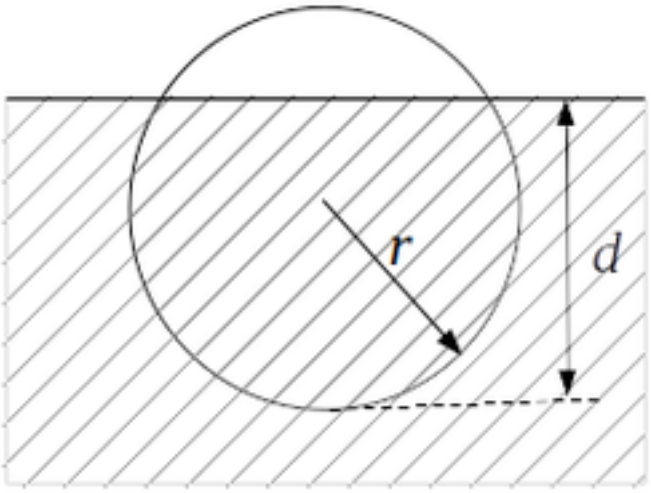
Considere o problema de calcular a temperatura final $T_f$ se $20kJ$ de energia forem absorvidos pelo gás, supondo estar contido em uma câmara com $n = 2mol$ de metano a temperatura $T_i = 300K$.

**Exemplo 2.1.2:** A área do segmento $A_S$ de um círculo de raio $r$ (ver figura abaixo) é dada por $A_S = \frac{1}{2}r^2(\theta - \text{sen}\,\theta)$.




Para se determinar o ângulo $\theta$ se $A_S$ e $r$ são conhecidos, a equação deve ser resolvida para $\theta$. Não é uma tarefa simples escrever $\theta$ explicitamente em termos de $A_S$ e $r$, portanto, pode não ser possível resolver essa equação analiticamente.

**Exemplo 2.1.3:** Seja uma esfera de madeira de raio $r$ colocada a flutuar sobre a água conforme figura. Qual é a altura $d$ da esfera que ficará submersa?




Para responder essa pergunta, observemos que, pelo *Princípio de Arquimedes* o peso da esfera deve ser equilibrado com o peso do volume de água deslocada, ou seja, $\mu_e V_e g = \mu_a V_a g$ onde $g$ é a aceleração da gravidade, $\mu_e$ é a densidade relativa da madeira e $\mu_a$ é a densidade relativa da água, $V_e$ e $V_a$ são os volumes. Essa equação nos leva a

$$\mu_e \frac{4\pi r^3}{3} = \mu_a \int_0^d \pi \left[r^2 - (x-r)^2)\right] dx$$

$$\mu_e \frac{4\pi r^3}{3} = \mu_a \frac{\pi d^2 (3r - d)}{3}$$

substituindo os valores $r = 10cm$, $\mu_e = 0,638g/cm^3$ e $\mu_a = 1g/cm^3$, com $\pi \approx 3.1415$, podemos chegar à seguinte equação aproximada

$$1.047d^3 - 31.415d^2 + 2672.369 = 0$$

Basta resolver essa equação para obter a resposta procurada.

No entanto, encontrar a solução analítica para esse problema não é tão simples, mas com uma rotina numérica simples podemos encontrar uma boa aproximação.

Observe que poderíamos ter multiplicado ambos os lados da equação por $\frac{3}{\pi}$, o que nos daria

$$d^3 - 30d^2 + 2552 = 0$$

que é uma equação mais precisa, uma vez que não tivemos que usar aproximações, no entanto esse tipo de simplificação requer manipular o número $\pi$ como um símbolo. Veremos que frequentemente soluções numéricas geram soluções aproximadas

**Exemplo 2.1.4:** Podemos utilizar uma abordagem numérica para obter soluções aproximadas de equações, vejamos por exemplo a equação $f(d) = 1.047d^3 - 31.415d^2 + 2672.369$ do exemplo descrito anteriormente. Dos dados do problema, podemos inferir que a solução é um número entre 0 e 20, então, calculamos valores da função nesse intervalo e tentar encontrar uma aproximação. Buscamos o valor de $d$ que anula a função $f$, então, observando os resultados na tabela abaixo, podemos concluir que a solução é um número entre 11 e 12 cm pois é o intervalo onde a função muda de sinal.

```
for d in range(0,20):
    y = 1.047*d**3-31.415*d**2+2672.369
    print(f'f({d}) = {round(y, 3)}')
```

```
f(0) = 2672.369
f(1) = 2642.001
f(2) = 2555.085
f(3) = 2417.903
f(4) = 2236.737
f(5) = 2017.869
f(6) = 1767.581
f(7) = 1492.155
f(8) = 1197.873
f(9) = 891.017
f(10) = 577.869
f(11) = 264.711
f(12) = -42.175
f(13) = -336.507
f(14) = -612.003
f(15) = -862.381
f(16) = -1081.359
f(17) = -1262.655
f(18) = -1399.987
f(19) = -1487.073
```

Seguindo o mesmo procedimento mas agora incrementando o valor de $d$ em 0.1 e calculando $f$ para valores de $d$ entre

11 e 12 observamos que a solução está entre 11.8 e 11.9.

```
for inc in range(0,11):
    d = 11+inc/10
    y = 1.047*d**3-31.415*d**2+2672.369
    print('f(',d,')=', round(y,3))
```

```
f( 11.0 )= 264.711
f( 11.1 )= 233.637
f( 11.2 )= 202.631
f( 11.3 )= 171.701
f( 11.4 )= 140.852
f( 11.5 )= 110.091
f( 11.6 )= 79.425
f( 11.7 )= 48.858
f( 11.8 )= 18.399
f( 11.9 )= -11.948
f( 12.0 )= -42.175
```

Por fim, podemos tomar como solução aproximada o ponto médio entre 11.8 e 11.9, ou seja, $d = 11.85$. A precisão é de 0.05.

**Exemplo 2.1.5:** (Usando a biblioteca SciPy) Encontrando a raiz de $A_S = \frac{1}{2}r^2(\theta - \text{sen}\,\theta)$ com $r = 1$ e $A_s = 3.5$.

```
from scipy.optimize import fsolve
import numpy as np

As = lambda x: 0.5*(x-np.sin(x))-3.5

sol = fsolve(As, 1)
print (sol)
```

```
[7.99067236]
```

Esses foram apenas alguns exemplos de como os métodos numéricos podem nos auxiliar na obtenção de soluções aproximadas quando não é fácil, ou mesmo possível, encontrar soluções analíticas. No entanto, sua aplicação vai muito além desses casos. Os métodos numéricos são amplamente utilizados na resolução de integrais definidas, sistemas lineares de grande porte, equações diferenciais, problemas de otimização e na visualização e análise de grandes conjuntos de dados, entre muitas outras aplicações.

Nos casos em que uma solução exata $\overline{x}$ da equação $f(x) = 0$ não puder ser encontrada por processos analíticos, uma solução aproximada poderá ser obtida por meio de métodos numéricos. Em geral, esses métodos produzem uma sequência de aproximações, obtidas por um processo iterativo que é encerrado quando a precisão desejada é atingida.

De forma geral, a determinação numérica de uma raiz pode ser dividida em duas etapas:

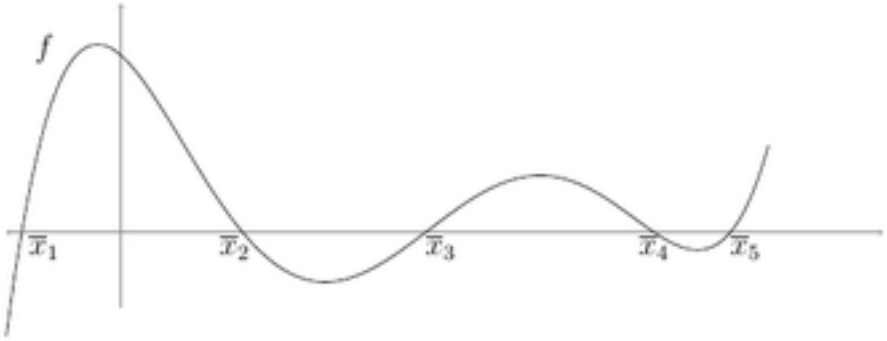
1. **Isolamento da raiz:** consiste em encontrar um intervalo $[a, b]$ que contenha uma única raiz da equação $f(x) = 0$;
2. **Refinamento da aproximação:** consiste em reduzir sucessivamente o intervalo que contém a raiz até que a precisão desejada seja alcançada.

Para isolar as raízes de uma função, podemos utilizar métodos gráficos ou avaliar a função em diferentes pontos do

domínio, identificando intervalos nos quais ocorre mudança de sinal.

## Isolamento da raiz: Localização por métodos gráficos

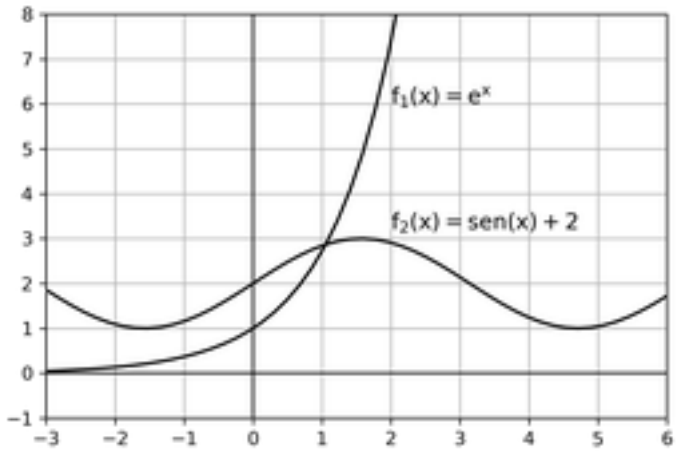
Graficamente as raízes da equação $f(x) = 0$ (ou zeros da função $f$) correspondem aos pontos onde o gráfico de $f$ intercepta o eixo $x$. Na figura abaixo, a função $f$ possui 5 zeros representados por $\overline{x}_1$, $\overline{x}_2$, $\overline{x}_3$, $\overline{x}_4$ e $\overline{x}_5$.




Nem sempre é fácil esboçar o gráfico de uma função sem a ajuda de um computador. Em muitos casos é conveniente reescrever a equação $f(x) = 0$ numa forma equivalente $f_1(x) = f_2(x)$ e esboçar os gráficos de $f_1$ e $f_2$ para verificar onde eles se interceptam. Outra abordagem seria atribuir valores arbitrários para a variável $x$ e encontrar um par $a$ e $b$ tal que $f(a) \cdot f(b) < 0$.

**Exemplo 2.1.6:**

A equação $f(x) = e^x - \text{sen}(x) - 2 = 0$ pode ser escrita como $e^x = senx + 2$ então, esboçando os gráficos de $f_1(x) = e^x$ e $f_2(x) = senx + 2$, observamos que a raiz procurada está entre $0.5$ e $1.5$, como pode ser visto no gráfico a seguir.




Vamos gerar esses gráficos usando Python. Para isso precisamos da seguinte biblioteca:

```
import matplotlib.pyplot as plt
```

A partir daí criamos uma lista de pontos para plotar. Fica fácil fazer isso usando as funções `arange` ou `linspace` da biblioteca `numpy`. Aqui vamos usar a primeira opção:

```
x = np.arange(-1, 6, 0.25) # início, fim, passo
print (x)
```

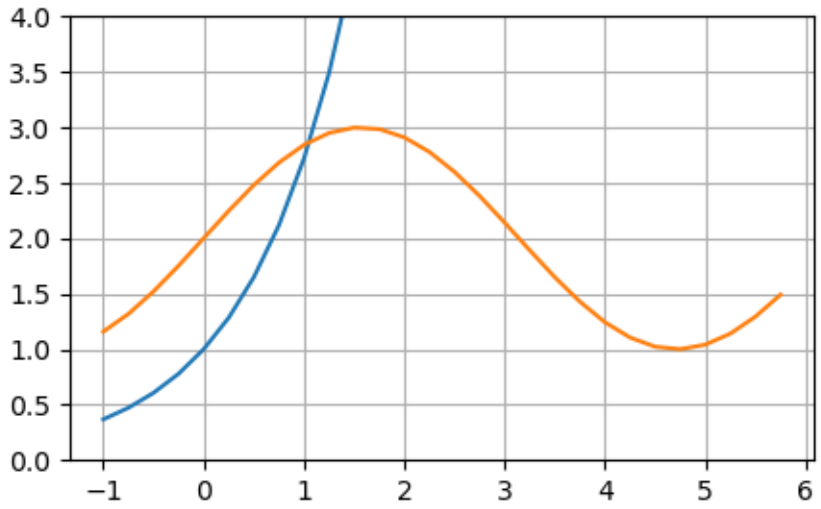
```
[-1.   -0.75 -0.5  -0.25  0.    0.25  0.5   0.75  1.    1.25  1.5   1.75
  2.    2.25  2.5   2.75  3.    3.25  3.5   3.75  4.    4.25  4.5   4.75
  5.    5.25  5.5   5.75]
```

Agora definimos as funções para obter os valores de $f_1(x)$ e $f_2(x)$. Isso pode ser feito usando as funções `lambda` ou `def`. Vamos usar a primeira opção:

```
f1 = lambda x: np.exp(x)
f2 = lambda x: np.sin(x)+2
```

Agora, basta plotar:

```
plt.figure(figsize=(5,3))
plt.plot(x, f1(x), x, f2(x))
plt.ylim(0, 4)
plt.grid()
plt.show()
```




Para manter o código simples e facilitar a compreensão dos primeiros exemplos, não vamos incluir títulos, legendas ou descrições nos eixos dos gráficos. Esses elementos serão introduzidos gradualmente ao longo do texto.

**Exemplo 2.1.7:** Atribuindo valores para $x$ e avaliando a função $f(x) = e^x - \text{sen}(x) - 2$ entre $x = 0$ e $x = 2.5$ com passo $0.5$. Para esses valores obtemos:

```
f(0.0) = -1.0000
f(0.5) = -0.8307
f(1.0) = -0.1232
f(1.5) = 1.4842
f(2.0) = 4.4798
f(2.5) = 9.5840
```

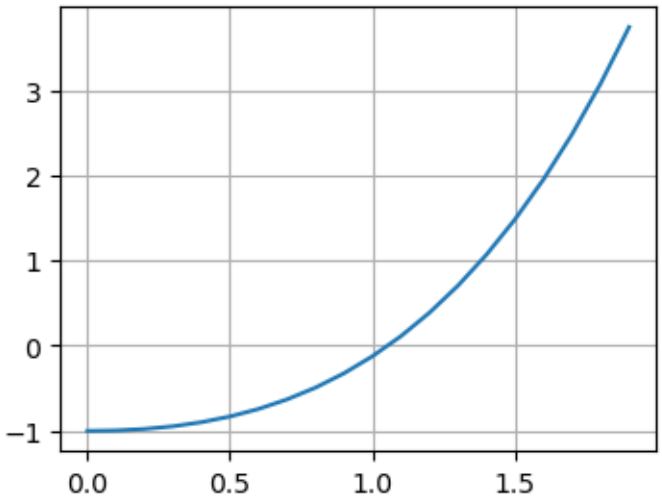
Como $f(1.0)f(1.5) < 0$, podemos concluir que existe pelo menos uma raiz entre 1.0 e 1.5.

Para plotar apenas o gráfico de $f(x) = e^x - \text{sen}(x) - 2$, com $0 \leq x \leq 2.5$, podemos usar o seguinte script:

```
plt.figure(figsize=(4,3))
x = np.arange(0.0, 2.0, 0.1)

f = lambda x: np.exp(x)-np.sin(x)-2

plt.plot(x, f(x))
plt.grid()
plt.show()
```




## Localização de raízes de polinômios

Para funções polinomiais, além dos métodos gráficos, existem teoremas da Álgebra que permitem investigar a existência e a localização de suas raízes. Alguns desses resultados serão apresentados a seguir.

Desejamos localizar as raízes $\overline{x}$ tais que $P(\overline{x}) = 0$, em que $P(x)$ é um polinômio de grau $n \in \mathbb{N}$, com coeficientes reais ou complexos, da forma

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n,$$

em que $a_n \neq 0$.

As raízes de um polinômio satisfazem as seguintes propriedades:

1. Um polinômio de grau $n$ possui exatamente $n$ raízes reais ou complexas, contando suas multiplicidades. Assim, algumas raízes podem coincidir.
2. Se os coeficientes do polinômio forem reais e $n$ for ímpar, então existe pelo menos uma raiz real.
3. Se os coeficientes do polinômio forem reais e existirem raízes complexas, elas ocorrerão aos pares conjugados. Ou seja, se $a + bi$ é uma raiz, então $a - bi$ também é raiz, em que $i = \sqrt{-1}$.

O valor de uma função polinomial pode ser calculado de forma mais eficiente quando o polinômio é reescrito na chamada **forma estruturada**, também conhecida como forma de **parênteses aninhados** (ou forma de Horner). Nessa representação, reduz-se significativamente o número de operações aritméticas necessárias para avaliar o polinômio.

Por exemplo, considere o polinômio de terceiro grau

$$f_3(x) = a_3x^3 + a_2x^2 + a_1x + a_0.$$

Se as potências de $x$ forem calculadas por multiplicações sucessivas, sua avaliação exige seis multiplicações e três adições. De modo geral, um polinômio de grau $n$ requer $\frac{n(n+1)}{2} + n$ multiplicações e $n$ adições.

Entretanto, se o polinômio for reescrito na forma

$$f_3(x) = ((a_3x + a_2)\,x + a_1)\,x + a_0,$$

serão necessárias apenas três multiplicações e três adições. Essa forma reduz o custo computacional e é conhecida como **método de Horner**.

**Exemplo 2.1.8:** Considere o polinômio $f(x) = 0.8x^5 - 1.3x^4 + 5x^3 + 17x^2 + 1.9x + 10$. Vamos calcular $f(0.5)$ usando um laço simples em que cada passo efetuamos uma multiplicação e uma soma.

```
a = [0.8, -1.3,5, 17,1.9,10] #coeficientes (do maior para o menor grau)
x = 0.5
```

A variável $p$ armazenará o valor parcial do polinômio obtido pelo método de Horner.

```
p = 0
for n in range(6):
    p = p*x+a[n]
print(p)
```

```
15.76875
```

Uma vantagem do método de Horner é que ele também permite calcular o valor da derivada com um custo computacional muito pequeno. Basta acrescentar uma variável para armazenar o valor parcial da derivada. Vamos fazer isso incluindo a linha `df = df*x+p`no laço acima.

```
p = 0
dp = 0

for n in range(6):
    dp = dp*x+p
    p = p*x+a[n]

print(f"f({x})  = {p}")
print(f"f'({x}) = {dp}")
```

```
f(0.5)  = 15.76875
f'(0.5) = 22.25
```

Podemos confirmar os valores obtidos calculando diretamente o polinômio e sua derivada.

```
p = 0.8*x**5-1.3*x**4+5*x**3+17*x**2+1.9*x+10
df = 5*0.8*x**4-1.3*4*x**3+3*5*x**2+2*17*x+1.9
print (p, df)
```

```
15.76875 22.25
```

Os valores coincidem com aqueles obtidos pelo método de Horner, confirmando a correção da implementação.

**Observação:** Seja $P(x)$ um polinômio de grau $n \geq 1$. Dizemos que $\bar{x}$ é uma raiz de multiplicidade " $m$ " da equação $P(x) = 0$ se:

$$P(\bar{x}) = P^{(1)}(\bar{x}) = ... = P^{(m-1)}(\bar{x}) = 0 \quad \text{e} \quad P^{(m)}(\bar{x}) \neq 0$$

em que $P^m(\bar{x})$ é a $m$-ésima derivada de $P(x)$ calculada no ponto $\bar{x}$.

Por exemplo, considere o polinômio $P(x) = x^3$. A raiz $\bar{x} = 0$ é uma raiz de multiplicidade três da equação $P(x) = 0$, pois:

$$P(x) = P^{(1)}(\bar{x}) = 0, \quad P^{(2)}(\bar{x}) = 0 \quad \text{e} \quad P^{(3)}(\bar{x}) = 6 \neq 0$$

Entendemos, neste caso, que a equação $P(x) = x^3 = 0$ possui três raízes idênticas, ou uma raiz de multiplicidade 3 .

A seguir apresentamos alguns teoremas que podem ser utilizados para investigar a localização das raízes de polinômios.

### Teorema: Localização de raízes no círculo

Sejam $P(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n$ e $A = \max\{|a_1|, ..., |a_n|\}$. Então, o módulo de todas as raízes $x_k$ , $k = 1, ..., n$ satisfaz a inequação:

$$|x_k| < 1 + \frac{A}{|a_0|} = R$$

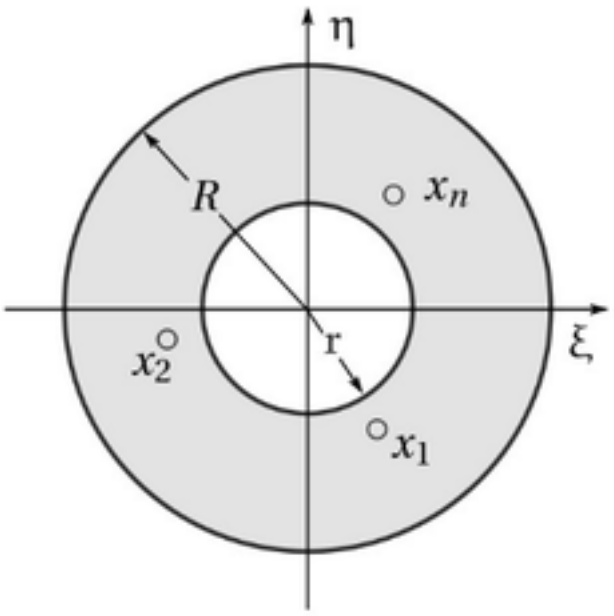
Para a demonstração, ver Arenales e Darezzo, 2016.

Como consequência desse teorema, tem-se o seguinte **corolário**:

Sejam $a_n \neq 0$ e $B = \max\{|a_0|, ..., |a_{n-1}|\}$, em que $a_k$,$k = 0, ..., n$ são os coeficientes de $P(x)$. Então, o módulo de todas as raízes $x_k$, $k = 1, ..., n$ satisfaz a inequação:

$$|x_k| > \frac{1}{1 + \frac{B}{|a_n|}} = r$$

(Para a demonstração, ver Arenales e Darezzo, 2015, p 123)




**Exemplo 2.1.9:** (Fonte: Arenales e Darezzo, 2016) As raízes de $P(x) = x^3 + 2x^2 - x - 2 = 0$ estão localizadas em $-3 < x_k < -1/2$ ou $1/2 < x_k < 3$, pois:

$$|x_k| < 1 - \frac{2}{|1|} = 3 = R \quad \text{e} \quad |x_k| > \frac{1}{1 + \frac{2}{|-2|}} = 1/2 = r$$

De fato, as raízes de $P(x) = 0$ são dadas por $x = 1$, $x = -1$ e $x = -2$.

Assim, no plano complexo as raízes de $P(x) = 0$ estão localizadas no anel $r < |x| < R$ como ilustra a figura abaixo.

## Teorema: regra de sinais de Descartes

Dada uma equação polinomial com coeficientes reais, o número de raízes reais positivas $p$ dessa equação não excede o número $v$ de variações de sinais dos coeficientes. Ainda mais, $(v - p)$ é inteiro par, não negativo.

Para determinar o número de raízes negativas, basta calcular o número de variações de sinais no polinômio $P(-x)$.

## Teorema: Budan-Fourier

Se os números $a$ e $b(a < b)$ não são raízes da equação $P(x) = 0$, então o número $N(a, b)$ de raízes reais da equação $P(x) = 0$ localizadas entre $a$ e $b$ é dado por $N(a, b) = \Delta\bar{N} - 2k \quad (k \text{ natural})$, sendo $\Delta\bar{N} = \bar{N}(a) - \bar{N}(b)$ em que $\bar{N}(x)$ é igual ao número de variações de sinais na sequência de sucessivas derivadas:

$$P(x), P^{(1)}(x), \ldots, P^{(n-1)}, 1$$

**Exemplo 2.1.10:** (Fonte: Arenales e Darezzo, 2016) Considere o polinômio $P(x) = x^3 + 2x^2 - x - 2$. Aplicando o teorema de Budan-Fourier, vemos que os valores 3 e -3 não são raízes de $P(x) = 0$, pois $P(3) = 40$ e $P(-3) = -8$. Desta forma, é possível calcular o número de raízes localizadas em $[-3, 3]$ :

Seja $P^{(i)}(x) \rightarrow i$-ésima derivada de $P(x)$. Assim, temos:

$$\begin{array}{cccc} P(3) = 40 & P^{(1)}(3) = 38 & P^{(2)}(3) = 22 & P^{(3)}(3) = 6 \\ P(-3) = -8 & P^{(1)}(-3) = 14 & P^{(2)}(-3) = -14 & P^{(3)}(-3) = 6 \end{array}$$

Logo,

$$\bar{N}(3) = 0 \quad \text{e} \quad \bar{N}(-3) = 3$$
$$\Delta\bar{N} = \bar{N}(-3) - \bar{N}(3) = 3 - 0 = 3$$
$$N(-3, 3) = 3 - 2k, \quad k = 0, 1$$

Portanto, temos uma ou três raízes no intervalo. Pela regra de sinais de Descartes, observamos que a equação possui uma raiz positiva e, portanto, a equação pode ter duas raízes no intervalo $(-3, 0)$ ou não ter nenhuma raiz real negativa.

## Exercícios

---

**1.** Localize graficamente e determine uma aproximação para as raízes reais das funções dadas.

a) $f(x) = -0.5x^2 + 2.5x + 4.5$

b) $f(x) = 5x^2 - 5x^2 + 6x - 2$

c) $f(x) = -25 + 82x - 90x^2 + 44x^3 - 8x^4 + 0.7x^5$

d) $f(x) = -12 - 21x + 18x^2 - 2.75x^3$

---

**2.** Determine um intervalo pequeno (de tamanho 0.1, por exemplo) que contenha as raízes reais das funções dadas.

a) $f(x) = 3x - e^x = 0$

b) $f(x) = \text{sen}(x) + x^2 + 1 = 0$

c) $f(x) = \text{sen}(x) - x + 2 = 0$

d) $f(x) = 2x - \text{tg}(x) = 0$

e) $f(x) = 3x - \cos(x) + 1 = 0$

f) $f(x) = \ln(x) - \text{sen}(x) = 0$

---

**3.** Localize as raízes dos seguintes polinômios:

a) $P(x) = x^3 - 7x^2 + 14x - 6 = 0$

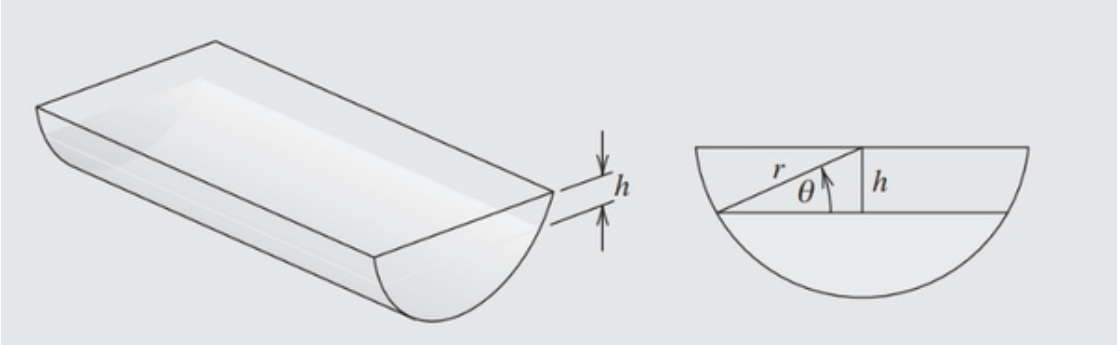
b) $P(x) = -25 + 82x - 90x^2 + 44x^3 - 8x^4 + 0.7x^5$

c) $P(x) = -12 - 21x + 18x^2 - 2.75x^3$

---

**4.** (Fonte) Uma gamela de comprimento $L$ tem seção transversal semicircular com raio $r$ (veja a figura abaixo). Quando a gamela está cheia com água até uma distância $h$ do topo, o volume $V$ de água é

$$V = L\left[0,5\pi r^2 - r^2 \,\text{arcsen}(h/r) - h\left(r^2 - h^2\right)^{1/2}\right].$$



Suponha que $L = 10ps$, $r = 1$ pé e $V = 12,4$ pés$^3$. Encontre uma aproximação para a profundidade da água na gamela (sem resolver a equação).

---

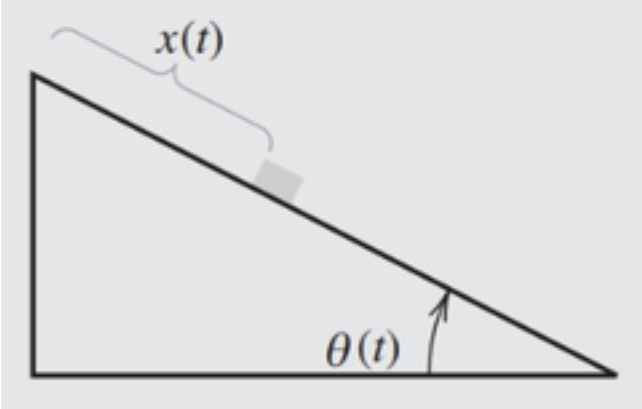
**5.** (Fonte) Uma partícula começa a se movimentar sobre um plano inclinado liso cujo ângulo $\theta$ está variando com velocidade constante

$$\frac{d\theta}{dt} = \omega < 0.$$

Depois de $t$ segundos, a posição do objeto é dada por

$$x(t) = -\frac{g}{2\omega^2}\left(\frac{e^{\omega t} - e^{-\omega t}}{2} - \operatorname{sen}\omega t\right).$$

Suponha que a partícula tenha se deslocado 1,7 pé em 1 segundo. Obtenha uma aproximação para a velocidade $\omega$ com a qual $\theta$ varia. Suponha que $g = 32,17$ pés $/\text{s}^2$.




---

## 2.2 Método da bisseção

Muitos problemas em Ciências e Engenharia envolvem a resolução de equações que podem ser escritas na forma

$$f(x) = 0.$$

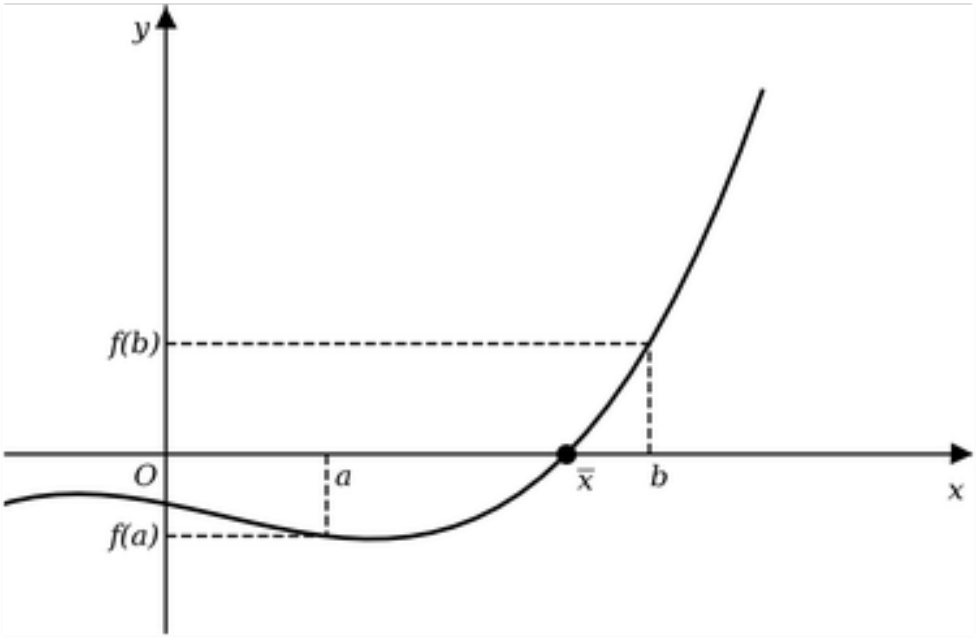
Em alguns casos, a solução exata $\overline{x}$ pode ser obtida por processos analíticos. Quando isso não é possível, uma solução aproximada pode ser determinada por meio de métodos numéricos.

Encontrar uma solução aproximada para a equação $f(x) = 0$ consiste em gerar uma sequência de aproximações. Se essa sequência se aproxima da solução exata $\overline{x}$, dizemos que o método converge para a solução.

O *Método da Bisseção* é um método iterativo para resolver equações da forma $f(x) = 0$. Partindo de um intervalo inicial $[a, b]$ em que $f(a)f(b) < 0$, e supondo que exista uma única raiz nesse intervalo, o método convergirá para essa raiz. Em cada iteração, o intervalo é dividido ao meio e utiliza-se o *Teorema de Bolzano* para identificar qual dos subintervalos contém a raiz.

O *Teorema de Bolzano* afirma que, se $f(x)$ é contínua no intervalo $[a, b]$ e $f(a)f(b) < 0$, então existe pelo menos uma raiz $\overline{x} \in (a, b)$ tal que $f(\overline{x}) = 0$.

Observe que a condição $f(a)f(b) < 0$ indica que a função assume sinais opostos nos extremos do intervalo. Como a função é contínua, ela deve assumir o valor zero em pelo menos um ponto de $(a, b)$.




O processo iterativo é repetido até que a amplitude do intervalo seja suficientemente pequena ou até que a aproximação obtida satisfaça um critério de parada previamente estabelecido.

1. Determinar um intervalo inicial $[a, b]$ contendo uma única raíz de $f(x)$;
2. Calcular o ponto médio $x_m = \frac{b+a}{2}$;
3. Se $f(x_m) = 0$, então a raiz é $\overline{x} = x_m$ e o algoritmo é encerrado;
4. Se $|b - a| < \epsilon$ (ou outro critério de parada for satisfeito), assumir $\overline{x} \approx x_m$ e encerrar o algoritmo;
5. Se $f(a)f(x_m) < 0$ fazer $b = x_m$, caso contrário, fazer $a = x_m$;
6. Voltar ao passo 2.

## Número de iterações

Podemos estimar o número de iterações necessárias para encontrar a aproximação para a raiz $\overline{x}$ com a precisão desejada. Considerando que a cada iteração o intervalo é dividido ao meio, então na n-ésima iteração tem-se um intervalo de amplitude $\frac{b-a}{2^n}$, então, queremos que

$$\frac{b-a}{2^{n+1}} < \epsilon$$

ou

$$log\left(\frac{b-a}{2^{n+1}}\right) < log(\epsilon)$$

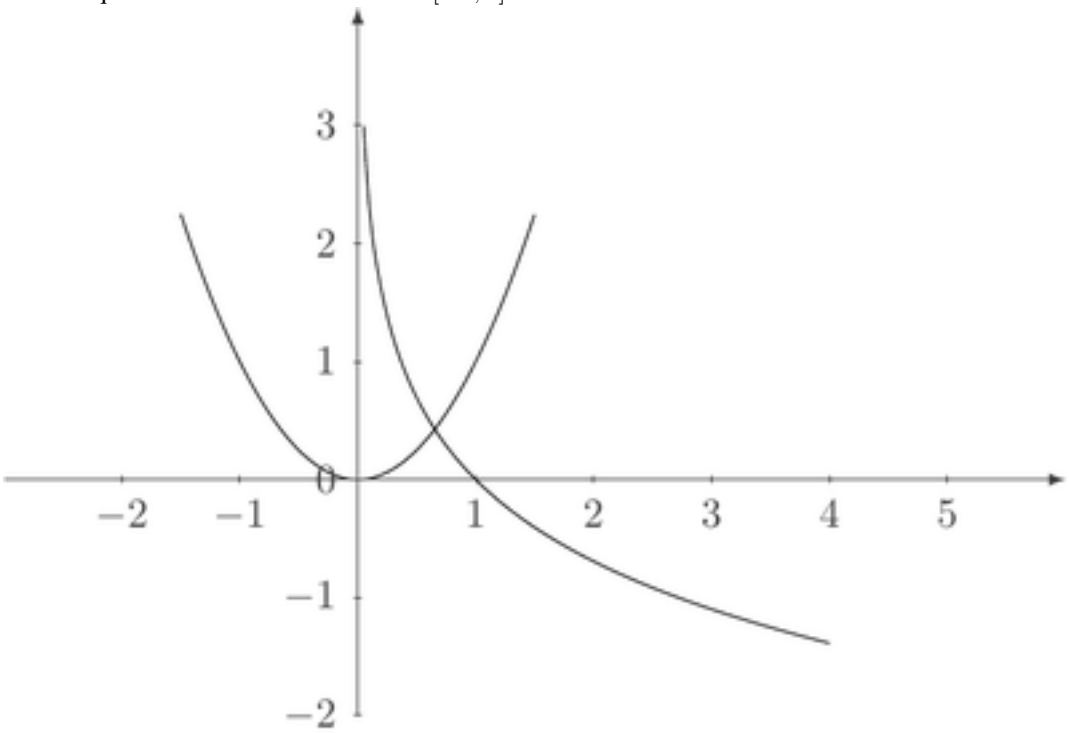
que leva a

$$n > \frac{log(b-a) - log(\epsilon)}{log2} - 1$$

Assim, $n$ é o número mínimo de iterações que devem ser realizadas para se obter $\overline{x}$ com precisão $\epsilon$.

**Exemplo 2.2.1:** Usando o método da bisseção, resolva a equação $x^2 + \ln(x) = 0$, com $\epsilon = 0.01$.

*Solução:* Reescrevendo a equação como $x^2 = -\ln(x)$ e esboçando os gráficos de $f_1(x) = x^2$ e $f_2(x) = -\ln(x)$ observamos que existe uma raiz no intervalo $[0.1, 1]$.




Então, na primeira iteração fazemos $x_m = \frac{a+b}{2} = \frac{1.0-0.1}{2} = 0.55$. Como $f(a) = f(0.1) = -2.293$, $f(b) = f(1.0) = 1$ e $f(x_m) = f(0.55) = -0.295$ temos que a raíz está no intervalo $[x_m, b]$. Além disso, $\frac{|x_1-x_0|}{|x_0|} = 0.2953$ é maior que $\epsilon = 0.01$. Então seguimos o processo iterativo obtendo a sequência abaixo.

| $i$ | $a$ | $b$ | $xm$ | $f(a)$ | $f(b)$ | $f(x_m)$ | $\frac{\lvert x_i - x_{i-1}\rvert}{\lvert x_i\rvert}$ |
|---|---|---|---|---|---|---|---|
| 1 | 0.1 | 1.00 | 0.55 | -2.2925 | 1 | -0.2953 | - |
| 2 | 0.5500 | 1.0000 | 0.7750 | -0.2953 | 1.0000 | 0.3457 | 0.2903 |
| 3 | 0.5500 | 0.7750 | 0.6625 | -0.2953 | 0.3457 | 0.0272 | 0.1698 |
| 4 | 0.5500 | 0.6625 | 0.6063 | -0.2953 | 0.0272 | -0.1329 | 0.0928 |
| 5 | 0.6063 | 0.6625 | 0.6344 | -0.1329 | 0.0272 | -0.0527 | 0.0443 |
| 6 | 0.6344 | 0.6625 | 0.6484 | -0.0527 | 0.0272 | -0.0127 | 0.0217 |
| 7 | 0.6484 | 0.6625 | 0.6555 | -0.0127 | 0.0272 | 0.0072 | 0.0107 |
| 8 | 0.6484 | 0.6555 | 0.6520 | -0.0127 | 0.0072 | -0.0027 | 0.0054 |

Em Python podemos fazer o gráfico como é mostrado a seguir

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
f1 = lambda x: x**2
```

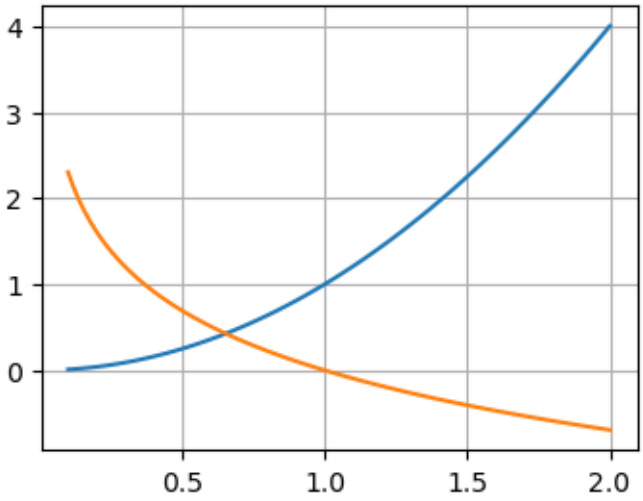
(continua na próxima página)

(continuação da página anterior)

```
f2 = lambda x: -np.log(x)

xi = np.linspace(0.1, 2, 100)

plt.plot(xi, f1(xi))
plt.plot(xi, f2(xi))
plt.grid()
```




As oito primeiras iterações para encontrar uma aproximação podem ser obtidas como é mostrado abaixo:

```
a = 0.1
b = 1.0

f = lambda x: x**2 + np.log(x)

for i in range(8):
    x = (a+b)/2.0
    if f(x)*f(a) < 0:
        b = x
    else:
        a = x
    print ("x =", round(x,4))
```

```
x = 0.55
x = 0.775
x = 0.6625
x = 0.6063
x = 0.6344
x = 0.6484
x = 0.6555
x = 0.652
```

**Exemplo 2.2.2:** Calcular, pelo método da bisseção, corretamente até a terceira casa decimal, a raiz da equação $\left(\frac{x}{2}\right)^2 - \text{sen}(x) = 0$, localizada no intervalo ($a_0 = 1,5$ e $b_0 = 2$).

```
a = 1.5
b = 2.0
err = 1.0
x_ant = 0.0

def f(x):
    y = (0.5*x)**2-np.sin(x)
    return y

while err>0.0001:
    x = (a+b)/2.0

    if f(x)*f(a) < 0:
        b = x
    elif f(x)*f(a) > 0:
        a = x
    else:
        print ("A raiz exata é x=", x)
        break

    err = abs(x-x_ant)
    x_ant = x

    print ("x = %.4f"%x,"| Err = %.4f"% err)
```

```
x = 1.7500 | Err = 1.7500
x = 1.8750 | Err = 0.1250
x = 1.9375 | Err = 0.0625
x = 1.9062 | Err = 0.0312
x = 1.9219 | Err = 0.0156
x = 1.9297 | Err = 0.0078
x = 1.9336 | Err = 0.0039
x = 1.9355 | Err = 0.0020
x = 1.9346 | Err = 0.0010
x = 1.9341 | Err = 0.0005
x = 1.9338 | Err = 0.0002
x = 1.9337 | Err = 0.0001
x = 1.9338 | Err = 0.0001
```

**Exemplo 2.2.3:** Use o método da bisseção para obter um zero da função $f(x) = x \log x - 1$ com erro relativo $\epsilon < 0.01$

```
from math import log10
```

(continua na próxima página)

(continuação da página anterior)

```
a = 2.0
b = 3.0
err = 1.0
x_ant = a
i = 0

f = lambda x:x*log10(x)-1

while err>=0.01:
    x = (a+b)/2.0
    err = abs(x - x_ant)/abs(x)
    print (i, a, b, x, round(f(a),5), round(f(b),5), round(f(x),5), round(err,5))

    if f(a)*f(x)<0:
        b = x
    else:
        a = x

    i = i+1
    x_ant = x
```

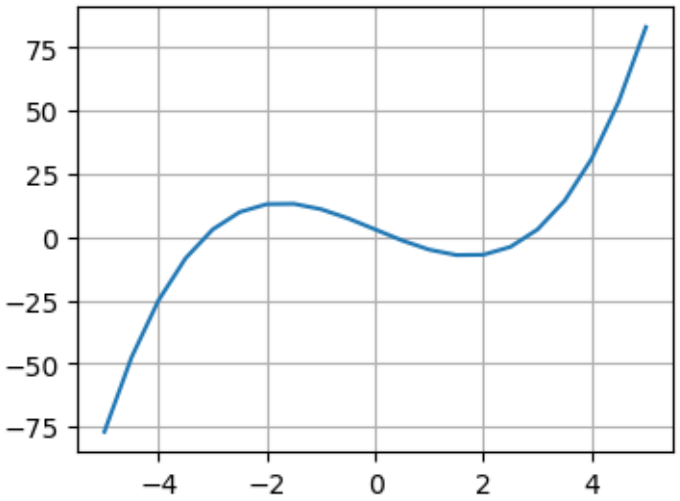
```
0 2.0 3.0 2.5 -0.39794 0.43136 -0.00515 0.2
1 2.5 3.0 2.75 -0.00515 0.43136 0.20816 0.09091
2 2.5 2.75 2.625 -0.00515 0.20816 0.10021 0.04762
3 2.5 2.625 2.5625 -0.00515 0.10021 0.0472 0.02439
4 2.5 2.5625 2.53125 -0.00515 0.0472 0.02094 0.01235
5 2.5 2.53125 2.515625 -0.00515 0.02094 0.00787 0.00621
```

**Exemplo 2.2.4:** Localize e as raízes e use o método da bisseção para obter os zero da função $f(x) = x^3 - 9x + 3$ com precisão $\epsilon = 10^{-3}$

**Solução:** Nesse exemplo, vamos começar fazendo o gráfico da função.

```
xi = np.linspace(-5, 5, 21)
f = lambda x: x**3-9*x+3

plt.plot(xi, f(xi))
plt.grid()
plt.show()
```




Pelo gráfico, podemos ver que parece existirem três raízes distintas $x_1 \in (-4, -3)$, $x_2 \in (0, 1)$ e $x_3 \in (2, 3)$. Vamos confirmar criando uma tabela de valores com o código a seguir.

```
xi =  np.linspace(-4,3, 15)
for x in xi:
    print (x, f(x))
```

```
-4.0 -25.0
-3.5 -8.375
-3.0 3.0
-2.5 9.875
-2.0 13.0
-1.5 13.125
-1.0 11.0
-0.5 7.375
0.0 3.0
0.5 -1.375
1.0 -5.0
1.5 -7.125
2.0 -7.0
2.5 -3.875
3.0 3.0
```

Pela tabela, confirmamos a existência das raízes nos segintes intervalos: $x_1 \in (-3.5, -3)$, $x_2 \in (0, 0.5)$ e $x_3 \in (2.5, 3)$. usaremos agora o método da bisseção para encontrar cada uma das raízes com erro inferior a $10^{-3}$. Faremos isso criando uma função para o método da bisseção, cujos argumentos de entrada são o intervalo inicial(a,b) e a precisão $\epsilon$.

```
def bissec(f, a, b, eps):
    err = 100
    x_ant = a
    while err>eps:
```

(continua na próxima página)

(continuação da página anterior)

```
        x = (a+b)/2.0
        err = abs(x - x_ant)/abs(x)
        if f(a)*f(x)<0:
            b = x
        else:
            a = x
        x_ant = x
    return (x)

eps = 10**(-3)

print ("x1 =", bissec(f, -3.5, -3, eps))
print ("x2 =", bissec(f, 0, 0.5, eps))
print ("x3 =", bissec(f, 2.5, 3, eps))
```

```
x1 = -3.154296875
x2 = 0.337646484375
x3 = 2.818359375
```

## Exercícios

**1.** Localize graficamente e determine as raízes reais das funções dadas. Estime o erro relativo a cada iteração e pare quando $\epsilon \leq 1\%$.

a) $f(x) = -0.5x^2 + 2.5x + 4.5$

b) $f(x) = 5x^2 - 5x^2 + 6x - 2$

c) $f(x) = -25 + 82x - 90x^2 + 44x^3 - 8x^4 + 0.7x^5$

d) $f(x) = -12 - 21x + 18x^2 - 2.75x^3$

e) $f(x) = 3x - e^x = 0$

f) $f(x) = \text{sen}(x) + x^2 + 1 = 0$

g) $f(x) = \text{sen}(x) - x + 2 = 0$

h) $f(x) = 2x - \text{tg}(x) = 0$

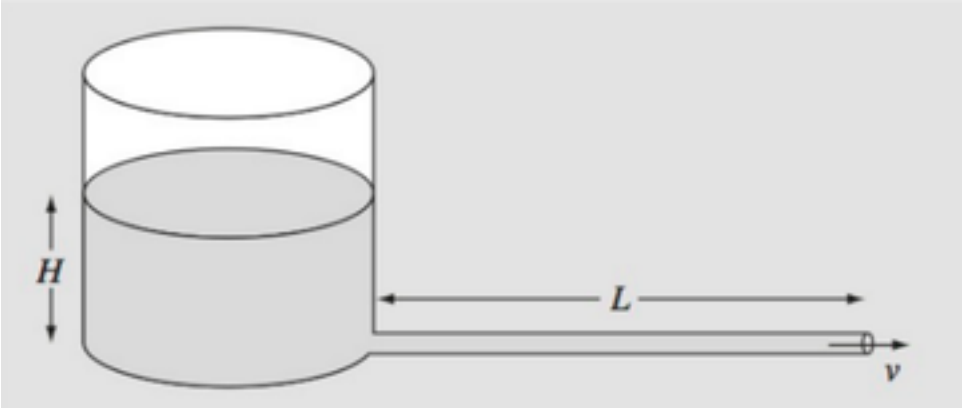
i) $f(x) = 3x - \cos(x) + 1 = 0$

j) $f(x) = \ln(x) - \text{sen}(x) = 0$

**2.** Como mostrado na figura abaixo, a velocidade da água, $v(m/s)$, descarregada de um tanque cilíndrico através de um cano longo pode ser calculada por

$$v = \sqrt{2gH}\tanh\left(\frac{\sqrt{2gH}}{2L}t\right)$$

em que $g = 9.81m/s^2$, $H$ é a altura inicial $(m)$, $L$ é o comprimento do cano $(m)$ e $t$ é o tempo decorrido $(s)$. Determine a altura necessária para atingir $v = 5m/s$ em $2.5s$ para um cano de $4m$ de comprimento pelo método da bisseção e pelo método do ponto-fixo com $\epsilon = 0.001$




**3.** Água está escoando em um canal trapezoidal a uma vazão de Q = 20 $m^3$/s. A profundidade crítica $y$ para tal canal deve satisfazer a equação

$$0 = 1 - \frac{Q^2}{gA_c^3}B$$

onde $g$ = 9.81 m/$s^2$, $A_c$ é a área da seção transversal ($m^2$) e $B$ é a largura do canal na superfície (m). Para esse caso, a largura e a área transversal podem ser relacionadas à profundidade $y$ por
Encontre a profundidade crítica usando o método gráfico da bissecção e itere até que o erro aproximado fique abaixo de 1%. Discuta seus resultados

## 2.3 Método do ponto fixo

Muitos problemas em ciências e engenharia envolvem a resolução de uma equação que pode ser colocada na forma $f(x) = 0$. Em alguns casos a solução exata $\overline{x}$ pode ser encontrada por processos analíticos, quando isso não for possível, uma solução aproximada pode ser obtida por métodos numéricos.

Em geral, a solução numérica é obtida por meio de um processo iterativo que gera uma sequência de soluções aproximadas $x_i$, $i = 1, 2, 3, ...$ a partir de uma aproximação inicial $x_0$. Se os valores obtidos se aproximam cada vez mais da solução exata $\overline{x}$, ou seja, $|\overline{x} - x_i| \to 0$ quando $i \to \infty$, dizemos que a sequência está convergindo para a solução exata.

O Método das aproximações sucessivas, ou método do ponto fixo, consiste em reescrever a equação $f(x) = 0$ em uma equação equivalente $x = \phi(x)$ em que $\phi(x)$ é a chamada função de iteração. A raiz $\overline{x}$ procurada é obtida por meio de uma sequência de de soluções aproximadas, geradas pelo processo iterativo dado por

$$x_{i+1} = \phi(x_i), \quad i = 0, 1, 2, ...$$

Existem muitas formas diferentes de se obter a função de iteração, como é mostrado nos exemplos a seguir.

**Exemplo 2.3.1:** Seja a equação $f(x) = x^2 - 5x = 0$, então, algumas possíveis funções de iteração são
(a) $x_{i+1} = \phi(x_i) = \sqrt{5x_i}$
(b) $x_{i+1} = \phi(x_i) = \frac{x_i^2}{5}$
(c) $x_{i+1} = \phi(x_i) = x^2 - 4$

**Exemplo 2.3.2:** Seja a equação $f(x) = x^3 - x - 7 = 0$, então, algumas possíveis funções de iteração são:
(a) $x_{i+1} = \phi(x_i) = x_i^3 - 7$
(b) $x_{i+1} = \phi(x_i) = \sqrt[3]{x + 7}$

(c) $x_{i+1} = \phi(x_i) = 7/(x^2 - 1)$

De modo geral, sempre é possível obter uma função de iteração multiplicando uma função qualquer $\theta(x)$ pela equação $f(x) = 0$ e somando $x$ a ambos os lados obtendo-se

$$x_{i+1} = \phi(x_i) = f(x)\theta(x_i) + x$$

O processo iterativo gera uma sequência de soluções aproximadas a partir de uma solução inicial ("chute" inicial) $x_0$, como é mostrado abaixo

$$x_1 = \phi(x_0)$$

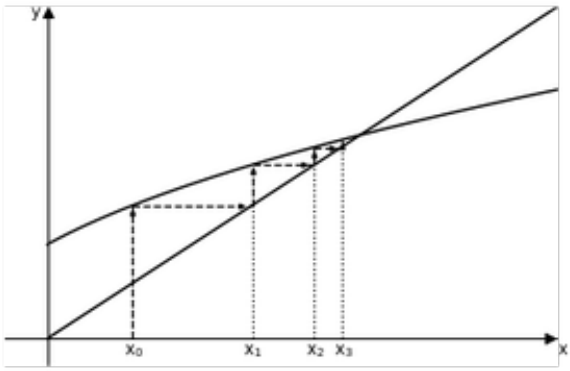
$$x_2 = \phi(x_1)$$

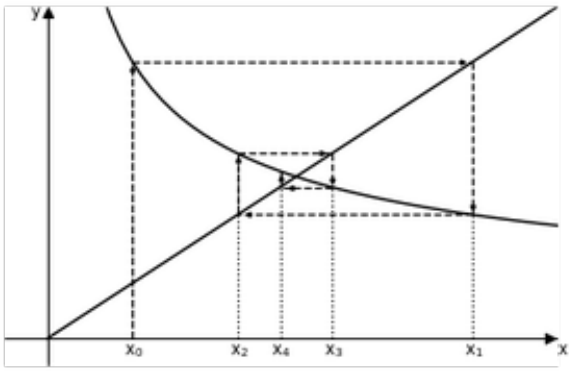
$$x_3 = \phi(x_2)$$

$$\vdots$$

$$x_n = \phi(x_{n-1}) \simeq \overline{x}$$

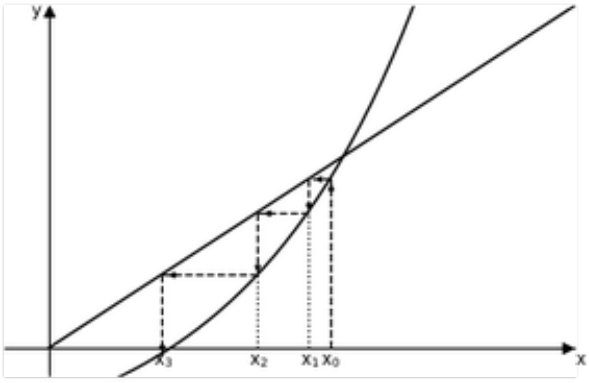
Geometricamente, interpretamos o processo iterativo como é mostrado nas figuras abaixo, para dois casos distintos. O ponto de intersecção entre o gráfico de $\phi(x)$ e a reta $y = x$ é o ponto cuja abcissa é a raix procurada $\overline{x}$.

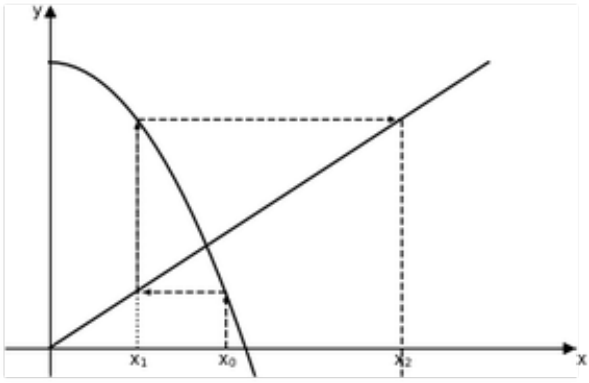



**Convergência:** Seja $\phi(x)$ uma função contínua e derivável em $I = [a, b]$ centrado em $\overline{x}$, então, se $|\phi'(x)| < 1$ em $I$, a sequência $x_k$, $k = 0, 1, 2, 3, ...$ gerada por $x_{k+1} = \phi(x_k)$ converge para a raiz $\overline{x}$ procurada.

No caso do processo iterativo gerar sequências divergentes, uma interpretação geométrica é mostrada na figura a seguir.

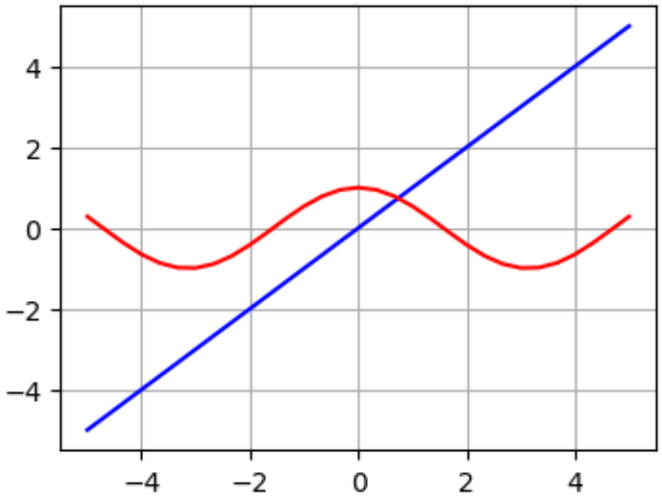


**Exemplo 2.3.3:** Encontre uma raiz para $f(x) = x - cosx$ com $\epsilon = 0.01$.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
f1 = lambda x: x
f2 = lambda x: np.cos(x)
X = np.linspace(-5, 5, 31)
plt.plot(X, f1(X), "b-", X, f2(X), "r-")
plt.grid()
plt.show()
```




```
from math import cos
phi = lambda x: cos(x)
x0 = 0.7
eps = 0.01

# Inicializando as variáveis
err = 10.0
x_ant = x0
i = 0

while err>eps:
    x = phi(x_ant)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    i=i+1
    print (i, "| x=%.9f"%x, "| err=%.9f"%err)
```

```
1 | x=0.764842187 | err=0.084778518
2 | x=0.721491640 | err=0.060084615
3 | x=0.750821329 | err=0.039063474
4 | x=0.731128773 | err=0.026934457
5 | x=0.744421184 | err=0.017856035
6 | x=0.735480200 | err=0.012156661
7 | x=0.741508652 | err=0.008129981
```

**Exemplo 2.3.4:** Encontrar a menor raíz positiva de $f(x) = 2x - \tan(x)$ com $\epsilon < 0.01$.

*Solução:* Inicialmente precisamos localizar a menor raiz positiva da função. Sabemos que há uma raíz em $x = 0$ e a outra é próxima de $\pi/2$. Para conseguir uma boa aproximação inicial, fazemos algumas contas e gráfico:

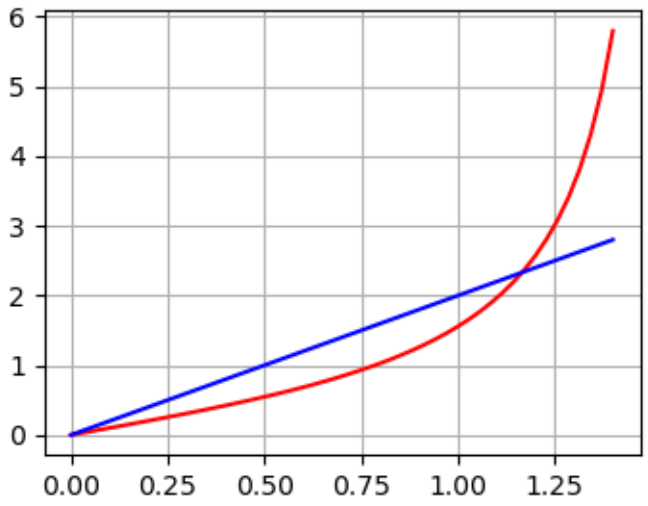
```
import numpy as np
import matplotlib.pyplot as plt

for x in np.linspace(0,np.pi,10):
    print ("f(",x,")=", 2*x-np.tan(x))
```

```
f( 0.0 )= 0.0
f( 0.3490658503988659 )= 0.33416146653152945
f( 0.6981317007977318 )= 0.5571637704181837
f( 1.0471975511965976 )= 0.3623442948243185
f( 1.3962634015954636 )= -2.8787550164267794
f( 1.7453292519943295 )= 9.16194032360637
f( 2.0943951023931953 )= 5.920841012355269
f( 2.443460952792061 )= 5.726021536761403
f( 2.792526803190927 )= 5.9490238406480564
f( 3.141592653589793 )= 6.283185307179586
```

Na tabela acima para valores da função entre 0 e $\pi$, já vemos que a menor raíz positiva está entre 1.04 e 1.39. Embora possa parecer qu existe outa raiz entre 1.39 e 1.75, essa é uma conclusão erradapois a função não é contínua em $x = \pi/2$. Fazendo o gráfico de $f_1(x) = 2x$ e $f_2(x) = \tan(x)$ podemos ter uma ideia melhor da localização dessa raiz e concluir que ela está próxima de $x = 1.2$.

```
X = np.linspace(0,1.4,50)
plt.plot(X, np.tan(X),"r",X, 2*X,"b")
plt.grid()
plt.show()
```




Usando o método do ponto fixo, a primeira função de iteração que imediatamente imaginamos é $\phi(x) = \tan(x)/2$. Mas nesse caso o método converge apenas para a raiz nula já que $|\phi'(x)| = |\tan(x)/2|$ é maior que 1 para valores próximos de $x = 1.2$ como podemos ver abaixo.

```
for x in np.linspace(0.8,1.2,6):
    print ('x =',x, "phi' =",  2-(1/(cos(x))**2 ))
```

```
x = 0.8 phi' = -0.06015555816475615
x = 0.88 phi' = -0.4632872860442183
x = 0.96 phi' = -1.0402051218775918
x = 1.04 phi' = -1.9023027471028362
x = 1.12 phi' = -3.268171142025814
x = 1.2 phi' = -5.615963967207052
```

Assim precisamos de uma outra função de iteração, talvez $\phi(x) = \tan^{-1}(2x)$. Podemos testar fazendo algumas iterações e verificando que está se aproximando de um valor fixo próximo de $x = 1.2$.

```
x = 1.0
for i in range(7):
    x = np.arctan(2*x)
    print (x)
```

```
1.1071487177940904
1.1466039045135867
1.1595864584933437
1.1636959143671342
1.1649805955397388
1.165380637446075
1.16550505598934
```

Lembrando que $\frac{d}{dx}(\tan^{-1}) = \frac{1}{1+x^2}$ fica fácil verificar a convergência.

A seguir a mesma equação resolvida pelo Método de Newton-Raphson, que é assunto da próxima seção.

```
from math import tan, cos
x = 1.16
for i in range(5):
    #x = tan(x)/2
    x = x-(2*x-tan(x))/(2-(1/(cos(x))**2))
    print (x, 2*x-tan(x))
```

```
1.1656668721422732 -0.00046879734426985564
1.1655612229823151 -1.6749979936392378e-07
1.165561185207216 -2.1316282072803006e-14
1.1655611852072112 4.440892098500626e-16
1.1655611852072112 4.440892098500626e-16
```

## Exercícios

---

**1.** Encontre uma aproximação inicial e utilize o método do ponto fixo para as raízes das equações a seguir, se possível, com precisão de $10^{-5}$.

a) $f(x) = 3x - e^x = 0$

b) $f(x) = \text{sen}(x) + x^2 + 1 = 0$

c) $f(x) = \text{sen}(x) - x + 2 = 0$

d) $f(x) = 2x - \text{tg}(x) = 0$

e) $f(x) = 3x - \cos(x) + 1 = 0$

f) $f(x) = \ln(x) - \text{sen}(x) = 0$

---

**2.** Utilize o método do ponto fixo para encontrar soluções com precisão de $10^{-5}$, se possível, para os problemas a seguir (Fonte: Burden et al., 2016):

a) $e^x + 2^{-x} + 2\cos x - 6 = 0$

b) $\ln(x-1) + \cos(x-1) = 0$

c) $2x\cos 2x - (x-2)^2 = 0$

d) $(x-2)^2 - \ln x = 0$

e) $e^x - 3x^2 = 0$

f) $\text{sen}\, x - e^{-x} = 0$

g) $x^3 + 3x^2 - 1 = 0$

h) $x^3 - 2x^2 - 5 = 0$

i) $x - 0,8 - 0,2\,\text{sen}\, x = 0$

---

## 2.4 Método de Newton-Raphson

Sabemos que muitos problemas em ciências e engenharia envolvem a resolução de uma equação que pode ser colocada na forma

$$f(x) = 0$$

Sabemos também que, em muitos casos, a solução exata $\overline{x}$ só pode ser encontrada por processos numéricos. Uma solução numérica é geralmente obtida por meio de um processo iterativo que gera uma sequência de soluções aproximadas, que converge para a solução exata, a partir de uma aproximação inicial $x_0$.

O *Método de Newton* (*Newton-Raphson* ou *método das tangentes*) para zeros de funções reais em uma variável pode ser deduzido a partir do método do ponto fixo, ao buscar uma função de interação $\phi(x)$ tal que $|\phi'(x)| < 1$. Dessa forma, escolhendo uma "boa" aproximação inicial $x_0$, garantimos que a sequência de soluções aproximadas irá convergir para a solução exata $\overline{x}$.

Para isso, estabelecemos que $|\phi'(\overline{x})| = 0$, assim, espera-se que em torno de $\overline{x}$ tenhamos $|\phi'(x)| < 1$. Para encontrar a função de iteração $\phi(x)$ que satisfaça essas condições, começamos multiplicando $f(x) = 0$ por uma função desconhecida $\theta(x)$ e somamos $x$ em ambos os lados da equação obtendo

$$x = x + \theta(x)f(x)$$

ou

$$\phi(x) = x + \theta(x)f(x)$$

nas quais $\theta(x)$ é uma função de $x$ tal que $\theta(\overline{x}) \neq 0$. Então, para o ponto fixo $x = \overline{x}$ tem-se $\phi(\overline{x}) = \overline{x}$.

Derivando a função $\phi(x)$ e avaliando no ponto fixo obtemos: $\$\phi'(x) = 1 + \theta'(x)f(x) + \theta(x)f'(x)\$$

avaliando em $\overline{x}$ (lembrando que $f(\overline{x}) = 0$ e $\theta'(\overline{x}) = 0$), obtemos:

$$\phi'(\overline{x}) = 1 + \theta'(\overline{x})f(\overline{x}) + \theta(\overline{x})f'(\overline{x})$$

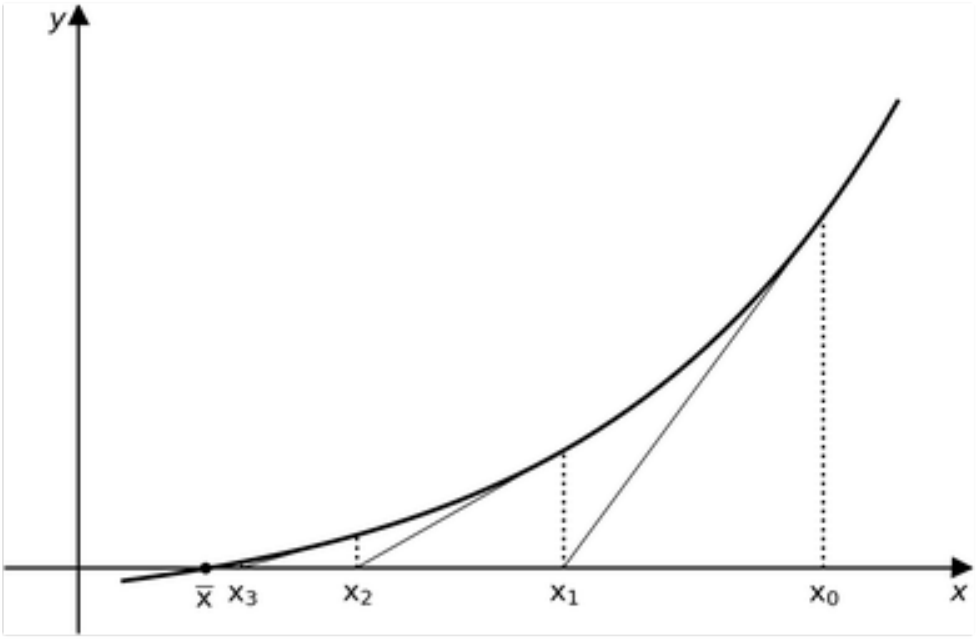
$$0 = 1 + \theta(\overline{x})f'(\overline{x})$$

então $\$\theta(\overline{x}) = -\frac{1}{f'(\overline{x})}\$$

Assim, escolhento $\theta(x) = -\frac{1}{f'(x)}$ e substituindo em $\phi(x) = x + \theta(x)f(x)$ obtemos a função de iteração $\$\phi(x) = x - \frac{f(x)}{f'(x)}\$$

que fornece o processo iterativo do **método de Newton-Raphson** : $\$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}\$$

a partir de uma aproximação inicial $x_0$, para $i = 1, 2, 3, ....$



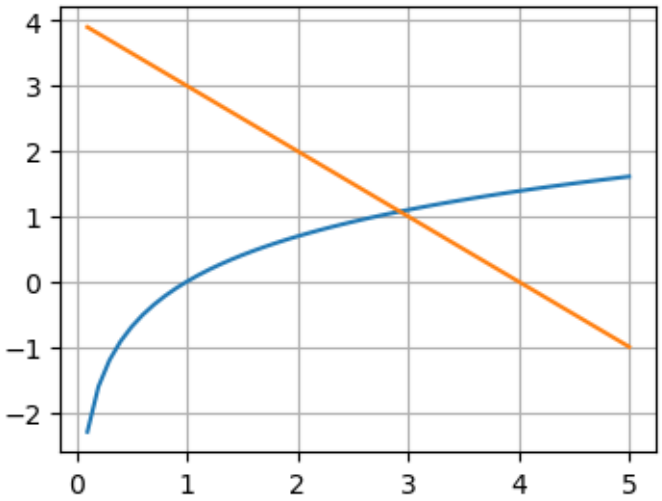

Vejamos agora alguns exemplos. Iniciamos importando as bibliotecas `numpy`e `matplotlib`.

```
import numpy as np
import matplotlib.pyplot as plt
```

**Exemplo 2.4.1:** Encontre a raiz de $f(x) = \ln(x) + x - 4$

Para obter uma aproximação inicial podemos usar um plot minimalista,fazendo.

```
xi = np.linspace(0.1, 5)
plt.figure(figsize=(4,3))
plt.plot(xi, np.log(xi), xi, 4-xi)
plt.grid()
plt.show()
```




O gráfico mostra que existe uma raiz entre 2 e 3, então, iniciando com $x_0 = 2$ obtemos pelo método de Newton:

$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 2 - \frac{ln(2)+2-4}{1/2+1} = 2.871235$

$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = 2 - \frac{ln(2.871235)+2.871235-4}{1/2.871235+1} = 2.926137$

$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = 2.926137 - \frac{ln(2.926137)+2.926137-4}{1/2.926137+1} = 2.926271$

Assim, em três interações chega-se a uma solução com $E = 0.000046$. Usando Python podemos chegar a esses resultados com o código mostrado abaixo:

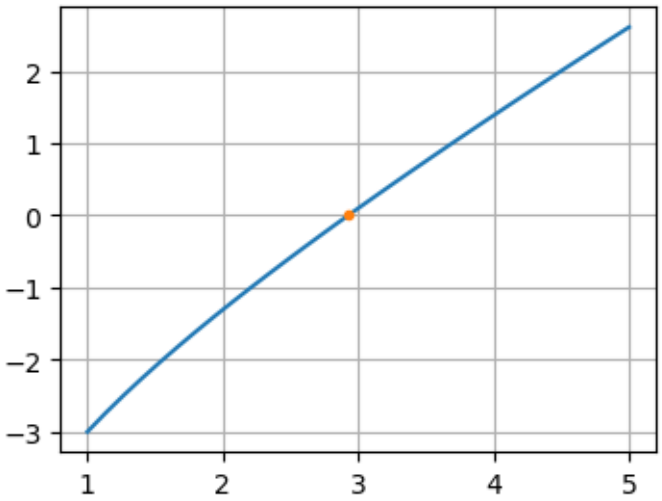
```
f = lambda x: np.log(x)+x-4.0
df = lambda x: 1./x +1
x = 2
x_ant = x
err = 10.0

while err>0.0001:
    x = x - f(x)/df(x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print ("| x=%.6f"%x, "| err=%.6f"%err)
```

```
| x=2.871235 | err=0.303436
| x=2.926137 | err=0.018762
| x=2.926271 | err=0.000046
```

Fazendo o gráfico apenas para conferir

```
f = lambda x: np.log(x)+x-4.0
xi = np.linspace(1, 5, 31)
plt.figure(figsize=(4,3))
plt.plot(xi, f(xi), "-")
plt.plot(x, f(x), ".")
plt.grid()
plt.show()
```




Use o exemplo 2.4.1 como modelo para resolver o Exercício 1 no final da seção.

## O método de Newton e a série te Taylor

O método de Newton também pode ser deduzido a partir do polinômio de Taylor em torno de $x_0$:

$$f(x) = f(x_0) + f^{(1)}(x_0)(x - x_0) + f^{(2)}(x_0)\frac{(x - x_0)^2}{2!} + f^{(3)}(x_0)\frac{(x - x_0)^3}{3!} + ...$$

desprezando os termos de maior grau, os quais assumem valores muito pequenos para $x$ suficientemente próximo de $x_0$, obtemos a seguinte aproximação para $f(x)$:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

fazendo $x = \overline{x}$, como $f(\overline{x}) = 0$, tem-se:

$$0 \approx f(x_0) + f'(x_0)(\overline{x} - x_0)$$

ou

$$\overline{x} \approx x_0 - \frac{f(x_0)}{f'(x_0)}$$

que fornece o processo iterativo

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad i = 1, 2, ...$$

Dizemos que um método iterativo apresenta convergência quadrática se

$$\lim_{i \to \infty} \frac{e_{i+1}}{e_i^2} = k$$

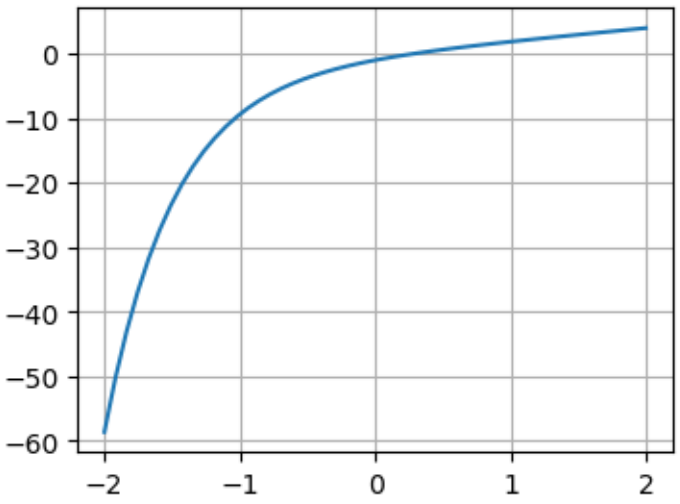
em que $k$ é chamada constante assintótica de proporcionalidade, $e_i = |x_i - \bar{x}|$ e $e_{i+1} = |x_{i+1} - \bar{x}|$ são os erros cometidos nas iterações correspondentes.

O método de Newton apresenta convergência quadrática, se $f'(\bar{x}) \neq 0$.

**Exemplo 2.4.2:** Usando o método de Newton, resolva a equação $2x - e^{-2x} = 0$, com $\epsilon = 0.0001$.

```
f = lambda x: 2*x-np.exp(-2*x)

xi = np.linspace(-2, 2)
plt.figure(figsize=(4,3))
plt.plot(xi, f(xi))
plt.grid()
plt.show()
```




Pelo gráfico podemos deduzir que a raiz está entre 0 e 1, tomemos $x_0 = 0,5$.

```
x0 = 0.5
df = lambda x: 2+2*np.exp(-2*x)

# Inicializando as variáveis
err = 10.0
x_ant = x0
eps = 0.0001

while err>eps:
    x = x_ant-f(x_ant)/df(x_ant)
    err = abs(x-x_ant)/x
    x_ant = x
    print ("x=%.5f, "%x, " err=%.5f"%err)
```

```
x=0.26894,   err=0.85914
x=0.28349,   err=0.05133
x=0.28357,   err=0.00028
x=0.28357,   err=0.00000
```

## Exercícios

**1.** Usando o método de Newton, resolva as equações a seguir com precisão de 5 casas decimais.
a) $f(x) = x^3 + 8x^2 - 4x - 2 = 0$
b) $f(x) = 2x - e^{-x} = 0$
c) $f(x) = \ln(x) - \text{sen}(x) = 0$

**2.** Utilize o método de Newton para encontrar soluções com precisão de $10^{-5}$ para os problemas a seguir (Fonte: Burden et al., 2016):
a) $e^x + 2^{-x} + 2\cos x - 6 = 0$
b) $\ln(x-1) + \cos(x-1) = 0$
c) $2x\cos 2x - (x-2)^2 = 0$
d) $(x-2)^2 - \ln x = 0$
e) $e^x - 3x^2 = 0$
f) $\text{sen}\, x - e^{-x} = 0$
g) $x^3 + 3x^2 - 1 = 0$
h) $x^3 - 2x^2 - 5 = 0$
i) $x - 0,8 - 0,2\,\text{sen}\, x = 0$

**3.** Use o método de Newton-Raphson para determinar uma raiz real de $f(x) = -1 + 5,5x - 4x^2 + 0,5x^3$ usando aproximações iniciais (a) 4,52 e (b) 4,54 . Discuta e use métodos gráficos e analíticos para explicar quaisquer peculiaridades nos seus resultados (Fonte: Chapra, 2016).

**4.** Use o método de Newton-Raphson para encontrar a raiz de

$$f(x) = e^{-0,5x}(4 - x) - 2$$

Use aproximações iniciais de (a) 2, (b) 6 e (c) 8 . Explique seus resultados (Fonte: Chapra, 2016)..

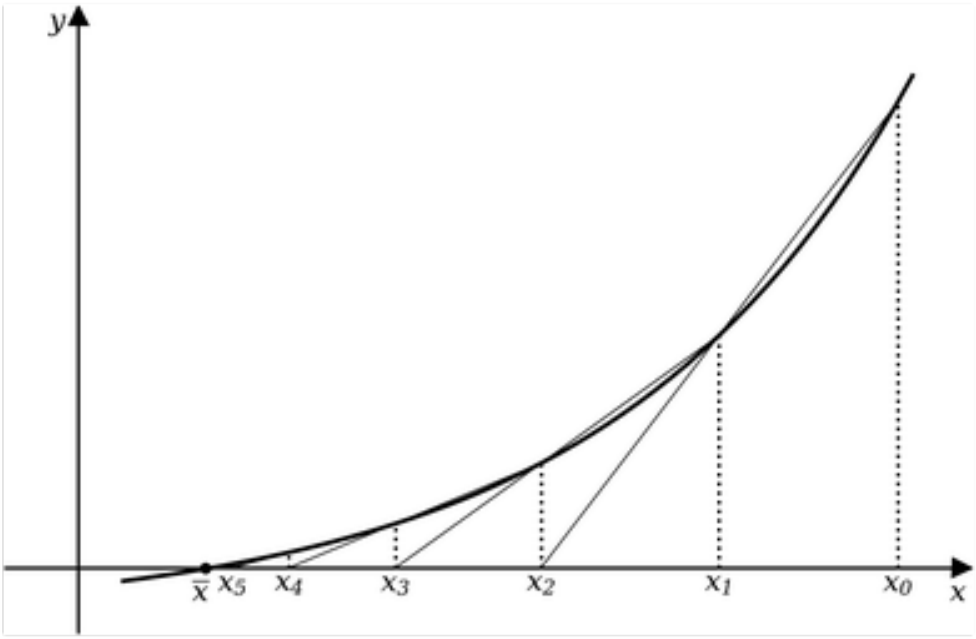
**5.** A função descrita por $f(x) = \ln\left(x^2 + 1\right) - \text{e}^{0,4x}\cos \pi x$ tem um número infinito de zeros (Fonte:Burden et al., 2016).
a) Determine, com precisão de $10^{-6}$, o único zero negativo.
b) Determine, com precisão de $10^{-6}$, os quatro menores zeros positivos.
c) Determine uma aproximação inicial razoável para determinar o enésimo menor zero positivo de $f$.
d) Utilize a parte (c) para determinar, com precisão de $10^{-6}$, o $25^\circ$ menor zero positivo de $f$.

## 2.5 Método das Secantes

O método das secantes consiste em aproximarmos a derivada da função $f'(x_i)$ que ocorre no método de Newton da seguinte forma:

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$

Observe que, neste caso, estamos trocando a inclinação da reta tangente pela inclinação da reta secante à curva, conforme a figura abaixo. Assim, dados os pontos $x_{i-1}$ e $x_i$ , onde a reta secante, passando por $(x_{i-1}, f(x_{i-1}))$ e $(x_i, f(x_i))$, cortar o eixo das abscissas, temos a aproximação $x_{i+1}$ para a raiz $\overline{x}$, conforme ilustrado




Assim, o método de Newton, dado por

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

é modificado da seguinte forma

$$x_{i+1} = x_i - \frac{f(x_i)}{\frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}}$$

Que pode ser reescrito como

$$x_{i+1} = x_i - \frac{x_{i-1}f(x_i) - x_i f(x_{i-1})}{f(x_i) - f(x_{i-1})}$$

**Exemplo 2.5.1:** Vamos encontrar a raiz da equação $\ln(x) + x - 4 = 0$ com tolerância $\epsilon = 0.0001$ usando o método das secantes.

Com base no que já foi feito no exemplo 2.4.1 podemos obter as iterações com o código mostrado a seguir.

```
import math
err = 10.0
x0 = 1.0
x1 = 2.0

f = lambda x: math.log(x)+x-4.0

while err> 0.0001:
    x = (x0*f(x1)-x1*f(x0))/(f(x1)-f(x0))
    err = abs(x-x1)/abs(x)    # calcula o erro relativo
    x0 = x1
    x1 = x
    print ("x=%.5f"%x, ", err=%.5f"%err)
```

```
x=2.77185 , err=0.27846
x=2.91848 , err=0.05024
x=2.92622 , err=0.00264
x=2.92627 , err=0.00002
```

Em Nogueira (2024) é disponibilizada uma implementação do método da secante como uma função cujos parÂmetros de entrada são a função (`f`), a precisão desejada (`e`) e as aproximações iniciais (`x_0`e `x_1`). O código abaixo mostra como utilizar a função `secant`.

```
# definindo a funcao secant
def secant(f, e, x0, x1):
    error = 1
    while error > e:
        x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
        error = abs(x2-x1)/abs(x2)
        x0 = x1
        x1 = x2
    return x2
```

```
# chamando a funcao secant
x = secant(f, 0.0001, 1.0, 2.0)
print("Método da Secante: ", round(x, 5))
```

```
Método da Secante:  2.92627
```

O método da **falsa posição** (também conhecido como *regula falsi*) gera aproximações da mesma maneira que o método da secante, mas inclui um teste para garantir que a raiz esteja sempre no intervalo delimitado por iterações sucessivas.

Primeiramente, escolha as aproximações iniciais $x_0$ e $x_1$ com $f(x_0) \cdot f(x_1) < 0$. Então, do mesmo modo que no método da secante, a aproximação $x_2$ é escolhida como a interseção da reta que liga os pontos $(x_0, f(x_0))$ e $(x_1, f(x_1))$ com o eixo $x$. O ponto de intersecção $x_2$ divide o intervalo $(x_0, x_1)$ em dois subintervalos. Para a próxima iteração escolhe-se o intervalo que contém a raiz, verificando se $f(x_0) \cdot f(x_2)$ ou $f(x_1) \cdot f(x_2) < 0$.

Em Nogueira (2024) é disponibilizada uma implementação do método da falsa posição, `false_pos`como uma função cujos parâmetros de entrada são os mesmos da função `secant`. O código abaixo mostra como utilizar a método da falsa posição para o mesmo problema do exemplo 2.5.1.

**Exemplo 2.5.2:** Vamos encontrar a raiz da equação $\ln(x) + x - 4 = 0$ com tolerância $\epsilon = 0.0001$ usando o método da falsa posição.

```
def false_pos(f, e, x0, x1):
    if f(x0) * f(x1) >= 0:
        return
    error = 1
    while error > e:
        x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
        if x2==0:
            print('raiz')
        if f(x0) * f(x2) < 0:
            x1 = x2
        else:
            x0 = x2
        error = abs(x1 - x0) / abs(x2)
    return x2
```

```
f = lambda x: math.log(x)+x-4.0
x = false_pos(f, 0.0001, 2.0, 4.0)
print("Método da Falsa Posição: ", round(x, 5))
```

```
Método da Falsa Posição:  2.92627
```

## Exercícios

---

**1.** Seja $f(x) = -x^3 - \cos x$. Com $x_0 = -1$ e $x_1 = 0$. Obtenha uma solução com precisão de 5 casas decimais utilizando o método da secante.

---

**2.** Obtenha uma solução para a equação $e^x + 2^{-x} + 2\cos x - 6 = 0$ utilizando o método das secantes com $\epsilon = 10^{-5}$ e faça um gráfico do erro relativo em função do número de iterações.

---

**3.** Resolva os exercícios 1 e 2 usando o método da falsa posição e compare os resultados.

---

**4.** Faça um programa para resolver equações utilizando os métodos da secante e da falsa posição, e plotar um gráfico do erro relativo em função do número de iterações para cada método. Use alguma das equações do exercício a seguir para testar.

---

**5.** Calcule, se possível, as raízes das seguintes funções com pelo menos 6 casas de precisão. Use diferentes métodos e compare a velocidade de convergência.

a) $f(x) = \sqrt{x} - 5^{-x}$
b) $f(x) = e^x + x$
c) $f(x) = xln(x) - 1$
d) $f(x) = \text{sen}(x) - 1/2$
e) $x^3 - 7x^2 + 14x - 6 = 0$
f) $2x\cos(2x) - (x+1)^2 = 0$
g) $x - 2^{-x} = 0$
h) $2 + \cos(e^x - 2) - e^x = 0$
i) $f(x) = –0.5x^2 + 2.5x + 4.5$
j) $f(x) = 5x^2–5x^2 + 6x - 2$
k) $f(x) = –25 + 82x–90x^2 + 44x^3–8x^4 + 0.7x^5$
l) $f(x) = –12–21x + 18x^2–2.75x^3$

---

# 2.6 Raízes de polinômios

## Método de Briot-Ruffini para divisão de polinômios

Dados os polinômios $$P(x) = a_0x^n + a_1x^{n-1} + \cdots + a_neQ(x) = b_0x^{n-1} + b_1x^{n-2} + \cdots + b_{n-1}$$

tal que $P(x) = (x-\alpha)Q(x) + P(\alpha)$, ou seja, $Q(x)$ é o polinômio resultante da divisão de $P(x)$ por $(x-\alpha)$ $P(\alpha)$ é o resto.

Podemos determinar $Q(x)$ e $P(\alpha)$ usando o método de Briot-Ruffini escrevendo

$$\begin{aligned}\left[a_0x^n + a_1x^{n-1} + \cdots + a_n\right] &= (x-\alpha)\left[b_0x^{n-1} + b_1x^{n-2} + \cdots + b_{n-1}\right] + b_n \\ &= b_0x^n + (b_1 - \alpha b_0)\,x^{n-1} + (b_2 - \alpha b_1)\,x^{n-2} + \\ &\quad + \cdots + (b_{n-1} - \alpha b_{n-2})\,x + (b_n - \alpha b_{n-1})\end{aligned}$$

que leva a

$$\begin{aligned} a_0 = b_0 \quad &\rightarrow \quad b_0 = a_0 \\ a_1 = b_1 - \alpha b_0 \quad &\rightarrow \quad b_1 = a_1 + \alpha b_0 \\ &\vdots \\ a_{n-1} = b_{n-1} - \alpha b_{n-2} \quad &\rightarrow \quad b_{n-1} = a_{n-1} + \alpha b_{n-2} \\ a_n = b_n - \alpha b_{n-1} \quad &\rightarrow \quad b_n = a_n + \alpha b_{n-1}\end{aligned}$$

Vejamos o seguinte exemplo:

**Exemplo 2.6.1:** (Arenales e Darezzo, 2016) Seja $P(x) = 6x^3 + x - 1$, calcule $P(3)$.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
def BriotRuffini(a,alpha):
    q = a.copy()
    for i in range(1,len(a)):
         q[i] = alpha*q[i-1] + a[i]
    return q
```

```
an = [6,0,1,-1]
bn = BriotRuffini(an , 3)

print ('Coeficientes de Q(x):', bn[:-1] )
print ('Resto: P(3)=', bn[-1])
```

```
Coeficientes de Q(x): [6, 18, 55]
Resto: P(3)= 164
```

Podemos também obter a derivada $P'(\alpha)$ pelo método de Briot-Ruffini, pois como

$$P(x) = (x - \alpha)Q(x) + P(\alpha)$$

temos que

$$P'(x) = (x - \alpha)Q'(x) + Q(x)$$

Assim, para $x = \alpha$, tem-se que $P'(\alpha) = Q(\alpha)$. Ou seja, para calcular $P'(\alpha)$ precisamos obter o resto da divisão de $Q(x)$ por $(x - \alpha)$, o que pode ser feito usando Briot-Ruffini uma segunda vez.

**Exemplo 2.6.2:** (Arenales e Darezzo, 2016) Seja $P(x) = 6x^3 + x - 1$, calcule $P'(3)$.

```
cn = BriotRuffini(bn[:-1] , 3)
print ("P'(3)=", cn[-1])
```

```
P'(3)= 163
```

## Método de Newton com Briot-Ruffini

Vamos agora obter zeros de polinômios pelo método iterativo de Newton

$$x_{i+1} = x_i - \frac{P\left(x_i\right)}{P'\left(x_i\right)}$$

usando Briot-Ruffini para calcular $P(x_i)$ e $P'(x_i)$ a cada iteração.

**Exemplo 2.6.3:** (Arenales e Darezzo, 2016) Determine uma raiz da equação $P(x) = x^3 + 2x^2 - x - 2 = 0$ usando o método de Newton mais Briot-Ruffini, com $\varepsilon = 0.001$

```
a = np.array([1.,2.,-1.,-2.])
x = 2.
x_ant = x
```

(continua na próxima página)

(continuação da página anterior)

```
def IteraNewton(a,x):
    q = BriotRuffini(a,x)
    m = BriotRuffini(q[0:-1],x)
    return (x - q[-1]/m[-1])
```

```
# uma iteração
IteraNewton(a,x)
```

```
1.368421052631579
```

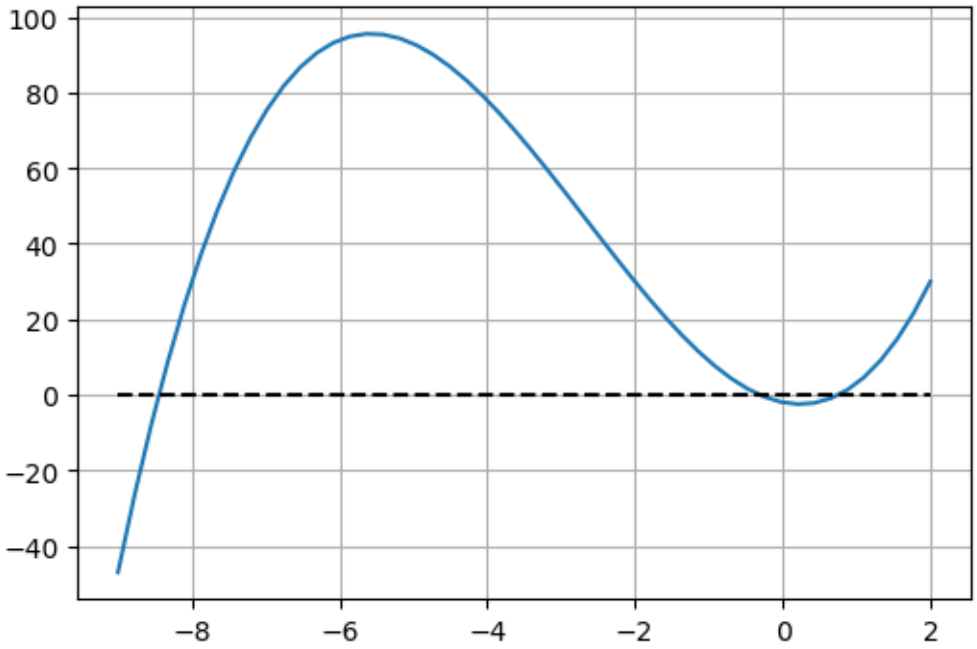
```
# iterando até atingir a precisão
err = 1.
x = 2.
while err>0.001:
    x = IteraNewton(a,x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print("x=",x, 'Erro:', err)
```

```
x= 1.368421052631579 Erro: 0.4615384615384615
x= 1.077163124665906 Erro: 0.27039351913946175
x= 1.0045201633201402 Erro: 0.07231608084965287
x= 1.0000169296346983 Erro: 0.004503157448631215
x= 1.0000000002388385 Erro: 1.6929395855800372e-05
```

**Exemplo 2.6.4:** (Arenales e Darezzo, 2016) Encontre as raízes de $P(x) = x^3 + 8x^2 - 4x - 2 = 0$.

```
a = [1,8,-4,-2]
x = np.linspace(-9,2)

plt.figure(figsize=(6,4))
plt.plot(x, np.polyval(a, x))
plt.plot(x, np.zeros(len(x)), 'k--')
plt.grid()
```




```
# iterando até atingir a precisão
err = 1.
x = 1.
while err>0.001:
    x = IteraNewton(a,x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print("x=",x, 'Erro:', err)
```

```
x= 0.8 Erro: 0.2500000002985481
x= 0.7597014925373134 Erro: 0.053045186640471545
x= 0.7579998221716971 Erro: 0.002244948238564167
x= 0.7579968013604806 Erro: 3.985255889050246e-06
```

```
# iterando até atingir a precisão
err = 1.
x = -1.
while err>0.001:
    x = IteraNewton(a,x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print("x=",x, 'Erro:', err)
```

```
x= -0.47058823529411764 Erro: 2.6107432028910216
x= -0.3279505432746347 Erro: 0.43493659316805666
x= -0.31260601722010856 Erro: 0.04908583075584857
```

(continua na próxima página)

(continuação da página anterior)

```
x= -0.3124159051406068 Erro: 0.000608522409946434
```

```
# iterando até atingir a precisão
err = 1.
x = -7.
while err>0.001:
    x = IteraNewton(a,x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    print("x=",x, 'Erro:', err)
```

```
x= -9.419354838709678 Erro: 0.9668325580158944
x= -8.609635740090033 Erro: 0.09404800888953495
x= -8.451477874162121 Erro: 0.0187136342640655
x= -8.4455889629375 Erro: 0.0006972765606359088
```

## Exercícios

---

**1.** Calcule, se possível, as raízes dos seguintes polinômios com pelo menos 5 casas de precisão:
a) $P(x) = x^3 - 7x^2 + 14x - 6 = 0$
b) $P(x) = -25 + 82x - 90x^2 + 44x^3 - 8x^4 + 0.7x^5$
c) $P(x) = -12 - 21x + 18x^2 - 2.75x^3$

---

**2.** Use o método de Briot-Ruffini para calcular $P(2)$ e $P'(2)$ do polinômio

$$P(x) = 2x^4 - 3x^3 + x^2 - 5x + 4$$

Mostre, a cada divisão, os coeficientes do polinômio quociente $Q(x)$ obtido. Confira o resultado comparando $P(2)$ com `np.polyval` e $P'(2)$ com a derivada calculada simbolicamente (por exemplo, usando `sympy` ou à mão).

---

**3.** O polinômio $P(x) = x^3 - 4x^2 + 5x - 2$ tem uma raiz simples em $x = 2$ e uma raiz dupla em $x = 1$ (verifique fatorando $P(x) = (x-1)^2(x-2)$).
a) Aplique o método de Newton com Briot-Ruffini partindo de $x_0 = 1{,}5$ com $\varepsilon = 10^{-4}$ e observe para qual raiz o método converge.
b) Repita partindo de $x_0 = 0{,}9$ e de $x_0 = 1{,}1$, comparando o número de iterações necessárias para atingir a mesma precisão nos dois casos.

---

**4.** O polinômio $P(x) = x^4 - 2x^3 - 13x^2 + 14x + 24$ tem quatro raízes reais e inteiras.
a) Plote o gráfico de $P(x)$ em um intervalo adequado para localizar visualmente as quatro raízes.
b) Encontre cada uma das raízes usando o método de Newton com Briot-Ruffini, escolhendo uma aproximação inicial diferente para cada uma.
c) Verifique o resultado fatorando $P(x)$ com os valores encontrados.

---

**5. (Deflação de polinômios)** Sabendo que $x = 2$ é raiz do polinômio $P(x) = x^3 - 3x^2 - 4x + 12$,
a) Use `BriotRuffini` para dividir $P(x)$ por $(x-2)$ e obter os coeficientes do polinômio quociente $Q(x)$, de grau 2.
b) Encontre as raízes restantes de $P(x)$ resolvendo $Q(x) = 0$ com a fórmula de Bhaskara.
c) Confirme que as três raízes encontradas satisfazem $P(x) = (x - r_1)(x - r_2)(x - r_3)$.

---

**6. (Convergência perto de uma raiz múltipla)** Considere $P(x) = (x-2)^2(x+1) = x^3 - 3x^2 + 4$, que tem uma raiz dupla em $x = 2$ e uma raiz simples em $x = -1$.
a) Aplique o método de Newton com Briot-Ruffini partindo de $x_0 = 3$ (próximo da raiz dupla) e de $x_0 = -2$ (próximo da raiz simples), ambos com $\varepsilon = 10^{-6}$, e imprima o erro a cada iteração.
b) Compare a velocidade de convergência nos dois casos. Qual delas converge mais lentamente?
c) Sabe-se que, perto de uma raiz de multiplicidade $m$, o método de Newton perde a convergência quadrática e passa a convergir linearmente. Os resultados obtidos são compatíveis com essa afirmação?

---

**7.** Considere o polinômio $P(x) = x^2 + 1$, que não possui raízes reais (suas raízes são $\pm i$).
a) Aplique o método de Newton com Briot-Ruffini partindo de $x_0 = 1$ por 10 iterações e observe o comportamento da sequência gerada.
b) O método converge? Explique, em termos geométricos (o gráfico de $P(x)$ nunca cruza o eixo $x$), por que isso acontece.

---

**8. (Eficiência computacional)** O método de Briot-Ruffini avalia $P(\alpha)$ usando apenas $n$ multiplicações e $n$ somas para um polinômio de grau $n$ (esquema de Horner).
a) Escreva uma função `avaliacao_ingenua(a, x)` que calcule $P(x) = \sum_{i=0}^{n} a_i x^{n-i}$ termo a termo, usando `x**k` para cada potência, e conte o número de multiplicações realizadas.
b) Compare esse número de multiplicações com o número de multiplicações usadas por `BriotRuffini` para o mesmo polinômio, para graus $n = 5, 10, 20$ e $50$.
c) Discuta como essa diferença de custo se torna relevante ao aplicar o método de Newton em polinômios de grau elevado.

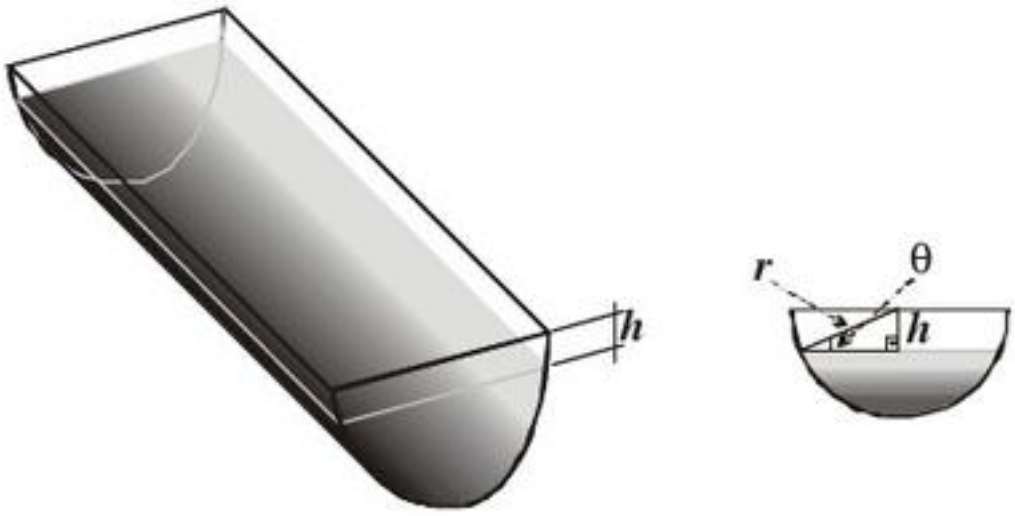
## 2.7 Exemplos de aplicações

```
import math
import numpy as np
import matplotlib.pyplot as plt
```

**Problema 1:** Um tanque de comprimento $L$ tem uma secção transversal no formato de um semicírculo com raio $r$ (veja a figura). Quando cheio de água até uma distância $h$ do topo, o volume $V$ da água é:

$$V = L\left[0.5\pi r^2 - r^2 arcsen\left(\frac{h}{r}\right) - h\sqrt{r^2 - h^2}\right]$$

Supondo que $L = 10ft$, $r = 1ft$ e $V = 12.4ft^3$, encontre a profundidade do tanque com precisão de $0.0001ft$ utilizando um programa em Python para o Método da Bisseção e Método de Newton. Apresente os códigos utilizados e comente os resultados obtidos.




*Resolução:* Substituindo os valores de $L$, $r$ e $V$ na equação

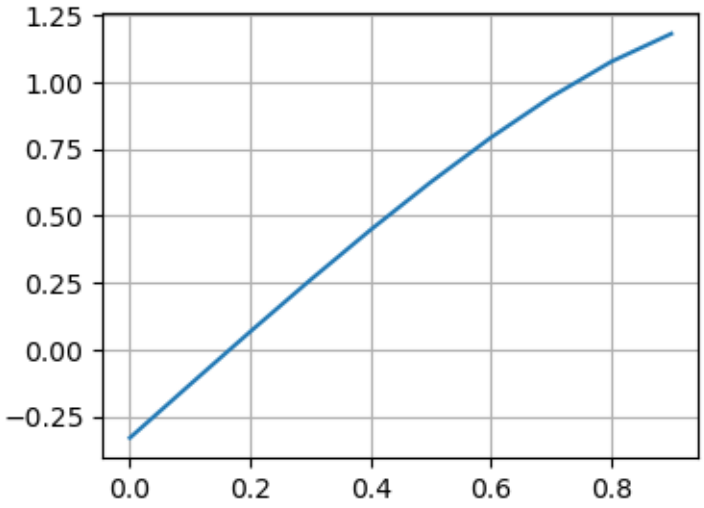
$$f(h) = \frac{V}{L} - \left[0.5\pi r^2 - r^2 arcsen\left(\frac{h}{r}\right) - h\sqrt{r^2 - h^2}\right] = 0$$

obtem-se a seguinte função, cuja raiz é a solução do problema

$$f(h) = arcsen(h) + h\sqrt{1 - h^2} - 0.3308$$

```
plt.figure(figsize=(4,3))
f = lambda h: np.arcsin(h)+h*(1-h**2)**0.5-0.3308

# Localização por método gráfico
h = np.arange(0.0, 1.0, 0.1)
plt.plot(h, f(h))
plt.grid()
plt.show()
```



Observa-se pelo gráfico que a raiz está entre 0,15 e 0,2.

```
f = lambda h: math.asin(h)+h*(1-h**2)**0.5-0.3308

#como é uma secção transversal no formato de um semicírculo de raio r=1, h vai de 0 a␣
↪1
a = 0.15
b = 0.2
eps = 0.0001
err = 10.0
h_ant = a

while err>eps:
    h = (a+b)/2.
    err = abs(h-h_ant)/abs(h)
    h_ant = h
    print ("|h=%.4f"%h,"|err=%.4f"%err)

    if f(h)*f(a)<0:
        b = h
    else:
        a = h
#foi calculada a distância h do topo, então a profundidade é o raio=1 menos h
print ("A profundidade aproximada pelo método da bisseção é ",1.-h)
```

```
|h=0.1750 |err=0.1429
|h=0.1625 |err=0.0769
|h=0.1687 |err=0.0370
|h=0.1656 |err=0.0189
|h=0.1672 |err=0.0093
|h=0.1664 |err=0.0047
|h=0.1660 |err=0.0024
|h=0.1662 |err=0.0012
|h=0.1661 |err=0.0006
|h=0.1662 |err=0.0003
|h=0.1662 |err=0.0001
|h=0.1662 |err=0.0001
A profundidade aproximada pelo método da bisseção é  0.8338256835937501
```

```
f = lambda h: math.asin(h)+h*(1-h**2)**0.5-0.3308
df = lambda h: 1./(1.-h**2)**0.5 + (1.-h**2)**0.5 + h*0.5*(1-h**2)**(-0.5)*(-2*h)

h0 = 0.0
eps = 0.0001
err = 10.
```

(continua na próxima página)

(continuação da página anterior)

```
while err>eps:
    h = h0 - f(h0)/df(h0)
    err = abs(h-h0)/abs(h)
    h0 = h
    print ( "h=%.5f"%h, "| err=%.5f"%err)
#foi calculada a distância h do topo, então a profundidade é o raio=1 menos h
print ("A profundidade aproximada pelo método de Newton é ",1.-h)
```

```
h=0.16540 | err=1.00000
h=0.16617 | err=0.00462
h=0.16617 | err=0.00000
A profundidade aproximada pelo método de Newton é  0.8338321028459801
```

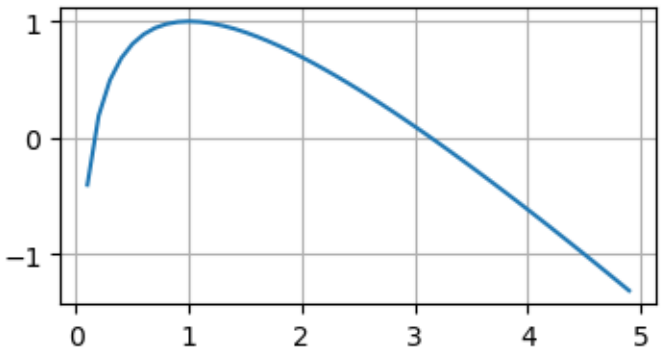
**Comentário** É possível observar ambos os métodos convergem para a raiz, no entanto o método de Newton converge mais rapidamente, com apenas 3 iterações, enquanto o método da bisseção precisou de 12 iterações para atingir a precisão de 0,0001.

**Problema 2:** Calcule todas as raízes de f(x)=lnx−x+2=0 pelo método do Aproximações Sucessivas (Ponto-Fixo) até que o erro relativo seja inferior a 0.0001. Em seguida, use o Método de Newton para resolver a mesma equação e compare os resultados comentando sobre o que você observou.

*Resolução:* Para o método do Ponto Fixo, precisamos encontrar uma função de interação ϕ(x) escrevendo uma equação equivalente a f(x)=lnx−x+2=0 na forma x=ϕ(x). Existem muitas formas de fazer isso, as funções obtidas podem gerar sequências convergentes ou não. Se |ϕ′(x)|≤1 em torno da raiz, a sequência será convergente. Entre as equações possíveis, tem-se ϕ1(x)=ln(x)+2 e ϕ2(x)=e(x−2).

```
plt.figure(figsize=(4,2))

# Localização por método gráfico
x = np.arange(0.1, 5.0, 0.1)
plt.plot(x, np.log(x)-x+2)
plt.grid()
plt.show()
```




Observa-se pelo gráfico que a função possui 2 raízes. Uma próxima de $x = 0.0$ e outra próxima de $x = 3.0$. Analisando a convergência do método do ponto fixo, vemos que $|\phi_1'(x)| = |\frac{1}{x}| \geq 1$ em torno de $0.0$ e $\leq 1$ em torno de $3.0$. Além disso, $|\phi_2'(x)| = e^{(x-2)} \leq 1$ em torno de $0.0$ e $\geq 1$ em torno de $3.0$. Assim, devemos usar $\phi_1$ para a maior

raiz e $\phi_2$ para a menor raiz.

```
phi1 = lambda x: math.log(x)+2
phi2 = lambda x: math.exp(x-2)

def ponto_fixo(phi, x0, eps):
    err = 1.0
    while err>eps:
        x = phi(x0)
        err = abs(x-x0)/abs(x)
        x0 = x
        print ( "| x=%.5f"%x, "| err=%.5f"%err)

print ("Raiz 1:")
ponto_fixo(lambda x: math.exp(x-2), 0.1, 0.0001)

print ("Raiz 2:")
ponto_fixo(lambda x: math.log(x)+2, 3.0, 0.0001)
```

```
Raiz 1:
| x=0.14957 | err=0.33141
| x=0.15717 | err=0.04836
| x=0.15837 | err=0.00757
| x=0.15856 | err=0.00120
| x=0.15859 | err=0.00019
| x=0.15859 | err=0.00003
Raiz 2:
| x=3.09861 | err=0.03182
| x=3.13095 | err=0.01033
| x=3.14134 | err=0.00331
| x=3.14465 | err=0.00105
| x=3.14570 | err=0.00033
| x=3.14604 | err=0.00011
| x=3.14614 | err=0.00003
```

Para o método de Newton, usamos a mesma função de iteração $x_{i+1} = x_1 - \frac{f(x_i)}{f'(x_i)}$ e variamos a aproximação inicial para obter cada uma das raízes. Para a menor fazemos $x_0 = 0,0$ e para a maior fazemos $x_0 = 3,0$.

```
def newton(x0, eps):
    err = 10.0
    x_ant = x0
    while err>eps:
        x = x_ant-f(x_ant)/df(x_ant)
        err = abs(x-x_ant)/x
        x_ant = x
        print ( "| x=%.5f"%x, "| err=%.5f"%err)
```

(continua na próxima página)

(continuação da página anterior)

```
f = lambda x: math.log(x) - x + 2.0
df = lambda x: 1.0/x - 1.0
eps = 0.0001

print ("Newton x0 = 0.01")
newton(0.01, eps)

print ("Newton x0 = 3.5")
newton(3.5, eps)
```

```
Newton x0 = 0.01
| x=0.03642 | err=0.72539
| x=0.08740 | err=0.58336
| x=0.13765 | err=0.36503
| x=0.15691 | err=0.12278
| x=0.15858 | err=0.01052
| x=0.15859 | err=0.00007
Newton x0 = 3.5
| x=3.15387 | err=0.10975
| x=3.14620 | err=0.00244
| x=3.14619 | err=0.00000
```

**Comentário** Para o método do ponto fixo precisamos de duas funções de iteração diferentes. Para Newton, basta variar a aproximação inicial. Newton converge mais rápido.

**Problema 3:**

A tensão elétrica sobre o capacitor de um circuito RLC-paralelo é dado por

$$u(t) = u_0 e^{-\alpha t} \cos(w_d t), t \geq 0$$

onde $u_0$ é a tensão inicial sobre o capacitor, $\alpha = \frac{1}{2RC}$ é a constante de amortecimento, $w_0 = \frac{1}{\sqrt{LC}}$ é a frequência angular de oscilação (natural), $w_d = \sqrt{w_0^2 - \alpha^2}$ é a frequência angular de oscilação (amortecida). A tensão $u$ é dada em volts e o tempo $t$ em segundos. Considere um circuito em que $R = 200\Omega$, $L = 0,3H$, $C = 40\mu F$ e $u_0 = 50V$. Substitua os valores na equação e determine os instantes $t_1$, $t_2$ e $t_3$ em que a tensão atinge o valor $10V$. Use o método das secantes e mais algum outro de sua escolha. Comente os resultados.

## Resolução

```
R = 200.0
L = 0.3
C = 0.000040
u0 = 50.0
#Ti = 300

alpha = 1./(2*R*C)
```

(continua na próxima página)

(continuação da página anterior)

```
w0 = 1./math.sqrt(L*C)
wd = math.sqrt(w0**2 - alpha**2)

f = lambda t: 10-u0*np.exp(-alpha*t)*np.cos(wd*t)

t = np.arange(0.0, 0.2, 0.001)

plt.figure(figsize=(6,3))

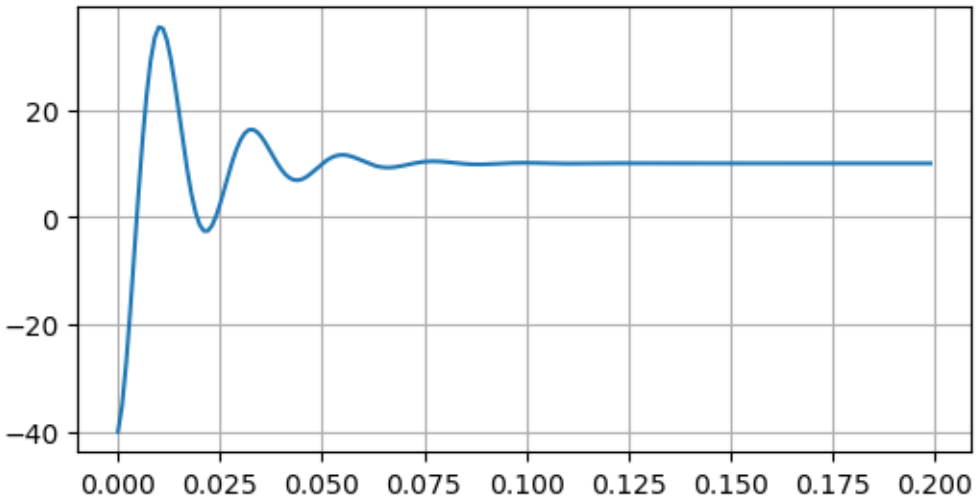
plt.plot(t, f(t))
plt.grid()
plt.show()

def secantes(x0,x1):
    err = 10.0
    while err> 0.0001:
        x = (x0*f(x1)-x1*f(x0))/(f(x1)-f(x0))
        err = abs(x-x1)/abs(x)    # calcula o erro relativo
        x0 = x1
        x1 = x
        print ("| x=%.10f"%x)


x0 = 0.0
x1 = 0.01
print ("Buscando t1:")
secantes(x0,x1)

print ("Buscando t2")
x0 = 0.0
x1 = 0.02
secantes(x0,x1)

print ("Buscando t3")
x0 = 0.0
x1 = 0.03
secantes(x0,x1)
```




```
Buscando t1:
| x=0.0053066982
| x=0.0040857456
| x=0.0046245866
| x=0.0046151446
| x=0.0046150592
Buscando t2
| x=0.0207432848
| x=0.0188853104
| x=0.0195069191
| x=0.0193390109
| x=0.0193208301
| x=0.0193216099
Buscando t3
| x=0.0220852171
| x=0.0232817083
| x=0.0243766968
| x=0.0239041166
| x=0.0239452287
| x=0.0239476194
```

**Comentário** Como pode ser visto no gráfico, a função possui 3 raízes, que correspondem aos tempos $t_1$, $t_2$ e $t_3$ em que a tensão atinge o valor de 10V. Assim, precisamos executar o método das secantes 3 vezes, com aproximações iniciais próximas a 0.01, 0.02 e 0.03 para obter as raízes.

**Problema 4:** Em Termodinâmica sob determinadas condições a relação entre o calor $Q$ fornecido a um gás e sua variação de temperatura $T_f - T_i$ é dada por $$Q = nR\left[A(T_f - T_i) + \frac{B}{2}(T_f^2 - T_i^2) + \frac{C}{3}(T_f^3 - T_i^3)\right]$$. $Para o gás metano$ R=8,314 J/mol.K,A=1,702,B=9,081 \times 10^{-3}K^{-1},C=-2,164 \times 10^{-6} K^{-2}. $Em uma amostra de tem - se$ n=2 mol $de metano a temperatura$ T_i = 300 K $qual será a temperatura final$ T_f $se$ 20 kJ de energia é absorvido pelo gás?

*Resolução:*

```
R = 8.314
A = 1.702
B = 0.009081
C = -2.164 * (10**(-6))
n = 2.0
T_i = 300.0
Q = 20000.0

f = lambda x: n*R*((A*(x - T_i))+((B/2)*(x*x - T_i * T_i)+((C/3)*(x*x*x - T_i*T_i*T_
↪i))))-Q

erro = 10.0
a = 500
b = 600
x_m = 1.0
x_ant = 1.0
i = 0

while erro > 0.001:
    x_m =((a+b)/2.0)
    if (f(a)*f(x_m)<0):
        a = a
        b = x_m
    else:
        a = x_m
        b = b
    print ( i, '| x =', x_m)
    erro = abs(x_m - x_ant)/abs(x_m)
    x_ant = x_m
    i = i+1
```

```
0 | x = 550.0
1 | x = 525.0
2 | x = 537.5
3 | x = 531.25
4 | x = 534.375
5 | x = 535.9375
6 | x = 535.15625
7 | x = 535.546875
```

**Comentário** Usando o método da bisseção, com a=500 e b=600 precisamos de 7 iteraçõespara chegar na solução com erro menor que 0,001.

**Problema 5:**

A resisitividade $\rho$ do silício dopado é baseada na carga $q$ de um elétron, na densidade eletrônica $n$ e na mobilidade de elétron $\mu$. A densidade eletrônica é dada em termos da densidade da dopagem $N$ e da densidade de transporte intríseca $n_i$. A mobilidade do elétron é descrita pela temperatura $T$, pela temperatura de referência $T_0$ e pela mobilidade de referência

$\mu_0$. As equações necessárias para o cálculo da resistividade são $\$\rho = \frac{1}{qn\mu}\$$ onde

$$n = \frac{1}{2}\left(N + \sqrt{N^2 + 4n_i^2}\right)$$

e

$$\mu = \mu_0 \left(\frac{T}{T_0}\right)^{-2,42}$$

Determine $N$, dado que $T_0 = 300K, T = 1000K, \mu_0 = 1300cm^2(Vs)^{-1}, q = 1,6 \times 10^{-19}C, n_i = 6,21 \times 10^9 cm^{-3}$ e uma resistividade desejada de $\rho = 6 \times 10^6\ V\ s\ cm/C$. Use o Método das Secantes e outro de sua escolha e compare os resultados.

*Resolução:*

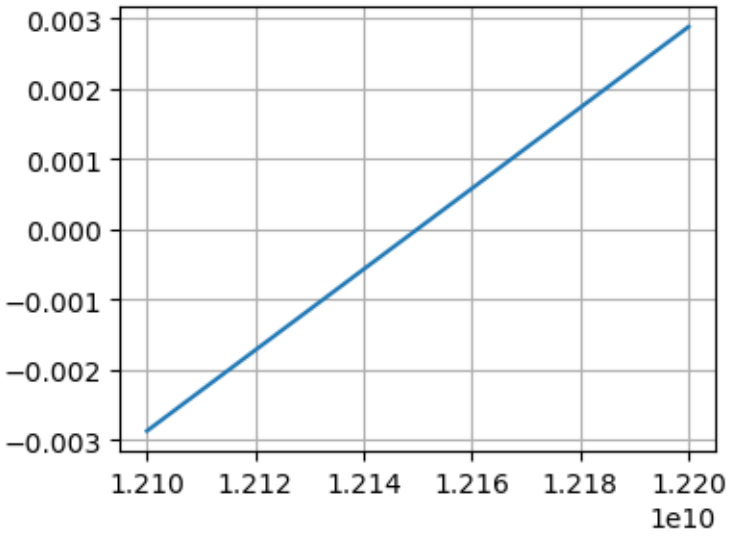
```
T0 = 300
T = 1000
mu0 = 1300
q = 1.6*10**(-19)
ni = 6.21*10**9
rho=6*10**6
mu = mu0*(T/T0)**(-2.42)

f = lambda N: (rho*q*mu*0.5*(N + np.sqrt(N**2+4*ni**2)) - 1)

N = np.arange(1.21*10**10, 1.22*10**10, 100)

plt.figure(figsize=(4,3))
plt.plot(N, f(N))
plt.grid()
plt.show()

a = 1.214*10**10
b = 1.216*10**10
x_ant = a
erro = 1000.
i=0
while erro > 0.0000001:
    x_m =((a+b)/2.0)
    if (f(a)*f(x_m)<0):
        a = a
        b = x_m
    else:
        a = x_m
        b = b
    print ( i, '| x =', x_m)
    erro = abs(x_m - x_ant)/abs(x_m)
    x_ant = x_m
    i = i+1
```




```
0 | x = 12150000000.0
1 | x = 12145000000.0
2 | x = 12147500000.0
3 | x = 12148750000.0
4 | x = 12149375000.0
5 | x = 12149687500.0
6 | x = 12149843750.0
7 | x = 12149921875.0
8 | x = 12149882812.5
9 | x = 12149863281.25
10 | x = 12149873046.875
11 | x = 12149877929.6875
12 | x = 12149880371.09375
13 | x = 12149881591.796875
14 | x = 12149880981.445312
```

```
T0 = 300
T = 1000
mu0 = 1300
q = 1.6*10**(-19)
ni = 6.21*10**9
rho=6*10**6
mu = mu0*(T/T0)**(-2.42)

f = lambda N: (rho*q*mu*0.5*(N + np.sqrt(N**2+4*ni**2)) - 1)
err = 10.0
x0 = 1.2*10**10
x1 = 1.3*10**10

while err> 0.000001:
```

(continua na próxima página)

(continuação da página anterior)

```
    x = (x0*f(x1)-x1*f(x0))/(f(x1)-f(x0))
    err = abs(x-x1)/abs(x)    # calcula o erro relativo
    x0 = x1
    x1 = x
    print ("| x=%.9f"%x)
```

```
| x=12148815088.399784088
| x=12149873683.437911987
| x=12149881211.855869293
```

**Comentário** Esse é um problema que possui uma escala atípica, mas como método de Newton chega-se a solução com apenas 3 iterações.

**Problema 6:** De acordo com o *princípio de Arquimedes*, a força de *flutuação* é igual ao peso do fluido deslocado pela parte submersa de um objeto. Para o tronco de cone mostrado abaixo, use o método da bisseção e mais algum método de sua escolha para determinar a altura $h_1$ da parte que está acima da água. Use os seguintes valores para seus cálculos: $r_1 = 0,5m$, $r_2 = 1m$, $h = 1m$, $\rho_f = densidadedotronco = 200kg/m^3$ e $\rho_w = densidadedagua = 1000kg/m^3$. O volume do trondo de cone é dado por: $$V = \frac{\pi h}{3}(r_1^2 + r_2^2 + r_1 r_2)$$

*Resolução:* A equação a ser resolvida é

$$200\frac{\pi 1.0}{3}(0.5^2 + 1.0^2 + 1.0 \times 0.5) = 1000\frac{\pi(1-h_1)}{3}((0.5 + h_1/2)^2 + 1.0^2 + 1.0 \times (0.5 + h_1/2))$$

```
import math
a = 0.1
b = 1.0
err = 10
x_ant = 0.0

f = lambda h: 1.75-5*(1-h)*(1.0+(0.5+h/2.)**2+(0.5+h/2.))

while err > 0.001:
    x = (a+b)/2.0
    if f(a)*f(x)<0:
        b = x
    else:
        a = x
    err = abs(x - x_ant)/abs(x)
    x_ant = x
    print ("| h1=%.10f"%x)
```

```
| h1=0.5500000000
| h1=0.7750000000
| h1=0.8875000000
| h1=0.8312500000
| h1=0.8593750000
```

(continua na próxima página)

(continuação da página anterior)

```
| h1=0.8734375000
| h1=0.8804687500
| h1=0.8769531250
| h1=0.8751953125
| h1=0.8760742188
| h1=0.8756347656
```

```
err = 1.0
x0 = 0.5
x1 = 1.0

f = lambda h: 1.75-5*(1-h)*(1.0+(0.5+h/2.)**2+(0.5+h/2.))

while err>0.001:
    x = (x0*f(x1)-x1*f(x0))/(f(x1)-f(x0))
    err = abs (x-x1)/abs(x)

    x0 = x1
    x1 = x

    print ("| h1=%.10f"%x)
```

```
| h1=0.8486486486
| h1=0.8740418082
| h1=0.8758028171
| h1=0.8757774319
```

**Problema 7:** Lei dos gases ideais e não ideais (engenharia química)

A lei dos gases ideais é dada por

$$pV = nRT$$

Essa equação é mais apropriada para alguns gases do que outros. Além disso, é acurada apenas para um intervalo limitado de pressão e temperatura.

Uma equação de estado alternativa, conhecida como *equação de Van der Waals* é dada por

$$\left(p + \frac{a}{v^2}\right)(v - b) = RT$$

onde $v = V/n$ é o volume molar, $a$ e $b$ são constantes empíricas que dependem do gás.

Considere o problema de se fazer uma estimattiva acurada do volume $v$ do dióxido de carbono e do oxigênio, para diversas combinações de temperatura e pressão. Também deseja-se examidar o quão precisamente esses gases obedecem à lei dos gases ideais.

São fornecidos os seguntes dados:

$R = 0,082054 L\,atm/(mol\,K)$

Dióxido de carbono: $a = 3,592$ e $b = 0,04267$

Oxigênio: $a = 1,360$ e $b = 0,03183$

As pressões de interesse são 1, 10 e 100 atm , e as temperaturas de 300, 500 e 700.

*Solução:*

Considerando a lei dos gases ideais tem-se

```
press = [1, 10,100]
temp = [300, 500,700]


def calcula_vol(p,T):
    v = 0.082054*float(T)/float(p)
    print ("Para T=",T, "e p=",p, "o volume é v=",v,"L/mol")

for T in temp:
    for p in press:
        calcula_vol(p, T)
```

```
Para T= 300 e p= 1 o volume é v= 24.6162 L/mol
Para T= 300 e p= 10 o volume é v= 2.46162 L/mol
Para T= 300 e p= 100 o volume é v= 0.246162 L/mol
Para T= 500 e p= 1 o volume é v= 41.027 L/mol
Para T= 500 e p= 10 o volume é v= 4.1027000000000005 L/mol
Para T= 500 e p= 100 o volume é v= 0.41027 L/mol
Para T= 700 e p= 1 o volume é v= 57.4378 L/mol
Para T= 700 e p= 10 o volume é v= 5.74378 L/mol
Para T= 700 e p= 100 o volume é v= 0.574378 L/mol
```

Considerando a *equação de Van der Waals*, tem-se, pelo método da bissecao

```
# bisseção
import math

p = 1
T = 300

f = lambda v: (p + 3.592/v**2)*(v-0.04267)-0.082054*T      #dioxido
#f = lambda v: (p + 1.360/v**2)*(v-0.03183)-0.082054*T     #oxigenio

a = 24.
b = 25.

v_ant = 0.0
err = 10.

while err > 0.0001:
    v = (a+b)/2.0
    if f(v)*f(a) < 0:
```

(continua na próxima página)

(continuação da página anterior)

```
        b = v
    else:
        a = v
        err = abs(v - v_ant)/abs(v)
        v_ant = v
    print ( v )
```

```
24.5
24.75
24.625
24.5625
24.53125
24.515625
24.5078125
24.51171875
24.513671875
24.5126953125
24.51220703125
```

Pelo método de Newton-Raphson

```
p = 1
T = 300

a = 3.592
b = 0.04267

#a = 1.360
#b = 0.03183

f = lambda v: (p + a/v**2)*(v-b)-0.082054*T
flinha = lambda v: p - a/v**2 + 2*a*b/v**3

x = 30.0
x_ant = x
eps = 0.0001
err = 10.0
i = 0

while err>eps:
    x = x - f(x)/flinha(x)
    err = abs(x-x_ant)/abs(x)
    x_ant = x
    i=i+1
    print (i, "| x=%.9f"%x, "| err=%.9f"%err)
```

```
1 | x=24.517487897 | err=0.223616389
2 | x=24.512588134 | err=0.000199888
3 | x=24.512588128 | err=0.000000000
```

Método da bissecao para todos os casos

```
# bisseção
import math
v=0.0
x_ant = 0.0

d = lambda v, p, T: (p + 3.592/v**2)*(v-0.04267)-0.082054*T
o = lambda v, p, T: (p + 1.360/v**2)*(v-0.03183)-0.082054*T

def bissecao(eps, a, b, p, T, f):
    v_ant = 0.0
    err = 10.
    while err > eps:
        v = (a+b)/2.0
        if f(v,p,T)*f(a,p,T) < 0:
            b = v
        else:
            a = v

        err = abs(v - v_ant)/abs(v)
        v_ant = v
    return v


print ("Dióxido de Carbono")
for T in temp:
    for p in press:
        v = bissecao(0.00001, 0.01, 100, p, T, d)
        print ("Para T=",T, "e p=",p, "o volume é v=",v,"L/mol")

print ("Oxigênio")
for T in temp:
    for p in press:
        v = bissecao(0.00001, 0.01, 100, p, T, o)
        print ("Para T=",T, "e p=",p, "o volume é v=",v,"L/mol")
```

```
Dióxido de Carbono
Para T= 300 e p= 1 o volume é v= 24.51259252548218 L/mol
Para T= 300 e p= 10 o volume é v= 2.354504963159561 L/mol
Para T= 300 e p= 100 o volume é v= 0.07951069049537182 L/mol
Para T= 500 e p= 1 o volume é v= 40.982235984802244 L/mol
```

(continua na próxima página)

(continuação da página anterior)

```
Para T= 500 e p= 10 o volume é v= 4.057775762081146 L/mol
Para T= 500 e p= 100 o volume é v= 0.36630179435014726 L/mol
Para T= 700 e p= 1 o volume é v= 57.418122749328596 L/mol
Para T= 700 e p= 10 o volume é v= 5.724178814888001 L/mol
Para T= 700 e p= 100 o volume é v= 0.5575539723038672 L/mol
Oxigênio
Para T= 300 e p= 1 o volume é v= 24.59269315719605 L/mol
Para T= 300 e p= 10 o volume é v= 2.4384079909324647 L/mol
Para T= 300 e p= 100 o volume é v= 0.2263596136868 L/mol
Para T= 500 e p= 1 o volume é v= 41.02571918487549 L/mol
Para T= 500 e p= 10 o volume é v= 4.1016403937339785 L/mol
Para T= 500 e p= 100 o volume é v= 0.4116146740317345 L/mol
Para T= 700 e p= 1 o volume é v= 57.446348686218265 L/mol
Para T= 700 e p= 10 o volume é v= 5.752118678092956 L/mol
Para T= 700 e p= 100 o volume é v= 0.5841945841908454 L/mol
```

| Temperatura, K | Pressão, atm | Volume molar (lei dos gases ideais), L/mol | Volume molar (van der Waals) dióxido de carbono, L/mol | Volume molar (van der Waals) oxigênio, L/mol |
|---|---|---|---|---|
| 300 | 1 | 24,6162 | 24,5126 | 24,5928 |
| | 10 | 2,4616 | 2,3545 | 2,4384 |
| | 100 | 0,2462 | 0,0795 | 0,2264 |
| 500 | 1 | 41,0270 | 40,9821 | 41,0259 |
| | 10 | 4,1027 | 4,0578 | 4,1016 |
| | 100 | 0,4103 | 0,3663 | 0,4116 |
| 700 | 1 | 57,4378 | 57,4179 | 57,4460 |
| | 10 | 5,7438 | 5,7242 | 5,7521 |
| | 100 | 0,5744 | 0,5575 | 0,5842 |

# 3. Sistemas Lineares

Sistemas de equações lineares surgem naturalmente na modelagem de problemas físicos, estruturais e computacionais. A resolução eficiente de sistemas $Ax = b$ é uma das tarefas centrais da computação científica.

### Objetivos de aprendizagem

- Representar sistemas lineares na forma matricial
- Aplicar eliminação gaussiana e escalonamento
- Utilizar decomposição LU para resolver múltiplos sistemas com a mesma matriz
- Calcular matrizes inversas pelo método de Gauss-Jordan
- Aplicar métodos iterativos de Jacobi e Gauss-Seidel

### Conteúdo


| Seção | Tema |
|---|---|
| 3.1 | Forma matricial e escalonamento |
| 3.2 | Decomposição LU |
| 3.3 | Gauss-Jordan e matriz inversa |
| 3.4 | Métodos iterativos |


## 3.1 Forma matricial

A resolução de sistema lineares é provavelmente o processo numérico mais utilizado para simular situações do mundo real. É uma etapa fundamental na busca de soluções de problemas que envolvam, por exemplo, equações diferenciais parciais, otimização, regressão linear e não linear e conjuntos de equações não lineares. Geralmente, é a fase que demanda a maior parte do tempo de processamento para resolver o problema. Portanto, é de extrema importância que se tenha uma implementação eficiente do método.

Nas seções seguintes, serão abordados os principais métodos diretos e alguns métodos iterativos usados na para a resolução de sistemas de equaçãoes algébricas lineares (ou simplesmente, sistemas lineares).

Considere a forma geral de um sistema linear de $n$ equações e $n$ incógnias

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

O qual pode ser escrito em notação compacta, na seguinte forma

$$\sum_{j=1}^{n} a_{ij}x_j = b_i, \quad i = 1, 2, \ldots, n$$

com $a_{ij}$ e $b_i$ sendo coeficientes reais conhecidos e $x_i$ as incógnitas.

Este mesmo sistema pode também ser escrito na forma de uma equação matricial

$$Ax = b$$

em que $A = [a_{ij}]$ com $i, j = 1, \ldots, n$, é a matriz dos coeficientes, $x = [x_1, x_2, \ldots, x_n]^t$ é o vetor de incógintas e $b = [b_1, b_2, \ldots, b_n]^t$ é o vetor de termos independentes de $x$. Então, tem-se

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Resolver o sistema dado, consiste em encontrar o vetor solução $\overline{x} = [\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_n]^t$ que satisfaça todas as equações simultaneamente. Uma condição suficiente para que este sistema possua solução única é que $\det(A) \neq 0$.

Como um primeiro exemplo vamos criar e realizar algumas operações básicas com vetores e matrizes usando a biblioteca `numpy` em Python. Para isso vamos utilizar a estrutura `array`, que é usada tanto para representar vetores quanto matrizes. Um vetor é representado por um `array` unidimensional e uma matriz por um `array` bidimensional. Para acessar os elementos em um `array` usamos índices (iniciando em zero) entre colcetes. Vejamos um exemplo.

```
import numpy as np
np.set_printoptions(precision = 4)
```

**Exemplo 3.1.1:** Dada a matriz $M = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 5 & 0 \\ 7 & -2 & 4 \end{bmatrix}$ e o vetor $v = \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix}$

vamos escrevê-los como `array`'s e fazer as seguintes operações:

a) somar o 1º elemento de $v$ com o 2º elemento da terceira linha de $M$

b) escrever a 3ª linha de $M$ somada à 1ª multiplicada por $-7$

c) multiplicar $M$ por $v$

d) obter os produtos escalar e vetorial de $v$ por $u = [5, -1, 4]^T$

e) calcular o módulo do vetor $v$

*Solução:* Para entrar com a matriz $M$ e o vetor $v$, primeiramente importamos a biblioteca `numpy` e então criamos os `array`'s. O produto entre matrizes pode ser obtido usando o operador `@` ou as funções `matmul` ou `dot` (veja a

documentação). O produto vetorial é feito usando a função `cross` (veja a documentação).

```
M = np.array([[1,2,3],[-1,5,0],[7,-2,4]])
v = np.array([3,-1,0])
```

```
# solução letra a)
print('a)', v[0] + M[1,2])

# solução letra b)
print('b)',M[2] - 7*M[0])

# solução letra c)
print('c)',np.dot(M,v))

# solução letra d)
print('d)',np.dot(v,[5,-1,4]))
print('  ',np.cross(v,[5,-1,4]))

# solução letra e)
print('e)',np.sqrt(v.dot(v)))
```

```
a) 3
b) [  0 -16 -17]
c) [ 1 -8 23]
d) 16
   [ -4 -12   2]
e) 3.1622776601683795
```

**Atividade 3.1.1:** Dada a matriz $N = \begin{bmatrix} 4 & -2 & 1 \\ 0 & 3 & 5 \\ -1 & 6 & 2 \end{bmatrix}$ e o vetor $w = \begin{bmatrix} -2 \\ 4 \\ 1 \end{bmatrix}$ vamos escrevê-los como `array`'s e realizar as seguintes operações:

a) Somar o 2º elemento de $w$ com o 3º elemento da primeira linha de $N$.

b) Escrever a 2ª linha de $N$ somada à 3ª multiplicada por 3.

c) Multiplicar $N$ por $w$.

d) Obter os produtos escalar e vetorial de $w$ por $z = [3, 2, -1]^T$.

e) Calcular o módulo do vetor $w$.

Observe que o sistema $Nx = w$, em que $N$ e $w$ são a matriz e o vetor da atividade 3.1.1, possui solução única pois $\det(N) \neq 0$.

```
N = np.array([[4,-2,1],[0,3,5],[-1,6,2]])
np.linalg.det(N)
```

```
-83.00000000000003
```

Além disso, observe que o vetor $x = [-0.90361446, -0.31325301, 0.98795181]^t$ é uma solução aproximada do sistema $Nx = w$. Podemos vertificar multiplicando $N$ por $x$ e obter que o resultado é $w$ (salvo erro de aproximação).

```
w = np.array([-2,4,1])
x = np.array([-0.90361446, -0.31325301,  0.98795181])
```

```
N.dot(x)
```

```
array([-2.,  4.,  1.])
```

```
np.round(N.dot(x),7)
```

```
array([-2.,  4.,  1.])
```

## Interpretação gráfica

No caso de um sistema $2 \times 2$, graficamente, em $R^2$, a solução representa a intersecção das retas definidas pelas equações que compõe o sistema. Vejamos os exemplos a seguir.

**Exemplo 3.1.2:** Os sistemas lineares

(a) $\begin{cases} -x_1 + 2x_2 = 3 \\ x_1 + x_2 = 3 \end{cases}$, (b) $\begin{cases} x_1 + x_2 = 1 \\ 2x_1 + 2x_2 = 2 \end{cases}$ e (c) $\begin{cases} x_1 + x_2 = 1 \\ 2x_1 + 2x_2 = 4 \end{cases}$

são tais que: (a) tem como única solução $\bar{x} = (1, 2)^t$, como pode ser visto no gráfico abaixo; (b) tem infinitas soluções já que as duas equações representam a retas coincidentes; e (c) não tem nenhuma solução (sistema inconsistente) já que as duas equações representam a retas paralelas.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-1, 3, 2)

fig, axs = plt.subplots(1, 3, figsize=(12, 3))

# Gráfico 1
axs[0].plot(x, -3 + 2*x, "b-")
axs[0].plot(x, 3 - x, "r-")
axs[0].grid(True)
axs[0].set_title("Sistema (a)")

# Gráfico 2
axs[1].plot(x, 1 - x, "b-", linewidth=2.0)
axs[1].plot(x, (2 - 2*x)/2, "r-", linewidth=1.0)
axs[1].grid(True)
axs[1].set_title("Sistema (b)")

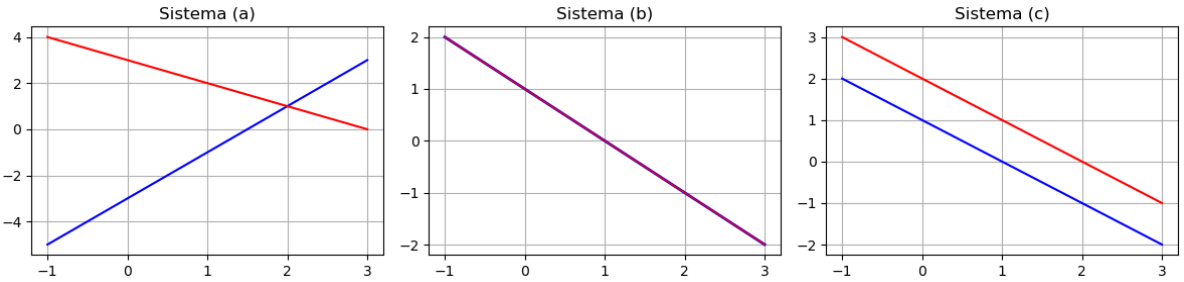
# Gráfico 3
axs[2].plot(x, 1 - x, "b-")
axs[2].plot(x, (4 - 2*x)/2, "r-")
```

(continua na próxima página)

(continuação da página anterior)

```
axs[2].grid(True)
axs[2].set_title("Sistema (c)")

plt.tight_layout()
plt.show()
```




Para sistema com três incógnitas, as equações envolvidas são geometricamente interpretadas como planos em $\mathbb{R}^3$. A figura abaixo mostra quatro possibilidades de posições relativas entre três planos distintos.

**Condicionamento de matrizes**

Sabemos que um sistema linear $n \times n$, $Ax = b$ tem solução única se $\det(A) \neq 0$, caso $\det(A) = 0$ dizemos que a matriz $A$ é singular e o sistema poderá ter infinitas soluções ou nenhuma solução. Mas o que acontece se a matriz é "quase singular" ($\det(A)$ é muito pequeno)?

Dizemos que $\det(A) = |A|$ é muito pequeno se $|A| << ||A||$ onde $||A||$ é a **norma** da matriz $A$. A norma é um número real usado para expressar a magnitude de um vetor ou de uma matriz.

## Norma de vetores e matrizes

No caso de um vetor $x$ de tamanho $n$, as normas são definidas em termos da *norma−p*

$$\|x\|_p = \left( \sum_{i=1}^{n} |x_i|^p \right)^{1/p}, \quad 1 \leq p < \infty$$

A partir dessa equação, definimos três normas vetoriais importantes:

| Norma soma de magnitudes | Norma Euclidiana | Norma de máxima magnitude |
|---|---|---|
| $\|x\|_1 = \sum_{i=1}^{n} \|x_i\|$ | $\|x\|_2 = \sqrt{\sum_{i=1}^{n} \|x_i\|^2}$ | $\|x\|_\infty = \max_{1 \leq i \leq n} \|x_i\|$ |

Normas de matrizes podem ser definidas forma semelhante, considerando $\mathrm{A} = \left(a_{ij}\right)_{i,j=1,\ldots,n}$, então temos as seguintes normas de matrizes:

| Norma Euclidiana | Norma linha | Norma coluna |
|---|---|---|
| $\|\mathbf{A}\|_2 = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{n} (a_{ij})^2}$ | $\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^{n} \|a_{ij}\|$ | $\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{n} \|a_{ij}\|$ |

O **número de condicionamento da matriz** é uma medida formal definida como

$$\text{cond}(A) = \|A\|\|A^{-1}\|$$

se esse número estiver próximo de 1, então a matriz é dita *bem condicionada*. Esse número aumenta com o mal condicionamento da matriz.

Infelizmente, o número de condicionamento é computacionalmente oneroso para matrizes grandes. Então, muitas vezes é suficiente estimar o condicionamento comparando o determinante com a magnitude dos elementos da matriz.

Se as equações estiverem mal condicionadas, pequenas mudanças nos coeficientes da matriz resultarão em grandes mudanças na solução.

**Exemplo 3.1.3:** Consideremos o sistema linear

$$\begin{cases} 2x_1 + x_2 = 3 \\ 2x_1 + 1.001x_2 = 0 \end{cases}$$

Cuja solução é dada por:

```
import scipy.linalg as la

A = [[2, 1], [2, 1.001]]
b = [3, 0]
print ('x =', la.solve(A,b))
```

```
x = [ 1501.5 -3000. ]
```

A matriz dos coeficientes desse sistema tem um número de condicionamento grande, ou seja, a matriz é mal condicionada

```
print (la.norm(A) * la.norm(la.inv(A)))
```

```
5001.000500000551
```

```
la.norm(A)
```

```
3.162594030222659
```

```
la.det(A)
```

```
0.0019999999999997797
```

Observe o que acontece com a solução do sistema quando variamos ligeiramente um dos coeficientes

```
A = [[2, 1], [2, 1.002]]
print ('x =',la.solve(A,b))
```

```
x = [  751.5 -1500. ]
```

Uma mudança no coeficiente na ordem de 1% gerou uma diferença na solução de 100%!

## Mais operações com matrizes

A seguir vamos ampliar nosso repertório de funções e operadores para criar, manipular e realizar cálculos com vetores e matrizes usando `numpy`. Lembrando que os índices dos *arrays* iniciam em 0, então uma matriz em Python tem uma forma geral como a matriz $A = [a_{ij}]$, $i = 0, \dots, n$ e $j = 0, \dots, m$ representada abaixo.

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0m} \\ a_{10} & a_{11} & \cdots & a_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n0} & a_{n1} & \cdots & a_{nm} \end{bmatrix}$$

Algumas operações úteis incluem tomar uma linha ou uma coluna de uma matriz, multiplicar uma matriz ou vetor por um número real não nulo, somar duas matrizes ou vetores, extrair a diagonal, etc. Vamos ver alguns exemplos:

**Exemplo 3.1.4:** Seja as matrizes

$$A = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 5 & 0 \\ 7 & -2 & 4 \end{bmatrix} \quad \text{e} \quad B = \begin{bmatrix} 0 & -2 & 1 \\ 1 & 0 & 5 \\ 1 & -2 & 2 \end{bmatrix}$$

vamos fazer algumas operações:

```
# definindo uma matriz A
A = np.array([[1,2,3],[-1,5,0],[7,-2,4]])
print(A)
```

```
[[ 1  2  3]
 [-1  5  0]
 [ 7 -2  4]]
```

```
# definindo uma matriz B
B = np.array([[0,-2,1],[1,0,5],[1,-2,2]])
print(B)
```

```
[[ 0 -2  1]
 [ 1  0  5]
 [ 1 -2  2]]
```

```
# somando A e B
A+B
```

```
array([[ 1,  0,  4],
       [ 0,  5,  5],
       [ 8, -4,  6]])
```

```
# multiplicando A por B
A@B
```

```
array([[  5,  -8,  17],
       [  5,   2,  24],
       [  2, -22,   5]])
```

```
# ou
A.dot(B)
```

```
array([[  5,  -8,  17],
       [  5,   2,  24],
       [  2, -22,   5]])
```

É fácil extrair uma linha da matriz, sua diagonal, obter sua transposta ou uma combinação linear. Veja os exemplos a seguir:

```
# pegando a segunda linha de A
A[1]
```

```
array([-1,  5,  0])
```

```
# pegando a diagonal de A
np.diag(A)
```

```
array([1, 5, 4])
```

```
# transpondo
A.T
```

```
array([[ 1, -1,  7],
       [ 2,  5, -2],
       [ 3,  0,  4]])
```

```
# combinação linear
2*A-3*B
```

```
array([[  2,  10,   3],
       [ -5,  10, -15],
```

(continua na próxima página)

(continuação da página anterior)

```
       [ 11,   2,   2]])
```

```
# combinação linear usando linhas da matriz A
2*A[0]-3*A[1]
```

```
array([  5, -11,   6])
```

## Sistemas triangulares e escalonamento de matrizes

Muitas vezes é útil escalonar a matriz ampliada de um sistema linear para obter um sistema triangular equivalente que seja mais fácil de resolver. Sistemas triagulares, em geral podem ser facilmente resolvidos por substituições sucessivas.

Um **sistema triangular inferior** representado por $Lx = b$, possui matriz de coeficientes tal que $[l_{ij}] = 0$ para $i < j$, com $i, j = 0, 1, ..., n$.

Um algoritmo para obter a sua solução é dado por:

$$x_1 = b_1/a_{11}$$

para $i = 2, 3 \ldots, n$, faça

$$x_i = \frac{b_i - \sum_{j=1}^{(i-1)} a_{ij}x_j}{a_{ii}}$$

Usando Python, um código que implementa esse algoritmo, é mostrado no exemplo abaixo.

**Exemplo 3.1.5:**

Seja o sistema linear

$$\begin{cases} 2x_1 = 2 \\ x_1 + 4x_2 = -3 \\ x_1 + x_2 + x_3 = 0 \end{cases}$$

um código em Python que implementa esse algoritmo e a solução obtida executando o programa são mostrados a seguir

```
A = np.array([[2,0,0],
              [1.,4.,0],
              [1.,1.,1.]])

b = np.array([2.,-3., 0])

M = np.hstack((A,b.reshape(3,1)))
M
```

```
array([[ 2.,  0.,  0.,  2.],
       [ 1.,  4.,  0., -3.],
       [ 1.,  1.,  1.,  0.]])
```

```
x1 = b[0]/A[0][0]
x = [x1]

for i in range(1,len(A)):
    soma = 0
    for j in range(0,i):
        soma +=  A[i][j]*x[j]
    x.append((b[i]-soma)/A[i][i])
print (x)
```

```
[1.0, -1.0, 0.0]
```

```
A@x
```

```
array([ 2., -3.,  0.])
```

Uma versão mais elegante do algoritmo para resolução de um sistema triangular inferior pode ser escrito como é mostrado abaixo

```
for i in range(len(b)):
    b[i] = (b[i] - np.dot(A[i,0:i],b[0:i]))/A[i][i]
print (b)
```

```
[ 1. -1.  0.]
```

Um **sistema triangular superior** representado por $Ux = b$, possui matriz de coeficientes tal que $[u_{ij}] = 0$ para $i > j$, com $i, j = 0, 1, \ldots, n$.

Um algoritmo para obter a sua solução é dado por:

$$x_n = b_n / a_{nn}$$

para $i = (n-1), (n-2), \ldots, 1$, faça

$$x_i = \frac{b_i - \sum_{j=i+1}^{(n)} a_{ij} x_j}{a_{ii}}$$

Usando Python, um código que implementa esse algoritmo, é mostrado abaixo.

```
def solve_U(U,b):
    n=len(b)
    x = np.empty(n)
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - np.sum(U[i,i+1:n+1]*x[i+1:n+1]))/U[i,i]
    return x
```

**Exemplo 3.1.6:** Seja o sistema linear

$$\begin{cases} 3x_1 + x_2 + x_3 = 4 \\ 2x_2 - x_3 = 2 \\ 3x_3 = 3 \end{cases}$$

um código em Python que implementa esse algoritmo e a solução obtida executando o programa são mostrados a seguir

```
A = np.array([[3,1,1],
              [0,2,-1],
              [0,0,3]]).astype(float)

b = np.array([4,2,3]).astype(float)
```

```
solve_U(A,b)
```

```
array([0.5, 1.5, 1. ])
```

```
x = np.linalg.solve(A,b)
x
```

```
array([0.5, 1.5, 1. ])
```

```
A@x
```

```
array([4., 2., 3.])
```

## Escalonamento de matrizes e resolução por eliminação de Gauss

O método de eliminação de Gauss para resolução de sistemas lineares consiste no processo de transformar um sistema $Ax = b$ em um sistema triangular equivalente, aplicando operações elementares sobre as linhas da matriz aumentada $[A \mid b]$. O sistema equivalente obtido ao final do processo, deve ser de resolução mais simples ou imediata, por substituição subsequente.

O processo de eliminação de Gauss com pivoteamento diagonal consiste em tomar, a cada passo, como pivô, os elementos da diadonal de $A$, desde que não seja nulo. Caso o algum elemento da diagonal seja nulo podemos permutar duas linhas da matriz a fim obter um pivô diferente de zero.

Para cada passo $k$ do processo, anulamos os termos abaixo do elemento da diagonal $a_{kk}$, subtraindo cada linha $i = k + 1$ da matriz $[A \mid b]$, pela linha linha anterior, $i = k$, multiplicada pelo fator $m_{ik} = \frac{a_{ik}}{a_{kk}}$.

Assim, no 1º passo ($k = 1$), fazemos $m_{i1} = \frac{a_{i1}}{a_{k1}}$, $i = 2, \ldots, n$ e cada linha de $[A \mid b]$, a partir da 2ª, será subtraída

da 1ª linha multiplicada por $m_{k1}$ obtendo-se a matriz $[A^{(1)} \mid b^{(1)}]$ tal que

$$= \left[\begin{array}{ccccc|c} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & \cdots & a_{3n}^{(2)} & b_3^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & a_{n2}^{(2)} & a_{n3}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{array}\right]$$

No 2º passo ($k = 2$), fazemos $m_{i2} = \frac{a_{i2}}{a_{k2}}$, $i = 3, \ldots, n$ e cada linha de $[A^{(1)} \mid b^{(1)}]$, a partir da 3ª, será subtraída da 2ª linha multiplicada por $m_{k2}$ obtendo-se a matriz $[A^{(2)} \mid b^{(2)}]$ tal que

$$= \left[\begin{array}{ccccc|c} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ 0 & 0 & a_{23}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ 0 & 0 & a_{33}^{(3)} & \cdots & a_{3n}^{(3)} & b_3^{(3)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & a_{n3}^{(3)} & \cdots & a_{nn}^{(3)} & b_n^{(3)} \end{array}\right]$$

Segundo os passos seguintes, até o passo $n - 1$, obtemos ou sistema triangular superior na forma

$$\begin{cases} a_{11}^{(1)}x_1 + a_{12}^{(1)}x_2 + a_{13}^{(1)}x_2 + \cdots + a_{1n}^{(1)}x_n = b_1^{(1)} \\ \qquad\qquad a_{22}^{(2)}x_2 + a_{23}^{(2)}x_2 + \cdots + a_{2n}^{(2)}x_n = b_2^{(2)} \\ \qquad\qquad\qquad\qquad a_{33}^{(3)}x_2 + \cdots + a_{3n}^{(3)}x_n = b_3^{(3)} \\ \qquad\qquad\qquad\qquad\qquad\qquad \vdots \\ \qquad\qquad\qquad\qquad\qquad\qquad a_{nn}^{(n)}x_n = b_n^{(n)} \end{cases}$$

O qual pode ser resolvido por substituição retroativa.

**Exemplo 3.1.7:** Usando o método de eliminação de Gauss, resolva o sistema de equações lineares:

$$\left[\begin{array}{ccc} 3 & 0 & 1 \\ 3 & 2 & 1 \\ -3 & 1 & 3 \end{array}\right] \left[\begin{array}{c} x_1 \\ x_2 \\ x_3 \end{array}\right] = \left[\begin{array}{c} 1 \\ 1 \\ 3 \end{array}\right]$$

Um exemplo de algoritmo para resolver sistemas lineares por eliminação de Gauss (sem pivoteamento e sem verificar valores nulos na diagonal).

```
# funcao que triangulariza o sistema
def escalona(M):
    n = len(M)
    for j in range(n-1):
        for i in range(j,n-1):
            m = M[i+1,j]/M[j,j]
            M[i+1] = M[i+1]-m*M[j]
    return (M)
```

```
A = np.array([[3.,0.,1.],
              [3.,2.,1.],
              [-3.,1.,3.]])
b = np.array([1.,1.,3.])
```

```
M = np.hstack((A,b.reshape(3,1)))
M
```

```
array([[ 3.,  0.,  1.,  1.],
       [ 3.,  2.,  1.,  1.],
       [-3.,  1.,  3.,  3.]])
```

```
Mesc = escalona(M)
Mesc
```

```
array([[3., 0., 1., 1.],
       [0., 2., 0., 0.],
       [0., 0., 4., 4.]])
```

```
Aesc =  Mesc[:,:-1]
Aesc
```

```
array([[3., 0., 1.],
       [0., 2., 0.],
       [0., 0., 4.]])
```

```
besc = Mesc[:,-1]
besc
```

```
array([1., 0., 4.])
```

```
x = solve_U(Aesc, besc)
x
```

```
array([0., 0., 1.])
```

**Exemplo 3.1.8:** Represente o sistema linear

$$\begin{aligned} x_1 - x_2 + 2x_3 - x_4 &= -8, \\ 2x_1 - 2x_2 + 3x_3 - 3x_4 &= -20, \\ x_1 + x_2 + x_3 &= -2, \\ x_1 - x_2 + 4x_3 + 3x_4 &= 4. \end{aligned}$$

e use a eliminação de Gauss para encontrar sua solução.

```
A = np.array([[1,-1,2,-1],
              [2,-2,3,-3],
              [1,1,1,0],
              [1,-1,4,3]]).astype(float)
b = np.array([-8, -20, -2, 4])

M = np.hstack((A,b.reshape(4,1)))
print (M)
```

```
[[  1.  -1.   2.  -1.  -8.]
 [  2.  -2.   3.  -3. -20.]
 [  1.   1.   1.   0.  -2.]
 [  1.  -1.   4.   3.   4.]]
```

```
Mesc = escalona(M)
```

```
/tmp/ipykernel_22090/3041863753.py:6: RuntimeWarning: divide by zero encountered␣
 ↪in scalar divide
  m = M[i+1,j]/M[j,j]
/tmp/ipykernel_22090/3041863753.py:7: RuntimeWarning: invalid value encountered in␣
 ↪multiply
  M[i+1] = M[i+1]-m*M[j]
/tmp/ipykernel_22090/3041863753.py:6: RuntimeWarning: invalid value encountered in␣
 ↪scalar divide
  m = M[i+1,j]/M[j,j]
```

Observe que aqui ocorreu um problema de divisão por zero. Vamos modificar o código da eliminação de gaus e incluir o pivoteamento. Para isso vamos usar a função `np.argsort` que devolve os índices dos elementos uma coluna ordenados. Vejamos um exemplo:

```
M = np.hstack((A,b.reshape(4,1)))
```

```
# ordem crescente dos elementos da 3ª coluna (em valor absoluto)
idx = np.argsort(abs(M[:,2]))
```

```
# trocando linhas para deixar a 3ª coluna em ordem decrescente
M = M[np.flip(idx)]
M
```

```
array([[  1.,  -1.,   4.,   3.,   4.],
       [  2.,  -2.,   3.,  -3., -20.],
       [  1.,  -1.,   2.,  -1.,  -8.],
       [  1.,   1.,   1.,   0.,  -2.]])
```

Agora vamos usar esse procedimento para incluir o pivoteamento na eliminação de Gauss.

```
M = np.hstack((A,b.reshape(4,1)))
```

```
def escalonap(M):
    n = len(M)
    for j in range(n-1):

        # pivoteamento
        m = M[j:,j:]
        idx=np.flip(np.argsort(abs(m[:,0])))
        M[j:,j:] = m[idx]

        for i in range(j,n-1):
            m = M[i+1,j]/M[j,j]
            M[i+1] = M[i+1]-m*M[j]

    return (M)
```

```
Mesc = escalonap(M)
Mesc
```

```
array([[  2. ,  -2. ,   3. ,  -3. , -20. ],
       [  0. ,   2. ,  -0.5,   1.5,   8. ],
       [  0. ,   0. ,   2.5,   4.5,  14. ],
       [  0. ,   0. ,   0. ,  -0.4,  -0.8]])
```

```
x = solve_U(Mesc[:,:-1], Mesc[:,-1])
x
```

```
array([-7.,  3.,  2.,  2.])
```

```
# checando se a solução está correta
np.allclose(np.dot(A, x), b)
```

```
True
```

Observe que, em um sistema

$$Ax = b,$$

a matriz de coeficientes $A$ é **quadrada** se e somente se **o número de equações do sistema é igual ao número de incógnitas**.

Algumas matrizes quadradas são **invertíveis**. Isto é, se $A$ é uma matriz quadrada $n \times n$, pode existir outra matriz de mesmo ordem, denotada por $A^{-1}$, tal que

$$AA^{-1} = A^{-1}A = I,$$

onde $I$ é a matriz identidade de ordem $n$.

No caso específico em que $A$ é quadrada, vale que o sistema $Ax = b$ tem **solução única** se e somente se $A$ **é invertível**.

Isso vale pois nesses casos podemos isolar o vetor $x$ multiplicando ambos os lados da equação matricial pela inversa $A^{-1}$:

$$Ax = b \quad \Longrightarrow \quad A^{-1}Ax = A^{-1}b \quad \Longrightarrow \quad x = A^{-1}b.$$

Ou seja, nesse caso, $x$ fica unicamente definido por $x = A^{-1}b$.

## Exercícios

---

**1.** Use a eliminação de Gauss para resolver os seguintes sistemas lineares. Não reordene as equações. (A solução exata para cada sistema é $(x_1 = 1, x_2 = -1, x_3 = 3)$.

$$\begin{aligned} 4x_1 - x_2 + x_3 &= 8, \\ 2x_1 + 5x_2 + 2x_3 &= 3, \\ x_1 + 2x_2 + 4x_3 &= 11. \end{aligned} \quad \text{e} \quad \begin{aligned} 4x_1 + x_2 + 2x_3 &= 9, \\ 2x_1 + 4x_2 - x_3 &= -5, \\ x_1 + x_2 - 3x_3 &= -9. \end{aligned}$$

---

**2.** Dada a matriz $M = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 5 & 0 \\ 7 & -2 & 4 \end{bmatrix}$ e o vetor $v = \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix}$

defina os *arrays* para a matriz e o vetor e faça as seguintes operações:

a) somar o 1º elemento de $v$ com o 2º elemento da terceira linha de $M$

b) escrever a 3ª linha de $M$ somada à 1ª multiplicada por $-7$

c) multiplicar $M$ por $v$

d) obter os produtos escalar e vetorial de $v$ por $u = [5, -1, 4]^T$

e) calcular o módulo (norma Euclidiana) do vetor $v$

f) escalonar a matriz $M$

---

**3.** Considere o sistema linear

$$\begin{cases} x_1 + 3x_2 - 2x_3 = -22 \\ 2x_1 - x_2 + 3x_3 = -9 \\ -2x_2 + 4x_3 = -2 \end{cases}.$$

(a) Defina a matriz de coeficientes $A$ e o vetor de termos independentes $b$ como *arrays* e verifique que $x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -1 \\ -11 \\ -6 \end{bmatrix}$ é uma solução do sistema.

(b) Verifique que $A^{-1} = \begin{bmatrix} -\frac{1}{7} & \frac{4}{7} & -\frac{1}{2} \\ \frac{4}{7} & -\frac{2}{7} & \frac{1}{2} \\ \frac{2}{7} & -\frac{1}{7} & \frac{1}{2} \end{bmatrix}$.

**4.** Dado o sistema

$$\begin{cases} -2x_1 + x_2 + 3x_3 = 2 \\ x_1 - 3x_2 - x_3 = -3 \\ -x_1 - 2x_2 + 2x_3 = -1 \end{cases},$$

mostre que $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ e $\begin{bmatrix} -7 \\ 0 \\ -4 \end{bmatrix}$ pertencem ao conjunto solução.

---

**5.** Verifique se o sistema linear

$$\begin{aligned} x_1 + x_2 - x_3 + x_4 - x_5 &= 2, \\ 2x_1 + 2x_2 + x_3 - x_4 + x_5 &= 4, \\ 3x_1 + x_2 - 3x_3 - 2x_4 + 3x_5 &= 8, \\ 4x_1 + x_2 - x_3 + 4x_4 - 5x_5 &= 16, \\ 16x_1 - x_2 + x_3 - x_4 - x_5 &= 32. \end{aligned}$$

possui solução única calculando o determinante da matriz dos coeficientes. Então obtenha a mariz inversa e encontre a solução fazendo $x = A^{-1}b$. Por fim, verifique a solução obtida comparando o produto $Ax$ com o vetor $b$.

---

**6.** Resolva o sistema linear

$$\begin{aligned} x_1 + x_2 - x_3 + x_4 - x_5 &= 2, \\ 2x_1 + 2x_2 + x_3 - x_4 + x_5 &= 4, \\ 3x_1 + x_2 - 3x_3 - 2x_4 + 3x_5 &= 8, \\ 4x_1 + x_2 - x_3 + 4x_4 - 5x_5 &= 16, \\ 16x_1 - x_2 + x_3 - x_4 - x_5 &= 32. \end{aligned}$$

---

## 3.2 Métodos diretos: decomposição LU

Os métodos diretos para resolução de sistemas lineares teoricamente dão a solução exata do sistema em um número finito de passos. Na prática, as soluções podem sofrer erros de arredondamento devido à aritmética que está sendo usada.

Os métodos diretos de eliminação consistem no processo de transformar um sistema de equações lineares $Ax = b$, em que $A = [a_{ij}]_i, j = 1, \ldots, n$, $x = [x_1, x_2, \ldots, x_n]^t$ e $b = [b_1, b_2, \ldots, b_n]^t$, em um sistema equivalente, aplicando operações elementares sobre as linhas da matriz aumentada $[A \mid b]$. O sistema equivalente obtido ao final do processo, deve ser de resolução mais simples ou imediata, por substituição direta.

O método de eliminação de Gauss é o método direto usado para escalonar a matriz que representa um sistema linear com o objetivo de obter um ou mais sistemas triangulares, os quais podem ser resolvidos por subtituições subsequentes. Outros métodos diretos podem envolver fatorar a matriz dos coeficientes do sistema em um produto de duas ou mais matrizes, como é o caso do método por fatoração LU, que será abordado nesse texto.

Antes disso, vamos relembrar de funções já desenvolvidas nas seções anteriores.

## Funções para escalonar uma matriz e para resolver sistemas triangulares

```
import numpy as np

# função para escalonar (sem pivoteamento)
def escalona(M):
    n = len(M)
    for j in range(n-1):
        for i in range(j,n-1):
            m = M[i+1,j]/M[j,j]
            M[i+1] = M[i+1]-m*M[j]
    return (M)

# função para sistema triangular inferior
def solve_tril(L,b):
    y = np.empty(len(b))
    for i in range(len(b)):
        y[i] = (b[i] - np.dot(L[i,0:i],y[0:i]))/L[i][i]
    return y

# função para sistema triangular superior
def solve_triu(U,y):
    n=len(y)
    x = np.empty(n)
    for i in range(n-1, -1, -1):
        x[i] = (y[i] - np.sum(U[i,i+1:n+1]*x[i+1:n+1]))/U[i,i]
    return x
```

**Exemplo 3.2.1:** Vamos neste exemplo revisar o sistema linear abaixo, já estudado anteriormente (fonte).

$$\begin{aligned} x_1 - x_2 + 2x_3 - x_4 &= -8, \\ 2x_1 - 2x_2 + 3x_3 - 3x_4 &= -20, \\ x_1 + x_2 + x_3 &= -2, \\ x_1 - x_2 + 4x_3 + 3x_4 &= 4. \end{aligned}$$

Para isso, vamos analisar o escalonamento, coluna por coluna.

```
# escrevendo a matriz ampliada
M = np.array([[1,-1,2,-1,-8],
              [2,-2,3,-3,-20],
              [1,1,1,0,-2],
              [1,-1,4,3,4]], dtype=float)
```

```
# zerando os elementos da primeira coluna
for i in range(3):
    m = M[i+1,0]/M[0,0]
```

(continua na próxima página)

(continuação da página anterior)

```
    M[i+1] = M[i+1] - m*M[0]
print (M)
```

```
[[ 1. -1.  2. -1. -8.]
 [ 0.  0. -1. -1. -4.]
 [ 0.  2. -1.  1.  6.]
 [ 0.  0.  2.  4. 12.]]
```

Observemos que o elemento pivô na segunda linha é zero, portanto, para que o processo de eliminação possa ser continuado, é necessário realizar uma troca de linhas.

```
M[[1,2]] = M[[2,1]]
print(M)
```

```
[[ 1. -1.  2. -1. -8.]
 [ 0.  2. -1.  1.  6.]
 [ 0.  0. -1. -1. -4.]
 [ 0.  0.  2.  4. 12.]]
```

Agora podemos seguir o escalonamento na terceira coluna.

```
# zerando os elementos da terceira coluna
m = M[3,2]/M[2,2]
M[3] = M[3] - m*M[2]
print (M)
```

```
[[ 1. -1.  2. -1. -8.]
 [ 0.  2. -1.  1.  6.]
 [ 0.  0. -1. -1. -4.]
 [ 0.  0.  0.  2.  4.]]
```

```
# solução do sistema
x = solve_triu(M[:,:-1], M[:,-1])
print(x)
```

```
[-7.  3.  2.  2.]
```

Uma função bastante útil para tratar esse tipo de situação é a função `numpy.roll()`, que "rola" os elementos do array ao longo do eixo especificado. Se o elemento a ser desclocado estiver na última posição, ele será levado de volta para a primeira posição. Veja um exemplo a seguir.

```
# rolando as linhas de M para baixo uma vez
np.roll(M, 1, axis=0)
```

```
array([[ 0.,  0.,  0.,  2.,  4.],
       [ 1., -1.,  2., -1., -8.],
       [ 0.,  2., -1.,  1.,  6.],
       [ 0.,  0., -1., -1., -4.]])
```

```
# rolando as linhas de M para cima duas vezes
np.roll(M, -2, axis=0)
```

```
array([[ 0.,  0., -1., -1., -4.],
       [ 0.,  0.,  0.,  2.,  4.],
       [ 1., -1.,  2., -1., -8.],
       [ 0.,  2., -1.,  1.,  6.]])
```

**Exemplo 3.2.2:** Vamos agora utilizar a função `roll` para modificar a função `escalona`, de modo que elementos nulos na posição dos pivôs sejam substituídos por meio de troca de linhas da matriz. Usaremos a matriz do Exemplo 3.2.1 para testar.

```
M = np.array([[1,-1,2,-1,-8],
              [2,-2,3,-3,-20],
              [1,1,1,0,-2],
              [1,-1,4,3,4]], dtype=float)
```

```
def escalona2(M):
    n = len(M)
    for j in range(n-1):
        for i in range(j,n-1):
            # rolando as linhas
            while M[j,j]==0:
                M[j:,:] = np.roll(M[j:,:], -1, axis=0)
            m = M[i+1,j]/M[j,j]
            M[i+1] = M[i+1]-m*M[j]
    return (M)
```

```
M = escalona2(M)
print(M)
```

```
[[ 1. -1.  2. -1. -8.]
 [ 0.  2. -1.  1.  6.]
 [ 0.  0.  2.  4. 12.]
 [ 0.  0.  0.  1.  2.]]
```

```
# solução do sistema
x = solve_triu(M[:,:-1], M[:,-1])
print(x)
```

```
[-7.  3.  2.  2.]
```

Observe que a matriz escalonada é diferente da matriz obtida no Exemplo 1 pois a troca de linha ocorreu de forma diferente durante o processo. Mesmo assim, o resultado final é o mesmo, como já era esperado.

**Exemplo 3.2.3:** Resolva o seguinte sistema pelo método de eliminação de Gauss.

$$
\begin{aligned}
2x_1 + x_2 - x_3 + x_4 - 3x_5 &= 7 \\
x_1 + 2x_3 - x_4 + x_5 &= 2 \\
-2x_2 - x_3 + x_4 - x_5 &= -5 \\
3x_1 + x_2 - 4x_3 + 5x_5 &= 6 \\
x_1 - x_2 - x_3 - x_4 + x_5 &= 3
\end{aligned}
$$

```
M = np.array([[2,1,-1,1,-3,7],
              [1,0,2,-1,1,2],
              [0,-2,-1,1,-1,-5],
              [3,1,-4,0,5,6],
              [1,-1,-1,-1,1,3]], dtype=float)
```

```
Mt = escalona(M.copy())
np.round(Mt,3)
```

```
array([[  2.   ,   1.   ,  -1.   ,   1.   ,  -3.   ,   7.   ],
       [  0.   ,  -0.5  ,   2.5  ,  -1.5  ,   2.5  ,  -1.5  ],
       [  0.   ,   0.   , -11.   ,   7.   , -11.   ,   1.   ],
       [  0.   ,   0.   ,   0.   ,  -3.182,  12.   ,  -3.455],
       [  0.   ,   0.   ,   0.   ,   0.   ,  -4.886,   5.543]])
```

```
# solução do sistema
x = solve_triu(Mt[:,:-1], Mt[:,-1])
print(x)
```

```
[ 1.91812865  1.96491228 -0.98830409 -3.19298246 -1.13450292]
```

```
M
```

```
array([[ 2.,  1., -1.,  1., -3.,  7.],
       [ 1.,  0.,  2., -1.,  1.,  2.],
       [ 0., -2., -1.,  1., -1., -5.],
       [ 3.,  1., -4.,  0.,  5.,  6.],
       [ 1., -1., -1., -1.,  1.,  3.]])
```

```
M[:,:-1]@x
```

```
array([ 7.,  2., -5.,  6.,  3.])
```

**Exemplo 3.2.4:** Use o algoritmo de eliminação de Gauss para resolver os seguintes sistemas lineares, se possível, e determine se são necessárias trocas de linhas (fonte)

a)
$$\begin{aligned} x_1 + x_2 + x_4 &= 2, \\ 2x_1 + x_2 - x_3 + x_4 &= 1, \\ 4x_1 - x_2 - 2x_3 + 2x_4 &= 0, \\ 3x_1 - x_2 - x_3 + 2x_4 &= -3. \end{aligned}$$

b)
$$\begin{aligned} x_1 - \frac{1}{2}x_2 + x_3 &= 4 \\ 2x_1 - x_2 - x_3 + x_4 &= 5 \\ x_1 + x_2 + \frac{1}{2}x_3 &= 2 \\ x_1 - \frac{1}{2}x_2 + x_3 + x_4 &= 5 \end{aligned}$$

c)
$$\begin{aligned} x_1 + x_2 + x_4 &= 2, \\ 2x_1 + x_2 - x_3 + x_4 &= 1, \\ -x_1 + 2x_2 + 3x_3 - x_4 &= 4, \\ 3x_1 - x_2 - x_3 + 2x_4 &= -3. \end{aligned}$$

```
M1 = np.array([[1,1,0,1,2],[2,1,-1,1,1],[4,-1,-2,2,0],[3,-1,-1,2,-3]],dtype=float)
M2 = np.array([[1,-0.5,1,0,4],[2,-1,-1,1,5],[1,1,0.5,0,2],[1,-0.5,1,1,5]],dtype=float)
M3 = np.array([[1,1,0,1,2],[2,1,-1,1,1],[-1,2,3,-1,4],[3,-1,-1,2,-3]],dtype=float)
```

*Solução letra a):*

```
M1e = escalona(M1)
M1e
```

```
array([[ 1.,  1.,  0.,  1.,  2.],
       [ 0., -1., -1., -1., -3.],
       [ 0.,  0.,  3.,  3.,  7.],
       [ 0.,  0.,  0.,  0., -4.]])
```

Sistema impossível.

*Solução letra b):*

```
M2e = escalona(M2.copy())
M2e
```

```
/tmp/ipykernel_22122/899190976.py:8: RuntimeWarning: divide by zero encountered in␣
 ↪scalar divide
  m = M[i+1,j]/M[j,j]
/tmp/ipykernel_22122/899190976.py:9: RuntimeWarning: invalid value encountered in␣
 ↪multiply
  M[i+1] = M[i+1]-m*M[j]
/tmp/ipykernel_22122/899190976.py:8: RuntimeWarning: invalid value encountered in␣
 ↪scalar divide
  m = M[i+1,j]/M[j,j]
```

```
array([[ 1. , -0.5,  1. ,  0. ,  4. ],
       [ 0. ,  0. , -3. ,  1. , -3. ],
       [ nan,  nan,  inf, -inf,  inf],
```

(continua na próxima página)

(continuação da página anterior)

```
       [ nan,  nan,  nan,  nan,  nan]])
```

```
M2e = escalona2(M2.copy())
M2e
```

```
array([[ 1. , -0.5,  1. ,  0. ,  4. ],
       [ 0. ,  1.5, -0.5,  0. , -2. ],
       [ 0. ,  0. , -3. ,  1. , -3. ],
       [ 0. ,  0. ,  0. ,  1. ,  1. ]])
```

```
# solução do sistema
x = solve_triu(M2e[:,:-1], M2e[:,-1])
print(x)
```

```
[ 2.22222222 -0.88888889  1.33333333  1.        ]
```

```
M2[:,:-1]@x
```

```
array([4., 5., 2., 5.])
```

*Solução letra c):*

```
M3
```

```
array([[ 1.,  1.,  0.,  1.,  2.],
       [ 2.,  1., -1.,  1.,  1.],
       [-1.,  2.,  3., -1.,  4.],
       [ 3., -1., -1.,  2., -3.]])
```

```
M3e = escalona2(M3.copy())
M3e
```

```
array([[ 1.,  1.,  0.,  1.,  2.],
       [ 0., -1., -1., -1., -3.],
       [ 0.,  0.,  3.,  3.,  3.],
       [ 0.,  0.,  0., -3., -3.]])
```

```
# solução do sistema
x = solve_triu(M3e[:,:-1], M3e[:,-1])
print(x)
```

```
[-1.  2.  0.  1.]
```

```
M3[:,:-1]@x
```

```
array([ 2.,  1.,  4., -3.])
```

## Decomposição LU

A fatoração $LU$ é útil para resolver sistemas lineares, calcular determinantes e encontrar inversas de matrizes de forma mais eficiente. Uma vez que a matriz $A$ é fatorada, resolver sistemas do tipo $Ax = b$ fica mais rápido, pois basta resolver dois sistemas triangulares mais simples, primeiramente $Ly = b$ e, então, $Ux = y$.

Além disso, é mais fácil obter o determinante de $A$ calculando o produto dos elementos da diagonal da matriz triangular. Por fim, evita fazer a eliminação gaussiana para obter a solução do sistema $Ax = b$ toda vez que muda o vetor $b$.

As mesmas operações com linhas utilizadas para resolver um sistema da forma $Ax = b$ por eliminação de Gauss podem ser utilizados para fatorar uma matriz $A$ em um produto de duas matrizes triangulares $L$ (triangular inferior) e $U$ (triangular superior). Vejamos um exemplo:

**Exemplo 3.2.5:** Seja o sistema

$$\begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 5 \end{bmatrix}$$

Vamos encontrar a solução usando o processo de fatoração $LU$.

```
# criamos as matrizes
A = np.array([[2,0,1],[0,2,1],[1,1,3]], dtype=float)
b = np.array([3,3,5], dtype=float)
```

Então, vamos elimintar os elementos da 1ª coluna, abaixo do pivô:

```
# segunda linha
m21 = A[1,0]/A[0,0]
A[1] = A[1] - m21*A[0]

#terceira linha
m31 = A[2,0]/A[0,0]
A[2] = A[2] - m31*A[0]

print (A)
```

```
[[2.  0.  1. ]
 [0.  2.  1. ]
 [0.  1.  2.5]]
```

Observe que o resultado é o mesmo que se multiplicar a matriz

$$M_1 = \begin{bmatrix} 1 & 0 & 0 \\ -m21 & 1 & 0 \\ -m31 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.5 & 0 & 1 \end{bmatrix}$$

pela matriz $A$ como é mostrado abaixo

```
M1 = np.array([[1,0,0],[0,1,0],[-0.5,0,1]])
A =np.array([[2,0,1],[0,2,1],[1,1,3]])
A = M1@A
print(A)
```

```
[[2.  0.  1. ]
 [0.  2.  1. ]
 [0.  1.  2.5]]
```

Por fim, concluímos a eliminação de Gauss fazendo:

```
#terceira linha segunda coluna
m32 = A[2,1]/A[1,1]
A[2] = A[2] - m32*A[1]
print (A)
```

```
[[2. 0. 1.]
 [0. 2. 1.]
 [0. 0. 2.]]
```

O resultado é o mesmo que obtemos ao multiplicar a matriz

$$M_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -m_{32} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix}$$

por $M_1 A$.

Ou seja, o escalonamento da matriz $A$ pode ser realizado pelo produto $M_2 M_1 A$, como podemos verificar a seguir:

```
A =np.array([[2,0,1],[0,2,1],[1,1,3]])
M1 = np.array([[1,0,0],[0,1,0],[-0.5,0,1]])
M2 = np.array([[1,0,0],[0,1,0],[0,-0.5,1]])
```

```
U = M2@M1@A
print(U)
```

```
[[2. 0. 1.]
 [0. 2. 1.]
 [0. 0. 2.]]
```

Então, em resumo, tem-se:

$$M_2M_1A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -m_{32} & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -m_{21} & 1 & 0 \\ -m_{31} & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ -m_{21} & 1 & 0 \\ -m_{31} & -m_{32} & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0,5 & -0,5 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix} = U$$

Observe que podemos obter a matriz $A$ a partir da matriz triangular superior $U$ fazendo $M_1^{-1}M_2^{-1}U = A$, pois

$$M_1^{-1}M_2^{-1}U = \begin{bmatrix} 1 & 0 & 0 \\ -m_{21} & 1 & 0 \\ -m_{31} & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -m_{32} & 1 \end{bmatrix}^{-1} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix} = A$$

```
from numpy.linalg import inv
```

```
inv(M1)@inv(M2)@U
```

```
array([[2., 0., 1.],
       [0., 2., 1.],
       [1., 1., 3.]])
```

Assim, fazendo

$$\begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0,5 & 0,5 & 1 \end{bmatrix} = L$$

podemos escrever a fatoração $LU$ como

$$A = LU \quad \rightarrow \quad \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0,5 & 0,5 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

Como é verificado abaixo

```
L = np.array([[1,0,0],[0,1,0],[0.5,0.5,1]])
U = np.array([[2,0,1],[0,2,1],[0,0,2]])
L@U
```

```
array([[2., 0., 1.],
       [0., 2., 1.],
       [1., 1., 3.]])
```

Por fim, para obter a solução do sistema linear $Ax = b$ por decomposição $LU$ fazemos

$$Ax = b \rightarrow LUx = b$$

escrevendo $Ux = y$ obtemos a solução resolvendo dois sitemas triangulares

Assim, a resolução do sistema $Ax = b$ é feita resolvendo-se o sistema triangular inferior e, em seguida, o sistema triagular superior como é mostrado a seguir:

```
y = solve_tril(L,b)
print(y)
```

```
[3. 3. 2.]
```

```
x = solve_triu(U,y)
print(x)
```

```
[1. 1. 1.]
```

Agora, podemos sintetizar os resultados escrevendo uma função para fatorar uma matriz por eliminação de Gauss , fazendo:

```
def fatoraLU(A):
    L = np.eye(len(A))
    U = A.copy()
    n = len(A)
    for j in range(n-1):
```

(continua na próxima página)

(continuação da página anterior)

```
        for i in range(j,n-1):
            m = U[i+1,j]/U[j,j]
            U[i+1] = U[i+1]-m*U[j]
            L[i+1,j] = m
    return (L,U)
```

```
# testando
A = np.array([[2,0,1],[0,2,1],[1,1,3]], dtype=float)
L,U = fatoraLU(A)
print(L)
print(U)
```

```
[[1.  0.  0. ]
 [0.  1.  0. ]
 [0.5 0.5 1. ]]
[[2. 0. 1.]
 [0. 2. 1.]
 [0. 0. 2.]]
```

```
y = solve_tril(L,b)
print("y=",y)
```

```
y= [3. 3. 2.]
```

```
x = solve_triu(U,y)
print("x=",x)
```

```
x= [1. 1. 1.]
```

**Exemplo 3.2.6:** Agora vamos considerar novamente o sistema do exemplo 1 e resolver usando fatoração LU.

```
M = np.array([[2,1,-1,1,-3,7],
              [1,0,2,-1,1,2],
              [0,-2,-1,1,-1,-5],
              [3,1,-4,0,5,6],
              [1,-1,-1,-1,1,3]], dtype=float)
```

```
A = M[:,0:5]
b = M[:,-1]
```

```
L,U = fatoraLU(A)
y = solve_tril(L,b)
x = solve_triu(U,y)
print(x)
```

```
[ 1.91812865  1.96491228 -0.98830409 -3.19298246 -1.13450292]
```

```
A@x
```

```
array([ 7.,  2., -5.,  6.,  3.])
```

## Fatoração LU por redução de Doolittle

Uma forma de obter a fatoração $LU$ da matriz $A$, conhecido como processo Redução de Doolittle será apresentado a seguir (fonte). Considere a fatoração da matriz $A = (a_{ij})_{i,j=1,\ldots,n}$ nas matrizes $L = (l_{ij})_{i,j=1,\ldots,n}$ e $U = (u_{ij})_{i,j=1,\ldots,n}$ como é mostrado a seguir

$$\begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ l_{n1} & l_{n2} & \cdots & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

o processo de múltiplicação de matrizes nos leva às seguintes equações, que fornecem a primeira linha da matriz $U$: $u_{11} = a_{11}, u_{12} = a_{12}, u_{12} = a_{12}, \ldots, u_{1n} = a_{1n}$. Seguindo o procedimento de multiplicar matrizes, podemos encontrar a 1ª coluna da matriz $L$: $l_{21} = a_{21}/u_{11}, l_{31} = a_{31}/u_{11}, \ldots, u_{n1} = a_{n1}/u_{11}$. Continuando o processo para a 2ª linha de $U$, 2ª coluna de $L$, 3ª linha de $U$, 3ª coluna de $L$, e assim por diante, chegamos nas seguintes fórmulas:

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj} \qquad i, j = 1, \ldots, n$$

e

$$l_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj}}{u_{jj}} \qquad i, j = 1, \ldots, n$$

Uma implementação desse algoritmo é mostrado a seguir.

```
def fatoracao_LU(A):
    n = len(A)
    U = np.zeros((n,n))
    L = np.identity(n)
    for m in range(n):
        for j in range(m, n):
            U[m,j] = A[m,j] - np.sum(L[m,0:m] * U[0:m,j])
        for i in range(m+1, n):
            L[i,m] = (A[i,m] - np.sum(L[i,0:m] * U[0:m,m]))/U[m,m]
    return L,U
```

Testando para a mesma matriz do exemplo 3.

```
A = np.array([[2,0,1],
```

(continua na próxima página)

(continuação da página anterior)

```
              [0,2,1],
              [1,1,3]])
L,U = fatoracao_LU(A)

print ("Matriz L:")
print (np.array(L))
print ("Matriz U:")
print(np.array(U))
```

```
Matriz L:
[[1.  0.  0. ]
 [0.  1.  0. ]
 [0.5 0.5 1. ]]
Matriz U:
[[2. 0. 1.]
 [0. 2. 1.]
 [0. 0. 2.]]
```

## Usando o módulo `scipy.linalg` para fatorar uma matriz

```
import scipy
import scipy.linalg
```

```
A = np.array([[1,2,4],[2,8,10],[4,10,26]])
```

### Fatoração LU

```
scipy.linalg.lu(A)
```

```
(array([[0., 0., 1.],
        [0., 1., 0.],
        [1., 0., 0.]]),
 array([[ 1.        ,  0.        ,  0.        ],
        [ 0.5       ,  1.        ,  0.        ],
        [ 0.25      , -0.16666667,  1.        ]]),
 array([[ 4., 10., 26.],
        [ 0.,  3., -3.],
        [ 0.,  0., -3.]]))
```

### Fatoração Cholesky

A fatoração de Cholesky é uma forma especial de decompor uma matriz simétrica e definida positiva. Ela escreve a matriz $A$ como:

$$A = LL^T$$

onde $L$ é uma matriz triangular inferior (com números reais positivos na diagonal), $L^T$ é a transposta de $L$. Nesse caso a solução de um sistema linear $Ax = b$ por fatoração de Cholesky pode ser ainda mais rápida do que por fatoração $LU$.

A decomposição Cholesky é especialmente útil em problemas de otimização, sistemas de equações diferenciais, simulações físicas e finanças.

```
L = np.linalg.cholesky(A)
L
```

```
array([[1., 0., 0.],
       [2., 2., 0.],
       [4., 1., 3.]])
```

```
#verificando
L@L.T
```

```
array([[ 1.,  2.,  4.],
       [ 2.,  8., 10.],
       [ 4., 10., 26.]])
```

### Fatoração QR

A fatoração $QR$ é uma decomposição que escreve uma matriz $A$ como o produto de duas matrizes, uma matriz ortogonal $Q$ e uma matriz triangular superior $R$. É útil na resolução de sistemas lineares superdeterminados, no cálculo de autovalores e na ortonormalização de vetores. Além disso, a fatoração $QR$ é mais estável numericamente que a $LU$, especialmente para sistemas mal condicionados.

```
Q,R = np.linalg.qr(A)
print(Q)
print(R)
```

```
[[-0.21821789  0.27263927 -0.93704257]
 [-0.43643578 -0.88607763 -0.15617376]
 [-0.87287156  0.374879    0.31234752]]
[[ -4.58257569 -12.65663763 -27.93188995]
 [  0.          -2.79455252   1.97663471]
 [  0.           0.           2.81112771]]
```

```
Q@R
```

```
array([[ 1.,  2.,  4.],
       [ 2.,  8., 10.],
       [ 4., 10., 26.]])
```

## Exercícios

---

**1.** Determine a fatoração $LU$ da matriz $A$ e resolva o sistema linear $Ax = b$, em que

$$A = \begin{bmatrix} 1 & 1 & 0 & 3 \\ 2 & 1 & -1 & 1 \\ 3 & -1 & -1 & 2 \\ -1 & 2 & 3 & -1 \end{bmatrix} \quad \text{e} \quad \mathbf{b} = \begin{bmatrix} 4 \\ 1 \\ -3 \\ 4 \end{bmatrix}$$

---

**2.** Use a fatoração do exercício anterior para resolver o sistema

$$\begin{aligned} x_1 + x_2 + 3x_4 &= 8, \\ 2x_1 + x_2 - x_3 + x_4 &= 7, \\ 3x_1 - x_2 - x_3 + 2x_4 &= 14, \\ -x_1 + 2x_2 + 3x_3 - x_4 &= -7. \end{aligned}$$

---

**3.** Obtenha, caso exista, a solução para os seguintes sistemas lineares utilizando os métodos estudados (eliminação de Gauss e decomposição LU).

a)
$$\begin{cases} 2x_1 + 3x_2 + x_3 + 5x_4 = 11 \\ x_1 + 3.5x_2 + x_3 + 7.5x_4 = 13 \\ 1.4x_1 + 2.7x_2 + 5.5x_3 + 12x_4 = 21.6 \\ -2x_1 + 1x_2 + 3x_3 + 28x_4 = 30 \end{cases}$$

b)
$$\begin{cases} 6.1x_1 + 0.32x_2 + 1.3x_3 + 2.1x_4 + 0.11x_5 = 19.52 \\ 0.82x_1 + 8.81x_2 + 1.01x_3 + 3x_4 + 3.12x_5 = 15.83 \\ 0.5x_1 + 1.78x_2 + 15.2x_3 + 4.2x_4 + 8.1x_5 = -22.14 \\ 4.2x_1 + 5.3x_2 + 1.8x_3 + 20.9x_4 + 7.51x_5 = 27.28 \\ 0.2x_1 + 9.1x_2 + 4.68x_3 + 4.3x_4 + 20.1x_5 = -21.78 \end{cases}$$

c)
$$\begin{cases} 12.1756x_1 + 4.0231x_2 - 2.1732x_3 + 5.1967x_4 = 17.1020 \\ -4.0231x_1 + 6.0030x_2 + 1.1973x_4 = -6.1593 \\ -1.0000x_1 - 5.2107x_2 + 11.1111x_3 = 3.0004 \\ 6.0235x_1 + 7.0000x_2 + -14.1561x_4 = 0.0000 \end{cases}$$

---

**4.** (Burden et al, 2016, p. 420) Resolva o seguinte sistema linear por eliminação de Gauss e decomposição LU (se possível).

$$\begin{aligned} 1,19x_1 + 2,11x_2 - 100x_3 + x_4 &= 1,12 \\ 14,2x_1 - 0,122x_2 + 12,2x_3 - x_4 &= 3,44 \\ 100x_2 - 99,9x_3 + x_4 &= 2,15 \\ 15,3x_1 + 0,110x_2 - 13,1x_3 - x_4 &= 4,16 \end{aligned}$$

---

**5.** (Burden et al, 2016, p.455) Use fatoração LU para resolver o sistema linear a seguir.

$$\begin{aligned} 2x_1 + x_2 &= 0, \\ -x_1 + 3x_2 + 3x_3 &= 5, \\ 2x_1 - 2x_2 + x_3 + 4x_4 &= -2, \\ -2x_1 + 2x_2 + 2x_3 + 5x_4 &= 6. \end{aligned}$$

---

## 3.3 Eliminação de Gauss-Jordan e matriz inversa

Os métodos diretos de eliminação consistem no processo de transformar um sistema de equações lineares $Ax = b$, em que $A = (a_{ij})_i, j = 1, \dots, n$, $x = (x_1, x_2, \dots, x_n)^t$ e $b = (b_1, b_2, \dots, b_n)^t$, em um sistema equivalente, aplicando operações elementares sobre as linhas da matriz $(A, b)$.

O sistema equivalente obtido ao final do processo, deve ser de resolução mais simples ou imediata, por substituição direta. No caso da eliminação de Gauss com pivoteamento diagonal ou pivoteamento parcial, obtém-se um sistema equivalente na forma triangular. No caso da eliminação de Gauss-Jordan o sistema equivalente obtido possui a matriz de coeficientes igual a matriz identidade (forma escalonada reduzida).

**Exemplo 3.3.1:** (Fonte) Use a eliminação de Gauss-Jordan para resolver o sistema

$$\begin{aligned} x - 2y + 3z &= 9 \\ -x + 3y &= -4 \\ 2x - 5y + 5z &= 17 \end{aligned}$$

A matriz aumentada associada ao sistema é:

$$\left[\begin{array}{rrrr} 1 & -2 & 3 & 9 \\ -1 & 3 & 0 & -4 \\ 2 & -5 & 5 & 17 \end{array}\right]$$

Substituindo a 2ª linha por ela mesma subtraída da 1ª linha ($L_2 = L_2 - L_1$) e a 3ª linha por ela mesma subtraida da 1ª linha multiplicada por 2 ($L_3 = L_3 - 2L_1$), obtém-se

$$\left[\begin{array}{rrrr} 1 & -2 & 3 & 9 \\ 0 & 1 & 3 & 5 \\ 0 & -1 & -1 & -1 \end{array}\right]$$

Agora, substituindo a 1ª linha por ela mesma somada da 2ª linha multiplicada por 2 ($L_1 = L_1 + 2L_2$) e a 3ª linha por ela mesma somada à 2ª ($L_3 = L_3 + L_2$), obtém-se

$$\begin{bmatrix} 1 & 0 & 9 & 19 \\ 0 & 1 & 3 & 5 \\ 0 & 0 & 2 & 4 \end{bmatrix}$$

Por fim, dividindo a 3ª linha por 2, então, substituindo a 1ª pela 3ª multiplicada por $-9$ ($L_1 = L_1 - 9L3$) e a 2ª por ela mesma multiplicada por $-3$, obtém-se:

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 2 \end{bmatrix}$$

A matriz está agora na forma escalonada reduzida. Convertendo de volta para um sistema de equações lineares, tem-se

$$x = 1$$
$$y = -1$$
$$z = 2.$$

Usando Python podemos fazer:

```
import numpy as np
```

```
def gaussJordan(M):
    n = len(M)
    for k in range(n):
        M[k] = M[k]/M[k,k]
        for i in range(n):
            if i!=k:
                M[i] = M[i] - M[i,k]*M[k]
    return (M)
```

```
Ab = np.array([[1,-2,3,9],[-1,3,0,-4],[2,-5,5,17]])
Ab
```

```
array([[ 1, -2,  3,  9],
       [-1,  3,  0, -4],
       [ 2, -5,  5, 17]])
```

```
gaussJordan(Ab)
```

```
array([[ 1,  0,  0,  1],
       [ 0,  1,  0, -1],
       [ 0,  0,  1,  2]])
```

```
print ('Solução:', Ab[:,-1])
```

```
Solução: [ 1 -1  2]
```

## Método Gauss-Jordan e matriz inversa

Considere o sistema de $n$ equações e $n$ variáveis dado por

$$Ax = b$$

Se a matriz $A$ dos coeficientes for não singular, ou seja, se o sistema possuir solução única a matriz é invertível e a solução do sistema pode ser obtida multiplicando-se a inversa $A^{-1}$ à esquerda de ambos os lados da equação, como segue:

$$A^{-1}Ax = A^{-1}b$$

logo

$$Ix = A^{-1}b$$

ou

$$x = A^{-1}b$$

Que é a solução do sistema linear.

Podemos obter a inversa da matriz $A$ por eliminação da Gauss-Jordam aplicando a eliminação à matriz aumentada $[A \mid I]$ em que $I$ é a matriz identidade de mesma ordem de $A$. Ao final da eliminação obtemos a matriz $[I \mid A^{-1}]$ em que $A^{-1}$ é a inversa da matriz $A$.

**Exemplo 3.3.2:** Resolva o sistema $Ax = b$ usando a matriz inversa de $A$, sendo $A = \begin{bmatrix} 3 & 0 & 1 \\ 3 & 2 & 1 \\ -3 & 1 & 3 \end{bmatrix}$ e $b = (1, 1, 3)^t$.

```
import numpy as np
```

```
A = np.array([[3,0,1],[3,2,1],[-3,1,3]])
b = np.array([1,1,3])
x = np.dot(np.linalg.inv(A),b)
print (np.round(x,4))
```

```
[-0.  0.  1.]
```

A obtenção inversa pode ser feita pelo processo de eliminação de Gauss-Jordan em que são realizadas operações elementares sobre as linhas da matriz $A$ até que ela se torne uma matriz identidade.

Essas mesmas operações são realizadas em uma matriz identidade de mesma ordem, ao final das operações na matriz identidade, o resultado é a inversa da matriz $A$. O exemplo a seguir implementa o processo de eliminação de Gauss-Jordan para obter a inversa de uma matriz $A$.

**Exemplo 3.3.2:** Vamos nesse exemplo utilizar o processo de eliminação de Gauss-Jordan para obter a matriz inversa

de

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 1 \\ 1 & 0 & 3 \end{bmatrix}$$

```
A = np.array([[2,1,3],[0,-1,1],[1,0,3]], float)
I = np.identity(len(A))
M = np.concatenate((A,I), axis=1)

print ('Matriz ampliada:')
print (M)

print ('Matriz após eliminação:')
Mgj = gaussJordan(M)
print (Mgj )

print ('Matriz inversa:')
Ainv = Mgj[:,3:6]
print(Ainv)
```

```
Matriz ampliada:
[[ 2.  1.  3.  1.  0.  0.]
 [ 0. -1.  1.  0.  1.  0.]
 [ 1.  0.  3.  0.  0.  1.]]
Matriz após eliminação:
[[ 1.   0.   0.   1.5  1.5 -2. ]
 [ 0.   1.   0.  -0.5 -1.5  1. ]
 [ 0.   0.   1.  -0.5 -0.5  1. ]]
Matriz inversa:
[[ 1.5  1.5 -2. ]
 [-0.5 -1.5  1. ]
 [-0.5 -0.5  1. ]]
```

## Exercícios

**1.** (Larson, 2017) Obtenha, caso exista, a solução para os seguintes sistemas lineares utilizando os métodos eliminação de Gauss-Jordan e Matriz inversa).

a)
$$\begin{aligned} 2x + y - z + 2w &= -6 \\ 3x + 4y + w &= 1 \\ x + 5y + 2z + 6w &= -3 \\ 5x + 2y - z - w &= 3 \end{aligned}$$

b)
$$\begin{aligned} x_1 + x_2 - 2x_3 + 3x_4 + 2x_5 &= 9 \\ 3x_1 + 3x_2 - x_3 + x_4 + x_5 &= 5 \\ 2x_1 + 2x_2 - x_3 + x_4 - 2x_5 &= 1 \\ 4x_1 + 4x_2 + x_3 - 3x_5 &= 4 \\ 8x_1 + 5x_2 - 2x_3 - x_4 + 2x_5 &= 3 \end{aligned}$$

c)
$$\begin{aligned} x_1 - x_2 + 2x_3 + 2x_4 + 6x_5 &= 6 \\ 3x_1 - 2x_2 + 4x_3 + 4x_4 + 12x_5 &= 14 \\ x_2 - x_3 - x_4 - 3x_5 &= -3 \\ 2x_1 - 2x_2 + 4x_3 + 5x_4 + 15x_5 &= 10 \\ 2x_1 - 2x_2 + 4x_3 + 4x_4 + 13x_5 &= 13 \end{aligned}$$

d)
$$\begin{aligned} x_1 + 2x_2 - 2x_3 + 2x_4 - x_5 + 3x_6 &= 0 \\ 2x_1 - x_2 + 3x_3 + x_4 - 3x_5 + 2x_6 &= 17 \\ x_1 + 3x_2 - 2x_3 + x_4 - 2x_5 - 3x_6 &= -5 \\ 3x_1 - 2x_2 + x_3 - x_4 + 3x_5 - 2x_6 &= -1 \\ -x_1 - 2x_2 + x_3 + 2x_4 - 2x_5 + 3x_6 &= 10 \\ x_1 - 3x_2 + x_3 + 3x_4 - 2x_5 + x_6 &= 11 \end{aligned}$$

---

**2.** Obtenha a matriz inversa de

$$A = \begin{bmatrix} 4 & -2 & 1 \\ 2 & 3 & -1 \\ -1 & 1 & 2 \end{bmatrix}$$

aplicando a eliminação de Gauss-Jordan à matriz aumentada $[A \mid I]$ (implemente usando a função `gaussJordan`). Verifique o resultado calculando $AA^{-1}$ e confirmando que se obtém a matriz identidade.

---

**3. (matriz singular)** Aplique o algoritmo de Gauss-Jordan à matriz aumentada $[A \mid I]$ para

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{bmatrix}$$

a) O que acontece durante a eliminação quando você tenta normalizar a segunda linha (dividir pelo pivô)?
b) Calcule o determinante de $A$ e relacione com o que você observou no item (a).
c) O sistema $Ax = b$ com $b = (6, 12, 2)^T$ possui solução única? Ele possui alguma solução? Justifique.

---

**4. (custo computacional com múltiplos vetores $b$)** Considere o sistema $Ax = b$ para a matriz

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 4 \end{bmatrix}$$

e os três vetores $b_1 = (1, 2, 3)^T$, $b_2 = (0, 1, 0)^T$ e $b_3 = (5, -1, 2)^T$.
a) Obtenha as três soluções aplicando a eliminação de Gauss-Jordan separadamente em cada matriz aumentada $[A \mid b_i]$.

b) Calcule a inversa $A^{-1}$ uma única vez e obtenha as três soluções fazendo $x_i = A^{-1}b_i$.
c) Explique por que, quando é necessário resolver o sistema para vários vetores $b$ diferentes com a mesma matriz $A$, calcular $A^{-1}$ uma única vez é mais vantajoso do que repetir a eliminação de Gauss-Jordan para cada $b_i$.

---

**5. (aplicação: circuito elétrico)** As correntes de malha $i_1$, $i_2$ e $i_3$ (em ampères) de um circuito elétrico satisfazem o sistema

$$10i_1 - 2i_2 - 3i_3 = 5$$
$$-2i_1 + 8i_2 - i_3 = 0$$
$$-3i_1 - i_2 + 9i_3 = 3$$

Resolva o sistema (a) pela eliminação de Gauss-Jordan e (b) pela matriz inversa, e confirme que as duas abordagens fornecem o mesmo resultado.

---

**6.** Considere um dos sistemas $Ax = b$ resolvidos na Seção 3.2 (decomposição LU) ou na Seção 3.4 (métodos iterativos).
a) Resolva-o também pela eliminação de Gauss-Jordan e pela matriz inversa.
b) Se você precisasse resolver esse mesmo sistema repetidamente, apenas alterando o vetor $b$ a cada vez, qual dos métodos estudados até aqui (Gauss-Jordan direto, matriz inversa, decomposição LU ou um método iterativo) você escolheria? Justifique sua resposta com base no número de operações necessárias em cada nova resolução.

## 3.4 Métodos iterativos

Os métodos iterativos são indicados para sistemas de grande porte com a matriz dos coeficientes esparsa, ou seja, com grande grande quantidade de zeros. Nesses casos os métodos iterativos são mais rápidos e demandam menos memória para armazenamento. Além disso, os métodos iterativos possuem a vantagem de se auto corrigir e também podem ser usados para reduzir os erros de arredondamento na solução obtida por métodos exatos. Sob certas condições, podem também ser usados para resolver um conjunto de equações não lineares.

No entanto, para que o método iterativo seja convergente, é necessário atender a certas condições. Dizemos que a sequência de vetores $x^{(k)}$, $k = 0, 1, 2, ...$ de um espaço vetorial $V$ converge para a solução exata $\bar{x} \in V$ se $\|x^{(k)} - \bar{x}\| \to 0$, quando $k \to \infty$ para alguma norma $\|.\|$ sobre o espaço vetorial $V$.

Para determinar a solução de um sistema linear por métodos iterativos, precisamos transformar o sistema dado em um outro sistema equivalente que estabeleça um processo iterativo. Suponha então, que o sistema $Ax = b$ seja reescrito na forma equivalente

$$x = Hx + g$$

tal que a solução $\bar{x}$ de $x = Hx + g$ é também, solução de $Ax = b$. A partir dessa nova forma podemos obter um processo iterativo que fornecerá a sequência de soluções aproximadas que buscamos. Assim, seja $x^{(0)}$ uma aproximação inicial para a solução $\bar{x}$, obtemos as aproximações sucessivas $x^{(k)}$ para a solução desejada usando o processo iterativo definido por:

$$x^{(k)} = Hx^{(k-1)} + g$$

**Convergência:** A condição necessária e suficiente para a convergência do processo iterativo definido por $x = Hx + g$ é que $\max\{|\lambda_i|\} < 1$, onde $\lambda_i$ são os autovalores da matriz $H$.

**Critério geral de convergência:** Como consequência do teorema acima, o processo será convergente se para qualquer norma de matrizes, $\|H\| < 1$.

*Prova:* Na k-ésima iteração o vetor erro, dado por $e^{(k)} = \bar{x} - x^{(k)}$ será

$$e^{(k)} = H^k e^{(0)}$$

Tomando normas consistentes, tem-se que

$$\|H^k e^{(0)}\| \leq \|H\|^k \|e^{(0)}\|$$

portanto $\$\|e^{(k)}\| \leq \|H\|^k \|e^{(0)}\|\$$

como $\|H\| < 1$ tem-se que $\|e^{(k)}\| \to 0$ quando $k \to \infty$.

Assim, se $\|H\| < 1$ para alguma norma, temos a garantia de que o processo iterativo dado por $x - x^{(k)} = H(x - x^{(k-1)})$ converge para a solução exata $\bar{x}$. A matriz $H$ é chamada **matriz de iteração**.

## Método iterativo de Jacobi-Richardson

Considere um sistema de equações lineares $Ax = b$ em que $\det(A) \neq 0$, com a diagonal principal $a_{ii} \neq 0$, $i = 1, \ldots, n$ como segue

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Dividindo cada linha do sistema dado pelo elemento da diagonal e isolando $x_1$ na $1^a$ equação, $x_2$ na $2^a$ equação até $x_n$ na n-ésima equação, temos o sistema escrito na forma equivalente:

$$\begin{cases} x_1 = \frac{1}{a_{11}}\left(b_1 - a_{12}x_2 - a_{13}x_3 - \cdots - a_{1n}x_n\right) \\ x_2 = \frac{1}{a_{22}}\left(b_2 - a_{21}x_1 - a_{23}x_3 - \cdots - a_{2n}x_n\right) \\ \vdots \\ x_n = \frac{1}{a_{nn}}\left(b_n - a_{n1}x_1 - a_{n2}x_2 - \cdots - a_{n\,n-1}x_{n-1}\right) \end{cases}$$

O método iterativo de Jacobi-Richardson é dado da seguinte forma:

$$\begin{cases} x_1^{(k+1)} = \frac{b_1}{a_{11}} - \frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} \\ x_2^{(k+1)} = \frac{b_2}{a_{22}} - \frac{a_{21}}{a_{22}}x_1^{(k)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} \\ \vdots \\ x_n^{(k+1)} = \frac{b_n}{a_{nn}} - \frac{a_{n1}}{a_{nn}}x_1^{(k)} - \frac{a_{n2}}{a_{nn}}x_3^{(k)} - \cdots - \frac{a_{n\,n-1}}{a_{nn}}x_{n-1}^{(k)} \end{cases}$$

### Convergência

Podemos verificar a convergência do método iterativo de Jacobi-Richardson avaliando a matriz $A$. Se a matriz $A = [a_{ij}]_{i,j=1.,,,,n}$ do sistema $Ax = b$ for estritamente diagonalmente dominante, ou seja,

$$|a_{ii}| > \sum_{j=1, i\neq j} |a_{ij}|, i = 1, ...n$$

então, a sequência será convergente.

### Critério de parada

Considerando que o processo iterativo está fornecendo uma sequência convergente, um critério de parada para o algoritmo pode ser dado por

$$\frac{\| x^{(k)} - x^{(k-1)} \|}{\| x^{(k)} \|} < \epsilon$$

para alguma norma vetorial $\| . \|: V \to R$ e alguma tolerância $\epsilon$ pré estabelecida.

Por conveniência, é comum utilizarmos a norma infinito:

$$\| x \|_\infty = \max\{|x_0|, |x_1|, ... , |x_n|\}$$

**Exemplo 3.4.1:** Usando o método interativo de Jacobi-Richardson, determine uma solução aproximada para o seguinte sistema de equações lineares, com aproximação inicial $x^{(0)} = (0, 0, 0)^T$ e precisão $\epsilon = 0.01$.

$$\begin{bmatrix} 10 & 2 & 1 \\ 1 & 5 & 1 \\ 2 & 3 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 14 \\ 11 \\ 8 \end{bmatrix}$$

Reescrevendo o sistema, obtemos:

$$\begin{cases} x_1^{(k+1)} = \frac{14}{10} - \frac{2}{10}x_2^{(k)} - \frac{1}{10}x_3^{(k)} \\ x_2^{(k+1)} = \frac{11}{5} - \frac{1}{5}x_1^{(k)} - \frac{1}{5}x_3^{(k)} \\ x_3^{(k+1)} = \frac{8}{10} - \frac{2}{10}x_1^{(k)} - \frac{3}{10}x_2^{(k)} \end{cases}$$

A partir da primeira iteração, $k = 0$, $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, x_3^{(0)})^T = (0, 0, 0)^T$obtem-se:

$x_1^{(1)} = \frac{14}{10} - \frac{2}{10}x_2^{(0)} - \frac{1}{10}x_3^{(0)} = 1,4 - 0,2(0) - 0,1(0) = 1,4$
$x_2^{(1)} = \frac{11}{5} - \frac{1}{5}x_1^{(0)} - \frac{1}{5}x_3^{(0)} = 2,2 - 0,2(0) - 0,2(0) = 2,2$
$x_3^{(1)} = \frac{8}{10} - \frac{2}{10}x_1^{(0)} - \frac{3}{10}x_2^{(0)} = 0,8 - 0,2(0) - 0,3(0) = 0,8$

Para $k = 0$ e tomando $x^{(0)} = (0, 0, 0)^t$, temos $x^{(1)} = (1,4; 2,2; 0,8)^t$ Teste de parada: $\frac{\|x^{(1)} - x^{(0)}\|_\infty}{\|x^{(1)}\|_\infty} = 1 > \varepsilon$

Para $k = 1$, temos:

$x_1^{(2)} = \frac{14}{10} - \frac{2}{10}x_2^{(1)} - \frac{1}{10}x_3^{(1)} = 1,4 - 0,2(2,2) - 0,1(0,8) = 0,88$
$x_2^{(2)} = \frac{11}{5} - \frac{1}{5}x_1^{(1)} - \frac{1}{5}x_3^{(1)} = 2,2 - 0,2(1,4) - 0,2(0,8) = 1,76$
$x_3^{(2)} = \frac{8}{10} - \frac{2}{10}x_1^{(1)} - \frac{3}{10}x_2^{(1)} = 0,8 - 0,2(1,4) - 0,3(2,2) = -0,14$

Portanto,

$$x^{(2)} = (0,8800; 1,7600; -0,1400)^t, \quad \frac{\left\|x^{(2)} - x^{(1)}\right\|_\infty}{\left\|x^{(2)}\right\|_\infty} = 0,5341 > \varepsilon$$

Assim, sucessivamente, calculamos $x^{(3)}$, $x^{(4)}$, $x^{(5)}$, ..., até que o critério de parada seja satisfeito em

$$x^{(6)} = (0,9967; 1,9955; -0,0047)^t$$

em que

$$\frac{\left\|x^{(6)} - x^{(5)}\right\|_\infty}{\left\|x^{(6)}\right\|_\infty} = 0,0091 < \varepsilon$$

Em python podemos fazer como é mostrado a seguir:

```
import numpy as np
np.set_printoptions(precision = 4)
```

```
# Dados iniciais
x = np.array([0.,0.,0.])

A =  np.array([[10.,2.,1.],
               [1.,5.,1.],
               [2.,3.,10.]])
b = np.array([14.,11.,8.])
```

```
# função para fazer as iterações
def itera(x):
    x1 = (b[0] - A[0,1]*x[1] - A[0,2]*x[2])/A[0,0]
    x2 = (b[1] - A[1,0]*x[0] - A[1,2]*x[2])/A[1,1]
    x3 = (b[2] - A[2,0]*x[0] - A[2,1]*x[1])/A[2,2]
    x = np.array([x1, x2, x3])
    return x
```

```
x = itera(x)
x
```

```
array([1.4, 2.2, 0.8])
```

```
#várias iterações e o erro
x = np.array([0.,0.,0.])

x_ant = x
for i in range(2,8):
    x = itera(x)
    err = np.max(abs(x-x_ant))/np.max(abs(x))
    x_ant = x
    print (i, x, np.round(err,4))

print ("Solução:", np.round(x,4))
```

```
2 [1.4 2.2 0.8] 1.0
3 [ 0.88  1.76 -0.14] 0.5341
4 [1.062 2.052 0.096] 0.1423
5 [ 0.98    1.9684 -0.028 ] 0.063
6 [1.0091 2.0096 0.0135] 0.0206
7 [ 0.9967  1.9955 -0.0047] 0.0091
Solução: [ 0.9967  1.9955 -0.0047]
```

Uma outra forma de expressarmos as equações usadas no processo iterativo de Jacobi-Richardson éusar a expressão compactada

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right),$$
$$i = 1, \ldots, n$$

em que o valor de $i$ identifica cada uma das equações do sistema. Então, podemos obter as soluções aproximadas como é mostrado a seguir:

**Exemplo 3.4.2:** Outra forma de obter as soluções do Exemplo 3.4.1 é mostrada a seguir.

```
A = np.array([[10., 2., 1.],[1.,5.,1.],[2.,3.,10.]])
b = np.array([14.,11.,8.])
x = np.array([0.,0.,0.])
n = len(A)

xk = np.array([0.,0.,0.])
for k in range(6):
    for i in range(n):
        xk[i] = (b[i]-np.sum(A[i,0:i]*x[:i])-np.sum(A[i,i+1:]*x[i+1:]))/A[i,i]
        # a linha acima é equivalente a:
        #xk[i] = (b[i]-np.dot(A[i,0:i],x[:i])-np.dot(A[i,i+1:],x[i+1:]))/A[i,i]
    print (xk)
    x = xk.copy()
```

```
[1.4 2.2 0.8]
[ 0.88  1.76 -0.14]
[1.062 2.052 0.096]
[ 0.98    1.9684 -0.028 ]
[1.0091 2.0096 0.0135]
[ 0.9967  1.9955 -0.0047]
```

```
print ("Solução:", x)
```

```
Solução: [ 0.9967  1.9955 -0.0047]
```

Na forma matricial, o método de Jacobi-Richardson é dada por:

$$x^{(k+1)} = Hx^{(k)} + g$$

onde

$$H = \begin{bmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \cdots & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & -\frac{a_{23}}{a_{22}} & \cdots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & -\frac{a_{n3}}{a_{nn}} & \cdots & 0 \end{bmatrix} \quad \text{e} \quad g = \begin{bmatrix} \frac{b_1}{a_{11}} \\ \frac{b_2}{a_{22}} \\ \vdots \\ \frac{b_n}{a_{nn}} \end{bmatrix}$$

ou, ainda,

$$\begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} = \begin{bmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \cdots & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & -\frac{a_{23}}{a_{22}} & \cdots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & -\frac{a_{n3}}{a_{nn}} & \cdots & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix} + \begin{bmatrix} \frac{b_1}{a_{11}} \\ \frac{b_2}{a_{22}} \\ \vdots \\ \frac{b_n}{a_{nn}} \end{bmatrix}$$

**Exemplo 3.4.3:** Resolvendo novamente o exemplo 3.4.1, mas agora usando operações matriciais.

```
A = np.array([[10., 2., 1.],[1.,5.,1.],[2.,3.,10.]])
b = np.array([14.,11.,8.])
```

```
H = np.eye(len(A))- A/np.diag(A).reshape(3,1)
print ('H =', H)
```

```
H = [[ 0.  -0.2 -0.1]
 [-0.2  0.  -0.2]
 [-0.2 -0.3  0. ]]
```

```
g = b/np.diag(A)
print ('g =', g)
```

```
g = [1.4 2.2 0.8]
```

```
# verificando a convergência com a norma inf
norma_inf = np.sum(abs(H), axis=1).max()
print ('Norma:', norma_inf)
```

```
Norma: 0.5
```

```
x = np.array([0,0,0])
```

```
x = H@x+g
x
```

```
array([1.4, 2.2, 0.8])
```

```
# fazendo as iterações
err = 1
```

(continua na próxima página)

(continuação da página anterior)

```
x = np.array([0,0,0])
x_ant = x

print ('Iterações:')
while err>0.01:
    x = np.dot(H,x)+g
    err = abs(max(x-x_ant)/max(x))
    x_ant = x
    print(np.round(x, 4), ", Err=", np.round(err,5))
```

```
Iterações:
[1.4 2.2 0.8] , Err= 1.0
[ 0.88  1.76 -0.14] , Err= 0.25
[1.062 2.052 0.096] , Err= 0.1423
[ 0.98    1.9684 -0.028 ] , Err= 0.04166
[1.0091 2.0096 0.0135] , Err= 0.02064
[ 0.9967  1.9955 -0.0047] , Err= 0.00621
```

### Método iterativo de Gaus-Seidel

Considere um sistema de equações lineares $Ax = b$ em que $\det(A) \neq 0$, com a diagonal principal $a_{ii} \neq 0$, $i = 1, \dots, n$ como segue

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Dividindo cada linha do sistema dado pelo elemento da diagonal e isolando $x_1$ na $1^a$ equação, $x_2$ na $2^a$ equação até $x_n$ na n-ésima equação, temos o sistema escrito na forma equivalente:

$$\begin{cases} x_1 = \frac{1}{a_{11}}\left(b_1 - a_{12}x_2 - a_{13}x_3 - \cdots - a_{1n}x_n\right) \\ x_2 = \frac{1}{a_{22}}\left(b_2 - a_{21}x_1 - a_{23}x_3 - \cdots - a_{2n}x_n\right) \\ \vdots \\ x_n = \frac{1}{a_{nn}}\left(b_n - a_{n1}x_1 - a_{n2}x_2 - \cdots - a_{n\,n-1}x_{n-1}\right) \end{cases}$$

O método iterativo de Gauss-Seidel é dado da seguinte forma:

$$\begin{cases} x_1^{(k+1)} = \frac{b_1}{a_{11}} - \frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} \\ x_2^{(k+1)} = \frac{b_2}{a_{22}} - \frac{a_{21}}{a_{22}}x_1^{(k+1)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} \\ \vdots \\ x_n^{(k+1)} = \frac{b_n}{a_{nn}} - \frac{a_{n1}}{a_{nn}}x_1^{(k+1)} - \frac{a_{n2}}{a_{nn}}x_3^{(k+1)} - \cdots - \frac{a_{n\,n-1}}{a_{nn}}x_{n-1}^{(k+1)} \end{cases}$$

De forma geral, o método de Gauss-Seidel pode ser resumido como:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right),$$
$$i = 1, \dots, n$$

ou ainda, de forma equivalente:

$$x_i^{(k+1)} = g_i - \sum_{j=1}^{i-1} h_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} h_{ij} x_j^{(k)},$$
$$i = 1, \dots, n$$

em que $g_i = \frac{b_i}{a_{ii}}$ e $h_{ij} = \frac{a_{ij}}{a_{ii}}$, $i = 1, \dots, n$.

### Convergência: critério de Sassenfield

Sejam as constantes $\beta_i$ definidas pelas seguintes fórmulas de recorrência:

$$\beta_i = \sum_{j=1}^{i-1} |h_{ij}| \beta_i + \sum_{j=i+1}^{n} |h_{ij}| \quad i = 1, \dots, n$$

e seja

$$\beta = \max_{1 \leq i \leq n} \beta_i$$

Então, se $\beta < 1$, a sequência $x^{(k)}$, gerada pelo método iterativo de Gauss-Seidel, converge para a solução $x$ do sistema dado.

**Exemplo 3.4.4:** (Fonte) Usando o método iterativo de Gauss-Seidel, determine uma solução aproximada para o sistema dado a seguir, com aproximação inicial $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, x_3^{(0)})^t = (0, 0, 0)^t$ e precisão $\epsilon = 0.01$.

$$\begin{bmatrix} 10 & 2 & 1 \\ 1 & 5 & 1 \\ 2 & 3 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 14 \\ 11 \\ 8 \end{bmatrix}$$

Uma possível solução

```
# Entrando com a matriz a e o vetor b
A = np.array([[10.0, 2.0, 1.0],
              [ 1.0, 5.0, 1.0],
              [ 2.0, 3.0, 10.0]])

b = np.array([14., 11., 8.])
```

```
# obtendo a matriz H
H = np.eye(len(A))-(A.T/np.diag(A)).T

print ('Matriz H:')
print(H)
```

```
Matriz H:
[[ 0.  -0.2 -0.1]
 [-0.2  0.  -0.2]
 [-0.2 -0.3  0. ]]
```

```
# verificando a convergência
beta = []
for i in range(len(H)):
    bi = np.dot(beta,abs(H[i,0:i]))+ np.sum(abs(H[i,i+1:]))
    beta.append(np.round(bi,4))

print('Convergência:')
print ('max{',beta,'} =',np.array(beta).max())
```

```
Convergência:
max{ [0.3, 0.26, 0.138] } = 0.3
```

```
n = len(A)
x = np.array([0.,0.,0.])
x_ant = x.copy()
err=10

while err>0.01:
    for i in range(len(A)):
        x[i] = (b[i] - np.dot(A[i,0:i],x[0:i])-np.dot(A[i,i+1:n],x[i+1:n]))/A[i,i]
    err = abs(max(x-x_ant)/max(x))
    x_ant = x
    print(x, ", Err=", np.round(err,5))

print("Solução:",np.round(x,4))
```

```
[ 1.4    1.92  -0.056] , Err= 1.0
[ 1.0216  2.0069 -0.0064] , Err= 0.0
Solução: [ 1.0216  2.0069 -0.0064]
```

Na forma matricial, o método de Gauss-Seidel pode ser escrito como

$$\begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ -\frac{a_{21}}{a_{22}} & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & \cdots & -\frac{a_{n\,n-1}}{a_{nn}} & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} + \begin{bmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \cdots & -\frac{a_{1n}}{a_{11}} \\ 0 & 0 & -\frac{a_{23}}{a_{22}} & \cdots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix} + \begin{bmatrix} \frac{b_1}{a_{11}} \\ \frac{b_2}{a_{22}} \\ \vdots \\ \frac{b_n}{a_{nn}} \end{bmatrix}$$

ou $$x^{(k+1)} = Px^{(k+1)} + Qx^{(k)} + g$$

$$(I - P)x^{(k+1)} = Qx^{(k)} + g$$

$$x^{(k+1)} = (I - P)^{-1}Qx^{(k)} + (I - P)^{-1}g$$

Fazendo-se $H' = (I - P)^{-1}Q$ e $g' = (I - P)^{-1}g$ o processo iterativo torna-se

$$x^{(k+1)} = H'x^{(k)} + g'$$

**Exemplo 3.4.5:** Repetindo o Exemplo 3.4.4 mas agora utilizando operções matriciais.

```
from numpy.linalg import inv
```

```
# Entrando com a matriz a e o vetor b
A = np.array([[10.0, 2.0, 1.0],
              [ 1.0, 5.0, 1.0],
              [ 2.0, 3.0, 10.0]])

b = np.array([14., 11., 8.])
```

```
H = np.eye(len(A))-(A.T/np.diag(A)).T
g = b/np.diag(A)
```

```
P = np.tril(H)
P
```

```
array([[ 0. ,  0. ,  0. ],
       [-0.2,  0. ,  0. ],
       [-0.2, -0.3,  0. ]])
```

```
Q = np.triu(H)
Q
```

```
array([[ 0. , -0.2, -0.1],
       [ 0. ,  0. , -0.2],
       [ 0. ,  0. ,  0. ]])
```

```
Hl = inv(np.eye(3)-P)@Q
gl = inv(np.eye(3)-P)@g
```

```
# fazendo as iterações
err = 1
x = np.array([0,0,0])
x_ant = x.copy()

print ('Iterações:')
while err>0.01:
    x = Hl@x + gl
    err = abs(max(x-x_ant)/max(x))
```

(continua na próxima página)

(continuação da página anterior)

```
    x_ant = x
    print(x, ", Err=", err)
```

```
Iterações:
[ 1.4    1.92  -0.056] , Err= 1.0
[ 1.0216  2.0069 -0.0064] , Err= 0.04329107868930881
[ 9.9926e-01  2.0014e+00 -2.7978e-04] , Err= 0.003049939954761819
```

## Exercícios

**1.** Usando algum método iterativo, determine uma solução aproximada para os sistemas dados, com aproximação inicial $x^{(0)} = (0, 0, ..., 0)^t$ e 10 iterações.

a)
$$\begin{bmatrix} 5 & 1 & 2 & 1 & -1 \\ 0 & 6 & 1 & 1 & -1 \\ 0 & 1 & -3 & 2 & 1 \\ 3 & 0 & 2 & 7 & 0 \\ 1 & 1 & 0 & 0 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 0 \\ 2 \\ 1 \end{bmatrix}$$

b)
$$\begin{aligned} 4x_1 + x_2 + x_3 + \quad x_5 &= 6, \\ -x_1 - 3x_2 + x_3 + x_4 &= 6, \\ 2x_1 + x_2 + 5x_3 - x_4 - x_5 &= 6, \\ -x_1 - x_2 - x_3 + 4x_4 &= 6, \\ 2x_2 - x_3 + x_4 + 4x_5 &= 6 \end{aligned}$$

c)
$$\begin{aligned} 4x_1 - x_2 &= 0, \\ -x_1 + 4x_2 - x_3 &= 5, \\ -x_2 + 4x_3 &= 0, \\ +4x_4 - x_5 &= 6, \\ -x_4 + 4x_5 - x_6 &= -2, \\ -x_5 + 4x_6 &= 6. \end{aligned}$$

**2.** Usando o método iterativo de Gauss-Seidel, determine uma solução aproximada para o sistema dado a seguir, com aproximação inicial $x^{(0)} = (0, 0, 0, 0, 0)^t$ e 10 iterações.

$$\begin{bmatrix} 5 & 1 & 2 & 1 & -1 \\ 0 & 6 & 1 & 1 & -1 \\ 0 & 1 & -3 & 2 & 1 \\ 3 & 0 & 2 & 7 & 0 \\ 1 & 1 & 0 & 0 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 0 \\ 2 \\ 1 \end{bmatrix}$$

**3.** (Burden et al, 2016, p.505) Use a técnica iterativa de Gauss-Seidel para encontrar soluções aproximadas de

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6, \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25, \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11, \\ 3x_2 - x_3 + 8x_4 &= 15, \end{aligned}$$

começando com $\mathbf{x} = (0, 0, 0, 0)^t$e iterando até que

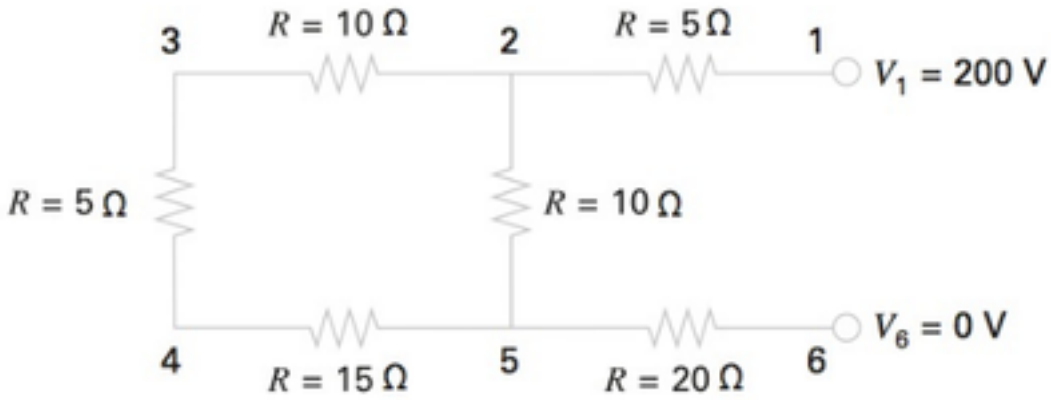
$$\frac{\left\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\right\|_\infty}{\left\|\mathbf{x}^{(k)}\right\|_\infty} < 10^{-3}.$$

---

**4.** (Chapra e Canale, 2016, p. 305)

Um problema comum na engenharia elétrica envolve a determinação das correntes e tensões em várias posições em circuitos de resistores.

Considere o curcuíto mostrado abaixo




A regra de corrente de Kirchhoff aplicada em cada nó do circuito abaixo fornece

$$\begin{aligned} i_{12} + i_{52} + i_{32} &= 0 \\ i_{65} - i_{52} - i_{54} &= 0 \\ i_{43} - i_{32} &= 0 \\ i_{54} - i_{43} &= 0 \end{aligned}$$

A aplicação da regra de tensão em cada um dos dois laços dá

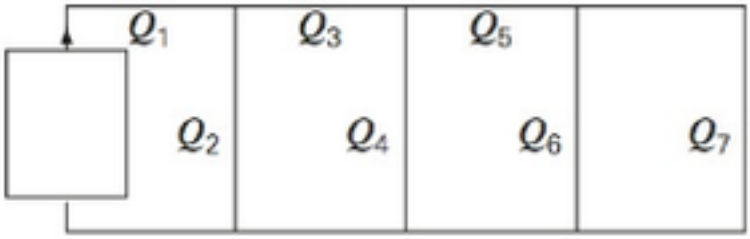
$$\begin{aligned} -i_{54}R_{54} - i_{43}R_{43} - i_{32}R_{32} + i_{52}R_{52} &= 0 \\ -i_{65}R_{65} - i_{52}R_{52} - i_{12}R_{12} - 200 &= 0 \end{aligned}$$

Substituindo o valor das resistências e movendo as constantes para o lado direito, o problema se transforma em resolver o conjunto de seis equações equações lineares para as seis correntes desconhecidas.

Obtenha o sistema e encontre as correntes usando algum método de eliminação.

---

**5.** (Chapra e Canale, 2016, p. 310) Uma bomba peristáltica produz um escoamento unitário ($Q_i$) de um fluido altamente viscoso. A rede está descrita na figura abaixo.




Cada seção de tubo tem os mesmos comprimento e diâmetro. Os balanços de massa e energia mecânica podem ser simplificados para se obter os escoamentos em cada tubo como solução do sistema de equações

$$
\begin{aligned}
Q_3 + 2Q_4 - 2Q_2 &= 0 \\
Q_5 + 2Q_6 - 2Q_4 &= 0 \\
3Q_7 - 2Q_6 &= 0 \\
Q_1 &= Q_2 + Q_3 \\
Q_3 &= Q_4 + Q_5 \\
Q_5 &= Q_6 + Q_7
\end{aligned}
$$

Resolva o sistema para obter o escoamento de cada corrente.

# 4. Interpolação Polinomial

Dado um conjunto de pontos $(x_i, y_i)$, a interpolação polinomial consiste em encontrar um polinômio que passe exatamente por todos esses pontos. É amplamente utilizada para reconstruir funções a partir de dados discretos e para aproximar valores entre pontos conhecidos.

### Objetivos de aprendizagem

- Construir o polinômio interpolador de Lagrange
- Aplicar a forma de Newton com diferenças divididas
- Estimar e controlar o erro de interpolação polinomial
- Identificar situações em que a interpolação de alto grau é inadequada

### Conteúdo


| Seção | Tema |
|---|---|
| 4.1 | Interpolação de Lagrange |
| 4.2 | Interpolação de Newton |
| 4.3 | Erro na interpolação polinomial |


## 4.1. Intepolação polinomial e fórmula de Lagrange

A interpolação polinomial é um método de aproximação de funções por polinômios. A interpolação é utilizada quando se conhece apenas alguns pontos da função ou quando a função é muito complicada e difícil de manejar. Os polinômios têm a vantagem de serem fáceis de derivar e integrar. Além disso, é possível provar que qualquer função contínua pode ser arbitrariamente aproximada por um polinômio (teorema de Weierstrass).

Outra particularidade é que o valor numérico de um polinômio

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

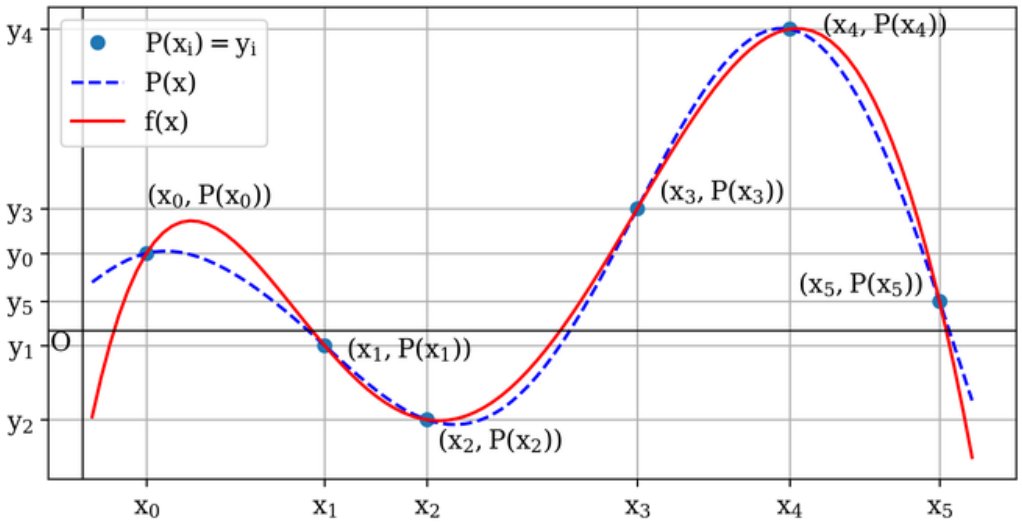
para algum valor real de $x$ pode ser calculado com um número reduzido de operações aritméticas escrevendo-o na forma

$$P_n(x) = a_0 + x(a_1 + x(a_2 + x(\ldots + x(a_{n-1} + x a_n)\ldots)))$$

o que apresenta algumas vantagens computacionalmente, como veremos mais para frente ao realizar a interpolação pela fórmula de Newton.

Interpolar uma função $f(x) = y$ em um conjunto de $n + 1$ pontos distintos $x_0, x_1, \dots, x_n$ de um intervalo $[a, b]$ por um polinômio de significa encontrar $P_n(x) = a_0 + a_1x + \cdots + ax^n$ tal que $P(x_i) = y_i$ com $i = 0, 1, \dots, n$. Ou seja, significa encontrar o polinômio de grau $n$ que "passa" por esses pontos.

Na figura abaixo é mostrada um polinômio que interpola uma função a partir de 6 pontos. É possível observar que o polinômio passa pelos pontos, ou seja, $P(x_i) = y_i$, $i = 0, 1, \dots, 5$, e que aproxima a função razoavelmente bem a função no intervalo de $x_1$ até $x_5$. No intervalo entre $x_0$ e $x_1$ a aproximação não é tão boa e fora do intervalo $(x_0, x_5)$ parece não funcionar muito bem.




O caso mais simples de interpolação que podemos analisar é o de encontrar a reta (polinômio de grau 1) que passa por dois pontos, $(x_0, y_0)$ e $(x_1, y_1)$. Nesse caso, o polinômio tem a forma $P(x) = a_0 + a_1x$ e para determiná-lo basta encontrar os coeficientes $a_0$ e $a_1$ da única reta que passa por esses dois pontos. Logo, os coeficientes podem ser obtidos resolvendo o sistema linear:

$$\begin{cases} y_0 = a_0 + a_1x_0 \\ y_1 = a_0 + a_1x_1 \end{cases} \quad \text{ou, na forma matricial} \quad \begin{bmatrix} 1 & x_0 \\ 1 & x_1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}$$

A solução pode ser obtida fazendo $a_1 = \frac{y_1 - y_0}{x_1 - x_0}$ e $a_0 = y_0 - a_1x_0$.

**Exemplo 4.1.1:** Vamos obter o polinômio que interpola os pontos $(1, 2), (5, 1)$.

_Solução:_Para isso, resolvemos o sistema linear

$$\begin{cases} a_0 + a_1 = 2 \\ a_0 + 5a_1 = 1 \end{cases}$$

que nos fornece:

$a_1 = \frac{y_1 - y_0}{x_1 - x_0} = \frac{1-2}{5-1} = -\frac{1}{4}$ e $a_0 = y_0 - a_1x_0 = 2 + \frac{1}{4} = \frac{9}{4}$

Assim, obtemos

$$P(x) = \frac{9}{4} - \frac{1}{4}x$$

A figura abaixo mostra o resultado

Assim, generalizando o caso anterior, para obter os coeficientes do polinômio $P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$,

de grau $n$, que interpola uma função a partir de $n+1$ pontos $(x_i, y_i)$, $i = 0, 1, \ldots, n$, podemos resolver o seguinte sistema

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \ldots & \cdots & \ldots & \cdots & \cdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \ldots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \ldots \\ y_n \end{bmatrix}$$

A matriz $n \times n$ mostrada acima é chamada matriz de Vandermonde e tem determinante não nulo para pontos $x_i$ distintos, ou seja, polinômio de coeficientes $a_0, a_1, \ldots, a_n$ existe e é único. Usando o módulo `polynomial` de Numpy a matriz de Vandermonde pode ser obtida usando a função `np.polynomial.polynomial.polyvander`.

**Exemplo 4.1.2:** Considere o problema de encontrar um polinômio interpolador para do conjunto de pontos $\{(0, 1)$, $(1, 6)$, $(2, 5)$, $(3, -8)\}$. Como o conjunto consiste de 4 pontos, o polinômio interpolador deve ser da forma: $p(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$

O que nos leva ao sistema linear:

$$\begin{cases} a_0 = 1 \\ a_0 + a_1 + a_2 + a_3 = 6 \\ a_0 + 2a_1 + 4a_2 + 8a_3 = 5 \\ a_0 + 3a_1 + 9a_2 + 27a_3 = -8 \end{cases}$$

cuja solução é $a_0 = 1$, $a_1 = 6$, $a_2 = 0$ e $a_3 = -1$. Portanto, o polinômio interpolador é $p(x) = 1 + 6x - x^3$.

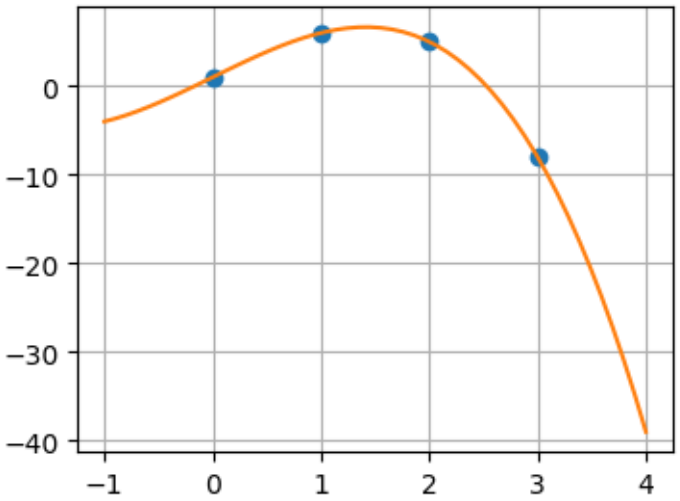
Podemos verificar o resultado plotando os pontos e o polinômio:

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
xi = np.array([0.,1.,2.,3.])
yi = np.array([1.,6.,5.,-8.])

x = np.linspace(-1,4)
y = 1+6*x-x**3

plt.plot(xi,yi,'o',x,y)
plt.grid()
```




Computacionalmente, podemos resolver esse exemplo como é mostrado a seguir, utilizando as operações vetorizadas da biblioteca `numpy` para montar a matriz do sistema e a função `solve` do módulo `linalg` para resolver, assim obtemos facilmente os coeficientes do polinômio interpolador. Também podemos usar a conveniente função `polyval` para avaliar o valores do polinômio em um conjunto de valores de $x$. Faremos isso para verificar se correspondem aos valores da função, como é esperado, ou seja, se $P(x_i) = y_i$, $i = 0, 1, 2, 3$.

```
A = np.array([xi**0,xi**1,xi**2,xi**3]).T
#ou  A = np.polynomial.polynomial.polyvander(xi,3)
ai = np.linalg.solve(A,yi)
coeffs = np.flipud(ai) # reverte a ordem

print ("Coeficientes:", coeffs)
print ("P(xi):", np.polyval(coeffs,xi))
```

```
Coeficientes: [-1. -0.  6.  1.]
P(xi): [ 1.  6.  5. -8.]
```

Como é possível notar, com poucas linhas de código, utilizando as convenientes funções disponíveis na biblioteca NumPy, é possível obter o polinômio que interpola os pontos dados. Podemos também verificar que os numéricos do polinômio em cada um dos pontos coincidem com os valores da função, ou seja, $f(x_i)$, $i = 0, 1, 2, 3$.

Embora sempre seja possível obter o polinômio interpolador de grau que passa por um conjunto de pontos distintos resolvendo um sistema linear, essa pode não ser a maneira mais eficiente de fazer isso. A seguir serão apresentadas duas fórmulas bem conhecidas para interpolação polinomial que requerem menos esforço computacional, as fórmula de Lagrange e de Newton. A primeira requer um número maior de operações aritméticas básicas mas ainda assim pode ser mais vantajosa quando várias funções precisam ser interpoladas no mesmo conjunto de pontos.

## Fórmula Interpolatória de Lagrange

Seja $f(x)$ definida em $x_0, x_1, \dots, x_n$, $(n+1)$ pontos distintos de um intervalo $[a,b]$ e $y_i = f(x_i)$, $i = 0, 1, \dots, n$, então o polinômio interpolador de Lagrange é dado por

$$P(x) = \sum_{k=0}^{n} y_k L_k(x)$$

com

$$L_k(x) = \prod_{j=0,\, j\neq k}^{n} \frac{(x - x_j)}{(x_k - x_j)}$$

**Exemplo 4.1.3:** Considere a função $f(x)$ definida nos pontos $\{(0.0, 1.9), (1.5, 2.5), (2.0, 0.9)\}$. Determine o polinômio interpolador, usando a fórmula de Lagrange, e estime $f(0.9)$.

*Solução:* Queremos o polinômio na forma $P(x) = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x)$. Então, obtemos os $L_k(x)$ fazendo

$$L_0 = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - 1,5)(x - 2,0)}{(0,0 - 1,5)(0,0 - 2,0)} = \frac{x^2 - 3,5x + 3,0}{3,0}$$

$$L_1 = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x - 0,0)(x - 2,0)}{(1,5 - 0,0)(1,5 - 2,0)} = \frac{x^2 - 2x}{-0,75}$$

$$L_2 = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{(x - 0,0)(x - 1,5)}{(2,0 - 0,0)(2,0 - 1,5)} = \frac{x^2 - 1,5x}{1,0}$$

Assim,

$$P(x) = 1,9\left(\frac{x^2 - 3,5x + 3,0}{3,0}\right) + 2,5\left(\frac{x^2 - 2x}{-0,75}\right) + 0.9\left(\frac{x^2 - 1,5x}{1,0}\right) = -1,8x^2 - 3,1x + 1,9$$

Portanto, tem-se

$$P(0,9) = -1,8(0,9)^2 + 3,1(0,9) + 1,9 = 3,2320.$$

O script Python abaixo plota o gráfico do polinômio obtido juntamente com os pontos tabelados.

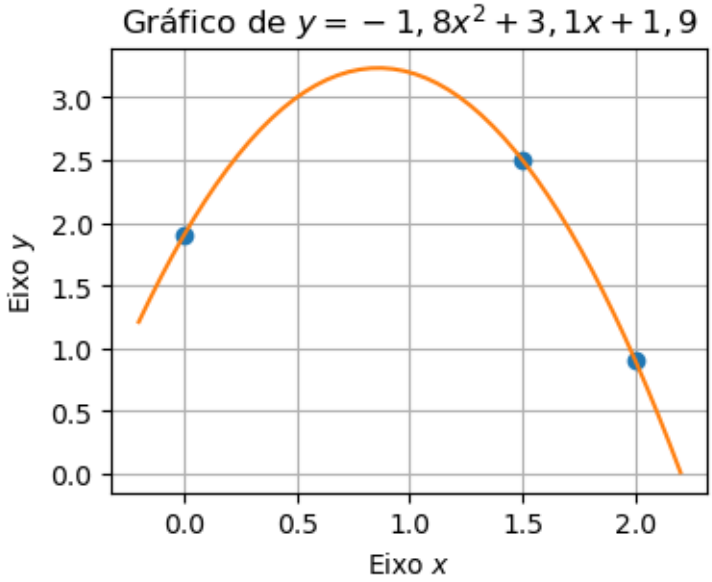
```
import matplotlib.pyplot as plt
import numpy as np

xi = [0.0, 1.5, 2.0]
yi = [1.9, 2.5, 0.9]

def P(x):
    return -1.8*x**2+3.1*x+1.9

xp= np.linspace(-0.2, 2.2)
plt.plot(xi, yi, 'o', xp, P(xp))

plt.title('Gráfico de $y=-1,8x^2+3,1x+1,9$')
plt.ylabel('Eixo $y$')
plt.xlabel('Eixo $x$')
plt.grid()
```



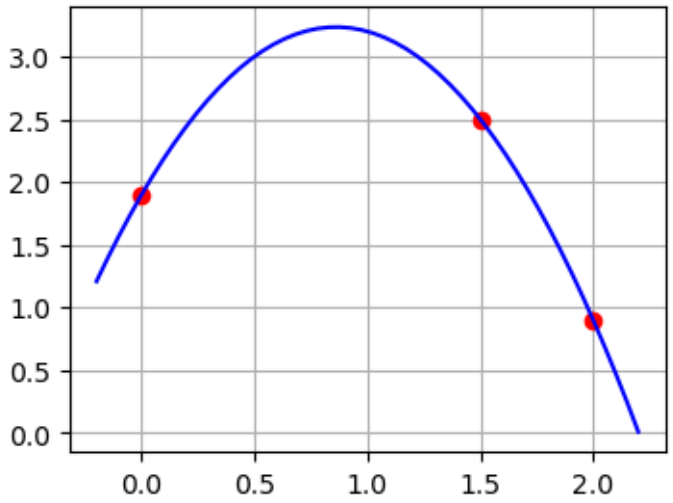

No código abaixo é mostrada uma função que calcula o valor numérico do polinômio interpolador usando a fórmula de Lagrange para um dado valor de x. Usando essa função vamos calcular o valor de $P(9)$ e, também, fazer um laço de repetição para calcular o calor do polinômio em um conjunto de pontos para construir o gráfico. Observe que, como é esperado, o gráfico é igual ao do polinômio que foi obtido analiticamente no Exemplo 4.1.3.

```
def calculaP(x,X,Y):
    valor = 0
    for k in range(len(X)):
        lk = 1.0
        for i in range(len(X)):
            if k != i:
                lk = lk*(x - X[i])/(X[k]-X[i])
        valor = valor + Y[k]*lk
    return valor
```

```
yp = []
for x in xp:
    y = calculaP(x, xi, yi)
    yp.append(y)

plt.plot(xi, yi, 'ro', xp, yp,'b-')
plt.grid()
plt.show()

print ("P(",0.9,") =", calculaP(0.9, xi,yi))
```



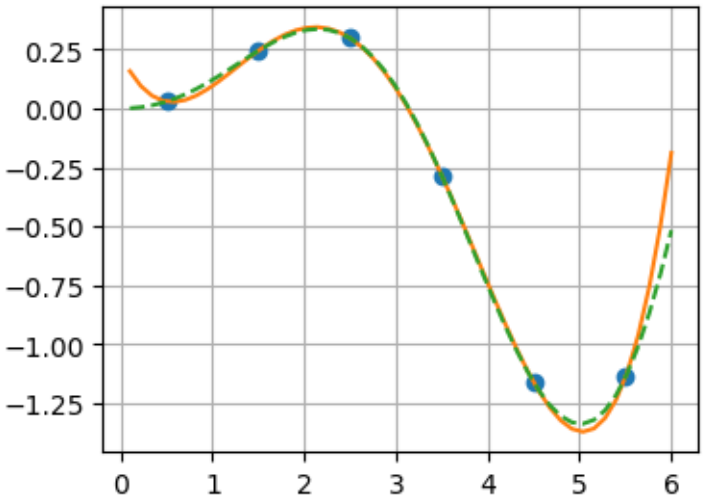

```
P( 0.9 ) = 3.232
```

**Exemplo 4.1.4:** Considere a função $f(x) = \frac{0{,}2x^2 sen(x)}{ln[(1+x)^2]}$ definida em seis pontos igualmente espaçados tais que $x_0 = 0.5$ e $x_5 = 5.5$. Vamos usar a função do exemplo 3.3 para plotar o gráfico do polinômio interpolador de Lagrage juntamente com o gráfico da função e os plontos utilizados.

```
f = lambda x:(0.2*x**2*np.sin(x))/np.log((1 + x)**2)
xi = np.linspace(0.5,5.5,6)
yi = f(xi)

xp = np.linspace(0.1, 6)
yp = []
for x in xp:
    y = calculaP(x, xi, yi)
    yp.append(y)

plt.plot(xi, yi, 'o',
         xp, yp,'-',
         xp,f(xp),'--')
plt.grid()
plt.show()
```




### Fórmula interpolatória de Lagrange para pontos equidistantes

Se os pontos $x_0, \ldots, x_n$ forem igualmente espaçados, ou seja, se $x_{i+1} - x_i = h$ para $i = 0, \ldots, n-1$, então é possível fazer uma mudança de variável e obter uma fórmula simplificada para o polinômio interpolador de Lagrange. Fazemos

$$u = \frac{x - x_0}{h}$$

que é o mesmo que $$x = x_o + uh$$

Com essas mudanças, a fórmula de Lagrange fica

$$P(u) = \sum_{k=0}^{n} y_k l_k(u)$$

com

$$l_k(u) = \prod_{j=0,\, j\neq k}^{n} \frac{(u-j)}{(k-j)}$$

Para avaliar o valor do polinômio de um determinado valor de $x$ precisamos obter o valor correspondente na variável $u$, fazendo, $u = \frac{x-x_0}{h}$ e calcular $P(u)$.

**Exemplo 4.1.5:**

Considere a função $f(x) = \cos(x)$. Obtenha uma aproximação para $f(0.3)$ usando um polinômio de grau 2 com $x_0 = 0.2$ e $h = 0.2$.

```
def calculaP_eq(u):
    valor = 0
    for k in range(len(X)):
        lk = 1.0
        for i in range(len(X)):
            if k != i:
                lk = lk*(u - i)/(k-i)
        valor = valor + Y[k]*lk
    return valor
```

```
X = np.linspace(0.2, 0.6, 3)
Y = np.cos(X)

h = X[1]-X[0]
u = (0.3 - X[0])/h

print (calculaP_eq(u))
```

```
0.9551537603289194
```

## Exercícios

---

**1.** Encontre polinômios interpoladores para as funções abaixo no intervalo de $x_0 = 1$ até $x_n = 3$ com 3, 4 e 5 pontos escolhidos arbitrariamente no intervalo.
(a) $\ln(x+1)$
(b) $\sqrt{1+x}$
(c)$e^{2x}\cos(3x)$

---

**2.** Dada a tabela a seguir de valores de uma função $f$,

| x | 0,15 | 0,17 | 0,19 | 0,21 | 0,23 | 0,25 | 0,27 | 0,29 | 0,31 |
|---|---|---|---|---|---|---|---|---|---|
| f(x) | 0,176 | 0,234 | 0,278 | 0,322 | 0,361 | 0,398 | 0,431 | 0,462 | 0,491 |

(a) Obtenha uma estimativa para $f(0,20)$ e para $f(0,22)$ utilizando interpolação polinomial. (b) Obtenha a expressão analítica do polinômio interpolador. Plote o gráfico. (c) Estime $f(0,20)$ e $f(0,22)$ utilizando uma polinomial de terceiro grau. Plote o gráfico e compare com o resultado do item anterior.

---

**3.** Considere a função $\$f(x) = \frac{3.21}{0.73+9.81x^2}\$$
(a) Aproxime o valor de $f(1.78)$ usando um polinômio interpolador de grau 3 no intervalo [1, 2] e compare com o valor da função. Mostre o gráfico do polinômio e da função nesse intervalo. (b) Repita o item anterior, mas dessa vez utilize um polinômio de grau 10 no intervalo [-2,2] e comente o que você observou. (c)Pesquise e responda o que é “Fenômeno de Runge”.

---

**4.** O concreto novo ganha resistência com o passar dos dias atingindo 99% de sua resistência máxima aos 28 dias. A tabela abaixo mostra os resultados de um ensaio de resistência à compressão simples (em MPa), obtidos em laboratório especializado, de três diferentes traços de concreto:

1. Dosagem padrão, sem a adição de agregado miúdo reciclado
2. Substituindo 20% de agregado miúdo natural por agregado miúdo reciclado
3. Substituindo 40% de agregado miúdo natural por agregado miúdo reciclado.

| Dias | Traço 1 | Traço 2 | Traço 3 |
|---|---|---|---|
| 3 | 14,4025 | 21,8450 | 19,0045 |
| 7 | 27,4350 | 31,2800 | 26,7450 |
| 14 | 34,4900 | 35,1600 | 31,3000 |
| 21 | 34,3750 | 38,1000 | 34,1800 |
| 28 | 36,1650 | 39,9800 | 35,6550 |

(a) Encontre o polinômio que interpola dos dados de cada um dos ensaios, faça o gráfico. (b) Estime quantos dias são necessários para que cada um dos concretos alcance 32 MPa. (c) Encontre a resistência de cada concreto após 16 dias.

---

**5.** Durante três dias consecutivos foi tomada a temperatura (em °C) numa região de uma cidade, por quatro vezes no período das 6 às 12 horas. Determine, usando todos os dados da tabela abaixo, a média das temperaturas dos três dias às 9 horas.

| Hora | 1º dia | 2º dia | 3ºdia |
|---|---|---|---|
| 6 | 18.5 | 17.2 | 18.1 |
| 8 | 20.3 | 20.1 | 21.4 |
| 10 | 24.0 | 25,1 | 22.9 |
| 12 | 28.2 | 27.5 | 23.7 |

---

**6.** (Fonte) Utilize polinômios interpoladores de Lagrange de grau três para aproximar cada um dos seguintes:
(a) $f(8,4)$ se $f(8,1) = 16,94410, f(8,3) = 17,56492, f(8,6) = 18,50515, f(8,7) = 18,82091$
(b) $f\left(-\frac{1}{3}\right)$ se $f(-0,75) = -0,07181250, f(-0,5) = -0,02475000, f(-0,25) = 0,33493750, f(0) = 1{,}10100000$
(c) $f(0,25)$ se $f(0,1) = 0,62049958, f(0,2) = -0,28398668, f(0,3) = 0,00660095, f(0,4) = 0,24842440$
(d) $f(0,9)$ se $f(0,6) = -0,17694460, f(0,7) = 0,01375227, f(0,8) = 0,22363362, f(1,0) = 0,65809197$

---

**7.** Os dados do Exercício 6 foram gerados a partir das funções a seguir. Faça o gráfico das funções e dos polinômios interpoladores e estime o erro máximo e o erro quadrático médio para os pontos usados no gráfico.
(a) $f(x) = x \ln x$
(b) $f(x) = x^3 + 4,001x^2 + 4,002x + 1,101$
(c) $f(x) = x \cos x - 2x^2 + 3x - 1$
(d) $f(x) = \text{sen}\,(e^x - 2)$

---

**8.** Construa os polinômios interpoladores de Lagrange para as funções a seguir e encontre um limitante para o erro absoluto no intervalo $[x_0, x_n]$.
(a) $f(x) = e^{2x} \cos 3x, \quad x_0 = 0, \quad x_1 = 0,3, \quad x_2 = 0,6, \quad n = 2$

(b) $f(x) = \text{sen}(\ln x), \quad x_0 = 2,0, \quad x_1 = 2,4, \quad x_2 = 2,6, \quad n = 2$
(c) $f(x) = \ln x, \quad x_0 = 1, \quad x_1 = 1,1, \quad x_2 = 1,3, \quad x_3 = 1,4, \quad n = 3$
(d) $f(x) = \cos x + \text{sen}\, x, \quad x_0 = 0, \quad x_1 = 0,25, \quad x_2 = 0,5, \quad x_3 = 1,0, \quad n = 3$

## 4.2 Fórmula de Newton

Seja $f(x)$ definida em $x_0, x_1, \ldots, x_n$, (n+1) pontos distintos de um intervalo $[a, b]$ e $y_i = f(x_i)$, $i = 0, 1, \ldots, n$. O polinômio interpolador de Newton é dado por

$$P(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \ldots$$

$$\ldots + (x - x_0)(x - x_1)\ldots(x - x_{n-1})f[x_0, \ldots, x_n]$$

onde $f[x_0]$, $f[x_0, x_1]$,...,$f[x_0, \ldots, x_n]$ são as **diferenças divididas**, definidas como é mostrado a seguir:

**Diferença dividida de ordem 0:** $\$f[x_i] = f(x_i), i = 0, 1, \ldots, n\$$

**Diferença dividida de ordem 1:** $\$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}, i = 0, 1, \ldots, n - 1\$$

**Diferença dividida de ordem 2:** $\$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}, i = 0, 1, \ldots, n - 2\$$

$$\vdots$$

**Diferença dividida de ordem n:** $\$f[x_0, \ldots, x_n] = \frac{f[x_1, \ldots, x_n] - f[x_0, \ldots, x_{n-1}]}{x_n - x_0}\$$

A seguir é apresentado um exemplo de como obter as diferenças divididas para um conjunto de 5 pontos distintos da função $f(x) = 1/x$.

**Exemplo 4.2.1 :** Considere os pontos

| $x_i$ | 1 | 2 | 4 | 5 | 8 |
|---|---|---|---|---|---|
| $f(x_i)$ | $1,00$ | $0,50$ | $0,25$ | $10,20$ | $1,125$ |

Diferenças divididas de ordem 0:

$f[x_0] = f(x_0) = f(1) = 1.00$
$f[x_1] = f(x_1) = f(2) = 0.50$
$f[x_2] = f(x_2) = f(4) = 0.25$
$f[x_3] = f(x_3) = f(5) = 0.20$
$f[x_4] = f(x_4) = f(8) = 0.125$

Diferenças divididas de ordem 1:

$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{0.5 - 1}{2 - 1} = -0.5$
$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1} = \frac{0.25 - 0.5}{4 - 2} = -0.125$
$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2} = \frac{0.2 - 0.25}{5 - 4} = -0.05$
$f[x_3, x_4] = \frac{f[x_4] - f[x_3]}{x_4 - x_3} = \frac{0.125 - 0.2}{8 - 5} = -0.025$

Diferenças divididas de ordem 2:

$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{-0.125 - (-0.5)}{4 - 1} = 0.125$
$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1} = \frac{-0.05 - (-0.125)}{5 - 2} = 0.025$
$f[x_2, x_3, x_4] = \frac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2} = \frac{-0.025 - (-0.05)}{8 - 4} = 0.00625$

Diferenças divididas de ordem 3:

$f[x_0, x_1, x_2, x_3] = \frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0} = \frac{0.025-0.125}{5-1} = -0.025$

$f[x_1, x_2, x_3, x_4] = \frac{f[x_2,x_3,x_4]-f[x_1,x_2,x_3]}{x_4-x_1} = \frac{0.00625-0.025}{5-2} = -0.003125$

Diferenças divididas de ordem 4:

$f[x_0, x_1, x_2, x_3, x_4] = \frac{f[x_1,x_2,x_3,x_4]-f[x_0,x_1,x_2,x_3]}{x_4-x_0} = \frac{-0.003125-(-0.025)}{8-1} = 0.003125$

Para facilitar o cálculo dessas diferencas divididas, é conveniente construir uma tabela em que cada coluna, a partir da terceira, é obtida usando as diferencas entre elementos consecutivos da coluna anterior e valores correspondentes da coluna de $x_i$. Para o exemplo anterior a tebela ficaria como é mostrado a seguir:

| $x_i$ | $f[x_i]$ | $f[x_i, x_{i+1}]$ | $f[x_i, \dots, x_{i+2}]$ | $f[x_i, \dots, x_{i+3}]$ | $f[x_i, \dots, x_{i+4}]$ |
|---|---|---|---|---|---|
| 1 | 1 | $-0,5$ | $0,125$ | $-0,025$ | $0,003125$ |
| 2 | $0,5$ | $-0,125$ | $0,025$ | $-0,003125$ | |
| 4 | $0,25$ | $-0,05$ | $0,00625$ | | |
| 5 | $0,2$ | $-0,025$ | | | |
| 8 | $0,125$ | | | | |

Uma vez calculadas as diferenças divididas, usa-se o seguinte método recursivo para avaliar o polinômio em um determinado valor de $x$

$P_0(x) = f[x_0, \dots, x_n]$

$P_1(x) = f[x_0, \dots, x_{n-1}] + (x - x_{n-1})P_0(x)$

$P_2(x) = f[x_0, \dots, x_{n-2}] + (x - x_{n-2})P_1(x)$

$\vdots$

$P_n(x) = f[x_0] + (x - x_0)P_n(x)$

**Exemplo 4.2.2** Considere a função $f(x) = e^x + \text{sen}(x)$ tabelada nos pontos $x_0 = 0,0$, $x_1 = 0,5$ e $x_2 = 1,0$. Determine o polinômio interpolador usando a fórmula de Newton e avalie $f(0,7)$.

Vamos fazer isso, inicialmente de um modo "não vetorizado".

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
# Definindo a lista de pontos a serem interpolados
X  = [0., 0.5, 1.0]

# Definindo a função a ser interpolada
f = lambda x: np.exp(x)+np.sin(x)

# Calculando os valores de f(xi)
Y = f(X)

print (np.around(X,4))
print (np.around(Y,4))
```

```
[0.  0.5 1. ]
[1.     2.1281 3.5598]
```

```
# Criando uma lista de listas para a tabela de diferencas divididas
dd=[Y] #ordem 0

# Gerando a tabela de dif. div. a partir da ordem 1 em diante
for o in range(1, len(X)):
    dd.append([])   # Adiciona uma lista vazia para armazenar as dds de ordem 1
    for k in range(0, len(X)-o, 1):
        valor = (dd[o-1][k+1]-dd[o-1][k])/(X[k+o]-X[k])
        dd[o].append(valor)
    print (np.around(dd[o],4))
```

```
[2.2563 2.8632]
[0.6069]
```

```
def produtorio(x,n):
    prod = 1.
    for i in range(n):
        prod = prod * (x-X[i])
    return prod
```

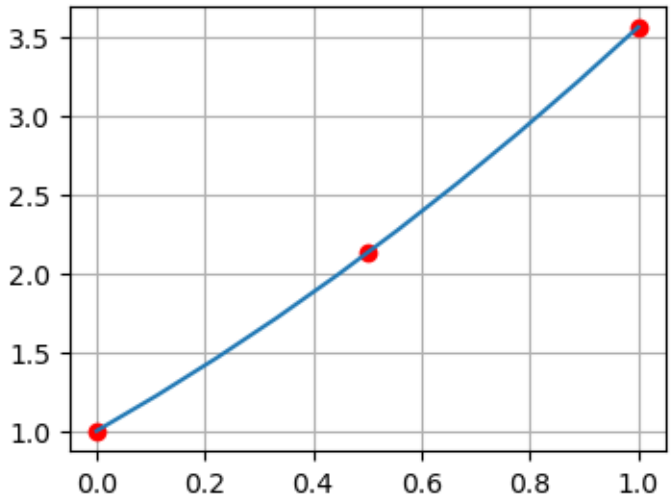
```
def calculaP(x):
    soma = dd[0][0]
    for i in range(1,len(X)):
        soma = soma + produtorio(x,i)*dd[i][0]
    return soma
```

```
print ("P(0.7) =", calculaP(0.7))

# Cria a lista de pontos e calcula os valores para p plot
Xp = np.linspace(X[0], X[-1], num=10)
Yp = []
for x in Xp:
    y = calculaP(x)
    Yp.append(y)


plt.plot(X, Y, 'ro', Xp, Yp,'-', )
plt.grid()
plt.show()
```

```
P(0.7) = 2.664374107530382
```




**Melhorando (vetorizando) nosso algoritmo**

Observe que as colunas da tabela de diferenças divididas mostradas no Exemplo 4.2.1 podem ser calculadas de forma bastante prática usando operações vetoriais entre `numpy.arrays`, por exemplo as diferencas divididas de ordem 1 podem ser obtidas fazendo `(f[1:]-f[0:-1])/(x[1:]-x[0:-1])` sendo `x` e `f` *arrays* contendo os dados de $x_i$ e $f(x_i)$ conforme é mostrado a seguir:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.array([1.,2.,4.,5.,8.])
f = 1/x
(f[1:]-f[0:-1])/(x[1:]-x[0:-1])
```

```
array([-0.5  , -0.125, -0.05 , -0.025])
```

fazendo um laço de repetição para as linhas seguintes podemos obter a tabela de diferenças divididas como é mostrado abaixo:

```
n = len(x)
for i in range(1,n+1):
    print (f)
    f= (f[1:]-f[0:-1])/(x[i:n]-x[0:n-i])
```

```
[1.    0.5   0.25  0.2   0.125]
[-0.5   -0.125 -0.05  -0.025]
[0.125   0.025   0.00625]
[-0.025    -0.003125]
[0.003125]
```

Por fim, como precisamos apenas do primeiro elemento de cada linha acima, bastaria armazenar os resultados em um

único `array`como é mostrado abaixo

```
f = 1/x
for i in range(1,n):
    f[i:n] = (f[i:n] - f[i-1])/(x[i:n] - x[i-1])
print (f)
```

```
[ 1.       -0.5       0.125    -0.025     0.003125]
```

Vamos agora melhorar nosso algoritmo usando a vetorização do código.

**Exemplo 4.2.3:**

Seja a função $f(x) = e^x cos(x)\sqrt{x}$ tabelada nos pontos $x_0 = 0,0$, $x_1 = 1,0$, $x_2 = 2,3$, $x_3 = 2,5$ e $x_4 = 4,0$. Vamos usar a fórmula de interpolatória de Newton para obter o gráfico do polinômio que interpola esses pontos calculando o valor numérico do poliômio em conjunto discreto de pontos. Também vamos obter uma forma analítica para esse polinônio usando Python.

```
# Definindo a lista de pontos a serem interpolados
func = lambda x: np.exp(x)*np.cos(x)*np.sqrt(x)
xi = np.array([ 0, 1.0, 2.3, 3.5, 4.0])
yi = func(xi)
```

```
# uma função para calcular as diferenças divididas
def difdiv(x,y):
    m = len(x)
    dd = y.copy()
    for k in range(1,m):
        dd[k:m]=(dd[k:m]-dd[k-1])/(x[k:m]-x[k-1])
    return dd
```

```
f = difdiv(xi,yi)
print(f)
```

```
[ 0.          1.46869394 -4.50049655 -2.26454299  2.25023479]
```

```
def calculaP(f,xi,x):
    n = len(xi) - 1
    p = f[n]
    for i in range(1,n+1):
        p = f[n-i] + (x -xi[n-i])*p
    return p
```

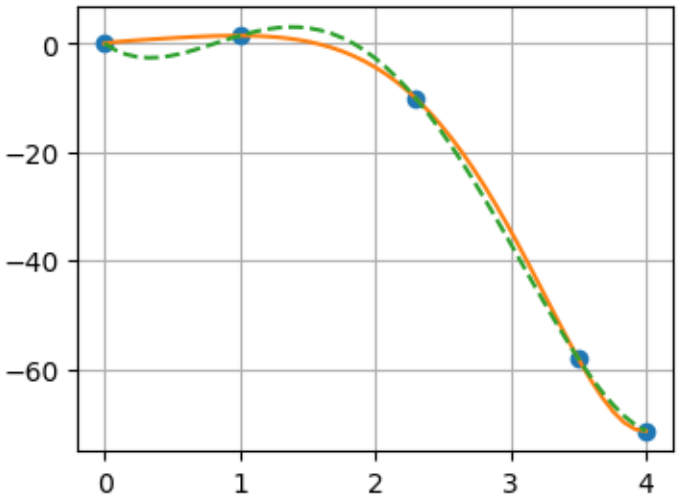
```
# Cria a lista de pontos para o plot
xp = np.linspace(0, 4, num=51)
yp = []
for x in xp:
```

(continua na próxima página)

(continuação da página anterior)

```
    yp.append(calculaP(f,xi,x))

# Plota
plt.plot(xi, yi, 'o',
         xp, func(xp),'-',
         xp, yp,'--')
plt.grid()
plt.show()
```




**Exemplo 4.2.4:**

Considere a função $f(x) = \frac{x}{(x+1)}$ tabelada nos pontos $x_0 = 0,0$, $x_1 = 1,0$ e $x_2 = 2,0$. Determine o polinômio interpolador usando a fórmula de Newton e avalie $f(1.3)$.

```
xi = np.array([0., 1.0, 2.0])
func = lambda x: np.divide(x, x + 1)
yi = func(xi)
```

```
f = difdiv(xi,yi)
print (f)
```

```
[ 0.          0.5         -0.16666667]
```

```
print ("P(1.3) =", calculaP(f,xi,1.3))

xp = np.linspace(-0.5, 3)
yp = []

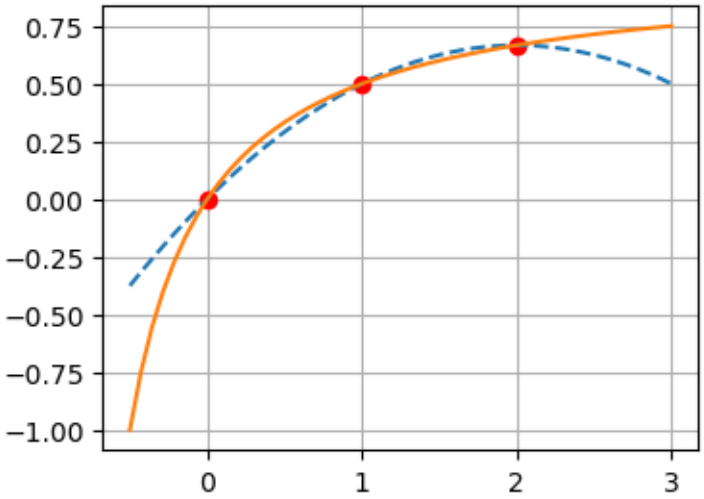
for x in xp:
    yp.append(calculaP(f,xi,x))
```

(continua na próxima página)

(continuação da página anterior)

```
plt.plot(xi, yi, "ro",
         xp, yp, "--",
         xp, func(xp),"-")
plt.grid()
plt.show()
```

```
P(1.3) = 0.5850000000000001
```



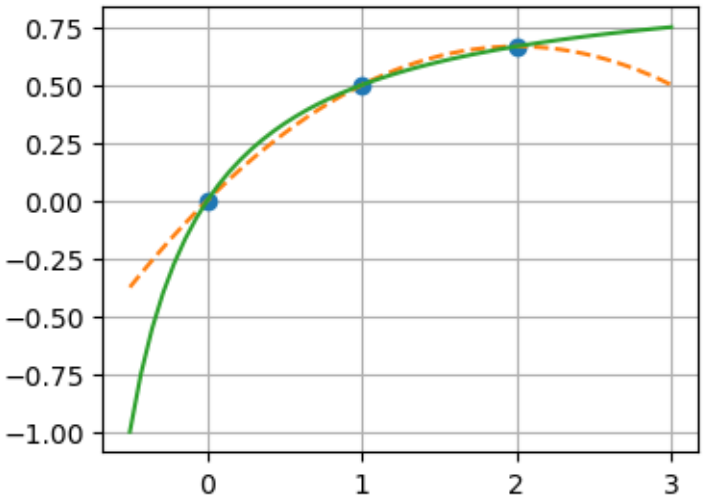

A biblioteca SciPy para computação científica em Python possui diversos algoritmos para interpolação uni e multi-dimensional, incluindo *splines*, métodos de Lagrange e baseados em série de Taylor. Veja como fica fácil com o exemplo abaixo.

```
from scipy.interpolate import lagrange

a = lagrange(xi, yi)
print(a.coef)
```

```
[-0.16666667  0.66666667  0.        ]
```

```
plt.plot(xi, yi, 'o',
        xp, np.polyval(a,xp),'--',
        xp, func(xp),"-")
plt.grid()
plt.show()
```




Para saber mais, pesquise sobre `scipy.interpolate` na *internet*.

## Interpolação Inversa

Denominamos interpolação inversa quando, conhecidos os valores de uma função $f(x)$ definida em $(n+1)$ pontos distintos $x_i$ ,$i = 0, \dots, n$, necessitamos calcular o valor numérico da variável $x$ correspondente a um valor $y = f(x)$ conhecido inicialmente.

Supondo que a função inversa de $f(x)$ exista no intervalo de interpolação, a qual denotamos por $f^{-1}(x)$, então para os pontos tabelados $y_i = f(x_i)$, $i = 0, \dots, n$ temos $x_i = f^{-1}(y_i)$, e o valor desejado $x$ tal que $y = f(x)$ é obtido por $x = f^{-1}(y)$.

Assim, simplesmente trocamos na tabela de dados os valores de $x$ e $f(x)$ e fazemos a interpolação de $f^{-1}(x)$, como visto anteriormente.

Lembramos, ainda, que a função inversa $x = f^{-1}(y)$ existe e é única se $f(x)$ é contínua e monótona crescente ou decrescente no intervalo de interpolação.

**Exemplo 4.2.5:**

Considere uma função $f(x)$ tabelada, como segue:

| $x_i$ | 0 | 1 | 2 |
|---|---|---|---|
| $f(x_i)$ | $1,31$ | $3,51$ | $3,78$ |

Usando interpolação inversa, determine $x$ tal que $f(x) = 3,63$.

```
yi = np.array([0.00, 1.00, 2.00])
xi = np.array([1.31, 3.51, 3.78])


def evalPoly(dd,xi,x):
    n = len(xi) - 1
    # Degree of polynomial
    p = dd[n]
    for k in range(1,n+1):
```

(continua na próxima página)

(continuação da página anterior)

```
        p = dd[n-k] + (x -xi[n-k])*p
    return p

def coeffts(xi,yi):
    m = len(xi)
    # Number of data points
    dd = yi.copy()

    for k in range(1,m):
        dd[k:m] = (dd[k:m] - dd[k-1])/(xi[k:m] - xi[k-1])
    return dd


dd = coeffts(xi,yi)
print ('dd',dd)

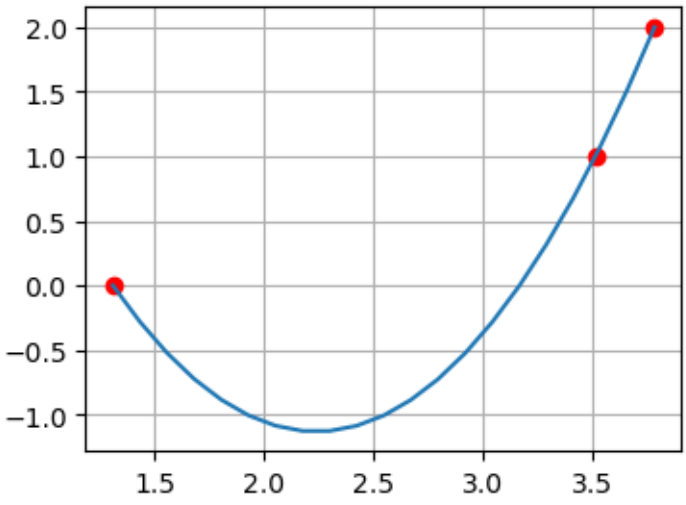
print("P(3.63)=",evalPoly(dd,xi,3.63))

Xplot = np.linspace(xi[0], xi[-1], 21)
Yplot = []

for x in Xplot:
    Yplot.append(evalPoly(dd,xi,x))

plt.plot(xi, yi, "ro", Xplot, Yplot, "-")
plt.grid()
plt.show()
```

```
dd [0.         0.45454545 1.31544868]
P(3.63)= 1.4207663681347895
```



## Exercícios

**1.** Considere os dados,

| $x$ | $1,6$ | $1,9$ | $2,5$ | $3,2$ | $3,9$ | $4,5$ |
|---|---|---|---|---|---|---|
| $f(x)$ | $2,0$ | $8,0$ | $14,0$ | $15,5$ | $9,0$ | $2,1$ |

(a) Calcule $f(2,9)$ usando polinômios interpoladores de Newton de segundo a quarto graus. Escolha a sequência de pontos que julgar mais adequada.
(b) Utilize a expressão

$$R_n \cong (x-x_0)(x-x_1)...(x-x_n)f[x_0,x_1,...,x_n,x]$$

para fazer uma estimativa do erro em cada uma das previsões.
(c) Para o polinômio que forneceu o melhor resultado, use a fórmula de Lagrange com os mesmos pontos e compare o resultado. Comente o que você observou.

---

**2.** O volume específico de um vapor superaquecido está listado nas tabelas de vapor para diversas temperaturas. por exemplo, na pressão absoluta de 3000 lb/pol$^2$:

| $T,^\circ F$ | $700$ | $720$ | $740$ | $760$ | $780$ |
|---|---|---|---|---|---|
| $v, ps^3/lb_m$ | $0,0977$ | $0,12184$ | $0,14060$ | $0,15509$ | $0,16643$ |

Use interpolação para obter uma tabela que forneça $v$ a cada 5° entre as temperaturas 720° e 760° usando a fórmula de Lagrange e a fórmula de Newton. Compare os resultados e comente o que você observou.

---

**3.** Após serem efetuadas medições num gerador de corrente contínua, foram obtidos os seguintes valores indicados por um voltímetro e um amperímetro.

| $I(A)$ | $1,58$ | $1,75$ | $2,10$ | $2,20$ | $2,50$ | $3,50$ | $3,75$ | $4,25$ | $4,50$ | $4,90$ | $3,12$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $V(v)$ | $210$ | $165$ | $154$ | $120$ | $85$ | $50$ | $73$ | $76$ | $110$ | $128$ | $60$ |

Faça um gráfico dos dados.(a) Ajuste os dados por polinômio de grau adequado. (b) Estime o valor a ser obtido no voltímetro quando o amperímetro estiver marcando $4.0A$.

---

**4.** Considere a função

$$f(x) = \frac{3,21}{0,73+9,81x^2}$$

(a) Aproxime o valor de $f(1,78)$ usando um polinômio interpolador de grau 3 no intervalo $[1,2]$ e compare com o valor da função. Mostre o gráfico do polinômio e da função nesse intervalo. (b) Repita o item anterior, mas dessa vez utilize um polinômio de grau 10 no intervalo $[-2,2]$ e comente o que você observou.
(c)Pesquise e responda o que é “Fenômeno de Runge”.

---

**5.** Dada a tabela a seguir, de valores de uma função $f$,

| $x$ | 0.15 | 0.17 | 0.19 | 0.21 | 0.23 | 0.25 | 0.27 |
|---|---|---|---|---|---|---|---|
| $f(x)$ | $0,1761$ | $0,2304$ | $0,2788$ | $0,3222$ | $0,3617$ | $0,3979$ | $0,4314$ |

(a) Utilize todos os pontos para obter uma estimativa para $f(0,22)$ usando um polinômio interpolador com a fórmula de Newton.
(b) Plote o gráfico do polinômio obtido juntamente com os pontos tabelados para verificar o resultado da interpolação.
(c) Escolha 4 pontos e estime $f(0,22)$ utilizando um polinomio de terceiro grau. Plote o gráfico e compare com o resultado do item anterior. Comente.

---

**6.** (Fonte) Use interpolação de Newton para construir polinômios interpoladores de grau três para os pontos espaçados de forma desigual fornecidos na tabela a seguir:

| $x$ | $-0,1$ | $0,0$ | $0,2$ | $0,3$ |
|---|---|---|---|---|
| $f(x)$ | $5,30$ | $2,00$ | $3,19$ | $1,00$ |

Acrescente $f(0,35) = 0,97260$ à tabela e construa o polinômio interpolador de grau quatro.

---

## 4.3. Erros na interpolação

### Um limitante superior para o erro usando a fórmula de Lagrange

(Burden,2016) Se $x_0, x_1, \ldots, x_n$ são $n+1$ números distintos, e $f$ é uma função cujos valores são dados por esses números, então existe um único polinômio $P(x)$ de grau no máximo $n$

$$f(x_k) = P(x_k), \quad \text{para cada } k = 0, 1, \ldots, n.$$

Esse polinômio é dado por

$$P(x) = f(x_0)\, L_{n,0}(x) + \cdots + f(x_n)\, L_{n,n}(x) = \sum_{k=0}^{n} f(x_k)\, L_{n,k}(x),$$

onde, para cada $k = 0, 1, \ldots, n$,

$$\begin{aligned} L_{n,k}(x) &= \frac{(x-x_0)(x-x_1)\cdots(x-x_{k-1})(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_0)(x_k-x_1)\cdots(x_k-x_{k-1})(x_k-x_{k+1})\cdots(x_k-x_n)} \\ &= \prod_{\substack{i=0 \\ i\neq k}}^{n} \frac{(x-x_i)}{(x_k-x_i)} \end{aligned}$$

Um esboço do gráfico de um típico $L_{n,k}$ quando $n$ é par é mostrado na Figura 4.3.1.

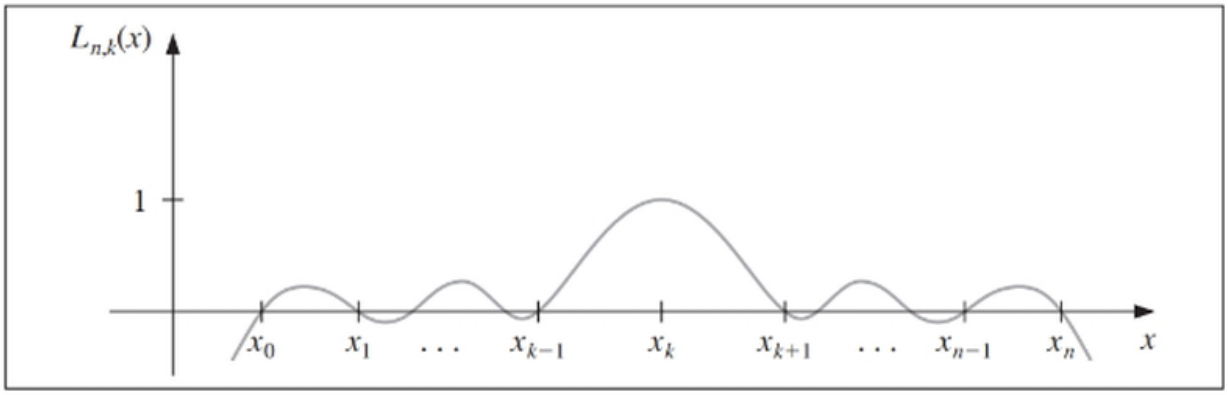



Figura 4.3.1 - Gráfico de um típico $L_{n,k}$ quando $n$ é par (Burden,2016).

Já vimos que essa fórmula é conhecida como *polinômio interpolados de Lagrange*.

O próximo passo é calcular um resto ou limitante para o erro envolvido na aproximação de uma função por um polinômio interpolador.

Suponha que $x_0, x_1, \ldots, x_n$ sejam números distintos no intervalo $[a, b]$ e que $f \in C^{n+1}[a, b]$. Então, para cada $x$ em $[a, b]$, existe umnúmero $\xi(x)$ (geralmente desconhecido) entre $\min\{x_0, x_1, \ldots, x_n\}$, e o $\max\{x_0, x_1, \ldots, x_n\}$ e consequentemente em $(a, b)$, com

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n)$$

onde $P(x)$ é o polinômio interpolador $P(x) = \sum_{k=0}^{n} f(x_k) L_{n,k}(x)$,

A demonstração desse resultado pode ser vista em Burden (2016, p. 138).

**Exemplo 4.3.1:** Vamos usar os pontos com $x_0 = 2, x_1 = 2,75$ e $x_2 = 4$ para determinar o polinômio de Lagrange de grau 2 que aproxima a função $f(x) = \frac{1}{x}$. Em seguida vamos determinar o erro máximo quando o polinômio for usado para aproximar $f(x)$ com $x \in [2, 4]$.

*Solução:* Primeiramente determinamos os coeficientes polinomiais $L_0(x), L_1(x)$ e $L_2(x)$.

$$L_0(x) = \frac{(x - 2,75)(x - 4)}{(2 - 2,75)(2 - 4)} = \frac{2}{3}(x - 2,75)(x - 4)$$
$$L_1(x) = \frac{(x - 2)(x - 4)}{(2,75 - 2)(2,75 - 4)} = -\frac{16}{15}(x - 2)(x - 4)$$

$$L_2(x) = \frac{(x - 2)(x - 2,75)}{(4 - 2)(4 - 2,75)} = \frac{2}{5}(x - 2)(x - 2,75)$$

Além disso, tem-se $f(x_0) = f(2) = 1/2, f(x_1) = f(2,75) = 4/11$ e $f(x^2) = f(4) = 1/4$, de modo que

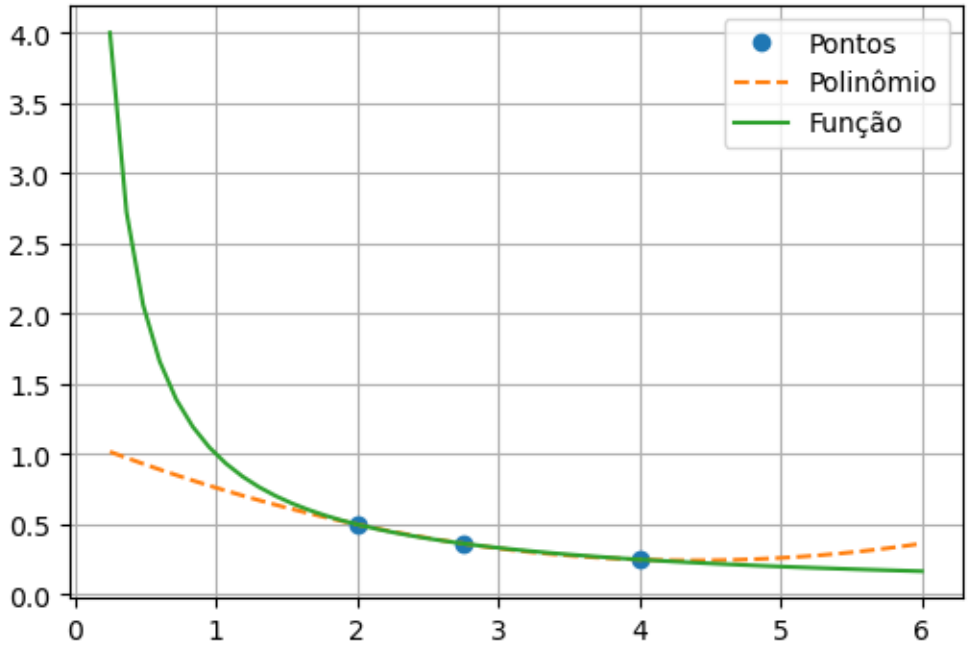
$$\begin{aligned} P(x) &= \sum_{k=0}^{2} f(x_k) L_k(x) \\ &= \frac{1}{3}(x - 2,75)(x - 4) - \frac{64}{165}(x - 2)(x - 4) + \frac{1}{10}(x - 2)(x - 2,75) \\ &= \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44} \end{aligned}$$

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [6, 4]
import numpy as np
```

```
xi = np.array([2, 2.75, 4])
yi =1/xi

x = np.linspace(0.25,6)
y = (1/22)*x**2 - (35/88)*x + 49/44

plt.plot(xi,yi,'o', label='Pontos')
plt.plot(x,y,'--', label='Polinômio')
plt.plot(x,1/x,'-', label='Função')
plt.legend()
plt.grid()
```




Para estimar o erro máximo, vamos primeiramente obter as derivadas de $f(x) = x^{-1}$ fazendo

$$f'(x) = -x^{-2}, \quad f''(x) = 2x^{-3} \quad \text{e} \quad f'''(x) = -6x^{-4}$$

Assim, a fórmula do erro fornece

$$\frac{f'''(\xi(x))}{3!}(x - x_0)(x - x_1)(x - x_2) = -(\xi(x))^{-4}(x - 2)(x - 2,75)(x - 4), \qquad \text{para } \xi(x) \text{ em } (2, 4)$$

O valor máximo de $(\xi(x))^{-4}$ no intervalo é $2^{-4} = 1/16$. Precisamos agora determinar o valor máximo neste intervalo do valor absoluto do polinômio

$$g(x) = (x - 2)(x - 2,75)(x - 4) = x^3 - \frac{35}{4}x^2 + \frac{49}{2}x - 22.$$

Já que

$$\frac{d}{dx}\left(x^3 - \frac{35}{4}x^2 + \frac{49}{2}x - 22\right) = 3x^2 - \frac{35}{2}x + \frac{49}{2} = \frac{1}{2}(3x-7)(2x-7),$$

os pontos críticos ocorrem em

$$x = \frac{7}{3}, \text{com } g\left(\frac{7}{3}\right) = \frac{25}{108}, \quad \text{e} \quad x = \frac{7}{2}, \text{com } g\left(\frac{7}{2}\right) = -\frac{9}{16}.$$

Assim, o erro máximo é

$$\frac{f'''(\xi(x))}{3!}\left|(x-x_0)(x-x_1)(x-x_2)\right| \leq \frac{1}{16}\left|-\frac{9}{16}\right| = \frac{9}{256} \approx 0,03515625.$$

**Exemplo 4.3.2:** Considerando o polinômio de Lagrange que aproxima a função $f(x) = 1/x$ no intervalo $[2, 4]$ pelos pontos $x_0 = 2, x_1 = 2,75$ e $x_2 = 4$. Vamos agora usar a função `calculaP` para calcular o valor numérico do polinômio **sem obter a expressão do polinômio** e verificar o erro em um conjunto de 21 pontos igualmente espaçados o londo do intervalo. Vamos verificar se o erro não ultrapassa o valor estimado no Exemplo 4.3.1.

```
def calculaP(x,X,Y):
    valor = 0
    for k in range(len(X)):
        lk = 1.0
        for i in range(len(X)):
            if k != i:
                lk = lk*(x - X[i])/(X[k]-X[i])
        valor = valor + Y[k]*lk
    return valor
```

```
xp = np.linspace(2,4,21)
yp = 1/xp
pi = []
for x in xp:
    pi.append(calculaP(x, xi, yi))
pi = np.array(pi)
```

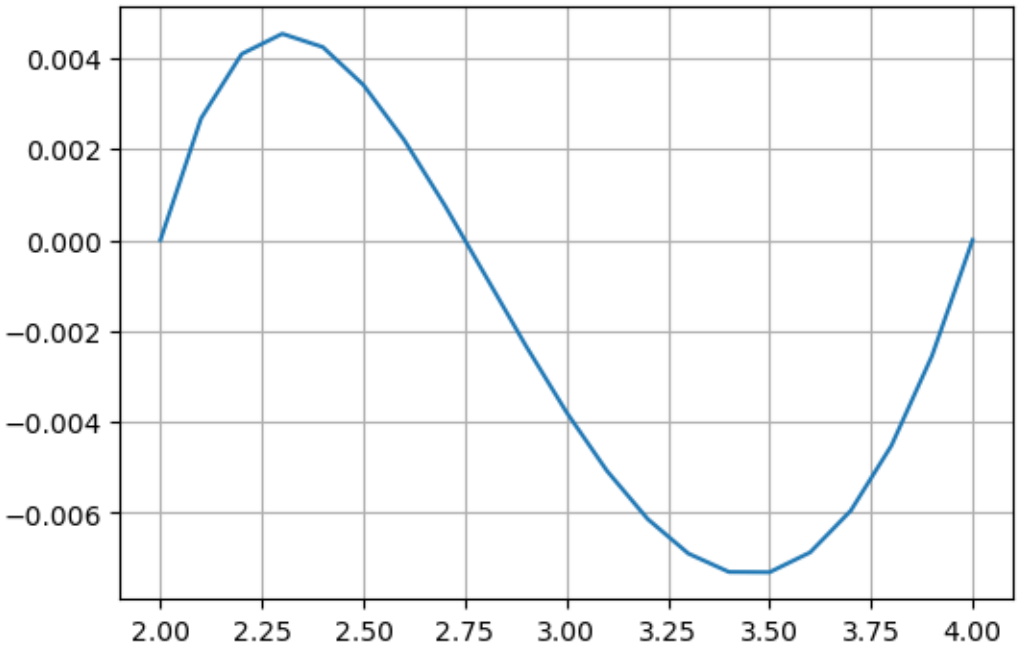
```
erro = pi-yp
print(erro)
```

```
[ 0.          0.00267316  0.00409091  0.00453557  0.00424242  0.00340909
  0.0022028   0.00076599 -0.00077922 -0.00232759 -0.00378788 -0.00508065
 -0.00613636 -0.00689394 -0.00729947 -0.00730519 -0.00686869 -0.00595209
 -0.00452153 -0.00254662  0.        ]
```

```
abs(erro).max()
```

```
0.007305194805194759
```

```
plt.plot(xp, erro)
#plt.plot(xp, yp)
#plt.plot(xp,pi)
plt.grid()
```




## Uma estimativa para o erro usando a fórmula de Newton de diferenças divididas

O polinômio interpolador de Newton é dado por

$$P(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + ...$$

$$... + (x - x_0)(x - x_1)...(x - x_{n-1})f[x_0, ..., x_n]$$

onde $f[x_0]$, $f[x_0, x_1]$,...,$f[x_0, ..., x_n]$ são as **diferenças divididas**

Uma tabela de diferenças divididas é mostrado na Figura 4.3.2.

| $x$ | $f(x)$ | Primeiras diferenças divididas | Segundas diferenças divididas | Terceiras diferenças divididas |
|---|---|---|---|---|
| $x_0$ | $f[x_0]$ | | | |
| | | $f[x_0,x_1]=\frac{f[x_1]-f[x_0]}{x_1-x_0}$ | | |
| $x_1$ | $f[x_1]$ | | $f[x_0,x_1,x_2]=\frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$ | |
| | | $f[x_1,x_2]=\frac{f[x_2]-f[x_1]}{x_2-x_1}$ | | $f[x_0,x_1,x_2,x_3]=\frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0}$ |
| $x_2$ | $f[x_2]$ | | $f[x_1,x_2,x_3]=\frac{f[x_2,x_3]-f[x_1,x_2]}{x_3-x_1}$ | |
| | | $f[x_2,x_3]=\frac{f[x_3]-f[x_2]}{x_3-x_2}$ | | $f[x_1,x_2,x_3,x_4]=\frac{f[x_2,x_3,x_4]-f[x_1,x_2,x_3]}{x_4-x_1}$ |
| $x_3$ | $f[x_3]$ | | $f[x_2,x_3,x_4]=\frac{f[x_3,x_4]-f[x_2,x_3]}{x_4-x_2}$ | |
| | | $f[x_3,x_4]=\frac{f[x_4]-f[x_3]}{x_4-x_3}$ | | $f[x_2,x_3,x_4,x_5]=\frac{f[x_3,x_4,x_5]-f[x_2,x_3,x_4]}{x_5-x_2}$ |
| $x_4$ | $f[x_4]$ | | $f[x_3,x_4,x_5]=\frac{f[x_4,x_5]-f[x_3,x_4]}{x_5-x_3}$ | |
| | | $f[x_4,x_5]=\frac{f[x_5]-f[x_4]}{x_5-x_4}$ | | |
| $x_5$ | $f[x_5]$ | | | |

Figura 4.3.2 - Tabela de diferenças divididas de Newton (Fonte: Chapra, 2016)

(Chapra, 2016, p. 455) Para um polinômio interpolador de grau $n$, uma fórmula para o erro é dada por

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x-x_0)(x-x_1)\ldots(x-x_n)$$

onde $\xi$ é algum ponto no intervalo contendo a variável e os dados. Para essa fórmula ser útil, a função em questão deve ser conhecida e $(n+1)$ vezes diferenciável. **Em geral, esse não é o caso**. Felizmente, está disponível uma formulação alternativa que não requer conhecimento anterior da função. Em vez disso, ela usa diferenças divididas finitas para aproximar a $(n+1)$-ésima derivada, $\$R_n = f[x, x_n, x_{n-1}, \ldots, x_0](x-x_0)(x-x_1)\cdots(x-x_n)\$$

onde $f[x, x_n, x_{n-1}, \ldots, x_0]$ é a $(n+1)$-ésima diferença dividida finita.

Como esta expressão contém a incógnita $f(x)$, ela não determina o erro. Entretanto, se estiver disponível um ponto dado adicional $f(x_{n+1})$, a equação pode ser usada para estimar o erro, como em

$$R_n \cong f[x_{n+1}, x_n, x_{n-1}, \ldots, x_0](x-x_0)(x-x_1)\cdots(x-x_n)$$

**Exemplo 4.3.3:** (Chapra, 2016, p. 451) Ajuste um polinômio de segundo grau aos três pontos da função $f(x) = \ln(x)$ dados, e use o polinômio para estimar $\ln(2)$. Calcule o erro absoluto.

Pontos: $\$ \begin{array}{ll} x_0 = 1 & f(x_0) = 0 \\ x_1 = 4 & f(x_1) = 1,386294 \\ x_2 = 6 & f(x_2) = 1,791759 \end{array} \$$

$f[x_0] = 0$

$f[x_0, x_1] = \frac{1,386294-0}{4-1} = 0,4620981$

$f[x_0, x_1, x_2] = \frac{\frac{1,791759-1,386294}{6-4} - 0,4620981}{6-1} = -0,0518731$

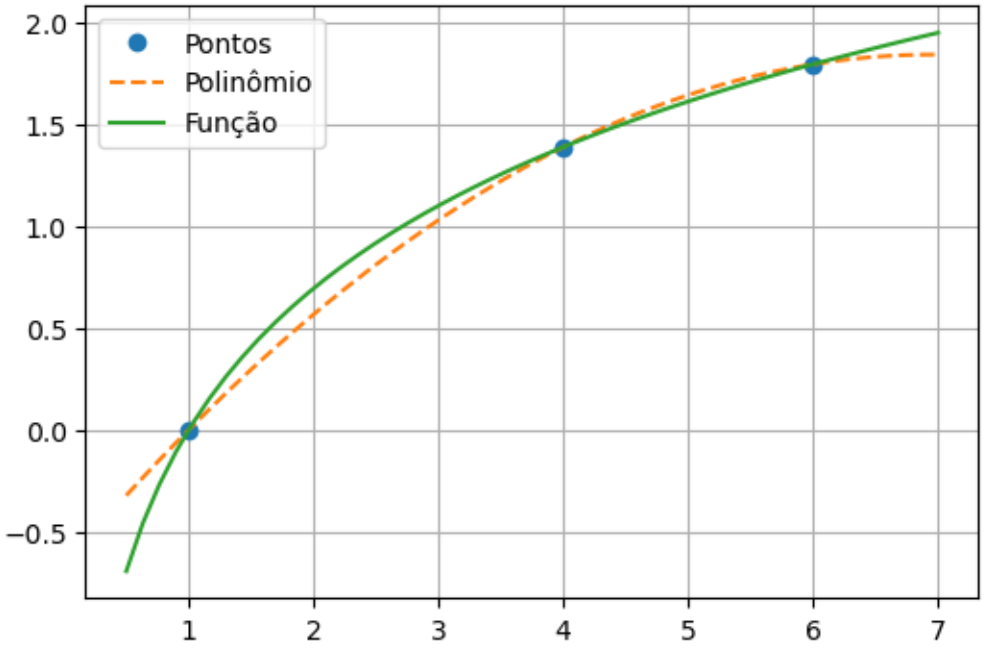
Substituindo os os valores obtidos para as diferenças divididas na fórmula de Newton, obtemos o polinômio

$$\begin{aligned} P_2(x) = &\ 0 + 0,4620981(x-1) - 0,0518731(x-1)(x-4) \\ = &\ -0.0518731x^2 + 0.7214636x - 0.6695905 \end{aligned}$$

```
xi = np.array([1, 4, 6])
yi = np.log(xi)

x = np.linspace(0.5,7)
y = -0.0518731*x**2 + 0.7214636*x - 0.6695905

plt.plot(xi,yi,'o', label='Pontos')
plt.plot(x,y,'--', label='Polinômio')
plt.plot(x,np.log(x),'-', label='Função')
plt.legend()
plt.grid()
```




```
x = 2
p2 = -0.0518731*x**2 + 0.7214636*x - 0.6695905
print('P(2) =',p2)
```

```
P(2) = 0.5658443
```

```
print('Erro absoluto =',abs(p2-np.log(2)))
```

```
Erro absoluto = 0.12730288055994532
```

Agora, adicionando um quarto ponto $x_3 = 5$ , $f(x_3) = 1,609438$, vamos obter uma estimativa do erro para o polinômio interpolador de segundo grau fazendo

$$R_2 = f\left[x_3, x_2, x_1, x_0\right]\left(x - x_0\right)\left(x - x_1\right)\left(x - x_2\right)$$

Para isso vamos precisar calcular as diferenças divididas incluindo o ponto $x_3$:

```
def difdiv(x,y):
    m = len(x)
    dd = y.copy()
    for k in range(1,m):
        dd[k:m]=(dd[k:m]-dd[k-1])/(x[k:m]-x[k-1])
    return dd
```

```
xi = np.array([1, 4, 5, 6])
yi = np.log(xi)
print(difdiv(xi, yi))
```

```
[ 0.          0.46209812 -0.05973864  0.00786553]
```

Assim, $f[x_0, x_1, x_2, x_3] = 0,00786553$. Então podemos escrever

$$R_2 = 0,007865529(x-1)(x-4)(x-6)$$

onde o valor da diferença dividida finita de terceira ordem é como calculado usando a função `difdiv` desenvolvida anteriormente. Essa relação pode ser calculada em $x = 2$ por

$$R_2 = 0,007865529(2-1)(2-4)(2-6) = 0,0629242$$

que é da mesma ordem de grandeza que o erro verdadeiro (Chapra, 2016, p. 455) .

Por exemplo, arredondando com uma casa decimal obtemos:

```
np.round(0.0629242,1)
```

```
0.1
```

```
np.round(0.12730288055994532,1)
```

```
0.1
```

## Exercícios

**1.** Para as funções $f(x)$ dadas, construa polinômios interpoladores de graus entre 2 e 5 para determinar uma aproximação de $f(0,45)$ e encontre o erro absoluto. Encontre uma estimativa para o erro.

a) $f(x) = \cos x$

b) $f(x) = \sqrt{1+x}$

c) $f(x) = \ln(x+1)$

d) $f(x) = \text{tg}\, x$

**2.** Para as funções $f(x)$, construa polinômios interpoladores de graus entre 2 e 5 para determinar uma aproximação de $f(1,4)$, e encontre o erro absoluto. Encontre uma estimativa para o erro.
a) $f(x) = \operatorname{sen} \pi x$
b) $f(x) = \sqrt[3]{x-1}$
c) $f(x) = \log_{10}(3x-1)$
d) $f(x) = e^{2x} - x$

**3.** (Burden, 2016, p. 125) Utilize polinômios interpoladores de Lagrange de graus dois e três adequados para aproximar cada um dos seguintes:
a) $f(8,4)$ se $f(8,1) = 16,94410, f(8,3) = 17,56492, f(8,6) = 18,50515, f(8,7) = 18,82091$
b) $f\left(-\frac{1}{3}\right)$ se $f(-0,75) = -0,07181250, f(-0,5) = -0,02475000, f(-0,25) = 0,33493750, f(0) = 1,10100000$
c) $f(0,25)$ se $f(0,1) = 0,62049958, f(0,2) = -0,28398668, f(0,3) = 0,00660095, f(0,4) = 0,24842440$
d) $f(0,9)$ se $f(0,6) = -0,17694460, f(0,7) = 0,01375227, f(0,8) = 0,22363362, f(1,0) = 0,65809197$
e) $f(0,43)$ se $f(0) = 1, f(0,25) = 1,64872, f(0,5) = 2,71828, f(0,75) = 4,48169$
f) $f(0)$ se $f(-0,5) = 1,93750, f(-0,25) = 1,332303, f(0,25) = 0,800781, f(0,5) = 0,687500$
g) $f(0,18)$ se $f(0,1) = -0,29004986, f(0,2) = -0,56079734, f(0,3) = -0,81401972, f(0,4) = -1,0526302$
h) $f(0,25)$ se $f(-1) = 0,86199480, f(-0,5) = 0,95802009, f(0) = 1,0986123, f(0,5) = 1,2943767$

**4.** (Burden, 2016, p. 125) Os dados do Exercício 3 foram gerados a partir das funções a seguir. Use a fórmula de erro para encontrar um limitante para o erro e compare o limitante ao erro real para os polinômiod de grau 2
a) $f(x) = x \ln x$
b) $f(x) = x^3 + 4,001x^2 + 4,002x + 1,101$
c) $f(x) = x \cos x - 2x^2 + 3x - 1$
d) $f(x) = \operatorname{sen}(e^x - 2)$
e) $f(x) = e^{2x}$
f) $f(x) = x^4 - x^3 + x^2 - x + 1$
g) $f(x) = x^2 \cos x - 3x$
h) $f(x) = \ln(e^x + 2)$

# 5. Ajuste por Mínimos Quadrados

Quando os dados experimentais contêm ruído ou erros de medição, não é adequado interpolar — é preferível encontrar uma curva que represente a tendência geral dos dados minimizando os desvios. O método dos mínimos quadrados é a ferramenta padrão para esse tipo de ajuste.

### Objetivos de aprendizagem

- Ajustar uma reta ou polinômio a dados pelo critério dos mínimos quadrados
- Resolver o sistema normal para obter os coeficientes do ajuste
- Aplicar linearizações para ajustar modelos não lineares
- Avaliar a qualidade do ajuste por meio de resíduos e coeficiente de determinação

### Conteúdo

| Seção | Tema |
|---|---|
| 5.1 | Mínimos quadrados discreto |
| 5.2 | Ajuste linear nos parâmetros |
| 5.3 | Ajuste não linear e linearização |

## 5.1 Minimos quadrados: caso discreto

O método dos mínimos quadrados é útil quando queremos aproximar uma função $f(x)$ por uma outra função $g(x)$, seja porque $f$ não tem expressão analítica conhecida, ou por ser de difícil tratamento ou manipulação. A função $g(x)$ pode ser uma combinação de funções polinomiais, exponenciais, logarítmicas, trigonométricas, etc.

De forma geral, escrevemos a função $g(x)$ como uma combinação linear das funções $g_1(x)$, $g_2(x)$,...,$g_n(x)$, ou seja:

$$g(x) = a_1 g_1(x) + a_2 g_2(x) + \cdots + a_n g_n(x)$$

O problema a ser resolvido, portanto, se resume em encontrar os parâmetros $a_1$, $a_2$, ..., $a_n$ para função $g(x)$ que melhor aproxima a função $f(x)$.

No **caso discreto** $f(x)$ é utilizado apenas num conjunto finito de pontos e no **caso contínuo** é usada a expressão analítica da função $f(x)$.

Para encontrar a função $g(x)$ que melhor "se ajusta" aos $n$ pontos conhecidos de uma função $f(x)$ devemos buscar minimizar os desvios, ou erros, $e(x_i) = f(x_i) - g(x_i)$, $i = 1, \dots, m$. No método dos mínimos quadrados, se a função

for conhecida apenas em um conjunto finito de pontos, fazemos isso minimizando a soma

$$E = \sum_{i=1}^{m} [e(x_i)]^2 = \sum_{i=1}^{m} [f(x_i) - g(x_i)]^2$$

Note que, diferentemente do que aproximar $f$ por interpolação, com mínimos quadrados, função $g(x)$ não necessariamente precisa passar pelos pontos mas sim se ajustar a eles.

Se $f$ for conhecida, buscamos minimizar a função

$$E = \int_a^b [e(x)]^2 dx = \int_a^b [f(x) - g(x)]^2 dx$$

ou seja, buscamos minimizar a área entra os gráficos de $g(x)$ e $f(x)$.

O caso discreto é útil quando queremos aproximar uma função complicada por outra mais simples, ou então, obter uma função que possa representar um conjunto de dados obtidos experimentalmente. Por exemplo, se quisermos encontrar a função linear que representa o comportamento de uma mola a partir de medidas da força $F$ aplicada para esticar essa mola e o do comprimento $x$ da mola esticada, podemos buscar a função linear $F(x) = kx$ (Lei de Hooke), encontrando constante elástica $k$ que determina a reta que melhor se ajusta aos dados.

## Caso discreto

Inicialmente vamos supor que queremos ajustar uma reta $g(x) = a_1 x + a_2$ aos pontos $(x_i, f(x_i))$, $i = 1, \dots, m$, como é mostrado na figura abaixo

Figura 5.1.1 - Ajuste de uma reta por um conjunto de pontos.

Então, buscamos os coeficientes $a_1$ e $a_2$ que determinam a reta que melhor se ajusta aos pontos, ou seja, os coeficientes que minimizam a função

$$E(a_1, a_2) = \sum_{i=1}^{m} [g(x_i) - f(x_i)]^2$$

Do cálculo infinitesimal, sabemos que se a função $E(a_1, a_2)$ possui um ponto de mínimo, então suas derivadas parciais nesse ponto devem ser nulas, ou seja

$$\frac{\partial E}{\partial a_1} = 0 \quad \frac{\partial E}{\partial a_2} = 0$$

Derivando $E(a_1, a_2)$ em relação a $a_1$ e $a_2$ obtemos

$$\frac{\partial E}{\partial a_1} = \frac{\partial}{\partial a_1} \left[ \sum_{i=1}^{m} \left( g(x_i) - f(x_i) \right)^2 \right] = 0$$

ou

$$\frac{\partial E}{\partial a_1} = \frac{\partial}{\partial a_1} \left[ \left( \sum_{i=1}^{m} a_1 x_i + \sum_{i=1}^{m} a_2 - \sum_{i=1}^{m} f(x_i) \right)^2 \right] = 0$$

ou, ainda

$$\frac{\partial E}{\partial a_1} = 2 \left[ a_1 \sum_{i=1}^{m} x_i^2 + a_2 \sum_{i=1}^{m} x_i \right] - 2 \left[ \sum_{i=1}^{m} f(x_i) x_i \right] = 0$$

e

$$\frac{\partial E}{\partial a_2} = 2\left[a_1\sum_{i=1}^{m} x_i + ma_2 - \sum_{i=1}^{m} f(x_i)\right] = 0$$

As equações obtidas nos permitem encontrar $a_1$ e $a_2$ por meio da resolução do sistema linear

$$\begin{cases} \left(\sum_{i=1}^{m} x_i^2\right) a_1 + \left(\sum_{i=1}^{m} x_i\right) a_2 = \sum_{i=1}^{m} f(x_i)x_i \\ \left(\sum_{i=1}^{m} x_i\right) a_1 + ma_2 = \sum_{i=1}^{m} f(x_i) \end{cases}$$

ou

$$\begin{bmatrix} \sum_{i=1}^{m} x_i^2 & \sum_{i=1}^{m} x_i \\ \sum_{i=1}^{m} x_i & m \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{m} f(x_i)x_i \\ \sum_{i=1}^{m} f(x_i) \end{bmatrix}$$

**Exemplo 5.1.1:** (Arenales, 2016) Ajuste uma reta à função $f(x)$, tabelada como segue

| $x_i$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $f(x_i)$ | 0.98 | $-3.01$ | $-6.99$ | $-11.01$ | $-15$ |

*Resolução:* Fazendo

$\sum_{i=1}^{5} x_i^2 = 0^2 + 1^2 + 2^2 + 3^2 + 4^2 = 30$
$\sum_{i=1}^{5} x_i = 0 + 1 + 2 + 3 + 4 = 10$
$\sum_{i=1}^{5} f(x_i)x_i = 0 \times 0.98 + 1 \times (-3.01) + 2 \times (-6.99) + 3 \times (-11.01) + 4 \times (-15) = -3.996$
$\sum_{i=1}^{5} x_i = 0.98 + (-3.01) + (-6.99) + 3 \times (-11.01) + (-15) = -3.996$

Então, fazendo

$$\begin{bmatrix} \sum_{i=1}^{5} x_i^2 & \sum_{i=1}^{5} x_i \\ \sum_{i=1}^{5} x_i & 5 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{5} f(x_i)x_i \\ \sum_{i=1}^{5} f(x_i) \end{bmatrix}$$

Obtem-se

$$\begin{bmatrix} 30 & 10 \\ 10 & 5 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} -110.02 \\ -35.03 \end{bmatrix}$$

cuja solução fornece

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} -3.996 \\ 0.986 \end{bmatrix}$$

Que são os coeficientes da reta procurada.

Vamos agora fazer as contas usando Python:

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [4,3]
```

```
# Pontos tabelados
X = np.array([0, 1, 2, 3, 4])
Y = np.array([0.98, -3.01, -6.99, -11.01, -15.0])

# Calcula os elementos das marizes
a11 = np.sum(X**2)
a12 = np.sum(X)
```

(continua na próxima página)

(continuação da página anterior)

```
a22 = len(X)
b1 = np.sum(X * Y)
b2 = np.sum(Y)

# Monta e resolve o sistema
A = np.array([[a11, a12], [a12, a22]])
print ("A =", A)
B = np.array([b1,b2])
print ("b = ",B)

a = np.linalg.solve(A, B)
print ("Solução:", a)
```

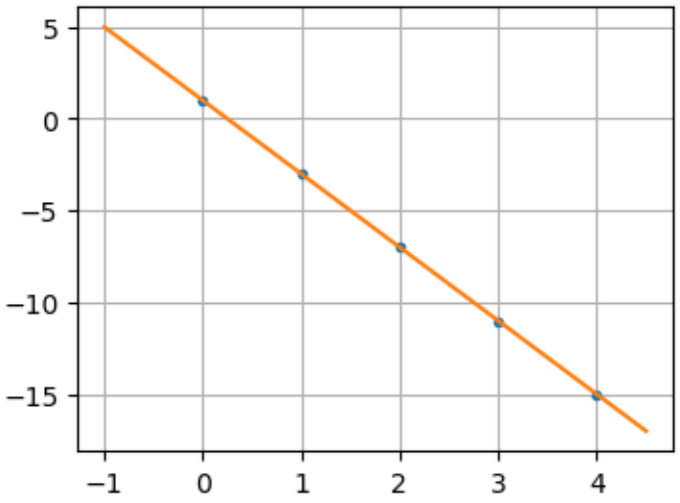
```
A = [[30 10]
 [10  5]]
b =  [-110.02  -35.03]
Solução: [-3.996  0.986]
```

Assim, a reta procurada é $g(x) = -3.996x + 0.986$

```
# Plotnado
# define a funcao g(x) para plotar
g = lambda x: a[0]*x+a[1]

# cria pontos (x, y) da reta
Xr = np.arange(-1, 5, 0.5)

# Plota os pontos e a reta
plt.plot(X, Y, ".", Xr, g(Xr), "-")
plt.grid()
plt.show()
```



Para aproximar a função f(x) por um **polinômio de grau 2**, usamos a seguinte forma geral para $g(x)$

$$g(x) = a_1 g_1(x) + a_2 g_2(x) + a_3 g_3(x) = a_1 x^2 + a_2 x + a_3$$

com $g_1(x) = x^2$, $g_2(x) = x$ e $g_3(x) = 1$.

$$E(a_1, a_2, a_3) = \sum_{i=1}^{m} [g(x_i) - f(x_i)]^2$$

Procuramos o ponto de mínimo, em que as derivadas parciais são nulas, ou seja

$$\frac{\partial E}{\partial a_1} = 0 \ , \ \frac{\partial E}{\partial a_2} = 0 \ e \ \frac{\partial E}{\partial a_3} = 0$$

Derivando $E(a_1, a_2, a_3)$ em relação a $a_1$, $a_2$ e $a_3$ obtemos

$$\frac{\partial E}{\partial a_1} = \frac{\partial}{\partial a_1} \left[ \sum_{i=1}^{m} \left( g(x_i) - f(x_i) \right)^2 \right] = 0$$

ou

$$\frac{\partial E}{\partial a_1} = \frac{\partial}{\partial a_1} \left\{ \left[ \left( \sum_{i=1}^{m} a_1 x_i^2 + a_2 x_i + a_3 \right) - \left( \sum_{i=1}^{m} f(x_i) \right) \right]^2 \right\} = 0$$

ou, ainda

$$\frac{\partial E}{\partial a_1} = 2 \left[ a_1 \sum_{i=1}^{m} x_i^4 + a_2 \sum_{i=1}^{m} x_i^3 + a_3 \sum_{i=1}^{m} x_i^2 \right] - 2 \left[ \sum_{i=1}^{m} f(x_i) x_i^2 \right] = 0$$

analogamente, derivando em relação a $a_2$

$$\frac{\partial E}{\partial a_2} = 2 \left[ a_1 \sum_{i=1}^{m} x_i^3 + a_2 \sum_{i=1}^{m} x_i^2 + a_3 \sum_{i=1}^{m} x_i \right] - 2 \left[ \sum_{i=1}^{m} f(x_i) x_i \right] = 0$$

e em relação a $a_3$

$$\frac{\partial E}{\partial a_3} = 2 \left[ a_1 \sum_{i=1}^{m} x_i^2 + a_2 \sum_{i=1}^{m} x_i + m a_3 \right] - 2 \left[ \sum_{i=1}^{m} f(x_i) \right] = 0$$

Assim, as equações obtidas nos permitem montar um sistema linear cuja solução dará os coeficientes $a_1$, $a_2$ e $a_3$ procurados

$$\begin{cases} \left( \sum_{i=1}^{m} x_i^4 \right) a_1 + \left( \sum_{i=1}^{m} x_i^3 \right) a_2 + \left( \sum_{i=1}^{m} x_i^2 \right) a_3 = \sum_{i=1}^{m} f(x_i) x_i^2 \\ \left( \sum_{i=1}^{m} x_i^3 \right) a_1 + \left( \sum_{i=1}^{m} x_i^2 \right) a_2 + \left( \sum_{i=1}^{m} x_i \right) a_3 = \sum_{i=1}^{m} f(x_i) x_i \\ \left( \sum_{i=1}^{m} x_i^2 \right) a_1 + \left( \sum_{i=1}^{m} x_i \right) a_2 + \left( \sum_{i=1}^{m} 1 \right) a_3 = \sum_{i=1}^{m} f(x_i) \end{cases}$$

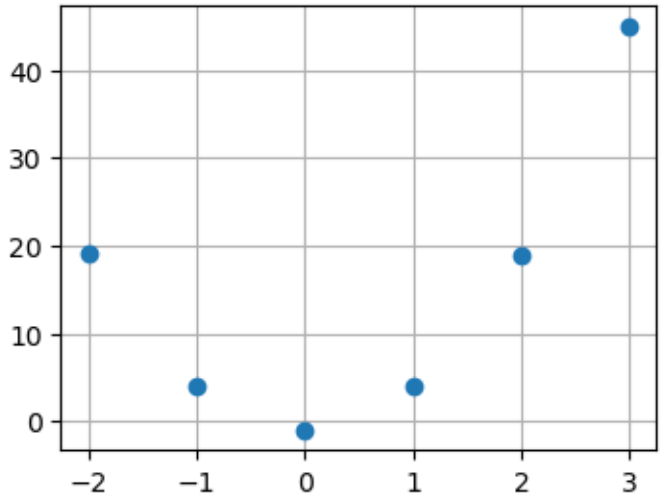
**Exemplo 5.1.2:** (Arenales, 2016) Considere uma função f (x) definida conforme tabela

| $x_i$ | $-2$ | $-1$ | $0$ | $1$ | $2$ | $3$ |
|---|---|---|---|---|---|---|
| $f(x_i)$ | $19.01$ | $3.99$ | $-1.00$ | $4.01$ | $18.99$ | $45.00$ |

Observando o gráfico, vemos que a função possui o comportamento de uma parábola, ou seja, podemos ajustar um polinômio de grau 2.

```
X = np.array([-2., -1., 0, 1., 2., 3.])
Y = np.array([19.01, 3.99, -1.00, 4.01, 18.99, 45.00])
```

```
plt.figure(figsize=(4,3))
plt.plot(X, Y, "o")
plt.grid()
plt.show()
```




Assim, tomamos $g(x) = a_1x^2 + a_2x + a_3$, isto é, $g_1(x) = x^2, g_2(x) = x, g_3(x) = 1$, e determinamos os parâmetros $a_1, a_2$ e $a_3$ de modo que $g(x)$ se ajuste aos dados. Substituindo no sistema de equações normais temos:

$$\begin{bmatrix} \sum_{i=1}^{6} x_i^4 & \sum_{i=1}^{6} x_i^3 & \sum_{i=1}^{6} x_i^2 \\ \sum_{i=1}^{6} x_i^3 & \sum_{i=1}^{6} x_i^2 & \sum_{i=1}^{6} x_i \\ \sum_{i=1}^{6} x_i^2 & \sum_{i=1}^{6} x_i & 6 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{6} f(x_i)\, x_i^2 \\ \sum_{i=1}^{6} f(x_i)\, x_i \\ \sum_{i=1}^{6} f(x_i) \end{bmatrix}$$

ou

$$\begin{bmatrix} 115 & 27 & 19 \\ 27 & 19 & 3 \\ 19 & 3 & 6 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 565.00 \\ 134.98 \\ 90.00 \end{bmatrix}$$

Vamos fazer essas contas em Python:

```
# Monta o sistema
A = np.array([[np.sum(X**4), np.sum(X**3), np.sum(X**2)],
              [np.sum(X**3), np.sum(X**2), np.sum(X)   ],
              [np.sum(X**2), np.sum(X),    len(X)]     ])

B = np.array([np.sum(X**2*Y),
              np.sum(X*Y)   ,
              np.sum(Y)     ])

print(A)
print(B)
```

```
[[115.  27.  19.]
 [ 27.  19.   3.]
 [ 19.   3.   6.]]
[565.   134.98  90.  ]
```

A solução do sistema nos fornece $a_1 = 5,0898$, $a_2 = 0,0519$ e $a_3 = -1,1437$ como podemos ver a seguir. Ou seja, a função que procuramos é $g(x) = 5,0898x^2 + 0,0519x - 1,1437$. Vamos plotar essa função junto com os pontos:

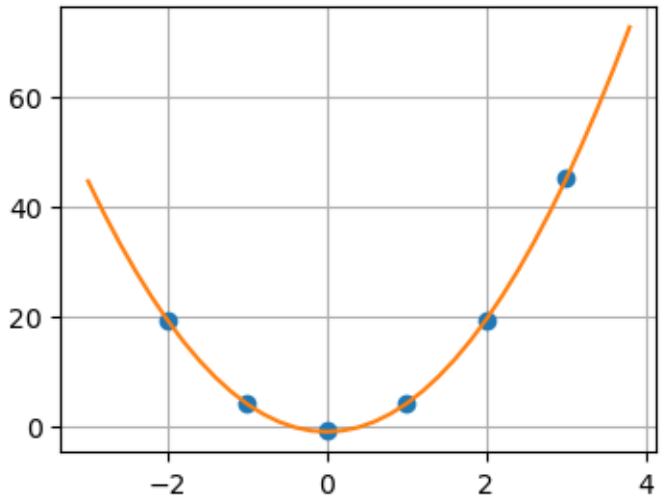
```
a = np.linalg.solve(A, B)
print (a)
```

```
[ 5.08982143  0.05189286 -1.14371429]
```

```
# define a funcao g(x) para plotar
g = lambda x: a[0]*x*x+a[1]*x+a[2]

# gera os pontos
Xr = np.arange(X[0]-1, X[-1]+1, 0.2)
Yr = []
for x in Xr:
    Yr.append(g(x))

# plota
plt.figure(figsize=(4,3))
plt.plot(X, Y, "o", Xr, Yr, "-")
plt.grid()
plt.show()
```




### Forma matricial do sistema

Observe que podemos obter o mesmo resultado se fizermos $A = V^T V$ e $B = V^T y_i$, onde $V$ é a matriz cujas linhas são formadas por $[g_1(x_i)\ g_2(x_i)\ \ ...\ \ g_n(x_i)\ ]$, $i = 1, ..., m$. Nesse caso, a solução pode ser obtida fazendo:

$$a = (V^T V)^{-1} V^T y$$

Observe os códigos a seguir:

```
V = np.array([X**2, X**1, X**0]).T
print (V)
```

```
[[ 4. -2.  1.]
 [ 1. -1.  1.]
 [ 0.  0.  1.]
 [ 1.  1.  1.]
 [ 4.  2.  1.]
 [ 9.  3.  1.]]
```

```
A = np.dot(V.T,V)
print (A)
```

```
[[115.  27.  19.]
 [ 27.  19.   3.]
 [ 19.   3.   6.]]
```

```
B = np.dot(V.T,Y)
print('B=',B)
```

```
B= [565.   134.98  90.  ]
```

```
a = np.linalg.solve(A,B)
print('solução:', a)
```

```
solução: [ 5.08982143  0.05189286 -1.14371429]
```

Ou ainda, de forma mais compacta,fazendo $a = (V^T V)^{-1} V^T y$:

```
# importando as funções para multiplicar e inverter
from numpy.linalg import inv
from numpy import dot
```

```
# resolvendo o sistema
a = inv(dot(V.T,V)).dot(dot(V.T,Y))
print('solução:', a)
```

```
solução: [ 5.08982143  0.05189286 -1.14371429]
```

### Generalizando o procedimento

Podemos determinar os coeficientes $a_1, a_2, \dots, a_n$ da função$g(x) = a_1 g_1(x) + a_2 g_2(x) + \cdots + a_n g_n(x)$ resolvendo o sistema linear formado pelas equações

$$\frac{\partial E}{\partial a_1} = 0, \ \frac{\partial E}{\partial a_2} = 0, \dots, \ \frac{\partial E}{\partial a_n} = 0$$

ou seja,

$$\begin{cases} \left(\sum_{i=1}^{m} g_1(x_i) g_1(x_i)\right) a_1 + \left(\sum_{i=1}^{m} g_1(x_i) g_2(x_i)\right) a_2 + \cdots + \left(\sum_{i=1}^{m} g_1(x_i) g_n(x_i)\right) a_n = \sum_{i=1}^{m} f(x_i) g_1(x_i) \\ \left(\sum_{i=1}^{m} g_2(x_i) g_1(x_i)\right) a_1 + \left(\sum_{i=1}^{m} g_2(x_i) g_2(x_i)\right) a_2 + \cdots + \left(\sum_{i=1}^{m} g_2(x_i) g_n(x_i)\right) a_n = \sum_{i=1}^{m} f(x_i) g_2(x_i) \\ \vdots \\ \left(\sum_{i=1}^{m} g_n(x_i) g_1(x_i)\right) a_1 + \left(\sum_{i=1}^{m} g_n(x_i) g_2(x_i)\right) a_2 + \cdots + \left(\sum_{i=1}^{m} g_n(x_i) g_n(x_i)\right) a_n = \sum_{i=1}^{m} f(x_i) g_n(x_i) \end{cases}$$

**Exemplo 5.1.3:** Dado o seguinte conjunto de dados

| $x_i$ | 0,0 | 0,1 | 0,2 | 0,3 | 0,4 | 0,5 | 0,6 | 0,7 | 0,8 | 0,9 | 1,0 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $y_i$ | 31 | 35 | 37 | 33 | 28 | 20 | 16 | 15 | 18 | 23 | 31 |

Encontre a função do tipo $f(x) = a + b\sin(2\pi x) + c\cos(2\pi x)$ que melhor aproxima os valores dados.

```
import numpy as np
from scipy.linalg import solve
import matplotlib.pyplot as plt
```

```
x = np.linspace(0.0,1.0, 11)
y = np.array([31.,35.,37,33.,28.,20.,16.,15.,18.,23.,31.])
```

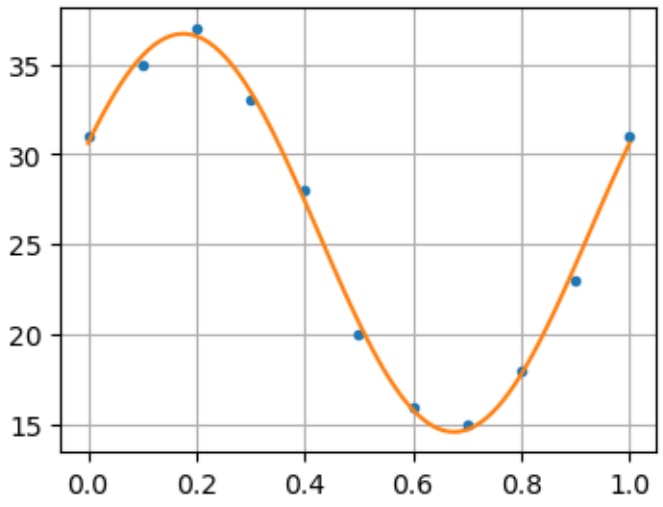
```
g1 = lambda x: np.ones(len(x))
g2 = lambda x: np.sin(2*np.pi*x)
g3 = lambda x: np.cos(2*np.pi*x)
```

```
V = np.array([g1(x), g2(x), g3(x)])
```

```
A = V.dot(V.T)
B = np.dot(V,y)
a = np.linalg.solve(A,B)
```

```
g = lambda x: a[0]*g1(x)+a[1]*g2(x)+a[2]*g3(x)
```

```
# Plota os pontos e a função g(x)
xp = np.linspace(0.0,1.0,100)
plt.plot(x, y, ".")
plt.plot(xp, g(xp), "-")
plt.grid()
plt.show()
```



## Ajuste linear não polinomial

Uma função do tipo

$$g(x) = a_1 g_1(x) + a_2 g_2(x) + \cdots + a_n g_n(x)$$

é dita um *modelo de regressão linear*. O adjetivo "linear" é usado para indicar que o modelo é linear nos parâmetros $a_1$, $a_2$,..., $a_n$ e não porque é um função linear de $x$. Por exemplo, uma expressão da forma

$$y = a_1 + a_2 \log(x) + a_3 x^3$$

é um modelo de regressão linear múltipla, assim como os ajustes polinomiais feitos nos capítulos anteriores, mas o mesmo não acontece com a equação

$$y = a_1 + a_2 x^{a_3} + a_4 x^2$$

Algumas maneiras de ajustar funções não lineares serão discutidos nas próximas seções. Por hora, apresentamos um exemplo de ajuste linear não polinomial.

**Exemplo 5.1.4:** Considere o segunte conjunto de pontos

| $x_i$ | -0.5 | 0.0 | 0.5 | 1.0 | 1.5 | 2.0 | 2.5 | 3.0 | 3.5 | 4.0 | 4.5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $f(x_i)$ | -0.25 | 0.5 | 0.25 | 0.0 | 0.75 | 1.5 | 1.25 | 1.00 | 1.75 | 2.5 | 2.25 |

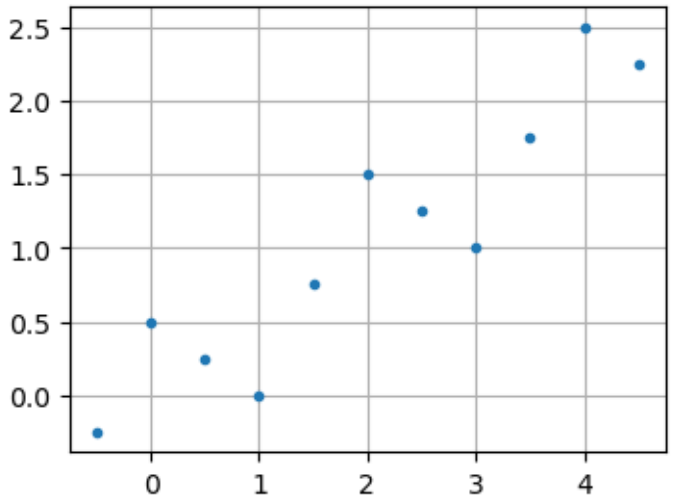
Usando o método dos mínimos quadrados, ajuste uma função $g(x)$ aos dados.

**Solução:** Primeiramente, construimos o diagrama de dispersão dos pontos dados:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([-0.5, 0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5])
Y = np.array([-0.25, 0.5, 0.25, 0.0, 0.75, 1.5, 1.25, 1.00, 1.75, 2.5, 2.25])

# Plota os pontos e a reta
plt.plot(X, Y, ".")
plt.grid()
plt.show()
```




É possível perceber que os pontos parecem crescer linearmente mas também parecem oscilar com período igual a 2. Essas características sugerem uma função do tipo $g(x) = a_1 x + a_2 \cos(\pi x)$ em que $g_1(x) = x$ e $g_2(x) = \cos(\pi x)$.

```
from scipy.linalg import solve

# Calcula os elementos das marizes
g1 = lambda X: X
g2 = lambda X: np.cos(np.pi*X)

print (g1(X))
a11 = np.sum(np.multiply(g1(X),g1(X)))
a12 = np.sum(np.multiply(g1(X),g2(X)))
a21 = a12
a22 = np.sum(np.multiply(g2(X),g2(X)))

print (a11, a12, a22)

b1 = np.sum(np.multiply(Y,g1(X)))
b2 = np.sum(np.multiply(Y,g2(X)))

print (b1,b2)


# Monta e resolve o sistema
A = np.array([[a11, a12],
              [a21, a22]])

B = np.array([b1,b2])

a = solve(A, B)
print (a)
```

(continua na próxima página)

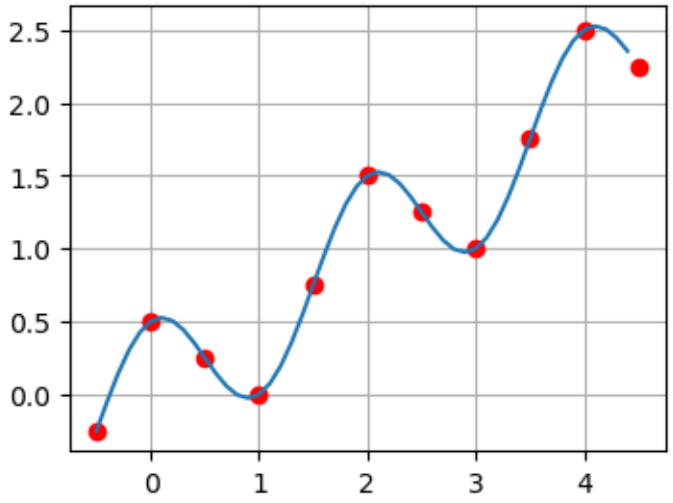
(continuação da página anterior)

```
# define a funcao g(x) para plotar
g = lambda X: a[0]*g1(X)+a[1]*g2(X)

# cria pontos (x, y) da reta
Xr = np.arange(X[0], X[-1], 0.1)
Yr = g(Xr)

# Plota os pontos e a reta
plt.plot(X, Y, "ro", Xr, Yr, "-")
plt.grid()
plt.show()
```

```
[-0.5  0.   0.5  1.   1.5  2.   2.5  3.   3.5  4.   4.5]
71.5 2.0000000000000018 5.0
36.75 3.5000000000000004
[0.5 0.5]
```




## Exercícios

---

**1.** Em um experimento foram obtidos os seguintes dados:

| $x_i$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $f(x_i)$ | $0,01$ | $1,01$ | $1,40$ | $3,81$ | $4,01$ |

a) Determine a reta que melhor se ajusta aos dados da tabela, usando o método dos mínimos quadrados
b) Calcule a soma do erro quadrático $\sum_{i=1}^{5}(f(x_i) - g(x_i))^2$

---

**2.** Encontre o polinômio $p(x) = a_1 + a_2x + a_3x^2$ que melhor se ajusta no sentido de mínimos quadrados aos pontos:

| $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $x_i$ | $-1,50$ | $-0,50$ | $1,25$ | $1,50$ |
| $y_i$ | $1,15$ | $-0,37$ | $0,17$ | $0,94$ |

*Resposta:* $a_1 = -0,67112$, $a_2 = -0,12123$, $a_3 = 0,73907$.

---

**3.** Encontrar a parábola $y = ax^2 + bx + c$ que melhor aproxima o seguinte conjunto de dados:

| $i$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $x_i$ | $0,01$ | $1,02$ | $2,04$ | $2,95$ | $3,55$ |
| $y_i$ | $1,99$ | $4,55$ | $7,20$ | $9,51$ | $10,82$ |

*Resposta:* $y = -0,0407898x^2 + 2,6613293x + 1,9364598$

---

**4.** Após serem efetuadas medições num gerador de corrente contínua, foram obtidos os seguintes

| $I(A)$ | $1,58$ | $1,75$ | $2,10$ | $2,20$ | $2,50$ | $3,50$ | $3,75$ | $4,25$ | $4,8$ | $4,9$ | $3,12$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $V(v)$ | 210 | 155 | 170 | 120 | 85 | 50 | 110 | 79 | 150 | 120 | 60 |

Faça um gráfico dos dados.(a) Ajuste os dados por polinômio de grau adequado. (b) Estime o valor a ser obtido no voltímetro quando o amperímetro estiver marcando $4.0A$.

---

**5.** Dados os seguintes pontos tabelados

| $x_i$ | $0,01$ | $0,99$ | $2,02$ | $3,01$ | $3,97$ | $5,01$ | $5,93$ | $6,99$ | $8,08$ |
|---|---|---|---|---|---|---|---|---|---|
| $f(x_i)$ | $0,000$ | $1,621$ | $1,782$ | $0,915$ | $-0,122$ | $-0,225$ | $1,099$ | $2,728$ | $3,534$ |

Encontre o polinômio de grau 3 que melhor se ajusta aos pontos dados.

---

**6.** Dados os seguintes pontos tabelados

| $x_i$ | $0,01$ | $0,99$ | $2,02$ | $3,01$ | $3,97$ | $5,01$ | $5,93$ | $6,99$ | $8,08$ |
|---|---|---|---|---|---|---|---|---|---|
| $f(x_i)$ | $0,000$ | $1,621$ | $1,782$ | $0,915$ | $-0,122$ | $-0,225$ | $1,099$ | $2,728$ | $3,534$ |

Encontre a função do tipo $g(x) = a_1x + a_2\,\text{sen}(x)$ que melhor se ajusta.

---

**7.** Dado o seguinte conjunto de dados

| $x_i$ | $0,0$ | $0,1$ | $0,2$ | $0,3$ | $0,4$ | $0,5$ | $0,6$ | $0,7$ | $0,8$ | $0,9$ | $1,0$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $y_i$ | 31 | 35 | 37 | 33 | 28 | 20 | 16 | 15 | 18 | 23 | 31 |

a) Encontre a função do tipo $f(x) = a + b\sin(2\pi x) + c\cos(2\pi x)$ que melhor aproxima os valores dados.
b) Encontre a função do tipo $f(x) = a + bx + cx^2 + dx^3$ que melhor aproxima os valores dados.
*Resposta:* a) $a = 25,638625$, $b = 9,8591874$, $c = 4,9751219$; b)$a = 31,475524$, $b = 65,691531$, $c = -272,84382$, $d = 208,23621$.

## 5.2 Minimos quadrados: caso contínuo

No caso da função $f(x)$ ser conhecida, podemos ajustar a função $g(x)$ de modo a minimizar

$$E = \int_a^b [e(x)]^2 dx = \int_a^b [f(x) - g(x)]^2 dx$$

o que leva a necessidade de resolução do sistema linear

$$\begin{bmatrix} \langle g_1, g_1 \rangle & \langle g_1, g_2 \rangle & \cdots & \langle g_1, g_n \rangle \\ \langle g_2, g_1 \rangle & \langle g_2, g_2 \rangle & \cdots & \langle g_2, g_n \rangle \\ \vdots & & & \\ \langle g_n, g_1 \rangle & \langle g_n, g_2 \rangle & \cdots & \langle g_n, g_n \rangle \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \langle f, g_1 \rangle \\ \langle f, g_2 \rangle \\ \vdots \\ \langle f, g_n \rangle \end{bmatrix}$$

Com $\langle f, g \rangle$ sendo o produto escalar entre as funções $f$ e $g$, definido como

$$\langle f, g \rangle = \int_a^b f(x) g(x) dx$$

**Exemplo 5.2.1:** (Arenales, 2016) Usando o método dos mínimos quadrados, aproxime a função $f(x) = e^{-x}$ no intervalo $[1, 3]$ por um polinômio de grau 1 , ou seja, $g(x) = a_1 + a_2 x$.

O sistema de equações normais é dado por:

$$\begin{bmatrix} \langle 1, 1 \rangle & \langle 1, x \rangle \\ \langle 1, x \rangle & \langle x, x \rangle \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \langle 1, e^{-x} \rangle \\ \langle x, e^{-x} \rangle \end{bmatrix}$$

Assim,

$$\langle 1, 1 \rangle = \int_1^3 1 dx = 2$$

$$\langle 1, x \rangle = \int_1^3 x dx = 4$$

$$\langle x, x \rangle = \int_1^3 x^2 dx = 26/3$$

$$\langle 1, e^{-x} \rangle = \int_1^3 e^{-x} dx = -\frac{1}{e^3} + e^{-1} \approx 0.3181$$

$$\langle x, e^{-x} \rangle = \int_1^3 x e^{-x} dx = -\frac{4}{e^3} + \frac{2}{e} \approx 0.5366$$

Portanto, o sistema de equações normais é dado por:

$$\begin{bmatrix} 2 & 4 \\ 4 & \frac{26}{3} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 0.3181 \\ 0.5366 \end{bmatrix}$$

Usando o método de eliminação de Gauss, obtemos:

$$a_1 = 0.4578 \quad a_2 = -1.1494$$

e a reta que melhor se ajusta à $f(x) = e^{-x}$ no intervalo $[1, 3]$ é dada por:

$$g(x) = a_1 + a_2 x = 0.4578 - 0.1494x$$

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)

# integrção numérica com scipy
from scipy.integrate import quad
```

```
A = np.array([[2, 4],
              [4, 26/3]])

# por conveniência, vamos obter b numericamente
b = np.array([quad(lambda x: -np.exp(-x), 1, 3)[0],
              quad(lambda x: -x*np.exp(-x), 1, 3)[0]])

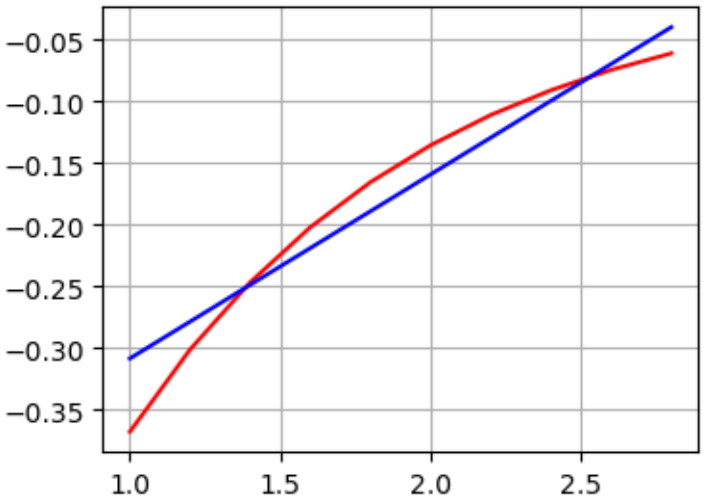
a = np.linalg.solve(A, b)
print (a)
```

```
[-0.4577686   0.14936121]
```

```
# define a funcao g(x) para plotar
g = lambda x: a[0]+a[1]*x

# cria pontos (x, y) da reta
Xr = np.arange(1, 3, 0.2)

# Plota os pontos e a reta
plt.plot(Xr, -np.exp(-Xr), "r-", Xr, g(Xr), "b-")
plt.grid()
plt.show()
```




**Exemplo 2:** (Burden, 2016) Encontre o polinômio de aproximação por mínimos quadrados de segundo grau para a função $f(x) = \text{sen}\,\pi x$ no intervalo $[0, 1]$.

*Solução:* As equações normais para $P_2(x) = a_2x^2 + a_1x + a_0$ são

$$\begin{aligned}
a_0\int_0^1 1dx + a_1\int_0^1 xdx + a_2\int_0^1 x^2dx &= \int_0^1 \text{sen}\,\pi x dx\\
a_0\int_0^1 xdx + a_1\int_0^1 x^2dx + a_2\int_0^1 x^3dx &= \int_0^1 x\,\text{sen}\,\pi x dx\\
a_0\int_0^1 x^2dx + a_1\int_0^1 x^3dx + a_2\int_0^1 x^4dx &= \int_0^1 x^2\,\text{sen}\,\pi x dx
\end{aligned}$$

Calculando as integrais, obtemos

$$\begin{aligned}
a_0 + \frac{1}{2}a_1 + \frac{1}{3}a_2 &= \frac{2}{\pi}\\
\frac{1}{2}a_0 + \frac{1}{3}a_1 + \frac{1}{4}a_2 &= \frac{1}{\pi}\\
\frac{1}{3}a_0 + \frac{1}{4}a_1 + \frac{1}{5}a_2 &= \frac{\pi^2-4}{\pi^3}
\end{aligned}$$

Estas três equações em três incógnitas podem ser resolvidas a fim de se obter

$$a_0 = \frac{12\pi^2 - 120}{\pi^3} \approx -0,050465 \quad \text{e} \quad a_1 = -a_2 = \frac{720 - 60\pi^2}{\pi^3} \approx 4,12251$$

Consequentemente, a aproximação polinomial por mínimos quadrados de segundo grau para $f(x) = \text{sen}\,\pi x$ em $[0, 1]$ é $P_2(x) = -4,12251x^2 + 4,12251x - 0,050465$.

```
A = np.array([[1,1/2,1/3],
              [1/2,1/3,1/4],
              [1/3,1/4,1/5]])

# por conveniência, vamos obter b numericamente
b = np.array([quad(lambda x: np.sin(np.pi*x), 0, 1)[0],
```

(continua na próxima página)

(continuação da página anterior)

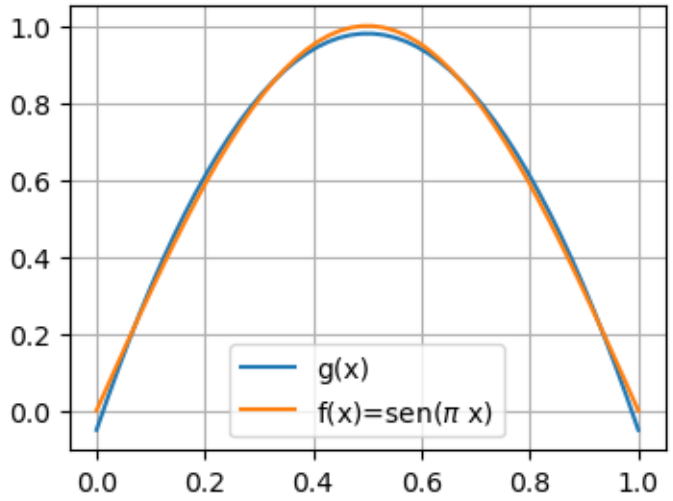
```
            quad(lambda x: x*np.sin(np.pi*x), 0, 1)[0],
            quad(lambda x: x**2*np.sin(np.pi*x), 0, 1)[0]])

a = np.linalg.solve(A,b)
print(a)
```

```
[-0.0504655   4.12251162 -4.12251162]
```

```
g = lambda x: a[2]*x**2+a[1]*x+a[0]
```

```
x = np.linspace(0,1)
plt.plot(x,g(x), label='g(x)')
plt.plot(x,np.sin(np.pi*x), label='f(x)=sen($\\pi$ x)')
plt.grid()
plt.legend()
plt.show()
```




Para obter a aproximação polinomial por mínimos quadrados, um sistema linear $(n+1) \times (n+1)$ nas incógnitas $a_0, \ldots, a_n$ deve ser resolvido e os coeficientes no sistema linear são da forma

$$\int_a^b x^{j+k} dx = \frac{b^{j+k+1} - a^{j+k+1}}{j+k+1}$$

A matriz no sistema linear é conhecida como matriz de Hilbert. O sistema gerado não tem uma solução numérica facilmente calculável. É um exemplo típico para ilustrar dificuldades causadas por erros de arredondamento (Burden, 2016).

### Exercícios

**1.** Encontre a aproximação polinomial por mínimos quadrados linear para $f(x)$ nos intervalos indicados se:
a. $f(x) = x^2 + 3x + 2$,$[0, 1]$;
b. $f(x) = x^3, \quad [0, 2]$;
c. $f(x) = \frac{1}{x}, \quad [1, 3]$;
d. $f(x) = e^x, \quad [0, 2]$;
e. $f(x) = \frac{1}{2}\cos x + \frac{1}{3}\operatorname{sen} 2x, [0, 1]$;
f. $f(x) = x \ln x, [1, 3]$.

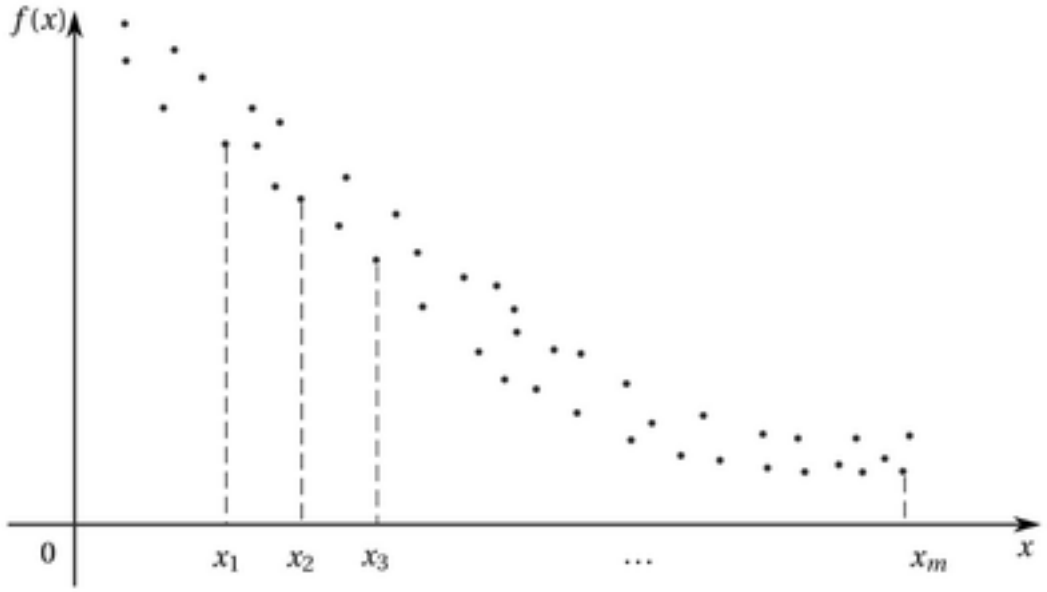
**2.** Encontre a aproximação polinomial por mínimos quadrados linear no intervalo $[-1, 1]$ para as seguintes funções:
a. $f(x) = x^2 - 2x + 3$
b. $f(x) = x^3$
c. $f(x) = \frac{1}{x+2}$
d. $f(x) = e^x$
e. $f(x) = \frac{1}{2}\cos x + \frac{1}{3}\operatorname{sen} 2x$
f. $f(x) = \ln(x + 2)$

## 5.3 Ajuste não linear e linearização

Muitas vezes, temos dados experimentais em que o ajuste como combinação linear nos parâmetros não é adequado e não pode ser considerado. Neste caso, necessitamos de outras famílias de funções para representar adequadamente uma função representada em uma tabela.

## Ajuste hiperbólico

Considere os dados obtidos experimentalmente, conforme ilustrado na figura abaixo




Observando a o diagrama de dispersão, vemos que a representação dos dados possui um comportamento do tipo:

$$g(x) = \frac{1}{a_1 x + a_2}$$

Para encontrar os parâmetros $a_1$ e $a_2$ da forma como fizemos anteriormente, podemos linearizar a função fazendo

$$h(x) = \frac{1}{g(x)} = a_1 x + a_2$$

que aproxima a função

$$\frac{1}{f(x)}$$

e temos agora o caso do ajuste linear, já desenvolvido anteriormente.

Assim, a partir da tabela original, construímos uma nova tabela

| $x_1$ | $x_2$ | ... | $x_m$ |
|---|---|---|---|
| $1/f(x_1)$ | $1/f(x_2)$ | ... | $1/f(x_m)$ |

O problema agora consiste em aproximarmos a função $\frac{1}{f(x)}$ por uma reta $h(x) = a_1 x + a_2$, e o sistema de equações normais é dado por

$$\begin{cases} \left(\sum_{i=1}^{m} x_i^2\right) a_1 + \left(\sum_{i=1}^{m} x_i\right) = \sum_{i=1}^{m} \frac{x_i}{f(x_i)} \\ \left(\sum_{i=1}^{m} x_i\right) a_1 + (m)\, a_2 = \sum_{i=1}^{m} \frac{1}{f(x_i)} \end{cases}$$

**Exemplo 5.3.1:** Considere a função f (x) tabelada nos pontos, como segue

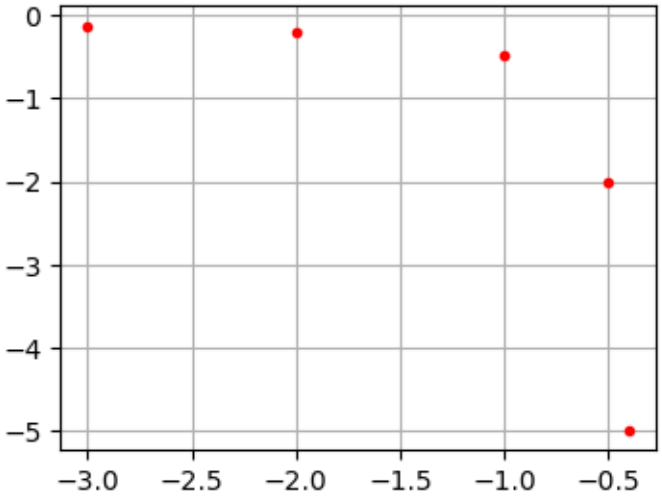
| $x_1$ | -3.0 | -2.0 | -1.0 | -0.5 | -0.4 |
|---|---|---|---|---|---|
| $f(x_1)$ | -0.13 | -0.20 | -0.49 | -2.01 | -4.99 |

*Solução:* Plotando os pontos tem-se

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)
```

```
import numpy as np
import matplotlib.pyplot as plt

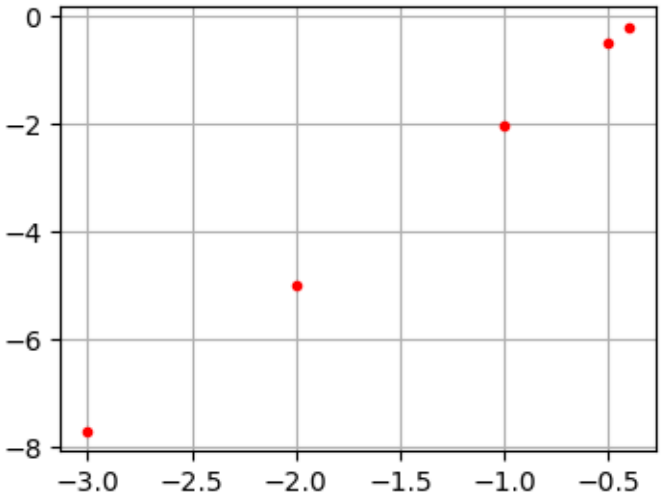
X = np.array([ -3.0, -2.0, -1.0, -0.5, -0.4])
Y = np.array([ -0.13, -0.20, -0.49, -2.01, -4.99])
# plotando
plt.plot(X, Y, "r.")
plt.grid()
plt.show()
```




O que sugere um comportamento de uma função do tipo $\$g(x) = \frac{1}{a_1 x + a_2}.Para confirmar, e verificar se a linearização do tipo h(x) = \frac{1}{g(x)} = a_1 x + a_2\$$ irá funcionar, construímos uma nova tabela e plotamos os pontos

```
Yl = np.power(Y, -1)
print(Yl)
# plotando
plt.plot(X, Yl, "r.")
plt.grid()
plt.show()
```

```
[-7.69230769 -5.         -2.04081633 -0.49751244 -0.2004008 ]
```



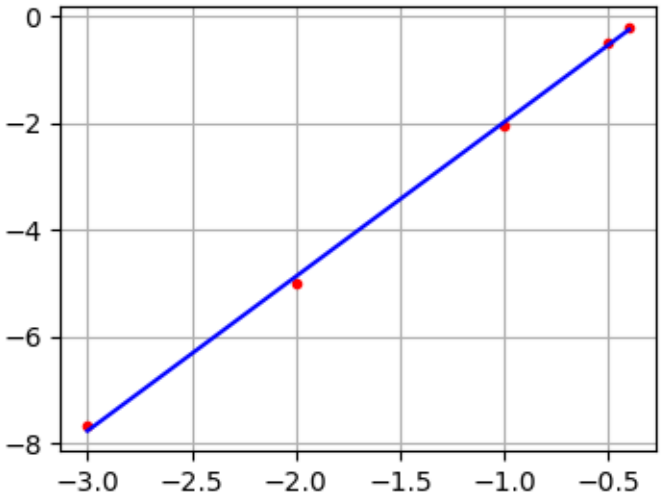

Notamos que os pontos parecem alinhados agora, o que sugere que podemos obter $a_1$ e $a_2$ ajustando uma reta a essa nova tabela

```
# montando a matriz do sistema
A = np.array([[np.sum(X * X), np.sum(X)],
              [np.sum(X),     len(X)]])
B = np.array([np.sum(X*Yl),
              np.sum(Yl)])
# resolvendo
a = np.linalg.solve(A, B)
print ("Parâmetros a1=", a[0]," e a2 =", a[1])

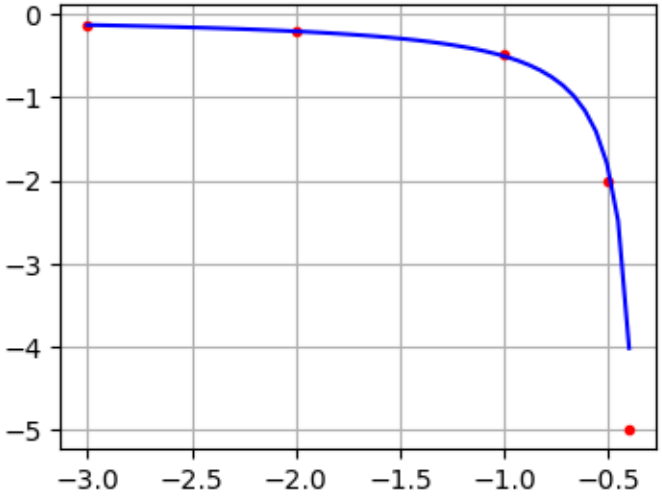
# lista de pontos para os plots
Xr = np.linspace(X[0], X[-1], 50)

#reta
h = lambda X: a[0]*X + a[1]
# plotando
plt.plot(X, Yl, "r.", Xr, h(Xr), "b-")
plt.grid()
plt.show()
```

```
Parâmetros a1= 2.8952177836767694  e a2 = 0.9091930898234505
```

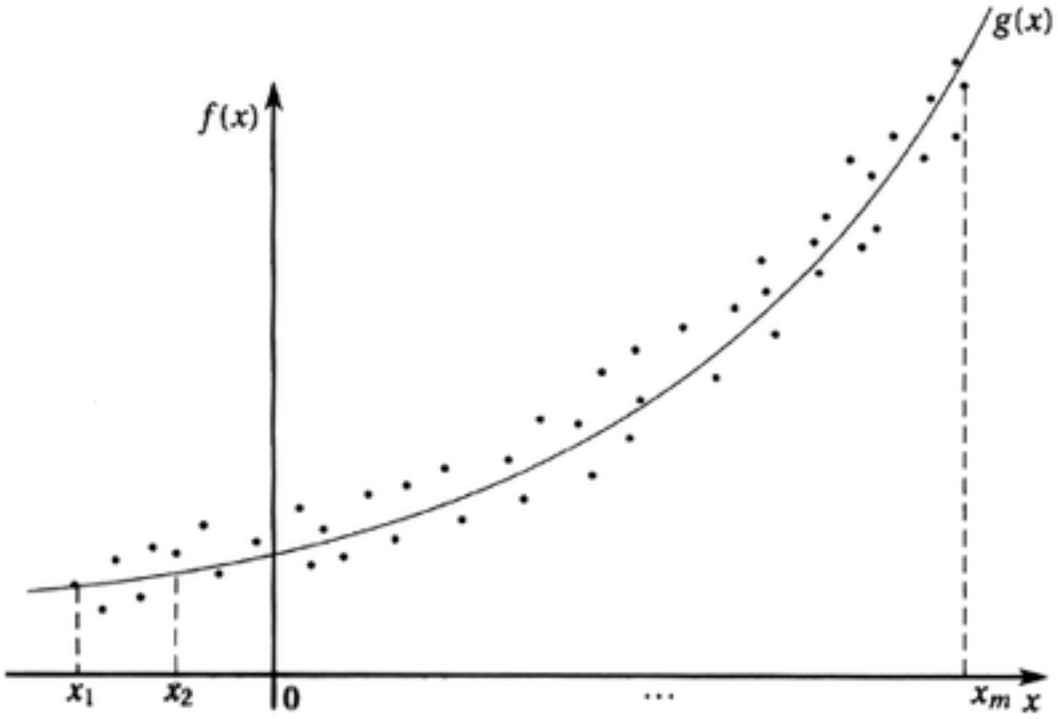



```
# hipérbole
g = lambda X: np.power(a[0]*X+a[1], -1)
Yr = g(Xr)
# plotando
plt.plot(X, Y, "r.", Xr, g(Xr), "b-")
plt.grid()
plt.show()
```



## Ajuste exponencial

Podemos obter dados experimentais dispostos conforme ilustrado na figura abaixo, a qual sugere que devemos aproximar a função observada por uma função $g(x)$ da forma $g(x) = a(b)^x$ com os parâmetros $a$ e $b$ positivos.




O método dos mínimos quadrados desenvolvido anteriormente pode ser usado fazendo-se a seguinte transformação

$$h(x) = \ln(g(x)) = \ln(a(b)^x) = \ln(a) + x\ln(b)$$

Então, fazendo

$$a_1 = \ln(a) \rightarrow e^{a_1} = a$$

$$a_2 = \ln(b) \rightarrow e^{a_2} = b$$

temos a linearização

$$h(x) = a_1 + a_2 x$$

Então, construímos uma nova tabela

| $x_1$ | $x_2$ | ... | $x_m$ |
|---|---|---|---|
| $\ln(f(x_1))$ | $\ln(f(x_2))$ | ... | $\ln(f(x_m))$ |

e buscamos os parâmetros $a_1$ e $a_2$ que ajustam uma reta a esses dados, ou seja, buscamos a solução do sistema

$$\begin{cases} \left(\sum_{i=1}^{m} x_i^2\right) a_1 + \left(\sum_{i=1}^{m} x_i\right) a_2 = \sum_{i=1}^{m} \ln(f(x_i))x_i \\ \left(\sum_{i=1}^{m} x_i\right) a_1 + (m)\, a_2 = \sum_{i=1}^{m} \ln(f(x_i)) \end{cases}$$

**Exemplo 5.3.2:** Considere uma função tabelada nos pontos, como segue:

| $x_i$ | $-1$ | $-0.9$ | $-0.8$ | $0$ | $1$ | $2$ |
|---|---|---|---|---|---|---|
| $f(x_i)$ | $6.01$ | $5.39$ | $4.80$ | $2.01$ | $0.65$ | $0.21$ |

Determine uma função $g(x)$ que melhor se ajuste aos dados da tabela de $f(x)$.

```
X = np.array([ -1, -0.9, -0.8, 0, 1, 2])
Y = np.array([ 6.01, 5.39, 4.8, 2.01, 0.65, 0.21])
Yl = np.log(Y)

# montando a matriz do sistema
A = np.array([[np.sum(X*X), np.sum(X)],
              [np.sum(X),   len(X)]])
B = np.array([np.sum(X*Yl), np.sum(Yl)])

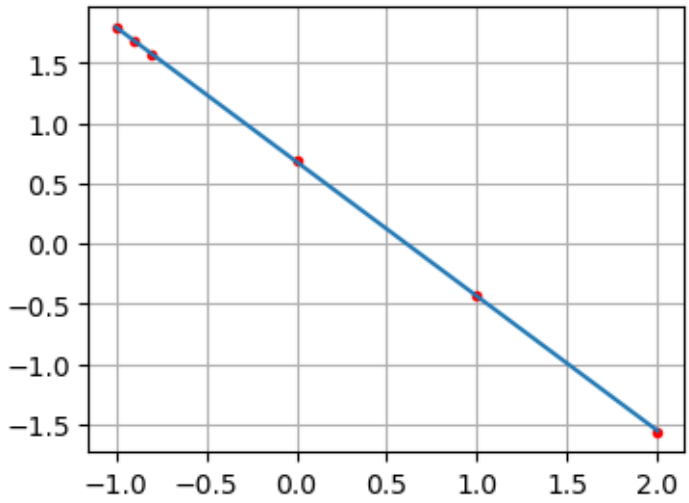
a = np.linalg.solve(A, B)
print (a)

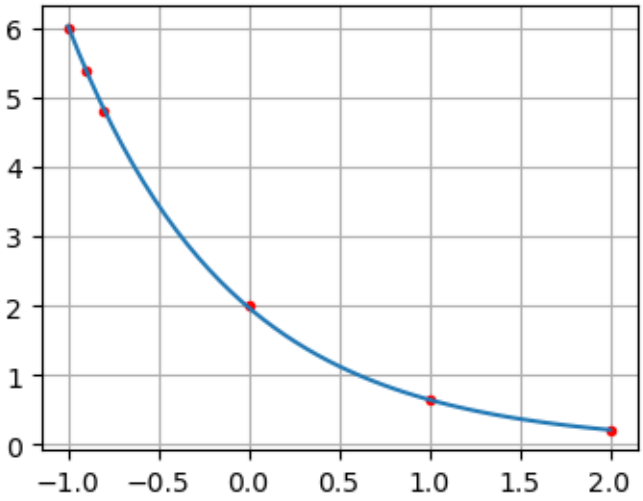
# lista de pontos para os plots
Xr = np.linspace(X[0], X[-1], 51)
#reta
h = lambda X: a[0]*X + a[1]
# plotando
plt.plot(X, Yl, "r.", Xr, h(Xr), "-")
plt.grid()
plt.show()

# hipérbole
k = np.exp(a[1])
b = np.exp(a[0])
g = lambda X: k*np.power(b,X)

Yr = g(Xr)
# plotando
plt.plot(X, Y, "r.", Xr, g(Xr), "-")
plt.grid()
plt.show()
```

```
[-1.11689997  0.68139335]
```



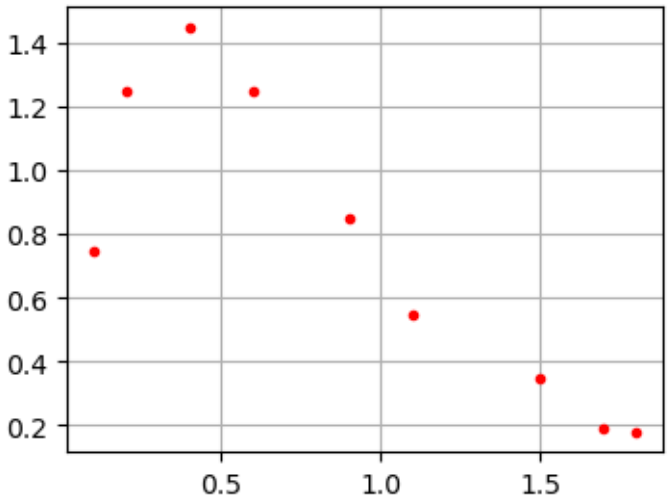

**Exemplo 5.3.3:** Linearize o modelo matemático

$$y = \alpha x e^{\beta x}$$

e encontre os parâmetros $\alpha$ e $\beta$ que fornecem o melhor ajuste com base nos seguintes dados.

| $x$ | 0,1 | 0,2 | 0,4 | 0,6 | 0,9 | 1,1 | 1,5 | 1,7 | 1,8 |
|---|---|---|---|---|---|---|---|---|---|
| $y$ | 0,75 | 1,25 | 1,45 | 1,25 | 0,85 | 0,55 | 0,35 | 0,19 | 0,18 |

```
x = np.array([0.1, 0.2, 0.4, 0.6, 0.9, 1.1, 1.5, 1.7, 1.8])
y = np.array([0.75, 1.25, 1.45, 1.25, 0.85, 0.55, 0.35, 0.19, 0.18])
# plotando
plt.plot(x,y, "r.")
plt.grid()
plt.show()
```



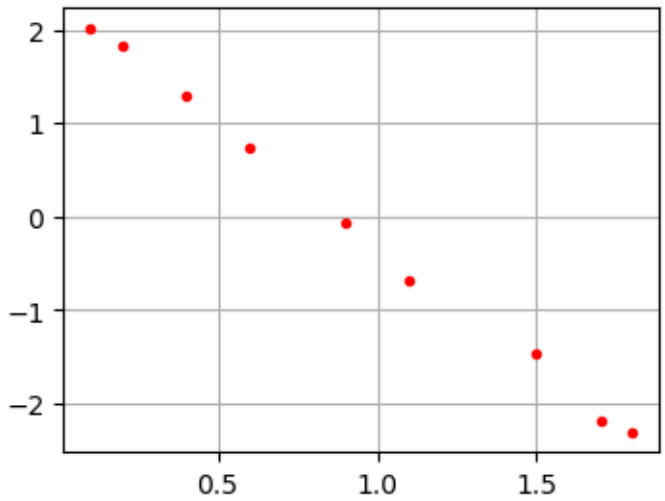

fazendo $\ln(y) = \ln(\alpha x e^{\beta x}) \ln(y) = \ln(\alpha) + \ln(x) + \beta x \ln(y) - \ln(x) = \beta x + \ln(\alpha)$
escrevendo $y' = a_1 x + a_2$
sendo $y' = \ln(y) - \ln(x)$

$$a_1 = \beta$$

$$a_2 = \ln(\alpha)$$

obtemos os pontos alinhados

```
yl = np.log(y)-np.log(x)
plt.plot(x,yl, "r.")
plt.grid()
plt.show()
```




```
A = np.array([[np.sum(x * x), np.sum(x)],
              [np.sum(x),     len(x)]])
```

(continua na próxima página)

(continuação da página anterior)

```
B = np.array([np.sum(x*yl),
              np.sum(yl)])

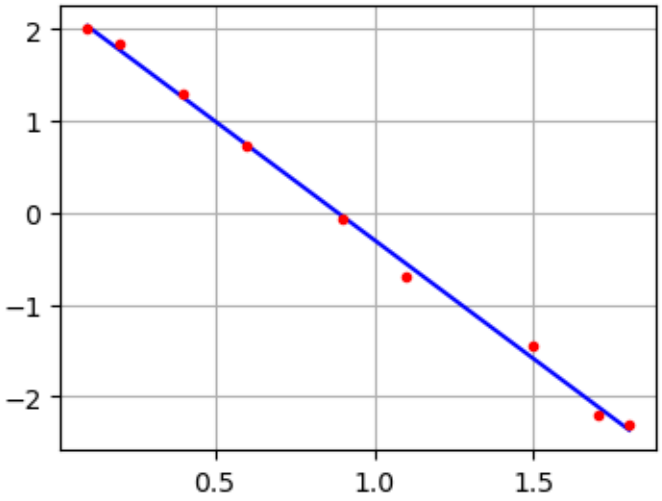
a = np.linalg.solve(A, B)

alpha = np.exp(a[1])
beta = a[0]
print(alpha, beta)
```

```
9.910716948702433 -2.5870818862156093
```

```
h = lambda x: a[0]*x + a[1]

xr = np.linspace(x[0], x[-1], 20)
plt.plot(xr,h(xr), "b-", x,yl, "r.")
plt.grid()
plt.show()
```




```
alpha = np.exp(a[1])
beta = a[0]
print(alpha, beta)
```

```
9.910716948702433 -2.5870818862156093
```

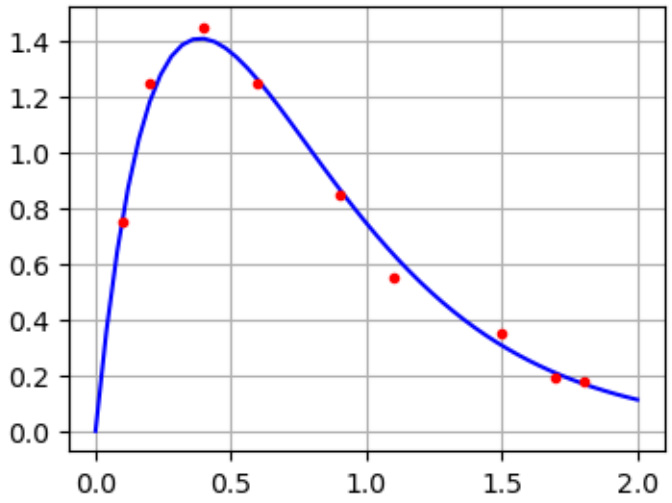
```
g = lambda x: alpha*x*np.exp(beta*x)
```

```
xr = np.linspace(0,2, 51)
plt.plot(xr,g(xr), "b-", x,y, "r.")
plt.grid()
```

(continua na próxima página)

(continuação da página anterior)

```
plt.show()
```




Resposta: $\alpha = 9.6618$ e $\beta = -2.4733$

Exemplos de linearização de algumas funções (Chapra e Canale, 2016):

## Exercícios

---

**1.** Considere os dados de um experimento conforme tabela: $

| $x_i$ | $-9$ | $-6$ | $-4$ | $-2$ | $0$ | $2$ | $4$ |
|---|---|---|---|---|---|---|---|
| $f(x_i)$ | $30.1$ | $10.1$ | $8.9$ | $5.9$ | $5.0$ | $3.9$ | $4.01$ |

$

Usando o método dos mínimos quadrados, ajustar aos dados da tabela funções:

a) Hiperbólica.

b) Exponencial.

c) Compare os resultados obtidos.

---

**2.** Ajuste os dados abaixo (número de bactérias por unidade de volume) à curva $y = ab^x$. Plote o gráfico da função obtida juntamente com os dados tabelados.

| nº de horas | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| nº de bactérias | 32 | 47 | 65 | 92 | 132 | 190 | 275 |

---

**3.** (Fonte)Fornecidos os dados:

| $x_i$ | $4,0$ | $4,2$ | $4,5$ | $4,7$ | $5,1$ | $5,5$ | $5,9$ | $6,3$ | $6,8$ | $7,1$ |
|---|---|---|---|---|---|---|---|---|---|---|
| $y_i$ | $102,56$ | $113,18$ | $130,11$ | $142,05$ | $167,53$ | $195,14$ | $224,87$ | $256,73$ | $299,50$ | $326,72$ |

a) Construa a aproximação por mínimos quadrados do tipo $be^{ax}$ e calcule o erro.

b) Construa a aproximação por mínimos quadrados do tipo $bx^a$ e calcule o erro.

---

**4.** (Fonte) Os dados tabulados a seguir foram gerados a partir de uma experiência que continha inicialmente cianeto de amônio puro ($NH_4OCN$). Sabe-se que tais mudanças de concen- tração podem ser modeladas pela seguinte equação:

$$c = \frac{c_0}{1 + kc_0t}$$

em que $c_0$ e $k$ são parâmetros. Use uma transformação para linearizar essa equação. A seguir, use regressão linear para prever a concentração em $t = 160$ min.

| t( min) | 0 | 20 | 50 | 65 | 150 |
|---|---|---|---|---|---|
| c(mols/L) | $0,381$ | $0,264$ | $0,180$ | $0,151$ | $0,086$ |

---

**5.** (Fonte) O modelo a seguir é usado frequentemente em engenharia ambiental para parametrizar o efeito da temperatura $T$ ($°C$) nas taxas de reações químicas $k$ (por dia),

$$k = k_{20}\theta^{T-20}$$

em que $k_{20}$ e $\theta$ são parâmetros. Use uma transformação para linearizar essa equação. A seguir, use regressão linear para estimar $k_{20}$ e $\theta$ e para prever a taxa de reação em $T = 17°C$.

| $T$ (°C) | 6 | 12 | 18 | 24 | 30 |
|---|---|---|---|---|---|
| $k$ (por dia) | $0,14$ | $0,20$ | $0,31$ | $0,46$ | $0,69$ |

---

**6.** (Fonte) A equação de Andrade foi proposta como um modelo para o efeito da temperatura sobre a viscosidade,

$$\mu = De^{B/T_a}$$

onde $\mu$ é a viscosidade dinâmica da água ($10^{-3}N \cdot s/m^2$), $T_a$ é a temperatura absoluta ($K$), e $D$ e $B$ são parâmetros. Ajuste esse modelo aos dados para a água

| $T$ | 0 | 5 | 10 | 20 | 30 | 40 |
|---|---|---|---|---|---|---|
| $\mu$ | $1,787$ | $1,519$ | $1,307$ | $1,002$ | $0,7975$ | $0,6529$ |

---

# 6. Integração Numérica

Muitas integrais definidas não podem ser calculadas analiticamente, ou o integrando só é conhecido em pontos discretos. Os métodos de integração numérica aproximam o valor de $\int_a^b f(x)\,dx$ utilizando somas ponderadas de valores da função.

### Objetivos de aprendizagem

- Compreender a formulação geral das fórmulas de Newton-Cotes
- Aplicar a regra dos trapézios simples e composta
- Aplicar as regras de Simpson 1/3 e 3/8
- Estimar e controlar o erro de integração numérica

### Conteúdo


| Seção | Tema |
|---|---|
| 6.1 | Introdução à integração numérica |
| 6.2 | Método de Newton-Cotes |
| 6.3 | Regra dos trapézios |
| 6.4 | Regras de Simpson |


## 6.1 Integração Numérica

A integração numérica é útil em situações nas quais não se conhece a expressão analítica da função a ser integrada ou quando a função primitiva é difícil de se obter. Uma forma de se calcular uma aproximação numérica para uma integral é por meio da integração do polinômio interpolador. Esse processo é conhecido como quadratura de Newton-Cotes e é desenvolvido por meio das sistemáticas conhecidas como *Regra dos Trapézios*, *Regra 1/3 de Simpson* e *Regra 3/8 de Simpson*. Antes de apresentarmos as fórmulas de Newton-Cotes, vamos relembrar um pouco do Cálculo.

Considere uma função $f(x)$ contínua no intervalo $[a, b]$. Então, sabemos que

$$I = \int_a^b f(x)dx = F(b) - F(a)$$

em que a função $F(x)$ é a primitiva (ou antiderivada) de $f$, ou seja, é tal que $F'(x) = f(x)$.

Graficamente, considerando a função $f(x) \geq 0$, para todo $x \in [a, b]$ podemos interpretar a intergral $I = \int_a^b f(x)dx$ como a área entre a curva de $f$ e o eixo das abcissas:

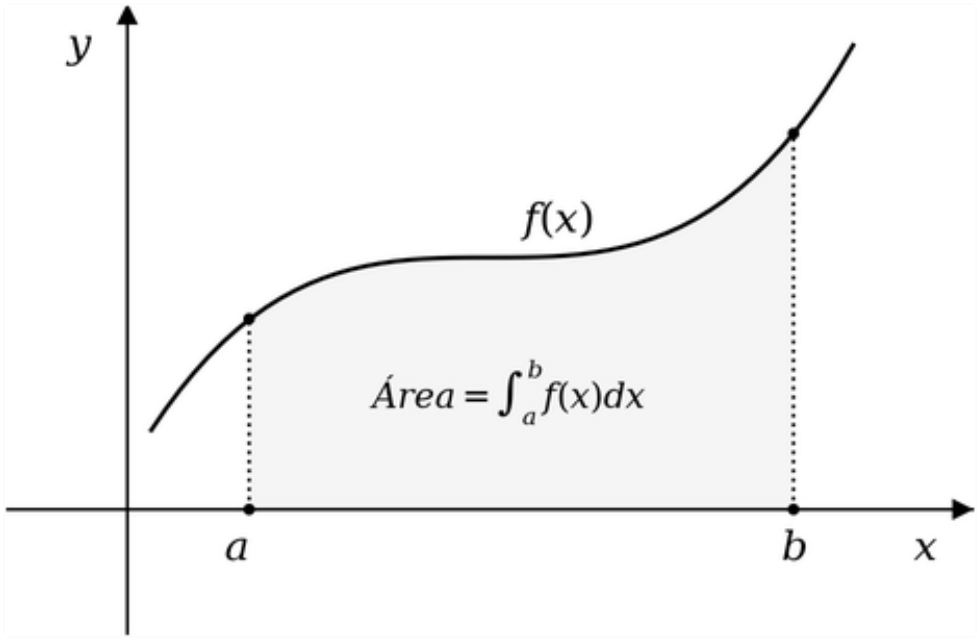



Figura 6.1.1 - A integral definida como área sob o gráfico

## Soma de Riemann

A forma mais simples de obter um valor aproximado para uma integral definida é por meio da soma de Riemann. Para isto, divide-se o intervalo de integração $(a, b)$ em $n$ subintervalos e somam-se as áreas dos retângulos definidos em cada subintervalo. A largura de cada retângulo será dado por $h = \frac{x_n - x_0}{n}$ e a altura será dada por $f(x)$ para algum $x$ dentro do subintervalo $[x_i, x_{i+1}]$, com $i = 0, 1, ..., n$. Uma escolha natural para o valor de $x$ é um dos extremos do intervalo, o extremo esquerdo $x_i$ (“pela esquerda”) ou o extremo direito $x_{i+1}$ (“pela direita”):

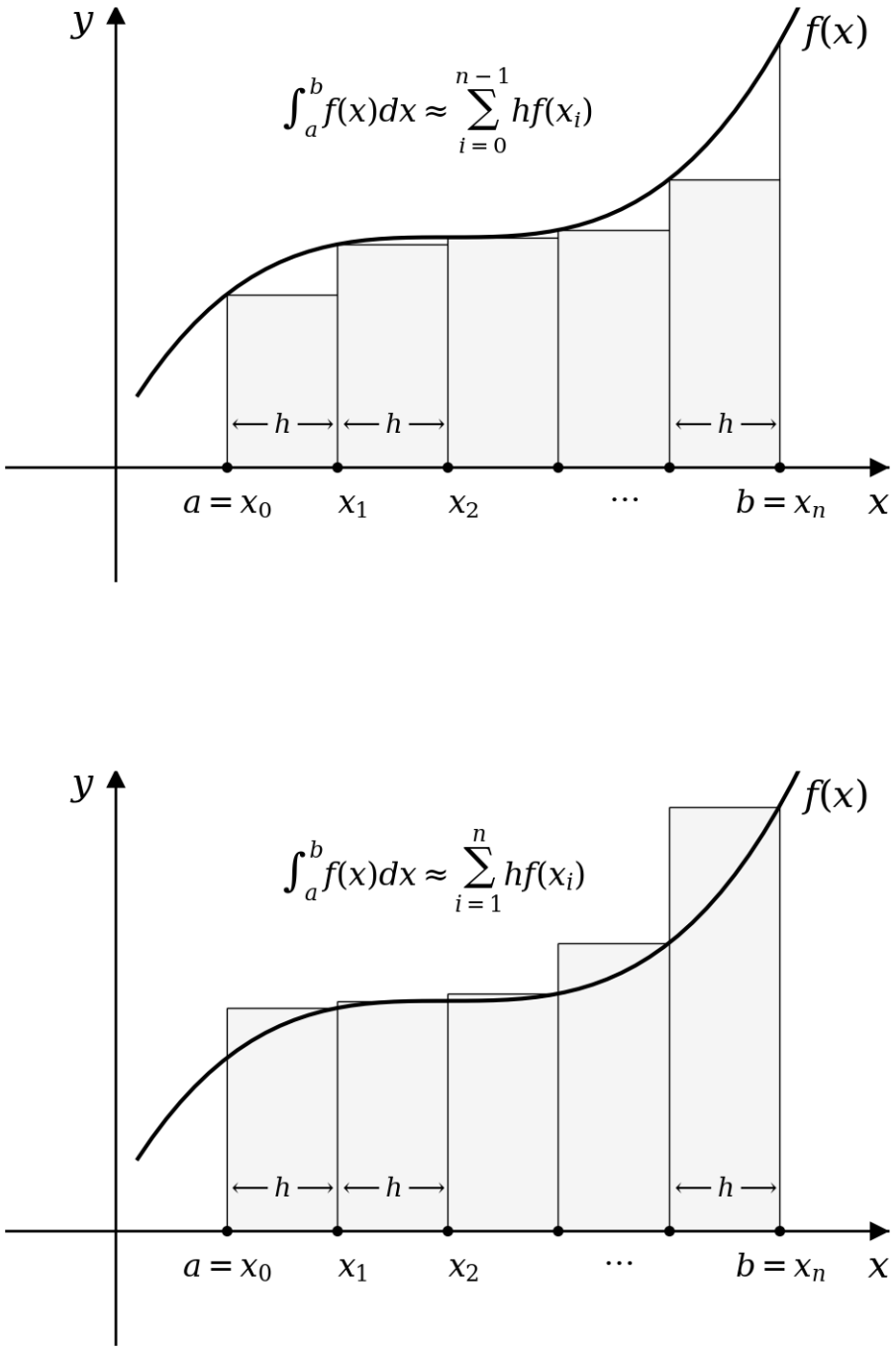



Figura 6.1.2 - Somas de Riemann “pela direita” e “pela esquerda”

Claramente podemos aumentar a acurácia da aproximação obtida aumentando o número de subintervalos. A *regra do ponto médio* considera a altura do retângulo definida em cada subintervalo como sendo o valor da função no ponto médio

entre $x_i$ e $x_{i+1}$:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf\left(\frac{x_i + x_{i+1}}{2}\right)$$

**Exemplo 6.1.1:** Nesse exemplo vamos calcular a integral $\int_{0.1}^{0.8} x^2 ln(x)dx$ usando aproximações da soma de Riemann pela esquerda, pela direita e pela regra do ponto médio, e comparar os resultados. Então vamos comparar com a solução exata ($\approx -0.0940934155044049$).

```
from math import log
```

```
F = lambda x: x**3*log(x)/3 - x**3/9
I = F(0.8)-F(0.1)
print(I)
```

```
-0.0940934155044049
```

Aproximado pela soma de Riemann com 7 subintervalos, ou seja, $h = 0.1$

```
import numpy as np
xi = np.linspace(0.1, 0.8, 8)
h = xi[1]-xi[0]
print ("h=",h)
```

```
h= 0.1
```

```
f = lambda x: x**2*np.log(x)
```

```
Ie = h*np.sum(f(xi[0:-1]))
print("Ie=",Ie, "Erro=",abs(I-Ie))
```

```
Ie= -0.08743221791422254 Erro= 0.006661197590182369
```

```
Id = h*np.sum(f(xi[1:]))
print("Id=",Id, "Erro=",abs(I-Id))
```

```
Id= -0.0994108201053379 Erro= 0.005317404600933009
```

```
Im = h*np.sum(f((xi[0:-1]+xi[1:])/2))
print("Im=",Im, "Erro=",abs(I-Im))
```

```
Im= -0.09443022564968762 Erro= 0.00033681014528270936
```

É possivel notar que a regra do ponto médio forneceu a melhor aproximação.

De modo geral, os métodos numéricos para cálculo de integrais definidas consistem em aproximar a função $f(x)$ pelo polinômio interpolador $P(x)$ e integrar o polinômio. Os casos ilustrados acima são polinômios de grau zero, mas

poderiamos utilizar polinômios de grau maior.

Assim, de forma geral temos:

$$\int_a^b f(x)dx \approx \int_{x_0}^{x_n} P(x)dx$$

Na sequência serão apresentados três métodos para integração numérica baseados na integração do polinômio interpolador conhecidos como Regras de Newton-Cotes. São eles, a regra dos trapézios, a regra 1/3 de Simpson e a regra 3/8 de Simpson.

## Soma dupla de Riemann

Considerando uma função $f$ de duas variáveis definida em um retângulo fechado $R = [a,b] \times [c,d] = \{(x,y) \in \mathbb{R}^2 \mid a \leq x \leq b, c \leq y \leq d\}$

e supondo que $f(x,y) \geq 0$. O gráfico de $f$ é a superficie com equação $z = f(x,y)$. Seja $S$ o sólido que está acima da região $R$ e abaixo do gráfico de $f$, isto é, $S = \{(x,y,z) \in \mathbb{R}^3 \mid 0 \leq z \leq f(x,y), (x,y) \in R\}$

então, o volume do sólido $S$ pode ser aproximado por

$$V \approx \sum_{i=1}^{m} \sum_{j=1}^{n} f\left(x_{ij}^*, y_{ij}^*\right) \Delta A$$

em que $\left(x_{ij}^*, y_{ij}^*\right)$ são *pontos de amostragem* sobre a superfíce e $f\left(x_{ij}^*, y_{ij}^*\right) \Delta A$ é o volume de uma caixa fina, ou "coluna", com base $R_{ij}$ de área $\Delta A = \Delta x \Delta y$ e altura $f\left(x_{ij}^*, y_{ij}^*\right)$.

A integral dupla de $f$ sobre o retângulo $R$ é definida como $\iint_R f(x,y)dA = \lim_{m,n\to\infty} \sum_{i=1}^{m} \sum_{j=1}^{n} f\left(x_{ij}^*, y_{ij}^*\right) \Delta A$ se esse limite existir.

A soma dupla de Riemann na qual o ponto de amostragem $\left(x_{ij}^*, y_{ij}^*\right)$ em $R_{ij}$ é tomado como o ponto central $\left(\bar{x}_i, \bar{y}_j\right)$ de $R_{ij}$, é dada por:

$$\iint_R f(x,y)dA \approx \sum_{i=1}^{m} \sum_{j=1}^{n} f\left(\bar{x}_i, \bar{y}_j\right) \Delta A$$

onde $\bar{x}_i$ é o ponto médio de $[x_{i-1}, x_i]$ e $\bar{y}_j$ é o ponto médio de $\left[y_{j-1}, y_j\right]$.

### Teoria de Fubini

Se $f$ for continua no retângulo

$$R = \{(x,y) \mid a \leq x \leq b, c \leq y \leq d\}$$

então, $\iint_R f(x,y)dA = \int_a^b \int_c^d f(x,y)dydx = \int_c^d \int_a^b f(x,y)dxdy$

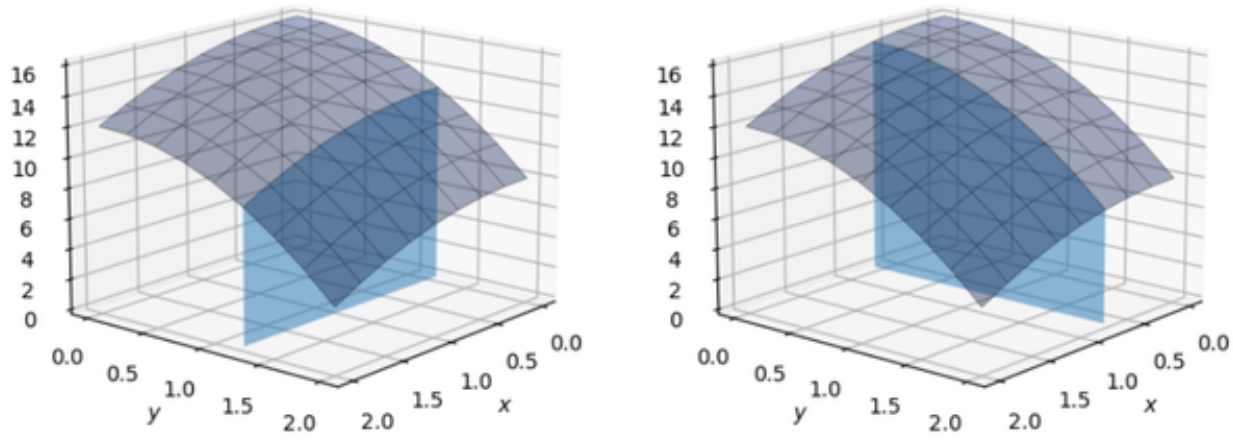


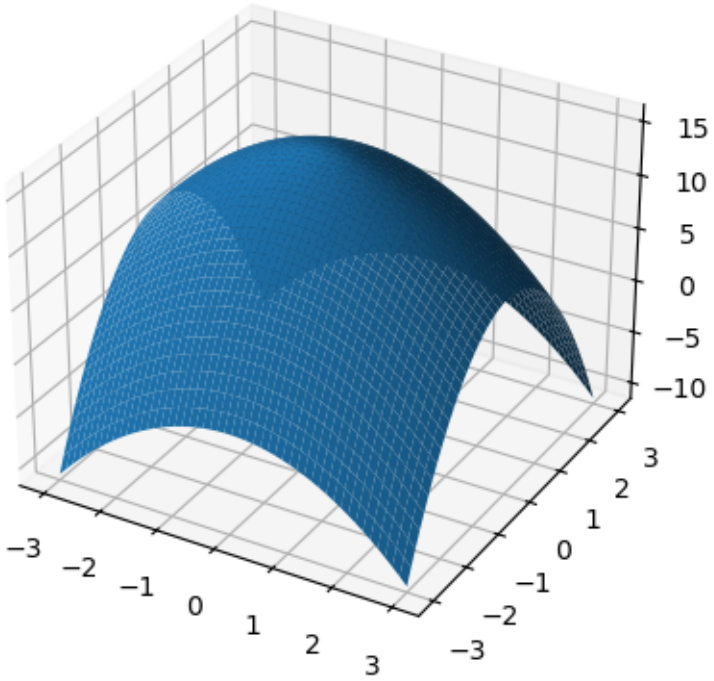

Figura 6.1.3 - Integral iterada

**Exemplo 6.1.2:** *(Integração dupla por soma de Riemann)* Vamos agora plotar o gráfico da função, estimar o volume do sólido usando somas de Riemann e comparar o resultado numérico com o valor exato de (48 unidades de volume).

```
import numpy as np
import matplotlib.pyplot as plt

xi, yi = np.meshgrid(np.linspace(-3,3),np.linspace(-3,3))
zi = 16- xi**2 - 2*yi**2

fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.plot_surface(xi, yi, zi)
plt.show()
```




Dividindo a região $R = \{(x, y) \mid 0 \leq x \leq 2, 0 \leq y \leq 2\}$ em $4 \times 4$ quadrados de lados $dx = dy = 0,5$ e fazendo a soma dos volumes das "caixas" com base $R_{ij}$, $i, j = 1, 2, 3, 4$ de áreas $\Delta A = \Delta x \Delta y = 0,5^2 = 0,25$ e altura $f\left(x_{ij}^*, y_{ij}^*\right)$ em que $(x_{ij}^*, y_{ij})$ são os pontos do canto superior direito de cada quadrado $R_{ij}$, obtemos uma aproximação.

```
dx=0.5; dy=0.5

# coordenadas dos pontos (x,y)
xi, yi = np.meshgrid(np.arange(dx,2+dx,dx), np.arange(dy,2+dy,dy))

# cálculo das alturas z = f(x,y)
f = lambda x,y: 16- x**2 - 2*y**2

# soma dupla de Riemann
V = np.sum(f(xi,yi))*dx*dy
print ('Soma dupla de Riemann:', V)
```

```
Soma dupla de Riemann: 41.5
```

Portanto a aproximação por soma de Riemann com 4 subintervalos na direção de $x$ e 4 subintervalos na direção de $y$ resultou em um volume aproximado de $41.5$, ou seja, um resultado com um erro relativo $\epsilon = 0.135417$, ou seja, na ordem de $13.5\%$.

A seguir é mostrada uma figura representando a superfície e a aproximação da integral por soma de Riemann.

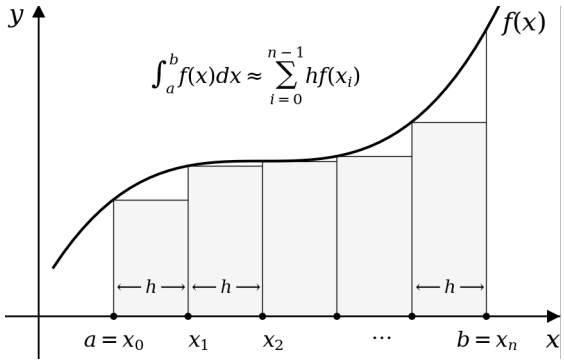



Figura 6.1.4 - Aproximação por soma de Riemann com 4 subintervalos

Quanto maior é o número de divisões da região $R$ melhor será a aproximação, como podemos ver na imagem abaixo:

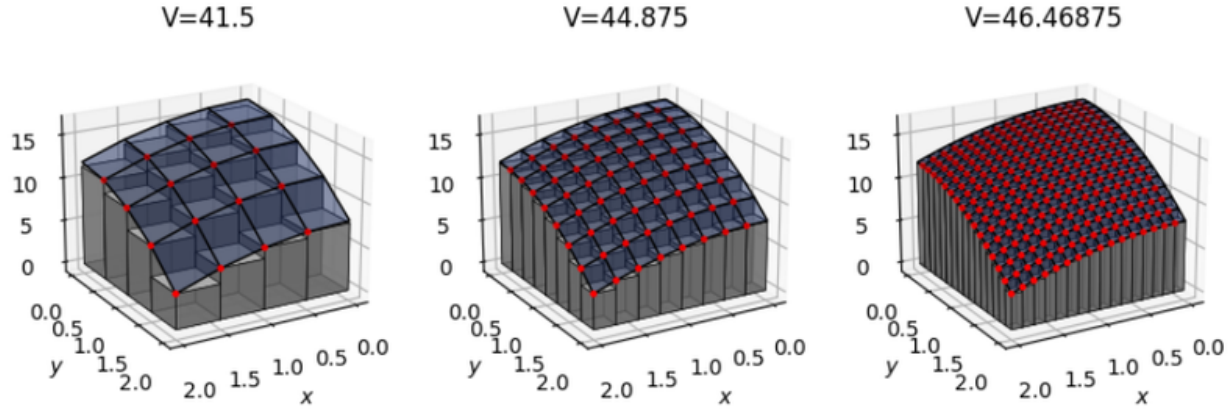



Figura 6.1.5 - Aproximação do volume com diferentes discretizações da região R (Ver ANEXO II)

Para obter a aproximação pela regra do ponto médio, podemos fazer:

```
V = np.sum(f(xi-dx/2,yi-dy/2))*dx*dy
print ('Pela regra do ponto médio: V=', V)
```

```
Pela regra do ponto médio: V= 48.25
```

## Exercícios

Fonte: Stewart (2022, v 1)

---

**1.** (a) Estime a área sob o gráfico $f(x) = 1/x$ de $x = 1$ até $x = 2$ usando oito retângulos aproximantes e extremidades direitas. Esboce o gráfico e os retângulos. Sua estimativa é uma subestimativa ou uma superestimativa? (b) Repita a parte (a) usando extremidades esquerdas.

---

**2.** (a) Estime a área sob o gráfico de $f(x) = \text{sen}\, x$ de $x = 0$ até $x = \pi/2$ usando oito retângulos aproximantes e extremidades direitas. Esboce o gráfico e os retângulos. Sua estimativa é uma subestimativa ou uma superestimativa? (b) Repita a parte (a) usando extremidades esquerdas.

---

**3.** (a) Estime a área sob o gráfico $f(x) = 1+x^2$ de $x = -1$ até $x = 2$ usando quatro retângulos aproximantes e extremidades direitas. Então, aperfeiçoe sua estimativa utilizando oito retângulos aproximantes. Esboce a curva e os retângulos aproximantes. (b) Repita a parte (a) usando extremidades esquerdas. (c) Repita a parte (a) empregando pontos médios. (d) A partir de seus esboços das partes (a), (b) e (c), qual parece ser a estimativa mais precisa?

---

**4.**(a) Faça o gráfico da função

$$f(x) = e^{x-x^2} \quad 0 \leq x \leq 2$$

(b) Estime a área sob o gráfico de $f$ usando quatro retângulos aproximantes e tomando como pontos amostrais (i) as extremidades direitas e (ii) os pontos médios. Em cada caso, esboce a curva e os retângulos. (c) Aperfeiçoe suas estimativas da parte (b) usando oito retângulos.
(Fonte: Stewart (2022, v2))

---

**5.** (a) Estime o volume do sólido que está abaixo da superfície $z = xy$ e acima do retângulo $$R = \{(x, y) \mid 0 \leq x \leq 6, 0 \leq y \leq 4\}$$
Utilize a soma de Riemann com $m = 6, n = 4$ e tome como ponto de amostragem o canto superior direito de cada sub-retângulo.
(b) Use a Regra do Ponto Médio para estimar o volume do sólido da parte (a).

---

**6.** Se $R = [0, 4] \times [-1, 2]$, use a soma de Riemann com $m = 4, n = 6$ para estimar o valor de $\iint_R (1 - xy^2)\, dA$. Tome os pontos de amostragem como (a) os cantos inferiores direitos e (b) como os cantos superiores esquerdos dos retângulos.

---

**7.** (a) Use uma soma de Riemann com $m = n = 8$ para estimar o valor de $\iint_R xe^{-xy} dA$, onde $R = [0, 2] \times [0, 1]$. Tome os pontos de amostragem como os cantos superiores direitos.
(b) Use a Regra do Ponto Médio para dar uma estimativa da integral do item (a).

---

**8.** (a) Estime o volume do sólido que está abaixo da superfície $z = 1 + x^2 + 3y$ e acima do retângulo $R = [1, 2] \times [0, 3]$. Use a soma de Riemann com $m = n = 8$ e escolha os pontos de amostragem como os cantos inferiores esquerdos.
(b) Use a Regra do Ponto Médio para estimar o volume do item (a).

---

## 6.2 Regra dos trapézios

### Regra dos trapézios em uma variável

Consiste am aproximar a integral da função em um intervalo [*a*,*b*] pela integral do polinômio de grau 1 que passa pelos pontos (*a*,*f*(*a*)) e (*b*,*f*(*b*)). Se *f*(*x*)>0 em [*a*,*b*] a área entre o gráfico da função e o eixo *x* é aproximado pela área do trapézio de altura *h*=*b*−*a* e bases *f*(*a*) e *f*(*b*).

Considere uma função $f(x)$ definida nos pontos $x_0$ e $x_1$ no intervalo $[a, b]$. O polinômio de Lagrange para esses pontos, considerando a mudança de variável $u = \frac{x-x_0}{h}$, é dado por

$$P(u) = f(x_0)(1-u) + f(x_1)(u)$$

e a integral pode ser aproximada como segue

$$\int_{x_0}^{x_n} f(x)dx \approx \int_{x_0}^{x_1} P_1(x)dx = h\int_0^1 P_1(u)du$$

ou, ainda,

$$\begin{aligned} h\int_0^1 P_1(u)du &= h\int_0^1 [f(x_0)(1-u) + f(x_1)(u)]du \\ &= hf(x_0)\int_0^1 (1-u)du + hf(x_1)\int_0^1 (u)du \end{aligned}$$

que resulta em

$$\int_{x_0}^{x_n} f(x)dx \approx \frac{h}{2}[f(x_0) + f(x_1)]$$

ou seja, tem-se aqui a fórmula da área do trapézio.

Graficamente, tem-se

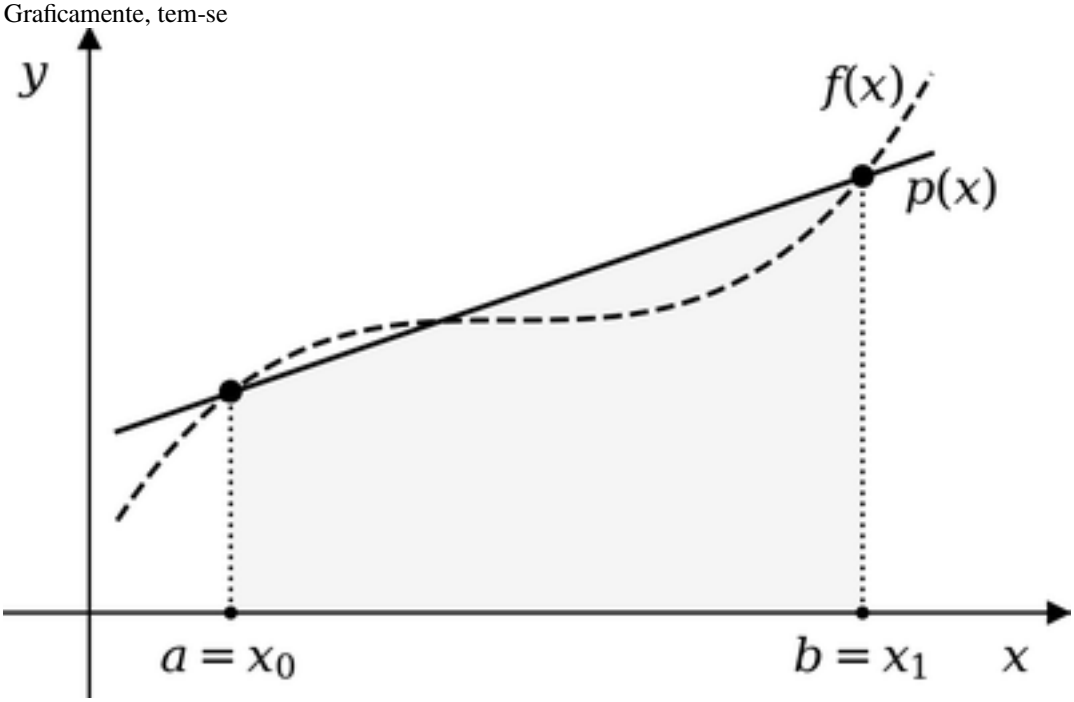



Figura 6.2.1 - Interpretação geométrica da regra dos trapézios

Um **limitante superior** para o erro será dado por

$$|E_1| \leq \frac{h^3}{12} \max .\{|f^{(2)}(x)|, x_0 \leq x \leq x_1\}$$

A **regra dos trapézios repetida** (ou generalizada), consiste na subdivisão do intervalo $[a, b]$ em $n$ subintervalos iguais, de amplitude $h = \frac{x_n - x_0}{n}$, $x_0 = a$ e $x_n = b$ e na aplicação da regra dos trapézios repetidamente a cada dois pontos

consecutivos.

Assim, temos

$$\begin{aligned}\int_{x_0}^{x_n} f(x)dx &\approx \frac{h}{2}[f(x_0)+f(x_1)]+\frac{h}{2}[f(x_1)+f(x_2)]+\cdots+\frac{h}{2}[f(x_{n-1})+f(x_n)]\\ &= \frac{h}{2}[f(x_0)+2f(x_1)+2f(x_2)+\cdots+2f(x_{n-1})+f(x_n)]\\ &= \frac{h}{2}\{f(x_0)+2[f(x_1)+f(x_2)+\cdots+f(x_{n-1})]+f(x_n)\}\end{aligned}$$

E o **limitante superior** para o erro será dado por

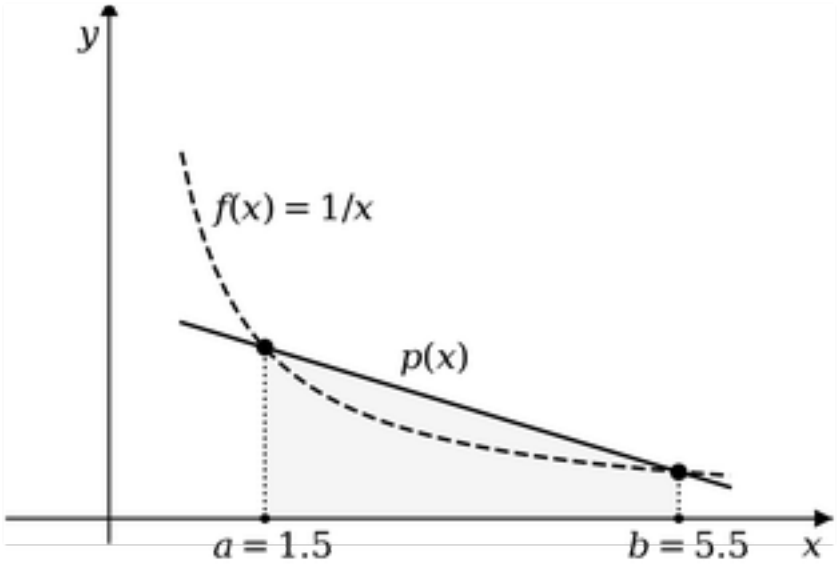
$$|E_t| \leq \frac{h^2}{12}(x_n - x_0)\max.\{|f^{(2)}(x)|, x_0 \leq x \leq x_n\}$$

**Exemplo 6.2.1:** Vamos calcular o valor aproximado da integral

$$\int_{1.5}^{5.5} \frac{1}{x}dx$$

usando a regra dos trapézios.

*Solução:* Basta fazer $\int_{1.5}^{5.5} \frac{1}{x}dx \approx \frac{4}{2}[f(1.5)+f(5.5)] = 2\left(\frac{1}{1.5}+\frac{1}{5.5}\right) \approx 1.697$




Resolvendo analiticamente o valor exato é $\ln(5.5)$ - $\ln(1.5)$ que pode ser calculado usando uma calculadora obtendo-se $1.2992829841302609$. O valor obtido pela regra do trapézio é maior, como pode ser verificado observando na figura, comparando-se as áreas. Vamos usar a regra dos trapézios repetida para 5, 10 e 100 subintervalos para ver como fica.

```
import numpy as np
```

```
x0 = 1.5; xn = 5.5
f = lambda x: 1/x

for n in [5,10,100]:
    h = (xn-x0)/n
    xi = np.linspace(x0, xn, n+1)
    It = h*((f(xi[0])+f(xi[-1]))/2 + np.sum(f(xi[1:-1])))
    print ("n=",n,", I=",It)
```

```
n= 5 , I= 1.3206255135651455
n= 10 , I= 1.304727563368882
n= 100 , I= 1.2993378314867685
```

A biblioteca `numpy` já traz uma implementação da regra dos trapézios. Veja como usar no exemplo a seguir com os pontos, o espaçameto e a função definidos acima.

```
np.trapz(f(xi),dx=h)
```

```
1.2993378314867685
```

A biblioteca `scipy` também possui funções para integrais definidas. Para saber mais pesquise por `scipy.integrate`.

**Exemplo 6.2.2:** Vamos calcular o valor aproximado da integral

$$\int_1^4 \sqrt{x}dx$$

usando a regra dos trapézios generalizada para 2, 4 e 6 subintervalos e obter um limitante superior para o erro.

```
f = lambda x: np.sqrt(x)
xi = np.linspace(1, 5, 6)
np.trapz(f(xi), xi)
```

```
6.772340295647981
```

```
xi = np.linspace(1, 5, 11)
np.trapz(f(xi), xi)
```

```
6.78322070363297
```

```
xi = np.linspace(1, 5, 101)
np.trapz(f(xi), xi)
```

```
6.786856407214704
```

## Regra dos trapézios para integral dupla

(Arenales e Darezo, 2016) Sabemos que a **regra dos trapézios** repetida (ou generalizada) para integração numérica de uma função de uma variável no intervalo $[a, b]$ é dada por

$$\int_{x_0}^{x_n} F(x)dx \approx \frac{h}{2}\{F(x_0) + 2[F(x_1) + F(x_2) + \cdots + F(x_{n-1})] + F(x_n)\}$$

em que $n$ é o número de subintervalos, de amplitudes iguais a $h = \frac{x_n - x_0}{n}$, com $x_0 = a$ e $x_n = b$.

Para integrais duplas fazemos

$$F(x_i) = \int_c^d f(x_i, y)\, dy \simeq \frac{h}{2}\left[f(x_i, y_0) + 2\left(f(x_i, y_1) + \ldots + f(x_i, y_{n-1})\right) + f(x_i, y_n)\right]$$

com $i = 0, \ldots, n$.

Os $(n + 1)$ valores $F(x_i)$ podem ser calculados por qualquer método de integração numérica e, com estes valores, podemos calcular o valor aproximado de

$$I = \int_a^b \int_c^d f(x, y)dydx$$

**Exemplo 6.2.3:** (Arenales e Darezo, 2016) Calcule o valor da integral dupla

$$\int_{0.1}^{1.0} \int_{0.1}^{0.5} \ln(x + y)dydx$$

usando a regra dos trapézios generalizada para $h_x = 0.3$, com os pontos $x_0 = 0.1$, $x_1 = 0.4, x_2 = 0.7$ e $x_3 = 1.0$ e $F(x_i), i = 0, \ldots, 3$, com $h_y = 0.1$ e $y_0 = 0.1, y_1 = 0.2, y_2 = 0.3, y_3 = 0.4, y_4 = 0.5$.

```
# dados do problema
x0 = 0.1; xn = 1.0
y0 = 0.1; yn = 0.5
nx = 3
ny = 4

hx = (xn-x0)/nx
hy = (yn-y0)/ny

x = np.arange(x0,xn+hx,hx)
y = np.arange(y0,yn+hy,hy)

print(x)
print(y)
```

```
[0.1 0.4 0.7 1. ]
[0.1 0.2 0.3 0.4 0.5]
```

```
f = lambda x,y: np.log(x+y)
```

```
Fx = []
for i in range(len(x)):
    Fxi = (hy/2)*(f(x[i],y[0]) +
                  2*np.sum(f(x[i],y[1:-1])) +
                  f(x[i],y[-1]))
    Fx.append(Fxi)

print(Fx)
```

```
[-0.3873542484860082, -0.14898979671278184, -0.0030461333113628686, 0.
↪1031545701838904]
```

```
It = (hx/2)*((Fx[0] + 2*np.sum(Fx[1:-1]) + Fx[-1]))
print ("I =",It)
```

```
I = -0.08824073075256109
```

**Exemplo 6.2.4:** *(Integração dupla pela regra dos trapézios)* Vamos agora usar a regra dos trapézios para calcular a integral dupla $\int_0^2 \int_0^2 (16 - x^2 - 2y^2) dx dy$.

Vamos criar uma função para a regra dos trapéxios generalizada:

```
def trapz2D(f,x0,xn,hx,y0,yn,hy):
    x = np.arange(x0,xn+hx,hx)
    y = np.arange(y0,yn+hy,hy)

    Fx = []
    for i in range(len(x)):
        Fxi = (hy/2)*(f(x[i],y[0]) +
                      2*np.sum(f(x[i],y[1:-1])) +
                      f(x[i],y[-1]))
        Fx.append(Fxi)
    Fx = np.array(Fx)
    It = (hx/2)*((Fx[0] + 2*np.sum(Fx[1:-1]) + Fx[-1]))
    return It
```

```
# com hx = hy = 0.5
trapz2D(lambda x,y: 16-x**2-2*y**2,0,2,0.5,0,2,0.5)
```

```
47.5
```

```
# com hx = hy = 0.2
trapz2D(lambda x,y: 16-x**2-2*y**2,0,2,0.2,0,2,0.2)
```

```
47.92
```

## Exercícios

**1.** (Fonte) Obtenha uma aproximação das seguintes integrais utilizando a regra dos trapézios.
a) $\int_{0,5}^{1} x^4 dx$
b) $\int_{0}^{0.5} \frac{2}{x-4} dx$
c) $\int_{1}^{1,5} x^2 \ln x dx$
d) $\int_{0}^{1} x^2 e^{-x} dx$
e) $\int_{1}^{1,6} \frac{2x}{x^2-4} dx$
f) $\int_{0}^{0,35} \frac{2}{x^2-4} dx$
g) $\int_{0}^{\pi/4} x \operatorname{sen} x dx$
h) $\int_{0}^{\pi/4} e^{3x} \operatorname{sen} 2x dx$

**2.** (Fonte) Suponha que uma força para cima de resistência do ar em um objeto em queda livre seja proporcional ao quadrado da velocidade. Nesse caso, a velocidade pode ser calculada por

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right)$$

em que $c_d$ é o coeficiente de arrasto de segunda ordem. (a) Se $g = 9,81\ \text{m/s}^2, m = 68,1\ \text{kg}$ e $c_d = 0,25\ \text{kg/m}$, use integração analítica para determinar quanto o objeto cai em 10 s . (b) Faça o mesmo cálculo, mas calcule a integral com a regra dos trapézios com segmentos múltiplos. Use um $n$ suficientemente grande para que você consiga uma acurácia de três algarismos significativos.

**3.** Calcule a integral dos seguintes dados tabulares com a regra dos trapézios

| $x$ | -2 | 0 | 2 | 4 | 6 | 8 | 10 |
|---|---|---|---|---|---|---|---|
| $f(x)$ | 35 | 5 | -10 | 2 | 5 | 3 | 20 |

**4.** A função $f(x) = 2e^{-1,5x}$ pode ser usada para gerar a seguinte tabela de dados desigualmente espaçados:

| $x$ | 0 | 0,05 | 0,15 | 0,25 | 0,35 | 0,475 | 0,6 |
|---|---|---|---|---|---|---|---|
| $f(x)$ | 2 | 1,8555 | 1,5970 | 1,3746 | 1,1831 | 0,9808 | 0,8131 |

Calcule a integral de $a = 0$ a $b = 0,6$ usando a regra do trapézio e calcule o erro relativo porcentual.

**5.** De um velocímetro de um automóvel foram obtidas as seguintes leituras de velocidade instantânea:

| $t$ (min) | 0 | 4 | 8 | 12 | 16 | 20 | 24 | 28 | 32 | 36 | 40 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $v$ (km/h) | 24 | 23 | 31 | 36 | 40 | 45 | 48 | 52 | 55 | 60 | 62 |

Calcule a distância, depois de 40 minutos, em quilômetros, percorrida pelo automóvel utilizando a regra dos trapézios generalizada.

---

**6.** Calcule a seguinte integral dupla:

$$\int_{-1}^{1}\int_{0}^{2}(x^2-2y^2+xy^3)\,dxdy$$

(a) analiticamente; (b) usando a aplicação múltipla da regra do trapézio com $n = 5$, 10 e 20.

---

**7.** Calcule o valor aproximado da integral dupla

$$\int_{0}^{1}\int_{0}^{5}e^{(x+y)}dydx$$

usando a regra dos trapézios generalizada com $h = 0.2$ e $h = 0.1$.

---

## 6.3 Regras de Simpson

### Regra 1/3 de Simpson

Consiste em aproximar a integral da função pela integral de um polinômio interpolador de grau 2. Para isso dividimos o intervalo de integração subintervalos e a cada 3 pontos usamos a integral do polinômio interplolador como aproximação da integral da função nesses dois subintervalos.

Então, considere uma função $f(x)$ definida em três pontos distintos $x_0$, $x_1$ e $x_2$ equidistantes no intervalo $[a, b]$. O polinômio de Lagrange para esses pontos, considerando a mudança de variável $u = \frac{x-x_0}{h}$ é dado por

$$P_2(u) = f(x_0)\left(\frac{u^2-3u+2}{2}\right) + f(x_1)\left(\frac{u^2-2u}{-1}\right) + f(x_2)\left(\frac{u^2-u}{2}\right)$$

A integral pode ser aproximada como segue

$$\int_{x_0}^{x_n} f(x)dx \approx \int_{x_0}^{x_2} P_2(x)dx = h\int_{0}^{2} P_2(u)du$$

ou, ainda,

$$h\int_{0}^{2} P_2(u)du = hf(x_0)(u)\Big|_0^2 + h(f(x_1)-f(x_0))\left(\frac{u^2}{2}\right)\Big|_0^2 + \frac{h}{2}(f(x_2)-2f(x_1)+f(x_0))\left(\frac{u^3}{3}-\frac{u^2}{2}\right)\Big|_0^2 =$$

$$= 2hf(x_0) + 2hf(x_1) - 2hf(x_0) + \frac{h}{3}(f(x_2)-2f(x_1)+f(x_0)) =$$

$$\frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)]$$

Assim, temos

$$\int_{x_0}^{x_n} f(x)dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)]$$

conhecida como **Regra 1/3 de Simpson**.

Graficamente, tem-se

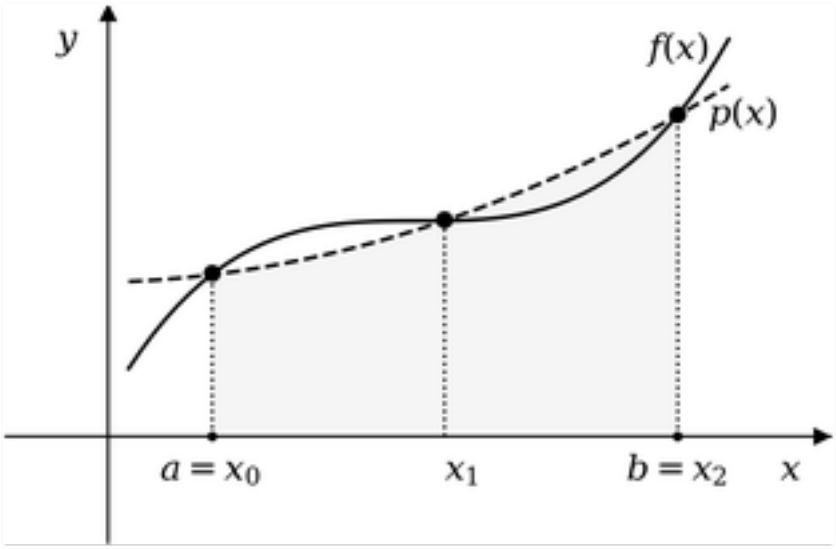



Figura 6.3.1 - Aproximação pela integral do polinômio interpolador de grau 2

Um **limitante superior** para o erro será dado por

$$|E_2| \leq \frac{h^5}{90} \max .\{|f^{(4)}(x)|, x_0 \leq \xi \leq x_2\}$$

**Exemplo 6.3.1:** Calcule o valor aproximado da integral $\int_{0.5}^{1.5} \cos(x)dx$ usando a regra 1/3 de Simpson.

```
import numpy as np
```

```
x0 = 0.5; xn = 1.5
n = 2
h = (xn-x0)/n

f = lambda x: np.cos(x)
xi = np.linspace(x0, xn, n+1)
yi = f(xi)

# Regra 1/3 de Simpson
Is = (h/3)*(yi[0] + 4*yi[1] + yi[2])
```

(continua na próxima página)

(continuação da página anterior)

```
print ("Resultado 1/3 Simpson: ", Is)
```

```
Resultado 1/3 Simpson:  0.5182548311717724
```

A biblioteca `scipy` contém diversas funções para integração numérica, dentre elas as regras de Simpson. Veja um exemplo de como utilizar a seguir:

```
import scipy.integrate as integrate
integrate.simpson(yi, x=xi)
```

```
0.5182548311717724
```

A **regra 1/3 de Simpson repetida** (ou generalizada) consiste em subdividirmos o intervalo de integração $[a, b]$ em $n$ subintervalos de ampliture $h$, que que $n$ é um número par de subintervalos, com $x_0 = a$ e $x_n = b$, então, aplicando a regra 1/3 de Simpson a cada 2 subintervalos consecutivos, obtemos

$$\int_{x_0}^{x_n} f(x)dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] + \frac{h}{3}[f(x_2) + 4f(x_3) + f(x_4)]$$
$$+ \cdots + \frac{h}{3}[f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)]$$

$$= \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)] =$$

$$= \frac{h}{3}\{f(x_0) + 4[f(x_1) + f(x_3) + \cdots + f(x_{n-1})] + 2[f(x_2) + f(x_4) + \cdots + f(x_{n-2})] + f(x_n)\}$$

Um **limitante superior** para o erro será dado por

$$|E_2| \leq \frac{h^4}{180}(x_n - x_0) \ \max . \left\{\left|f^{(4)}(x)\right|, x_0 \leq \xi \leq x_n\right\}$$

**Exemplo 6.3.2:** Calcule o valor aproximado da integral $\int_1^4 (xe^x + 3)dx$ usando a regra 1/3 de Simpson para 10, 20 e 100 subintervalos

```
x0 = 1.0; xn = 4.0
f = lambda x: x*np.exp(x)+3

for n in [10,20,100]:
    h = (xn-x0)/n
    x = np.linspace(x0, xn, n+1)
    y = f(x)
    I = (h/3)*(y[0] + 4*np.sum(y[1:n:2]) + 2*np.sum(y[2:n:2])+ y[-1])
    print ("n=",n,", I=",I)
```

```
n= 10 , I= 172.8109327815904
n= 20 , I= 172.79549085115607
n= 100 , I= 172.79445177011632
```

```
xi = np.linspace(1, 4, 11)
print(integrate.simpson(f(xi), x=xi))
```

```
172.81093278159045
```

```
xi = np.linspace(1, 4, 21)
print(integrate.simpson(f(xi), x=xi))
```

```
172.79549085115613
```

```
xi = np.linspace(1, 4, 101)
print(integrate.simpson(f(xi), x=xi))
```

```
172.79445177011632
```

## Regra 3/8 de Simpson

Consiste em aproximar a integral da função pela integral de um polinômio interpolador de grau 3. Para isso dividimos o intervalo de integração subintervalos e a cada 4 pontos usamos a integral do polinômio interpolador como aproximação da integral da função nesses 3 subintervalos.

Então, considere uma função $f(x)$ definida em quatro pontos distintos $x_0$, $x_1$, $x_2$ e $x_3$ equidistantes no intervalo $[a, b]$. O polinômio interpolador de Lagrange para esses pontos, considerando a mudança de variável $u = \frac{x-x_0}{h}$ é dado por

$$P_3(u) = f(x_0)\left[\frac{(u-1)(u-2)(u-3)}{-6}\right] + f(x_1)\left[\frac{u(u-2)(u-3)}{2}\right] + f(x_2)\left[\frac{u(u-1)(u-3)}{-2}\right] + f(x_3)\left[\frac{u(u-1)(u-2)}{6}\right]$$

ou

$$P_3(u) = f(x_0)\left[\frac{u^3-6u^2+11u-6}{-6}\right] + f(x_1)\left[\frac{u^3-5u^2+6u}{2}\right] + f(x_2)\left[\frac{u^3-4u^2+3u}{-2}\right] + f(x_3)\left[\frac{u^3-3u^2+2u}{6}\right]$$

O que fornece

$$\int_{x_0}^{x_3} f(x)dx \approx h\int_0^3 P_3(u)du = \frac{3}{8}h\left[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)\right]$$

Então

$$\int_{x_0}^{x_3} f(x)dx \approx \frac{3}{8}h\left[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)\right]$$

que consiste na regra 3/8 de Simpson.

Graficamente, tem-se

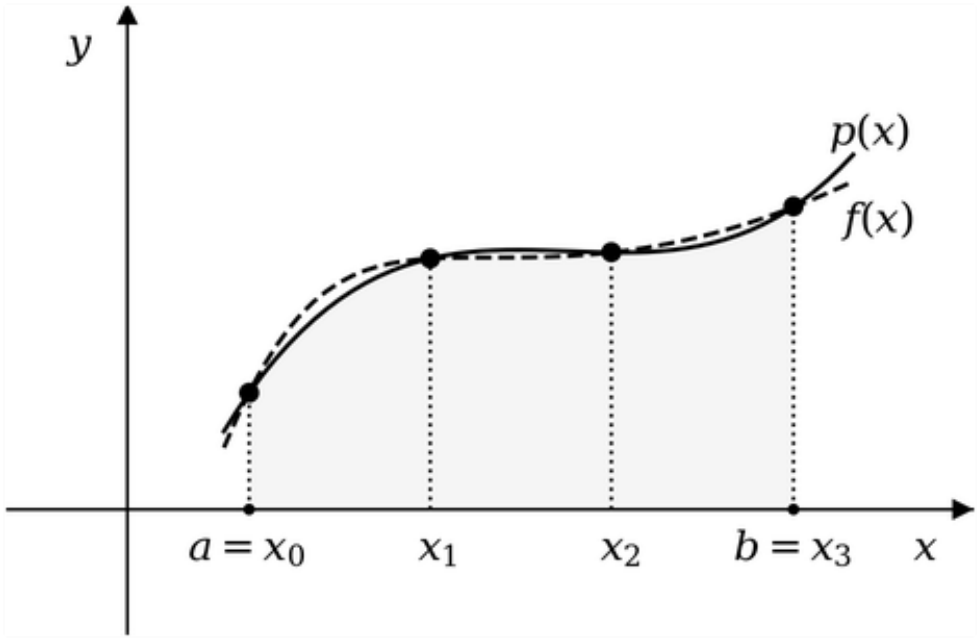



Figura 6.3.2 - Aproximação pela integral do polinômio interpolador de grau 3

Um *limitante superior* para o erro será dado por

$$|E_3| \leq \frac{3h^5}{80} \max.\{|f^{(4)}(x)|, x_0 \leq \xi \leq x_3\}$$

**Exemplo 6.3.3:** Calcule o valor aproximado da integral $\int_1^7 \ln(x+9)dx$ usando a regra 3/8 de Simpson.

```
x = np.linspace(1,7,4)
h = x[1]-x[0]
f = lambda x: np.log(x+9)
I = (3*h/8)*(f(x[0]) + 3*f(x[1])+3*f(x[2])+f(x[3]))
print(I)
```

```
15.335299315082702
```

```
print(integrate.simpson(f(x), x=np.linspace(1,7,4)))
```

```
15.335928614229697
```

A **regra 3/8 de Simpson repetida** (ou generalizada) consiste em subdividir o intervalo de integração $[a, b]$ em $n$ subintervalos de ampliture $h$, em que $n$ é um número múltiplo de 3, com $x_0 = a$ e $x_n = b$, então, aplicar a regra 3/8 de

Simpson repetidamente a cada 3 subintervalos consecutivos. Com isso obtém-se:

$$\begin{aligned}\int_{x_0}^{x_n} f(x)dx \approx & \frac{3}{8}h\Big[f(x_0)+3f(x_1)+3f(x_2)+f(x_3)\Big]+\\ & \frac{3}{8}h\Big[f(x_3)+3f(x_4)+3f(x_5)+f(x_6)\Big]+\\ & \quad\vdots\\ & \frac{3}{8}h\Big[f(x_{n-3})+3f(x_{n-2})+3f(x_{n-1})+f(x_n)\Big]\\ = & \\ & \frac{3}{8}h\Big\{f(x_0)+\\ & 3\Big[f(x_1)+f(x_2)+f(x_4)+f(x_5)+...+f(x_{n-2})+f(x_{n-1})\Big]+\\ & 2\Big[f(x_3)+f(x_6)+...+f(x_{n-3})\Big]+\\ & f(x_n)\Big\}\end{aligned}$$

que consiste na regra 3/8 de Simpson repetida.

Um *limitante superior* para o erro é dado por

$$|E_3| \leq \frac{h^4}{80}(x_n - x_0)\ \max.\left\{\left|f^{(4)}(x)\right|, x_0 \leq \xi \leq x_n\right\}$$

**Exemplo 6.3.4:** Calcule o valor aproximado da integral $\int_1^6 \ln(x+9)dx$ usando a regra 3/8 de Simpson para 8, 20 e 40 subintervalos.

```
x0 = 1.0; xn = 6.0
f = lambda x: np.log(x+9)

for n in [4,20,40]:
    h = (xn-x0)/n
    x = np.linspace(x0, xn, n+1)
    y = f(x)
    # Regra 3/8 de Simpson usando funções do numpy
    I = (3*h/8)*(y[0] + 3*np.sum(y[1:n-1:3]+y[2:n:3]) +\
                 2*np.sum(y[3:n:3])+ y[-1])
    print ("n=",n,", I=",I)
```

```
n= 4 , I= 11.761407778686316
n= 20 , I= 11.75388556350441
n= 40 , I= 12.510405538009266
```

O sub-pacote `scipy.integrate`, de SciPy, provê diversas funções para integração numérica, incluindo as regras de Newton-Cotes `trapezoid`, `cumulative_trapezoid` e `simpson`. Além destas, outras funções disponíveis são `quad`, `dblquad` e `tplquad` para integrais simples, duplas e triplas, respectivamente; `fixed_quad` e `quadrature`para quadratura Gaussiana e; `romberg`para integração de Romberg.

## Exercícios

**1.** Encontre, se possível, a primitiva das seguintes funções, então obtenha a solução exata e compare com soluções numéricas usando somas de Riemann e os métodos de Newton-Cotes para diferentes números de subintervalos. Compare os resultados.

a) $\int_1^{1.6} \frac{2x}{x^2-4}dx$ b) $\int_0^1 x^2 e^{-x}dx$ c) $\int_0^{\pi/4} e^{x^3}\,\text{sen}(2x)dx$

d) $\int_0^{0.5} \frac{2}{x-4}dx$ e) $\int_0^1 x^{-1/4}\,\text{sen}(x)dx$ f) $\int_0^1 \frac{e^{2x}}{\sqrt[5]{x^2}}dx$

g) $\int_0^1 \frac{ln(x)}{(1-x)^{1/5}}dx$ h) $\int_0^1 \frac{cos(2x)}{x^{1/3}}dx$ i) $\int e^{x^2}dx$

j) $\int \frac{e^x}{x}dx$ k) $\int \text{sen}(x^2)dx$ l) $\int \ln(lnx)dx$

m) $\int e^{-\frac{x^2}{2}}dx$ n) $\int \cos(x^2)dx$ o) $\int \cos(e^x)dx$

p) $\int \frac{1}{ln(x)}dx$

**2.** A vazão de um rio (em m$^3$/s) foi registrada em intervalos de 2 horas ao longo de um período de 12 horas:

| $t$ (h) | 0 | 2 | 4 | 6 | 8 | 10 | 12 |
|---|---|---|---|---|---|---|---|
| $Q$ (m$^3$/s) | 12,0 | 14,5 | 18,2 | 20,1 | 17,6 | 13,4 | 10,8 |

a) Verifique que o número de subintervalos permite aplicar diretamente a regra 1/3 de Simpson repetida.

b) Calcule o volume total de água escoado no período, em m$^3$ (integre $Q$ em relação a $t$, convertendo horas em segundos).

c) Compare o resultado com o que seria obtido pela regra dos trapézios generalizada.

**3.** Quando o número de subintervalos $n$ é ímpar, não é possível aplicar diretamente a regra 1/3 de Simpson repetida (que exige $n$ par) nem a regra 3/8 isoladamente sobre todo o intervalo (que exige $n$ múltiplo de 3). Uma estratégia comum é aplicar a regra 3/8 aos três últimos subintervalos e a regra 1/3 repetida aos demais, em número par.

a) Use essa estratégia para calcular $\int_0^{\pi} \text{sen}(x)\,dx$ com $n = 7$ subintervalos.

b) Compare o resultado com o valor exato (igual a 2) e com o resultado de `scipy.integrate.simpson` aplicado aos mesmos 8 pontos.

**4. (ordem de convergência)** Para a integral $\int_0^2 e^x\,dx$, cujo valor exato é $e^2 - 1$,

a) calcule a aproximação pela regra 1/3 de Simpson repetida para $n = 4$, 8, 16 e 32 subintervalos;

b) calcule o erro absoluto para cada valor de $n$;

c) faça um gráfico de $\log(h)$ versus $\log(\text{erro})$ e estime a ordem de convergência do método a partir da inclinação da reta. O resultado é compatível com a ordem teórica $O(h^4)$?

**5. (integral dupla)** Calcule a integral dupla

$$\int_0^1 \int_0^1 \frac{1}{1+x+y}\,dx\,dy$$

(a) analiticamente (o valor exato é $3\ln 3 - 4\ln 2 \approx 0{,}5232$) e (b) usando aplicação repetida da regra 1/3 de Simpson nas duas direções, com $n = 4$ e $n = 8$ subintervalos em cada uma. Compare os erros obtidos nos dois casos.

---

**6. (comparação de métodos)** Para a integral $\displaystyle\int_0^1 \frac{4}{1+x^2}\,dx$, cujo valor exato é $\pi$, calcule aproximações usando a regra dos trapézios, a regra 1/3 de Simpson e a regra 3/8 de Simpson, todas com $n = 6$ subintervalos (mesmo número de avaliações da função). Compare os erros absolutos de cada método e discuta qual apresenta melhor precisão para o mesmo custo computacional.

---

**7.** Retome o problema do objeto em queda livre do Exercício 2 da Seção 6.2, com velocidade

$$v(t) = \sqrt{\frac{gm}{c_d}}\,\tanh\left(\sqrt{\frac{gc_d}{m}}\,t\right), \qquad g = 9{,}81,\ m = 68{,}1,\ c_d = 0{,}25.$$

Use a regra 1/3 de Simpson repetida com $n = 6$, 12 e 24 subintervalos para estimar a distância percorrida em 10 s, e compare o erro relativo em cada caso com o valor exato obtido por integração analítica e com os resultados já obtidos pela regra dos trapézios naquele exercício.

# 7. Equações Diferenciais Ordinárias

Equações diferenciais ordinárias (EDOs) descrevem a variação de grandezas em relação a uma variável independente e aparecem em praticamente todos os ramos da ciência e engenharia. Quando a solução analítica não é viável, métodos numéricos permitem obter aproximações precisas.

### Objetivos de aprendizagem

- Formular problemas de valor inicial (PVI) para EDOs
- Aplicar o método de Euler explícito e analisar seu erro
- Implementar métodos de série de Taylor de ordem superior
- Utilizar os métodos de Runge-Kutta de 2ª e 4ª ordem
- Comparar precisão e custo computacional dos métodos

### Conteúdo


| Seção | Tema |
|---|---|
| 7.1 | Introdução às EDOs |
| 7.2 | Método de Euler |
| 7.3 | Métodos de série de Taylor |
| 7.4 | Métodos de Runge-Kutta |


## 7.1 Introdução

As **equações diferenciais** constituem uma das aplicações mais importantes do cálculo. Essas equações relacionam uma função desconhecida com uma ou mais de suas derivadas e são utilizadas para descrever como uma grandeza varia em função de outra, normalmente em relação ao tempo ou ao espaço

As leis fundamentais da Física, como as da mecânica, do eletromagnetismo e da termodinâmica, são, em geral, formuladas a partir de observações empíricas que descrevem a evolução das propriedades físicas e dos estados dos sistemas. Em vez de expressarem diretamente o estado de um sistema, essas leis são normalmente formuladas em termos das variações espaciais e temporais das grandezas envolvidas, isto é, em termos de derivadas.

Alguns exemplos dessas leis são apresentados na tabela a seguir.

| Lei | Expressão matemática | Variáveis e parâmetros |
|---|---|---|
| Segunda lei de movimento de Newton | $\frac{dv}{dt} = \frac{F}{m}$ | Velocidade ( V ), força ( f ) e massa $(m)$ |
| Lei do calor de Fourier | $q = -k' \frac{dT}{dx}$ | Fluxo de calor (q), condutividade térmico $(k')$ e temperatura ( 7 ) |
| Lei de difusão de Fick | $J = -D\frac{dc}{dx}$ | Fluxo de massa (), coeficiente de difusão ( $D$ ) e concentração (c) |
| Lei de Faraday (queda de tensão em um indutor) | $\Delta V_i = l\frac{di}{dt}$ | Queda de tensão ( $\Delta V_1$ ), indutância (L) e corrente (1) |

Em conjunto com os princípios de conservação da energia, da massa e da quantidade de movimento, essas leis dão origem a modelos matemáticos formulados como equações diferenciais. A resolução dessas equações permite obter funções que descrevem a evolução espacial e temporal de grandezas como velocidade, temperatura, concentração e energia (Chapra e Canale, 2016).

A importância dos métodos numéricos para a resolução de equações diferenciais ordinárias (EDOs) decorre do fato de que essas equações surgem com frequência na modelagem matemática de fenômenos em diversas áreas do conhecimento, como engenharia, física, química, biologia, economia e biomedicina.

De modo geral, problemas que relacionam uma função de uma variável com suas derivadas conduzem a uma equação diferencial ordinária. Em muitos casos, não é possível obter uma solução analítica para essas equações ou ela é excessivamente difícil de ser obtida. Nesses casos, recorre-se aos métodos numéricos, que fornecem aproximações da solução com precisão suficiente para aplicações práticas.

Uma equação diferencial ordinária é uma equação que relaciona uma função desconhecida de uma única variável independente, $y(x)$, com uma ou mais de suas derivadas em relação a essa variável.

A ordem de uma equação diferencial corresponde à maior ordem da derivada da função desconhecida presente na equação.

$$y' = \frac{dy}{dx}, \qquad y'' = \frac{d^2y}{dx^2}, \qquad \ldots$$

A seguir, apresentam-se alguns exemplos de equações diferenciais ordinárias:

- $\frac{dy}{dx} = 3x - 1$ — EDO de primeira ordem;
- $y' = -xy$ — EDO de primeira ordem;
- $\frac{d^2y}{dx^2} = x^2 - y + e^x$ — EDO de segunda ordem;
- $y'' - 4y' + 3y = x$ — EDO de segunda ordem.

Resolver uma equação diferencial consiste em encontrar uma função $y(x)$ que, ao ser substituída na equação juntamente com suas derivadas, torne a igualdade verdadeira.

**Exemplo 7.1.1:** A equação diferencial $y' = -xy$ tem solução $y = c\,e^{\frac{-x^2}{2}}$ com $c$ constante, pois

$$y' = c\,e^{\frac{-x^2}{2}}(-x) = -xy$$

ou seja, a função $y(x)$ satisfaz a equação.

Podemos observar que esta equação apresenta infinitas soluções, pois, para cada valor diferente de $c$ temos uma solução particular diferente.

A seguir, vamos usar Python para plotar algumas das soluções, com a constante $c$ assumindo alguns valores entre 0 e

2.

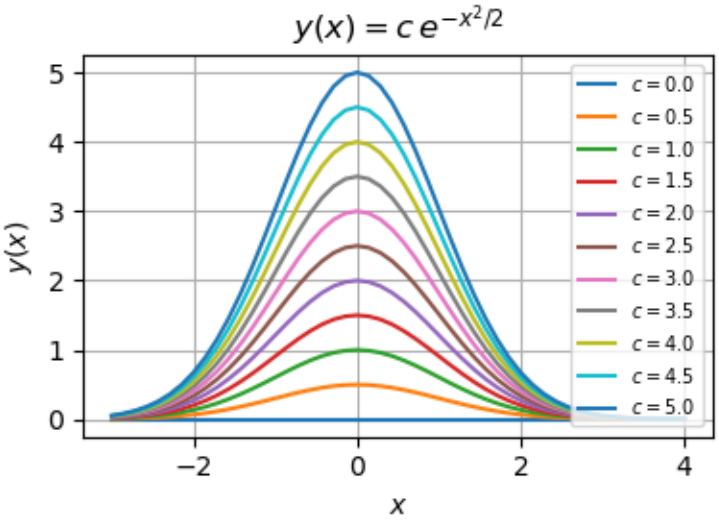
```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)
```

```
y = lambda x: c*np.exp((-x**2)/2)

xi = np.linspace(-3,4)

for c in np.arange(0,5.5,0.5):
    plt.plot(xi,y(xi),label=f"$c={c:.1f}$")

plt.xlabel("$x$")
plt.ylabel("$y(x)$")
plt.title(r"$y(x)=c\,e^{-x^2/2}$")
plt.grid(True)
plt.legend(fontsize=7)
plt.tight_layout()
plt.show()
```




**Exemplo 7.1.2:** A equação diferencial $\frac{du}{dt} = t + 2$ tem solução obtida facilmente integrando ambos os lados fazendo

$$\int \frac{du}{dt} = \int (t+2)dt$$

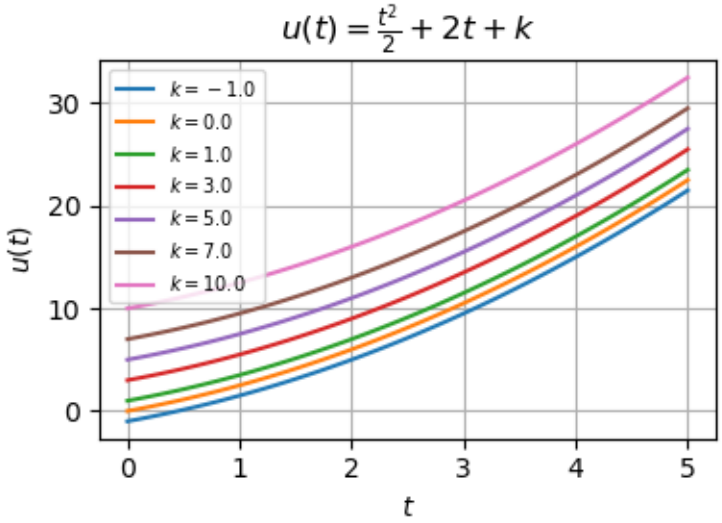
$$u(t) = \frac{t^2}{2} + 2t + k$$

em que $k$ é a constante de integração. Nesse caso a família de soluções, para $k = \{-1, 0, 1, 3, 5, 7, 10\}$ pode ser visualizada como é mostrado abaixo.

```
u = lambda t: (t**2/2)+2*t+k
t = np.linspace(0,5)

for k in [-1,0,1,3,5,7,10]:
    plt.plot(t,u(t),label=f"$k={k:.1f}$")

plt.xlabel("$t$")
plt.ylabel("$u(t)$")
plt.title(r"$u(t)=\frac{t^2}{2}+2t+k$")
plt.grid(True)
plt.legend(fontsize=7,loc="upper left")
plt.tight_layout()
plt.show()
```




Se impomos uma condição, como por exemplo $u(0) = 8$, obtemos o valor $k = 8$, que restringe a solução a uma única curva e não mais à uma família de curvas. Uma equação diferencial com condições iniciais como essa compõem o que chamamos de um **problema de valor inicial (PVI)**.

Ou seja, o problema de valor inicial descrito acima pode ser enunciado como

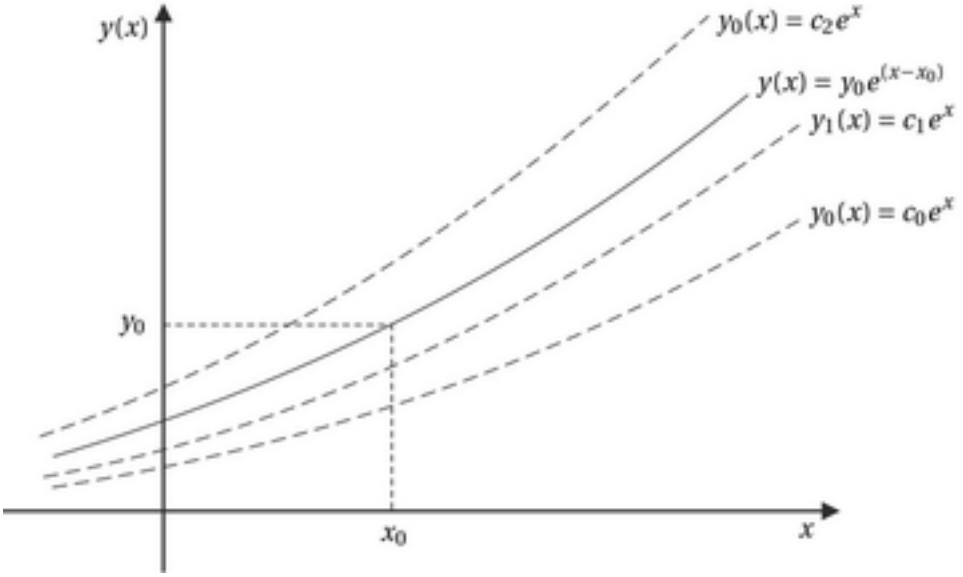
$$\begin{cases} \frac{du}{dt} = t + 2 \\ \\ u(0) = 8 \end{cases}$$

E existência e unicidade da solução de um PVI é garantida, sob certas condições, pelos teoremas *Condição de Lipschits*, *Pincard-Lindelof* e *Dependência contínua na condição inicial* enunciados e demonstrados em Medeiros e Oliveira (2020, p.24)

## Resolução numérica de um PVI

Considere a equação diferencial ordinária, $y' = y$, cuja solução é dada por $y(x) = ce^x$, $c \in R$. Sabemos que essa equação diferencial apresenta infinitas soluções, uma vez que, para cada valor escolhido para a constante $c$, temos uma solução.

Por exemplo, $y(x) = c_0e^x$, $y(x) = c_1e^x$, $y(x) = c_2e^x$, sendo $c_0$, $c_1$ , $c_2$ valores fixados, são soluções dessa equação conforme é ilustrado na figura abaixo.




Fonte: Arenales e Darezzo (2016)

Se conhecermos o valor da função $y(x)$ em um ponto $x = x_0$, ou seja, conhecermos $y_0$ etal que $y(x_0) = y_0$, então, podemos obter uma única solução para a equação proposta a partir da solução geral aplicando o ponto $x = x_0$ e obtendo o valor da constante $c$. Ou seja, fazemos

$$y(x) = ce^x \rightarrow ce^{x_0} = y_0 \rightarrow c = y_0e^{0-x_0}.$$

Assim, a solução única para a equação dada, juntamente com o valor inicial $y(x_0) = y_0$, é:

$$y(x) = ce^x = y_0e^{-x_0}e^x = y_0e^{(x-x_0)}$$

Portanto, $y(x) = y_0e^{(x-x_0)}$, conforme exibido na figura.

Um **PVI de primeira ordem** consiste de uma equação diferencial $y' = f(x, y)$, $x \geq x_0$ e uma condição inicial $y(x_0) = y_0$, em que $y_0$ é um valor dado, chamado de valor inicial.

Neste caso, podemos escrever o PVI da seguinte forma:

$$\begin{cases} y' = f(x, y) \\ \\ y(x_0) = y_0 \end{cases}$$

Resolver o PVI consiste em determinar uma função $y(x)$ que satisfaça a equação diferencial $y' = f(x, y)$ e a condição inicial $y(x_0) = y_0$. Se a função $f$ e $\frac{\partial f}{\partial y}$ são contínuas, temos que o PVI apresenta solução única.

**Resolver numericamente um PVI** consiste em calcular aproximações para $y = y(x)$ em pontos discretos $x_0, x_1, x_2, \ldots, x_N$ de um intervalo $[a, b]$.

Para discretizar o intervalo $[a, b]$, tomamos $N$ subintervalos $(N \geq 1)$ e fazemos $x_n = x_0 + nh$, $n = 0, 1, 2, \ldots, N$ com $x_0 = a$ e $x_N = b$,sendo $h = (x_N - x_0)/N$.

A este conjunto de pontos $x_0, x_1, x_2, \ldots, x_N$ denominamos rede ou malha de pontos discretos, e calculamos aproximações para a função $y(x)$ nestes pontos, isto é, determinamos $y_n$ tal que $y_n \approx y(x_n)$, $n = 0, 1, \ldots, N$.

A partir de um ponto inicial dado $y(x_0) = y_0$ (valor inicial), calculamos passo a passo, nos pontos

$$x_1 = x_0 + h,$$

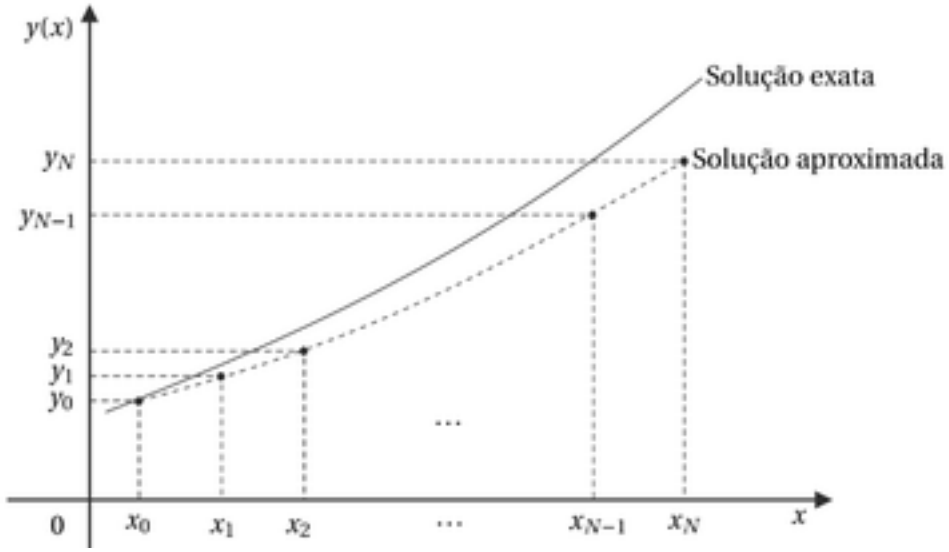
$$x_2 = x_0 + 2h,$$

$$x_3 = x_0 + 3h,$$

$$\vdots$$

$$x_n = x_0 + nh$$

soluções aproximadas $y_n$ para a solução exata $y(x_n)$, $n = 0, 1, 2, \ldots, N$, conforme ilustrado na figura a seguir.




Nas seções seguintes serão estudados e implementados alguns dos principais métodos numéricos para a resolução de problemas de valor inicial (PVIs), como os métodos de Euler, Taylor e Runge-Kutta. Muitos desses métodos já estão implementados em bibliotecas computacionais e podem ser utilizados por meio de funções prontas. Como exemplo, a seguir é apresentada a utilização da função `solve_ivp`, disponível no módulo `scipy.integrate` da biblioteca SciPy (Virtanen et al., 2020), para resolver o PVI apresentado no Exemplo 7.1.2.

**Exemplo 7.1.3:** Resolução de um PVI **usando SciPy** (Referência):

```
from scipy.integrate import solve_ivp

def du(t, u):
    return t+2
```

(continua na próxima página)

(continuação da página anterior)

```
t0 = 0.0
tf = 5.0
u0 = [8]
t = np.linspace(t0,tf,20)

sol = solve_ivp(du, (t0,tf), u0, t_eval=t)
print(sol.y)
```

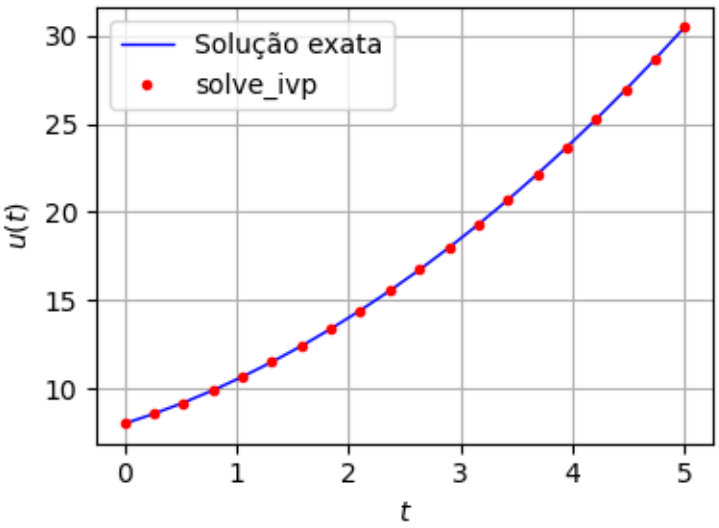
```
[[ 8.          8.56094183  9.19113573  9.89058172 10.65927978 11.49722992
  12.40443213 13.38088643 14.4265928  15.54155125 16.72576177 17.97922438
  19.30193906 20.69390582 22.15512465 23.68559557 25.28531856 26.95429363
  28.69252078 30.5       ]]
```

```
# Gráfico

u_exata = lambda t: (t**2/2)+2*t+8

plt.figure(figsize=(4,3))
plt.plot(t, u_exata(t), 'b-', lw=1, label='Solução exata')
plt.plot(sol.t, sol.y[0], 'r.', label='solve_ivp')

plt.xlabel('$t$')
plt.ylabel('$u(t)$')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



## Exercícios

**1.** Considere a equação diferencial

$$\frac{du}{dt} = 2t - e^{-t},$$

cuja solução geral é

$$u(t) = t^2 + e^{-t} + C.$$

a) Plote as soluções correspondentes a $C = -4, -2, 0, 2, 4$.

b) Resolva numericamente o problema de valor inicial

$$u(0) = 2.$$

c) Plote, no mesmo gráfico, a solução numérica e a solução analítica particular.

---

**2.** Considere a equação diferencial

$$\frac{du}{dt} = 3t^2 + \cos(t),$$

cuja solução geral é

$$u(t) = t^3 + \sin(t) + C.$$

a) Plote as soluções correspondentes a $C = -6, -3, 0, 3, 6$.

b) Resolva numericamente o problema de valor inicial

$$u(0) = 1.$$

c) Compare, em um mesmo gráfico, a solução numérica e a solução analítica.

---

**3.** Considere a equação diferencial

$$\frac{du}{dt} = e^t - 2t,$$

cuja solução geral é

$$u(t) = e^t - t^2 + C.$$

a) Plote cinco soluções distintas da família de soluções.

b) Resolva numericamente o problema de valor inicial

$$u(0) = 4.$$

c) Compare a solução numérica com a solução analítica.

---

**4.** Considere a equação diferencial

$$\frac{du}{dt} = 2\cos(2t) + 3,$$

cuja solução geral é

$$u(t) = \sin(2t) + 3t + C.$$

a) Plote as soluções correspondentes a $C = -3, -1, 1, 3, 5$.

b) Resolva numericamente o problema de valor inicial

$$u(0) = -2.$$

c) Compare a solução numérica com a solução analítica.

---

**5.** Considere a equação diferencial

$$\frac{du}{dt} = \cos(t) + 2t.$$

a) Determine a solução geral da equação diferencial.

b) Plote as soluções correspondentes a $C = -4, -2, 0, 2, 4$.

c) Resolva numericamente o problema de valor inicial

$$u(0) = 3.$$

d) Compare a solução numérica com a solução analítica particular.

---

**6.** Considere a equação diferencial

$$\frac{du}{dt} = 4t^3 - e^{-t}.$$

a) Determine a solução geral da equação diferencial.

b) Plote cinco soluções distintas da família de soluções.

c) Resolva numericamente o problema de valor inicial

$$u(0) = 1.$$

d) Compare a solução numérica com a solução analítica particular.

---

**7.** Considere a equação diferencial

$$\frac{du}{dt} = \frac{1}{1+t^2} + 2.$$

a) Determine a solução geral da equação diferencial.

b) Plote as soluções correspondentes a $C = -3, -1, 1, 3, 5$.

c) Resolva numericamente o problema de valor inicial

$$u(0) = 0.$$

d) Compare a solução numérica com a solução analítica particular.

---

**8.** Considere a equação diferencial

$$\frac{du}{dt} = 2e^t - 3\sin(t).$$

a) Determine a solução geral da equação diferencial.

b) Plote cinco soluções distintas da família de soluções.

c) Resolva numericamente o problema de valor inicial

$$u(0) = 2.$$

d) Compare a solução numérica com a solução analítica particular.

---

**9.** Considere a equação diferencial

$$\frac{du}{dt} = 3t^2 + 2\cos(2t).$$

a) Determine a solução geral da equação diferencial.

b) Plote as soluções correspondentes a $C = -5, -2, 0, 2, 5$.

c) Resolva numericamente o problema de valor inicial

$$u(0) = 4.$$

d) Compare a solução numérica com a solução analítica particular.

---

**10.** Considere a equação diferencial

$$\frac{du}{dt} = t^4 - 2t + \cos(t).$$

a) Determine a solução geral da equação diferencial.

b) Plote cinco soluções distintas da família de soluções.

c) Resolva numericamente o problema de valor inicial

$$u(0) = -1.$$

d) Compare a solução numérica com a solução analítica particular.

## 7.2 Método de Euler

Vamos agora estudar a solução de equações diferenciais ordinárias da forma

$$\frac{dy}{dx} = f(x, y)$$

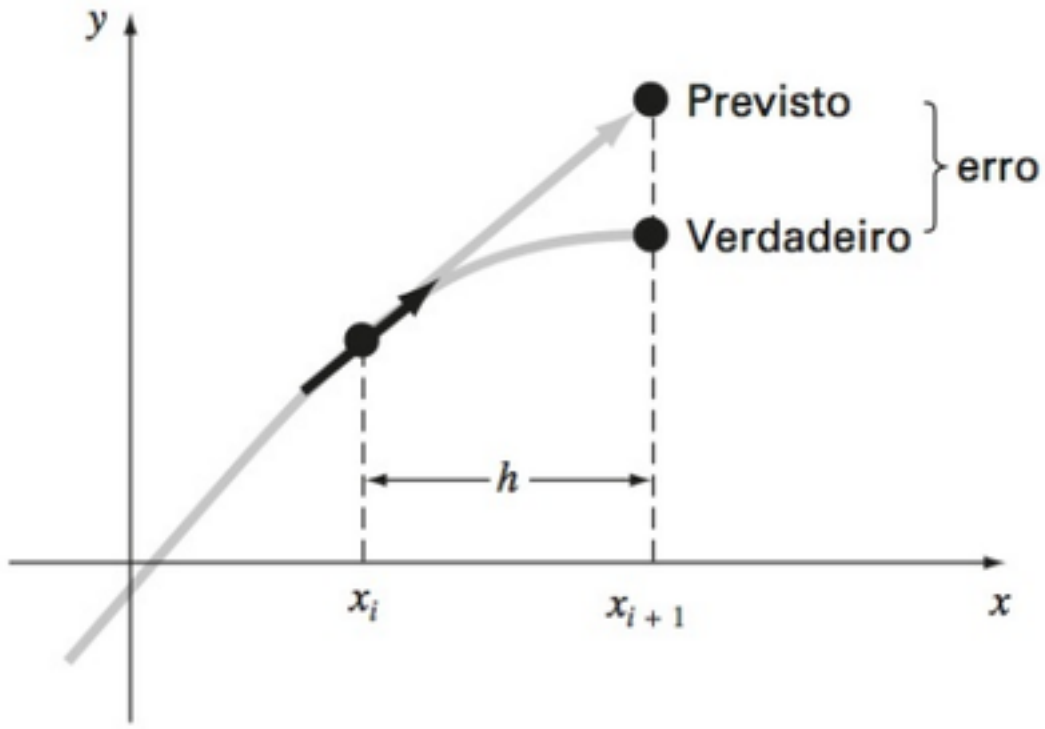
usando um método numérico de *passo único* em que obtemos um processo iterativo para obter a solução para cada ponto $y_{i+1}$ a partir de um ponto $y_i$ conhecido. Fazemos isso considerando a derivada no ponto $x_i$ para obter o valor da função no ponto $x_{i+1}$ em que o tamanho de cada *passo* é dado por $h = x_{i+1} - x_i$. Matematicamente, escrevemos

$$y_{i+1} = y_i + \phi h$$

Assim, a estimativa da inclinação é usada para extrapolar de um valor antigo $y_i$ para um valor novo $y_{i+1}$ em uma distância $h$.




A inclinação no início do intervalo é tomada como uma aproximação da inclinação média em todo o intervalo, com isso um erro é inserido. Em geral, quanto maior o tamanho do passo, maior é o erro nessa apoximação.




No método de Euler a inclinação é tomada como sendo a derivada primeira da função $f(x, y)$, ou seja, considerando

que $\frac{dy}{dx} = f(x, y)$, escrevemos

$$y_{i+1} = y_i + f(x_i, y_i)h$$

Assim, a partir de um valor inicial $y_0$ conhecido obtemos estimativas para os valores subsequentes $y_1, y_2, y_3, ...$ da função.

Esse método também é chamado de método de *Euler-Cauchy* ou *ponto-inclinação*.

**Exemplo 7.2.1:** Use o método de Euler para integrar numericamente a equação:

$$\frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5$$

de $x = 0$ a $x = 4$ com um tamanho de passo de $0.5$. A condição inicial em $x = 0$ é $y = 1$.

A solução exata é dada por $y = -5.x^4 + 4x^3 - 10x^2 + 8.5x + 1$.

*Solução:* Com um passo $h = 0.5$, temos $x_0 = 0.0$, $x_1 = 0.5$, $x_2 = 1.0$, $x_3 = 1.5$, ..., $x_8 = 4.0$. Assim, a partir da equação

$$y_{i+1} = y_i + f(x_i, y_i)h$$

com

$$f(x, y) = \frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5$$

e sabendo que em $x_0 = 0$ temos $y_0 = 1$ obtemos os valores de $y_1, y_2, y_3, ..., y_8$

Vamos fazer isso computacionalmente, em Python.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)
```

```
# Dados do exercício
h = 0.2 #passo
x0 = 0.0
xn = 4.0

# criamos a lista com os pontos xi
xi = np.arange(x0, xn+h, h)
print ("pontos xi:", xi)

# lista com y0, a ser preenchida pelos yi
yi = [1.0]

# entramos com f(x,y)
f = lambda x: -2*x**3 + 12*x**2 - 20*x + 8.5
```

```
pontos xi: [0.  0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8 2.  2.2 2.4 2.6 2.8 3.  3.2 3.4
 3.6 3.8 4. ]
```

```
# laço para ir calculando os yi e preenchendo a lista
for x in xi[0:-1]:
    yi.append(yi[-1] + f(x)*h ) #yi[-1] é o último yi calculado
```

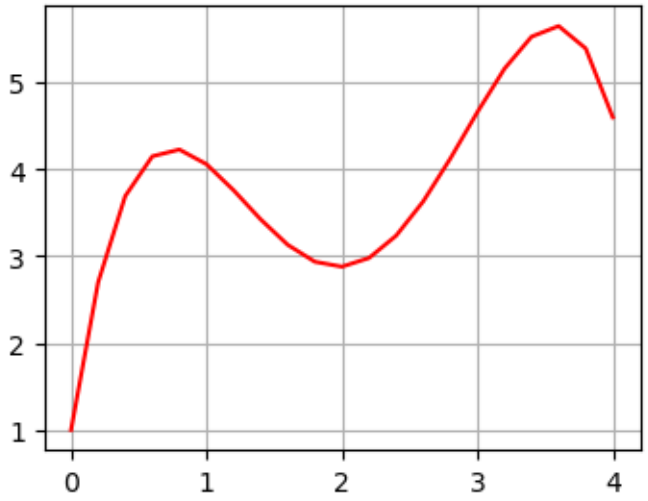
(continua na próxima página)

(continuação da página anterior)

```
# imprime
print ("pontos yi:", np.round(yi,4))

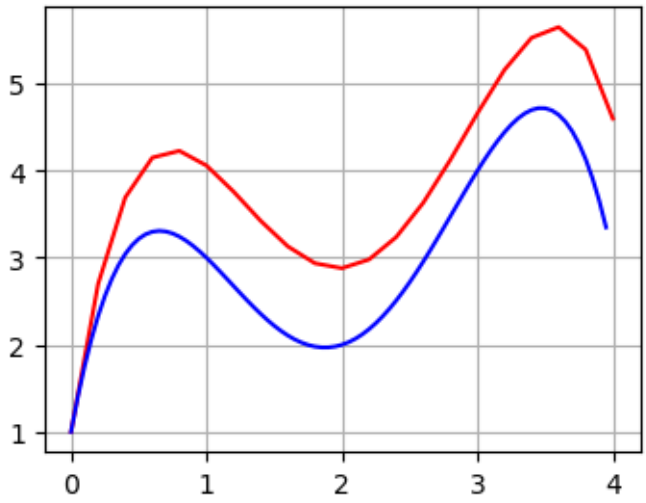
# plota
plt.plot(xi, yi, "r-")
plt.grid()
plt.show()
```

```
pontos yi: [1.     2.7    3.6928 4.1512 4.2288 4.06   3.76   3.4248 3.1312 2.9368
 2.88   2.98   3.2368 3.6312 4.1248 4.66   5.16   5.5288 5.6512 5.3928
 4.6   ]
```




```
# plotando a solução exata para comparar
x = np.arange(0, 4, 0.05)
y = -0.5*x**4 + 4*x**3 - 10*x**2 + 8.5*x + 1.0

plt.plot(xi, yi, "r-", x, y, "b-")
plt.grid()
plt.show()
```




Nesse exemplo usamos um polinômio simples para a equação diferencial para facilitar. Um caso mais geral envolveria EDOs que dependam de $x$ e $y$,

$$\frac{dy}{dx} = f(x, y)$$

como o que será apresentado no exemplo a seguir.

**Exemplo 7.2.2:** Use o método de Euler para integrar numericamente a equação:

$$\frac{dy}{dx} = x - y + 2$$

de $x = 0$ a $x = 1$ com um tamanho de passo de $0.25$. A condição inicial em $x = 0$ é $y = 2$.

```
h = 0.20 #passo
x0 = 0.0
xn = 1.0

# criamos a lista com os pontos xi
X = np.arange(x0, xn+h, h)
print ("pontos xi:", X)

# lista com y0, a ser preenchida pelos yi
Y = [2.0]

# entramos com f(x,y)
f = lambda x,y: x-y+2

# laço para ir calculando os yi e preenchendo a lista
for n in range(len(X)-1):
    Y.append(Y[n] + f(X[n],Y[n])*h ) #Y[-1] é o último yi calculado

# imprime
print ("pontos yi:", Y)
s = lambda x: np.exp(-x) + x + 1
```

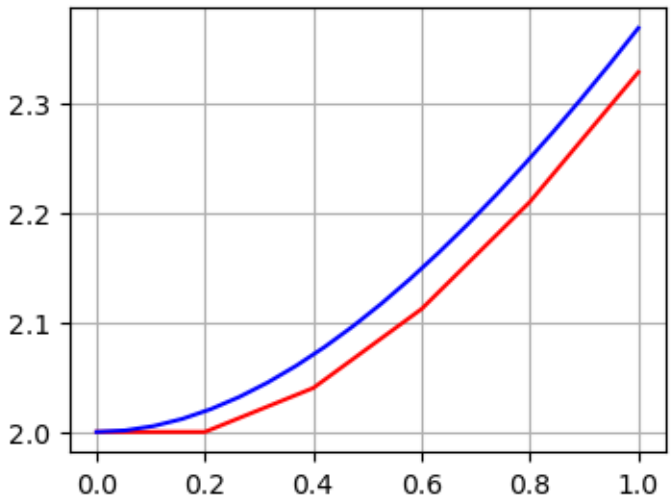
(continua na próxima página)

(continuação da página anterior)

```
Xe = np.linspace(x0,xn, 20)
Ye = s(Xe)


# plota
plt.plot(X, Y, "r-", Xe, Ye, "b-")
plt.grid()
plt.show()
```

```
pontos xi: [0.  0.2 0.4 0.6 0.8 1. ]
pontos yi: [2.0, 2.0, 2.04, 2.112, 2.2096, 2.32768]
```




## Uma melhoria no método de Euler (método de Heun)

*Adaptado de Chapra e Canalle (2016, p.651).*

Uma melhoria nos resultados do método de Euler pode ser obtida melhorando a estimativa da inclinação $y_i'$. No método a inclinação em $y_i$ é tomada como sendo constante e igual a $dy/dx$ ao longo do intervalo. O método de Heun consiste em usar a inclinação média $\bar{y}'$ entre a inclinação nos dois extremos do intervalo, ou seja:

$$\bar{y}' = \frac{y_i' + y_{i+1}'}{2}$$

em que $y_{i+1}'$ é estimada fazendo $y_{i+1}' = f\left(x_{i+1}, y_{i+1}^0\right) \quad \text{com} \quad y_{i+1}^0 = y_i + f(x_i, y_i)h$

Aqui, $y_{i+1}^0$ é a primeira aproximação obtida pelo método de Euler, uma estimativa preditiva da inclinação, então, uma correção nessa aproximação é realizada usando a média das inclinações estimadas em $x_i$ e $x_{i+1}$.

Essa inclinação média é, então, usada para extrapolar linearmente de $y_i$ a $y_{i+1}$

$$y_{i+1} = y_i + \bar{y}'h$$

O método de Heun é uma abordagem do tipo *preditor-corretor* e pode ser usada repetidamente realizando múltiplas iterações de correção da inclinação.

**Exemplo resolvido:** (Chapra e Canale, 2016, p. 653)

Use o método de Heun para integrar $y' = 4e^{0.8x} - 0,5y$ de $x = 0$ a $x = 4$ com tamanho do passo 1 . A condição inicial em $x = 0$ é $y = 2$.

```
import numpy as np

#Método de Heun
def heun(a,b,h,f,n):
    yi = [2]
    xi = np.arange(a,b+2*h,h)
    for i in range(len(xi)-1):
        err = 10.0
        dy = f(xi[i],yi[i])
        y = yi[i]+h*dy            #previsor (Euler)
        y_ant = y                 #guarda iteração anterior
        #n iterações
        for k in range(n):           #iterações
            dy0 = f(xi[i+1],y)        #corretor
            y = yi[-1]+h*(dy+dy0)/2   #metodo de Heun
            err = abs(y-y_ant)/abs(y) #calcula o erro
            y_ant = y                 #atualiza iteração anterior
        yi.append(y)                        #inclui o resultado na lista
        print (xi[i], yi[i], err)  #imprime
    return (yi)
```

```
a = 0.0                                 #início
b = 4.0                                 #fim
h = 1.0                                 #passo
f = lambda x,y: 4*np.exp(0.8*x)-0.5*y   #função
yi = heun(a,b,h,f,15)
```

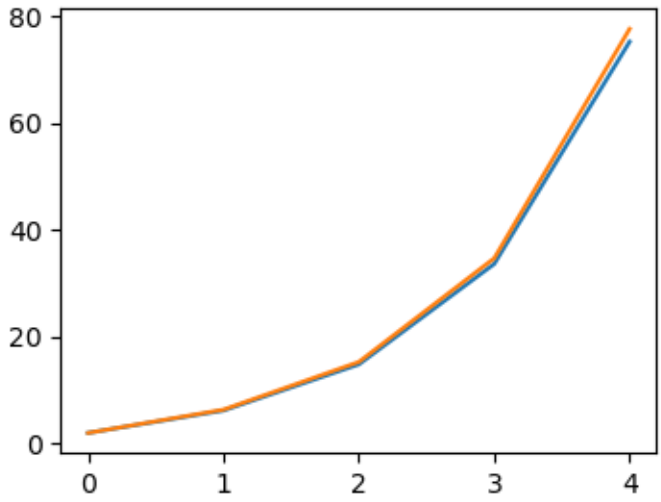
```
0.0 2 9.9625174649684e-10
1.0 6.360865486855353 9.797664900539922e-10
2.0 15.302236659731868 9.75736166799688e-10
3.0 34.743276090677924 9.746769595727495e-10
4.0 77.73509619396161 9.743931514353492e-10
```

```
# solução exata
xi = np.arange(a,b+h,h)
y_ex = (4/1.3)*(np.exp(0.8*xi)-np.exp(-0.5*xi))+2*np.exp(-0.5*xi)
print (y_ex)
```

```
[ 2.          6.19463138 14.84392191 33.67717177 75.33896261]
```

```
import matplotlib.pyplot as plt
plt.plot(xi,y_ex, xi, yi[0:-1])
```

```
[<matplotlib.lines.Line2D at 0x7fd66f3abe60>,
 <matplotlib.lines.Line2D at 0x7fd66d274230>]
```




```
a = 0.0                                 #início
b = 4.0                                 #fim
h = 1.0                                 #passo
f = lambda x,y: 4*np.exp(0.8*x)-0.5*y   #função
heun(a,b,h,f,15)
```

```
0.0 2 9.9625174649684e-10
1.0 6.360865486855353 9.797664900539922e-10
2.0 15.302236659731868 9.75736166799688e-10
3.0 34.743276090677924 9.746769595727495e-10
4.0 77.73509619396161 9.743931514353492e-10
```

```
[2,
 6.360865486855353,
 15.302236659731868,
 34.743276090677924,
 77.73509619396161,
 173.25014611854544]
```

## Exercícios

**1.** Considere o problema de valor inicial (Boyce, 2020, p.54)

$$\frac{dy}{dt} = 3 - 2t - 0,5y, \quad y(0) = 1.$$

Use o método de Euler com passos de tamanho $h = 0,2$ para encontrar valores aproximados da solução do problema de valor inicial em $t = 0,2$; $0,4$; $0,6$; $0,8$ e $1$. Compare-os com os valores correspondentes da solução exata $y = 14 - 4t - 13e^{-t/2}$.
Em seguida, use diversos tamanhos de passos para calcular valores aproximados da solução para $0 \leq t \leq 5$. Compare os resultados calculados com os valores correspondentes da solução exata. Plote os gráficos.

---

**2.** Considere o problema de valor inicial (Boyce, 2020, p.56)

$$\frac{dy}{dt} = 4 - t + 2y, \quad y(0) = 1.$$

Use o método de Euler e o método de Heun (Euler melhorado), com o mesmo número de pontos na discretização, para encontrar valores aproximados da solução no intervalo $0 \leq t \leq 5$. Compare os resultados com os valores correspondentes da solução

$$y = -\frac{7}{4} + \frac{1}{2}t + \frac{11}{4}e^{2t}.$$

Plote os gráficos.

---

**3.** Para cada um dos problemas a seguir, encontre a solução exata e soluções numéricas com três diferentes discretizações no intervalo $0,1 <= t <= 0.5$. Plote os gráficos (Boyce, 2020, p.57).

$(a)$ $y' = 3 + t - y, \quad y(0) = 1$

$(b)$ $y' = 2y - 1, \quad y(0) = 1$

$(c)$ $y' = 0,5 - t + 2y, \quad y(0) = 1$

$(d)$ $y' = 3\cos t - 2y, \quad y(0) = 0$

---

**4.** Resolva o seguinte problema de valor inicial no intervalo de $t = 0$ a $2$ em que $y(0) = 1$. Mostre todos os seus resultados em um mesmo gráfico

$$\frac{dy}{dt} = yt^2 - 1,1y$$

(a) Analiticamente. (b) Método de Euler com $h = 0,5$, $0,25$ e $0,1$.

---

**5.** Resolva o seguinte problema de valor inicial no intervalo de $t = 0$ a $1$ usando um tamanbo de passo de $0,1$ , em que $y(0) = 1$. Mostre todos os seus resultados cm um mesmo gráfico

$$\frac{dy}{dt} = (1 + 4t)\sqrt{y}$$

(a) Analiticamente. (b) Método de Euler. (c) Método de Heun.

**6.** Use os métodos de (a) Euler e (b) Heun para resolver

$$\frac{d^2y}{dt^2} - 0,5t + y = 0$$

em que $y(0) = 2$ e $y'(0) = 0$. Resolva de $x = 0$ a 4 usando $h = 0,1$. Compare os métodos fazendo os gráficos da solução.

## 7.3 Métodos de Série de Taylor

Consideremos o Problema de Valor Inicial (PVI) descrito por $\begin{cases} y' = f(x,y) \\ y(x_0) = y_0 \end{cases}$

Supondo que a solução $y(x)$ do PVI possui todas as derivadas contínuas no intervalo $[a,b]$ que contém os pontos de discretização $x_0$,$x_1$,...,$x_n$ , o desenvolvimento em série de Taylor da função $y(x)$ em torno do ponto $x_n$ é dado por

$$y(x) = y(x_n) + y'(x_n)(x - x_n) + y''(x_n)\frac{(x-x_n)^2}{2!} + \cdots + y^{(p)}(x_n)\frac{(x-x_n)^p}{p!} + y^{(p+1)}(\xi)\frac{(x-\xi)^{p+1}}{(p+1)!}$$

com $x_0 < \xi < x$.

Tomando $x = x_n + h$, denotado por $x_n + 1$ , assim, $h = x_n + 1 - x_n$ e temos:

$$y(x) = y(x_n) + y'(x_n)h + y''(x_n)\frac{h^2}{2!} + \cdots + y^{(p)}(x_n)\frac{h^p}{p!} + y^{(p+1)}(\xi)\frac{(x-\xi)^{p+1}}{(p+1)!}$$

Se truncarmos a série de Taylor no p-ésimo termo, temos:

$$y(x) = y(x_n) + y'(x_n)h + y''(x_n)\frac{h^2}{2!} + \cdots + y^{(p)}(x_n)\frac{h^p}{p!}$$

com um **erro de truncamento** dado pelo último termo da série de Taylor

$$e(x_n) = y^{(p+1)}(\xi)\frac{(x-\xi)^{p+1}}{(p+1)!}$$

Se a função $y(x)$ possui a derivada de ordem $(p+1)$ contínua no intervalo $[a,b]$, então existe $M = \max\left\{\left|y^{(p+1)}(x)\right|, x \in [a,b]\right\}$, e temos uma estimativa para o erro dada por um limitante superior, de ordem $(p+1)$, como segue:

$$|E| \leq \frac{h^{p+1}}{(p+1)!}M$$

Assim, usando a série de Taylor, podemos calcular aproximações para $y(x)$ da seguinte forma:

$$y(x_{n+1}) = y_n + y_n'h + y_n''\frac{h^2}{2!} + \cdots + y_n^{(p)}\frac{h^p}{p!}$$

onde

$y_n' = f(x_n, y_n)$

$$y_n'' = f_x(x_n, y_n) + f_y(x_n, y_n)y_n'$$
$$y_n''' = f_{xx}(x_n, y_n) + 2f_{xy}(x_n, y_n)f(x_n, y_n) + f^2(x_n, y_n)f_{yy}(x_n, y_n) + f_y^2(x_n, y_n)f(x_n, y_n)$$
$$\vdots$$

## Método de Taylor de ordem p=2

**Exemplo 7.3.1:** Use o método de Taylor de ordem 2 para integrar numericamente a equação:

$$\begin{cases} \frac{dy}{dx} = x - y + 2 \\ y(0) = 2 \end{cases}$$

de $x = 0$ a $x = 1$ com $N = 5$ subintervalos. Compare os valores e plote o resultado considerando a solução exata $y(x) = e^{-x} + x + 1$.

*Solução:*

Precisamos usar a fórmula de recorrência

$$y_{n+1} = y_n + y_n'h + y_n''\frac{h^2}{2!}$$

em que

$$y' = x - y + 2$$

$$y'' = 1 - y' = 1 - (x - y + 2) = -x + y - 1$$

então

$$y_{n+1} = y_n + (x_n - y_n + 2)h + (-x_n + y_n - 1)\frac{h^2}{2!}$$

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)
```

```
f = lambda x,y: x-y+2
df = lambda x,y: -x+y-1

x0 = 0
xn = 1
n = 5 #subintervalos

X = np.linspace(x0,xn,n+1)
Y = np.zeros(n+1)
Y[0] = 2
h=(xn-x0)/n
for i in range(n):
    Y[i+1] = Y[i] + h*f(X[i],Y[i]) + 0.5*(h**2)*df(X[i],Y[i])
    print ("Y",i+1,"=",Y[i+1])
print (X)
print ('Sol. aprox.:',np.round(Y,4))
```

(continua na próxima página)

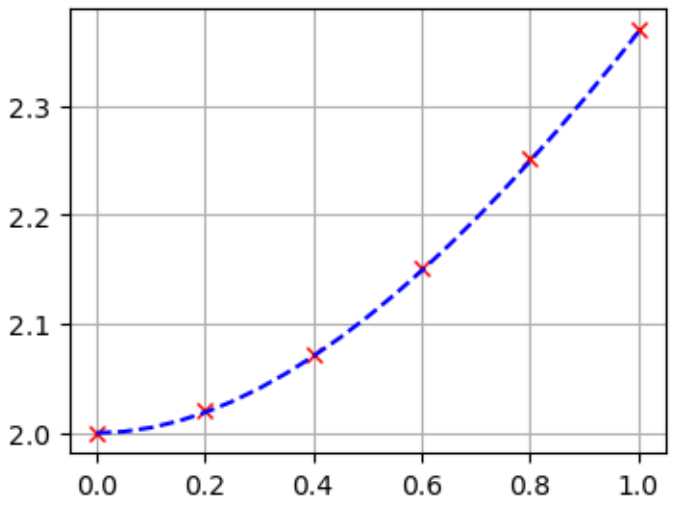
(continuação da página anterior)

```
print ('Sol. exata.:',np.round(np.exp(-X) + X + 1, 4))

Xe = np.linspace(X[0],X[-1],21)
Ye = np.exp(-Xe) + Xe + 1


# plotando
plt.plot(X, Y, "xr",Xe, Ye, 'b--')
plt.grid()
plt.show()
```

```
Y 1 = 2.02
Y 2 = 2.0724
Y 3 = 2.1513679999999997
Y 4 = 2.2521217599999996
Y 5 = 2.3707398431999995
[0.  0.2 0.4 0.6 0.8 1. ]
Sol. aprox.: [2.     2.02   2.0724 2.1514 2.2521 2.3707]
Sol. exata.: [2.     2.0187 2.0703 2.1488 2.2493 2.3679]
```




## Exercícios

**1.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = 2x - y \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = 2x - 2 + 3e^{-x}$.

a) Calcule analiticamente $y''(x)$ em função de $x$ e $y$.

b) Implemente o método de Taylor de ordem 2 com $N = 5$ subintervalos no intervalo $[0, 2]$.

c) Compare a solução aproximada com a solução exata plotando ambas no mesmo gráfico.

---

**2.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = x^2 - y \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = x^2 - 2x + 2 - e^{-x}$.

a) Obtenha $y''(x)$ derivando a equação diferencial em relação a $x$.

b) Use o método de Taylor de ordem 2 com $h = 0,2$ para aproximar $y$ no intervalo $[0, 2]$.

c) Plote a solução numérica e a solução exata no mesmo gráfico e calcule o erro absoluto em cada ponto.

---

**3.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = x + 2y \\ y(0) = 0 \end{cases}$$

cuja solução exata é $y(x) = \dfrac{1}{4}e^{2x} - \dfrac{x}{2} - \dfrac{1}{4}$.

a) Calcule $y''(x)$.

b) Implemente o método de Taylor de ordem 2 com $N = 8$ subintervalos em $[0, 1]$.

c) Compare, em um mesmo gráfico, a solução numérica com a solução exata.

---

**4.** Considere o problema de valor inicial não linear

$$\begin{cases} \dfrac{dy}{dx} = xy^2 \\ y(1) = 1 \end{cases}$$

cuja solução exata é $y(x) = \dfrac{2}{3 - x^2}$, definida para $x < \sqrt{3}$.

a) Calcule $y''(x)$ usando as derivadas parciais de $f(x, y) = xy^2$.

b) Implemente o método de Taylor de ordem 2 com $h = 0,1$ no intervalo $[1;\ 1,7]$.

c) Compare a solução numérica com a solução exata e discuta o que ocorre quando $x$ se aproxima de $\sqrt{3}$.

---

**5.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = y - x^2 + 1 \\ y(0) = 0,5 \end{cases}$$

cuja solução exata é $y(x) = (x + 1)^2 - 0,5e^x$.

a) Obtenha $y''(x)$.

b) Resolva numericamente usando o método de Taylor de ordem 2 com $N = 10$ subintervalos em $[0, 2]$.

c) Plote a solução numérica, a solução exata e o erro absoluto ao longo do intervalo.

**6.** Para o problema de valor inicial do Exercício 1,

a) Resolva numericamente com $h = 0,4; 0,2; 0,1$ e $0,05$.

b) Para cada tamanho de passo, calcule o erro máximo em relação à solução exata.

c) Faça um gráfico de $\log(h)$ versus $\log(\text{erro máximo})$ e estime a ordem de convergência do método a partir da inclinação da reta. O resultado é compatível com a ordem teórica $O(h^2)$?

---

**7.** Considere novamente o problema de valor inicial do Exercício 2.

a) Resolva-o usando o método de Euler, o método de Heun (ambos da Seção 7.2) e o método de Taylor de ordem 2, todos com o mesmo passo $h = 0,25$.

b) Plote as três soluções numéricas junto com a solução exata no mesmo gráfico.

c) Compare os erros máximos dos três métodos e discuta as vantagens e desvantagens do método de Taylor em relação aos demais, considerando a necessidade de calcular $y''$ analiticamente.

---

**8.** Considere o problema de valor inicial do Exercício 3.

a) Calcule $y'''(x)$ derivando $y''(x)$ em relação a $x$.

b) Implemente o método de Taylor de ordem 3, isto é,

$$y_{n+1} = y_n + y'_n h + y''_n \frac{h^2}{2!} + y'''_n \frac{h^3}{3!}$$

com $N = 8$ subintervalos em $[0, 1]$.

c) Compare, no mesmo gráfico, as soluções obtidas pelos métodos de Taylor de ordem 2 e de ordem 3 com a solução exata, e compare os respectivos erros máximos.

---

**9.** A lei do resfriamento de Newton estabelece que a temperatura $T(t)$ de um corpo varia proporcionalmente à diferença entre sua temperatura e a temperatura ambiente $T_{amb}$, isto é,

$$\frac{dT}{dt} = -k(T - T_{amb})$$

Considere $k = 0,4$, $T_{amb} = 25°C$ e a condição inicial $T(0) = 90°C$.

a) Calcule $T''(t)$ em função de $T$.

b) Use o método de Taylor de ordem 2 com $h = 0,5$ para estimar a temperatura do corpo entre $t = 0$ e $t = 10$ minutos.

c) Compare com a solução exata $T(t) = T_{amb} + (T_0 - T_{amb})e^{-kt}$ e determine, numericamente, em quanto tempo o corpo atinge $30°C$.

---

**10.** Implemente uma função `taylor_ordem2(f, df, x0, y0, xn, n)` que receba a função $f(x, y)$, sua derivada $y'' = df(x, y)$ (calculada analiticamente por você), o ponto inicial, o ponto final e o número de subintervalos, retornando os arrays `X` e `Y` da solução aproximada.

a) Teste sua função nos problemas de valor inicial dos Exercícios 1, 3 e 5, fornecendo em cada caso as expressões de $f$ e $y''$ já obtidas.

b) Verifique se os resultados coincidem com os obtidos ao resolver cada exercício individualmente.

c) Generalize sua função para aceitar também $y'''$ como parâmetro opcional, implementando o método de Taylor de ordem 3 quando esse argumento for fornecido, e teste-a no problema do Exercício 8.

## 7.4 Métodos de Runge-Kutta

O método de Runge-Kutta mais simples é o de ordem 1. Neste caso, o método coincide com o método de Euler, isto é, o método de Taylor de ordem 1, apresentado na seção 7.2.

Assim, considerando, novamente, o Problema de Valor Inicial (PVI) dado por

$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

tem-se o Método de Runge-Kutta como é apresentado a seguir.

### Método de Runge-Kutta de ordem 1

O processo iterativo será dado por:

$$y_{n+1} = y_n + hk_1$$

com $\$k_1 = f(x_n, y_n)\$$

(Ver exempos de seção 7.2).

### Método de Runge-Kutta de ordem 2

O processo iterativo será dado por:

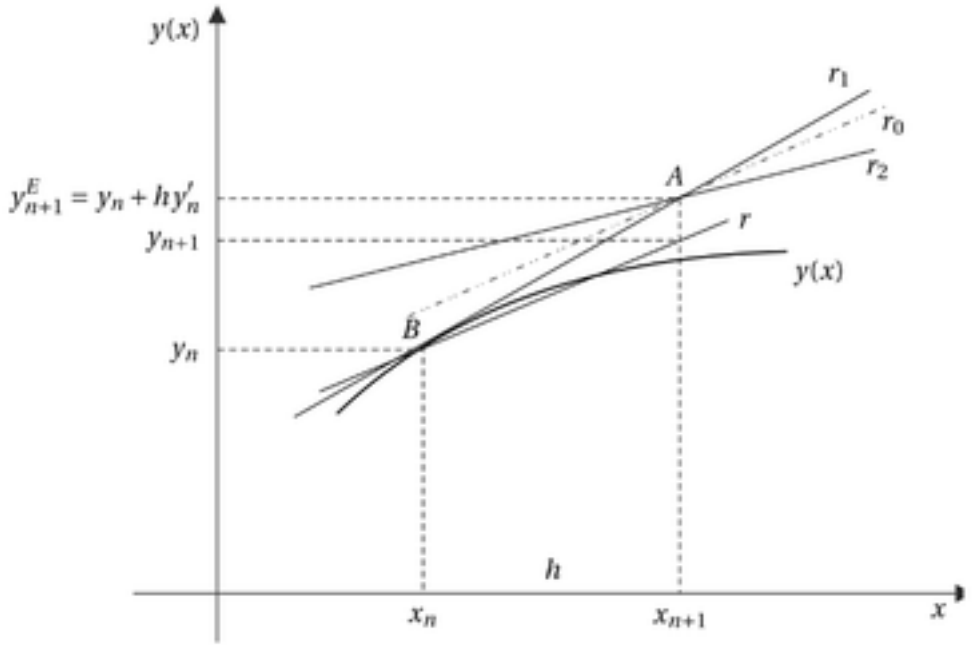
$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2)$$

com $\$k_1 = f(x_n, y_n)\$$

$$k_2 = f(x_n + h, y_n + hk_1)$$

o qual também é conhecido como **método de Euler aperfeiçoado**.

A figura abaixo ilustra uma interpretação gráfica do método.




**Exemplo 7.4.1:** Usando o método de Euler aperfeiçoado, calcule a solução do PVI definido por:

$$\begin{cases} y' = f(x,y) = x - y + 2 \\ y(x_0) = y(0) = 2 \end{cases}$$

com $x \in [a,b] = [0,1]$ e $N = 5$.

A solução exata é $y(x) = e^{-x} + x + 1$.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=(4,3)
```

```
f = lambda x,y: x-y+2
x0 = 0.0
y0 = 2.0
N = 5

a = 0.0
b = 1.0
X = np.linspace(a,b,N+1)
print (X)
Y = [y0]

h = (b-a)/N

for n in range(len(X)-1):
    k1 = f(X[n], Y[n])
    k2 = f(X[n]+h, Y[n]+h*k1)
```

(continua na próxima página)

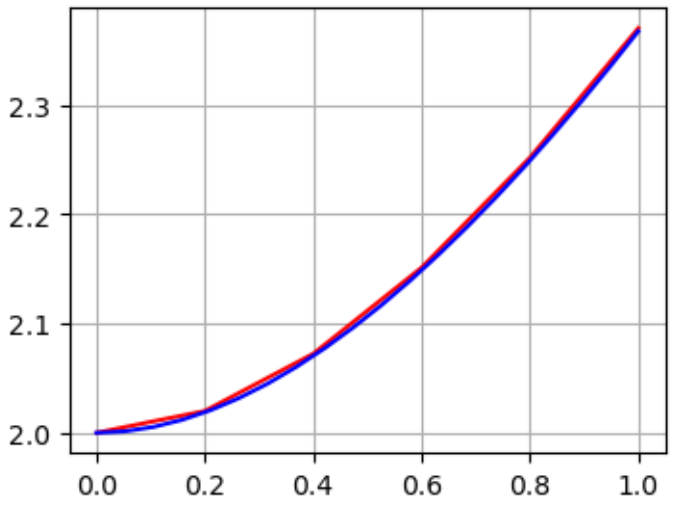
(continuação da página anterior)

```
    Y.append(Y[n]+0.5*h*(k1+k2))
print (Y)

s = lambda x: np.exp(-x) + x + 1
Xe = np.linspace(a,b, 20)
Ye = s(Xe)
print (s(X))

# plota
plt.plot(X, Y, "r-", Xe, Ye, "b-")
plt.grid()
plt.show()
```

```
[0.  0.2 0.4 0.6 0.8 1. ]
[2.0, 2.02, 2.0724, 2.151368, 2.25212176, 2.3707398432]
[2.         2.01873075 2.07032005 2.14881164 2.24932896 2.36787944]
```




Uma outra forma para o Método de Runge-Kutta de ordem 2 é dado pelo seguinte processo iterativo:

$$y_{n+1} = y_n + hf(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1)$$

com $$k_1 = f(x_n, y_n)$$

$$k_2 = f(x_n + h, y_n + hk_1)$$

o qual também é conhecido como **método de Euler modificado**.

**Exemplo 7.4.2:** Usando o método de Euler modificado, calcule a solução do PVI definido por:

$$\begin{cases} y' = f(x,y) = x - y + 2 \\ y(x_0) = y(0) = 2 \end{cases}$$

com $x \in [a,b] = [0,1]$ e $h = 0.2$.

A solução exata é $y(x) = e^{-x} + x + 1$.

```
f = lambda x,y: x-y+2
x0 = 0.0
y0 = 2.0
N = 5

a = 0.0
b = 1.0
X = np.linspace(a,b,N+1)
print (X)
Y = [y0]

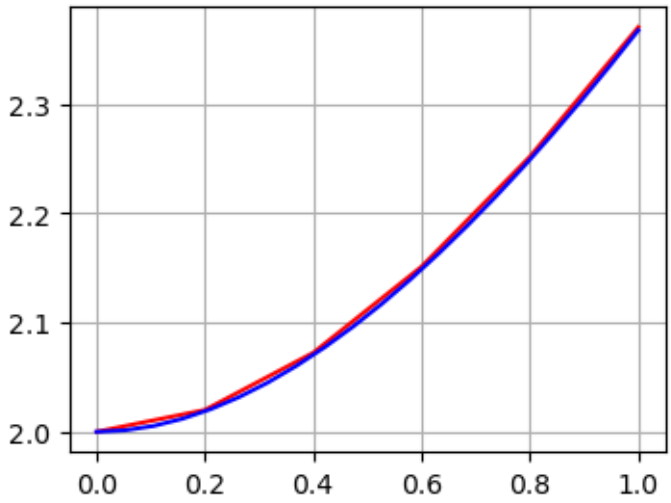
h = (b-a)/N

for n in range(len(X)-1):
    k1 = f(X[n], Y[n])
    Y.append(Y[n] + h * f(X[n]+0.5*h, Y[n]+0.5*h*k1))
    #print (k1,Y[-1])
print (np.round(Y,4))

s = lambda x: np.exp(-x) + x + 1
Xe = np.linspace(a,b, 20)
Ye = s(Xe)
print (np.round(s(X),4))

# plota
plt.plot(X, Y, "r-", Xe, Ye, "b-")
plt.grid()
plt.show()
```

```
[0.  0.2 0.4 0.6 0.8 1. ]
[2.     2.02   2.0724 2.1514 2.2521 2.3707]
[2.     2.0187 2.0703 2.1488 2.2493 2.3679]
```




## Método de Runge-Kutta de ordem 3

O método de Runge-Kutta ordem 3 é dado po

$$y_{n+1} = y_n + \frac{h}{9}(2k_1 + 3k_2 + 4k_3)$$

com

$$k_1 = f(x_n, y_n)$$

$$k_2 = f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1)$$

$$k_3 = f(x_n + \frac{3}{4}h, y_n + \frac{3}{4}hk_2)$$

**Exemplo 7.4.3:** Usando o método de Runge-Kutta de ordem 3, calcule a solução do PVI definido por:

$$\begin{cases} y' = f(x,y) = x - y + 2 \\ y(x_0) = y(0) = 2 \end{cases}$$

com $x \in [a,b] = [0,1]$ e $h = 0.2$.

A solução exata é $y(x) = e^{-x} + x + 1$.

```
f = lambda x,y: x-y+2
x0 = 0.0
y0 = 2.0
N = 5

a = 0.0
b = 1.0
X = np.linspace(a,b,N+1)
print (X)
Y = [y0]
```

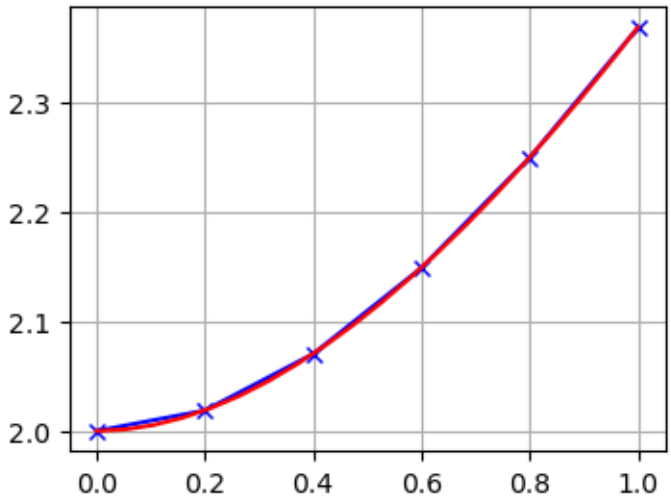
(continua na próxima página)

(continuação da página anterior)

```
h = (b-a)/N
for n in range(len(X)-1):
    k1 = f(X[n], Y[n])
    k2 = f(X[n]+0.5*h, Y[n]+0.5*h*k1)
    k3 = f(X[n]+0.75*h, Y[n]+0.75*h*k2)
    Y.append(Y[n] + (h/9) * (2*k1 + 3*k2 + 4*k3))
    #print (k1,Y[-1])
print (np.round(Y,4))

# plota
plt.plot(X, Y, "xb-", Xe, Ye, "r-")
plt.grid()
plt.show()
```

```
[0.  0.2 0.4 0.6 0.8 1. ]
[2.     2.0187 2.0702 2.1487 2.2492 2.3677]
```




## Método de Runge-Kutta de ordem 4

O método de Runge-Kutta ordem 4 é dado po

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k4)$$

com

$$k_1 = f(x_n, y_n)$$

$$k_2 = f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1)$$

$$k_3 = f(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2)$$

$$k_4 = f(x_n + h, y_n + hk_3)$$

**Exemplo 7.4.4:** Usando o método de Runge-Kutta de ordem 4, calcule a solução do PVI definido por:

$$\begin{cases} y' = f(x,y) = x - y + 2 \\ y(x_0) = y(0) = 2 \end{cases}$$

com $x \in [a,b] = [0,1]$ e $h = 0.2$.

A solução exata é $y(x) = e^{-x} + x + 1$.

```
f = lambda x,y: x-y+2
x0 = 0.0
y0 = 2.0
N = 5

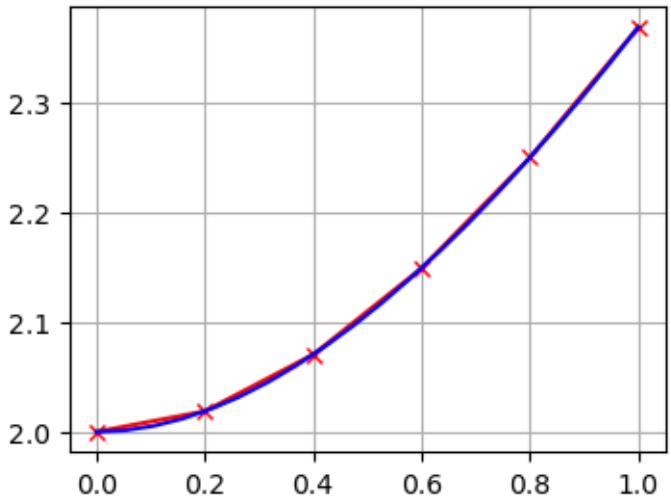
a = 0.0
b = 1.0
X = np.linspace(a,b,N+1)
print (X)
Y = [y0]

h = (b-a)/N
for n in range(len(X)-1):
    k1 = f(X[n], Y[n])
    k2 = f(X[n]+0.5*h, Y[n]+0.5*h*k1)
    k3 = f(X[n]+0.5*h, Y[n]+0.5*h*k2)

    k4 = f(X[n]+h, Y[n]+h*k3)
    Y.append(Y[n] + (h/6) * (k1 + 2*k2 + 2*k3 + k4))
    #print (k1,Y[-1])
print (np.round(Y,4))

# plota
plt.plot(X, Y, "xr-", Xe, Ye, "b-")
plt.grid()
plt.show()
```

```
[0.  0.2 0.4 0.6 0.8 1. ]
[2.     2.0187 2.0703 2.1488 2.2493 2.3679]
```




## Exercícios

---

**1.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = x^2 - y \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = x^2 - 2x + 2 - e^{-x}$.

a) Implemente o método de Runge-Kutta de ordem 2 (Euler aperfeiçoado) com $N = 5$ subintervalos em $[0, 2]$.

b) Plote, no mesmo gráfico, a solução obtida, a solução exata e a solução calculada pelo método de Taylor de ordem 2 no Exercício 2 da Seção 7.3.

c) Compare os erros máximos dos dois métodos para o mesmo $N$.

---

**2.** Considere o problema de valor inicial não linear

$$\begin{cases} \dfrac{dy}{dx} = xy \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = e^{x^2/2}$.

a) Implemente as duas variantes do método de Runge-Kutta de ordem 2 apresentadas nesta seção — Euler aperfeiçoado e Euler modificado — com $h = 0,2$ em $[0, 2]$.

b) Calcule o erro absoluto de cada variante em relação à solução exata em cada ponto.

c) Os dois métodos produzem exatamente os mesmos resultados? Explique por que, sendo $f$ não linear em $x$ e $y$, isso pode ocorrer ou não.

---

**3.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = y - x^2 + 1 \\ y(0) = 0,5 \end{cases}$$

cuja solução exata é $y(x) = (x+1)^2 - 0,5e^x$.

a) Implemente o método de Runge-Kutta de ordem 3 com $N = 8$ subintervalos em $[0, 2]$.

b) Compare a solução obtida com a solução exata e com a solução calculada pelo método de Taylor de ordem 2 no Exercício 5 da Seção 7.3.

c) Qual dos dois métodos apresenta menor erro máximo para o mesmo número de subintervalos?

---

**4.** Considere o problema de valor inicial não linear

$$\begin{cases} \dfrac{dy}{dx} = xy^2 \\ y(1) = 1 \end{cases}$$

cuja solução exata é $y(x) = \dfrac{2}{3 - x^2}$, definida para $x < \sqrt{3}$.

a) Implemente o método de Runge-Kutta de ordem 4 com $h = 0,05$ no intervalo $[1;\ 1,7]$.

b) Compare a solução obtida com a solução exata e com a solução calculada pelo método de Taylor de ordem 2 no Exercício 4 da Seção 7.3.

c) Discuta o comportamento dos dois métodos à medida que $x$ se aproxima de $\sqrt{3}$.

---

**5.** Para o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = 2x - y \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = 2x - 2 + 3e^{-x}$,

a) resolva numericamente usando o método de Runge-Kutta de ordem 4 com $h = 0,4$; $0,2$; $0,1$ e $0,05$;

b) calcule o erro máximo em relação à solução exata para cada tamanho de passo;

c) faça um gráfico de $\log(h)$ versus $\log(\text{erro máximo})$ e estime a ordem de convergência do método. O resultado é compatível com a ordem teórica $O(h^4)$?

---

**6.** Retome o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = x - y + 2 \\ y(0) = 2 \end{cases}$$

utilizado nos exemplos desta seção, cuja solução exata é $y(x) = e^{-x} + x + 1$.

a) Resolva-o com $N = 5$ em $[0, 1]$ usando o método de Euler e o método de Heun (Seção 7.2), o método de Taylor de ordem 2 (Seção 7.3) e os métodos de Runge-Kutta de ordens 2, 3 e 4.

b) Monte uma tabela comparando o erro máximo de cada método em relação à solução exata.

c) Cada passo dos métodos de Euler, Heun/RK2, RK3 e RK4 exige, respectivamente, 1, 2, 3 e 4 avaliações da função $f$. Discuta a relação entre esse custo computacional e o ganho de precisão obtido por cada método.

**7.** O crescimento de uma população é modelado pela equação logística

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

com $r = 0,5$, $K = 1000$ e $P(0) = 50$.

a) Escreva a solução exata do PVI, dada por $P(t) = \dfrac{KP_0e^{rt}}{K + P_0(e^{rt} - 1)}$.

b) Implemente o método de Runge-Kutta de ordem 4 com $h = 0,5$ para simular a população de $t = 0$ a $t = 20$.

c) Compare a solução numérica com a solução exata e determine, numericamente, em que instante a população atinge 90% da capacidade de suporte $K$.

---

**8.** Considere o problema de valor inicial

$$\begin{cases} \dfrac{dy}{dx} = -50y \\ y(0) = 1 \end{cases}$$

cuja solução exata é $y(x) = e^{-50x}$, uma função que decai rapidamente para zero.

a) Resolva o problema usando os métodos de Euler, Runge-Kutta de ordem 2 e Runge-Kutta de ordem 4, primeiro com $h = 0,1$ e depois com $h = 0,02$.

b) Plote, para cada tamanho de passo, as três soluções numéricas junto com a solução exata.

c) Algum dos métodos apresenta oscilações ou valores absurdos com $h = 0,1$? Discuta a relação entre o tamanho do passo, a magnitude de $f_y$ e a estabilidade da solução numérica.

---

**9.** Implemente uma função `rk4(f, x0, y0, xn, n)` que aplique o método de Runge-Kutta de ordem 4 a um PVI genérico $y' = f(x, y)$, $y(x_0) = y_0$, retornando os arrays `X` e `Y` da solução aproximada.

a) Teste sua função nos problemas de valor inicial dos Exercícios 1, 3 e 7.

b) Verifique se os resultados coincidem com os obtidos ao resolver cada exercício individualmente.

c) Generalize sua função para aceitar um parâmetro `ordem` (2, 3 ou 4) que selecione qual dos métodos de Runge-Kutta estudados nesta seção deve ser usado, mantendo a mesma interface de chamada.

---

**10.** Retome o problema de valor inicial de segunda ordem do Exercício 6 da Seção 7.2,

$$\frac{d^2y}{dt^2} - 0,5t + y = 0, \qquad y(0) = 2, \quad y'(0) = 0.$$

a) Reescreva o problema como um sistema de duas equações de primeira ordem, definindo $z = y'$.

b) Implemente o método de Runge-Kutta de ordem 4 para resolver simultaneamente $y$ e $z$, de $t = 0$ a $t = 4$ com $h = 0,1$.

c) Compare, no mesmo gráfico, a solução obtida por Runge-Kutta de ordem 4 com as soluções por Euler e por Heun encontradas no Exercício 6 da Seção 7.2.

---

# 8. Equações Diferenciais Parciais

Equações diferenciais parciais (EDPs) envolvem funções de múltiplas variáveis independentes e descrevem fenômenos como condução de calor, difusão, ondas e eletrostática. Métodos numéricos de discretização permitem obter soluções aproximadas em domínios complexos.

### Objetivos de aprendizagem

- Representar funções periódicas por séries de Fourier
- Classificar EDPs em elípticas, parabólicas e hiperbólicas
- Discretizar domínios espaciais e temporais por diferenças finitas
- Resolver a equação de Laplace (elíptica) por métodos iterativos
- Aplicar esquemas explícitos e implícitos à equação do calor (parabólica)
- Aproximar a equação de onda (hiperbólica) por diferenças finitas

### Conteúdo


| Seção | Tema |
|---|---|
| 8.1 | Introdução e séries de Fourier |
| 8.2 | Classificação e discretização |
| 8.3 | EDPs elípticas |
| 8.4 | EDPs parabólicas |
| 8.5 | EDPs hiperbólicas |


## 8.1 Introdução e Séries de Fourier

Problemas físicos importantes tais como aqueles ligados à condução de calor, à propagação de ondas e à teoria do potencial envolvem duas ou mais variáveis independentes e, portanto, são modelados por meio de Equações Diferenciais Parciais (EDPs).

Uma EDP de ordem $m$ é uma igualdade envolvendo uma função de $n$ variáveis ( $n \geq 2$ ) e suas respectivas derivadas parciais de até ordem $m(m \geq 1)$, ou seja é uma igualdade do tipo

$$F\left(x_1, \ldots, x_n, u, \frac{\partial u}{\partial x_1}, \ldots, \frac{\partial u}{\partial x_n}, \frac{\partial^2 u}{\partial x_1^2}, \frac{\partial^2 u}{\partial x_1 \partial x_2}, \ldots, \frac{\partial^m u}{\partial x_n^m}\right) = 0$$

onde $u = u\left(x_1, \ldots, x_n\right)$ e $F$ é uma função qualquer.

As equações diferenciais parciais básicas de condução de calor, propagação de ondas e teoria do potencial, estão associadas a três tipos distintos de fenômenos físicos: processos de difusão, processos oscilatórios e processos independentes do tempo ou estacionários.

Um método importante para resolver equações diferenciais parciais, conhecido como **método de separação de variáveis**, transforma a equação diferencial parcial por um conjunto de equações diferenciais ordinárias. A resolução muitas vezes requer condições iniciais ou de contorno e série em senos e/ou cossenos conhecidas como **séries de Fourier** (Boyce e Diprima, 2024).

## Séries de Fourier

As funções

$$\operatorname{sin}\left(\frac{n\pi x}{L}\right) \quad \text{e} \quad \cos\left(\frac{n\pi x}{L}\right), \quad n = 1, 2, \ldots,$$

são periódicas com período fundamental $T = 2L/n$ e formam, juntamente com a função constante 1, um conjunto ortogonal no intervalo $[-L, L]$. Assim, toda função periódica de período $2L$ que satisfaça as condições usuais de Fourier pode ser representada por sua **série de Fourier**

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty}\left(a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \operatorname{sin}\left(\frac{n\pi x}{L}\right)\right),$$

com os coeficientes de Fourier de $f(x)$ dados pelas **fórmulas de Euler**

$$a_0 = \frac{1}{2L}\int_{-L}^{L} f(x)dx,$$

$$a_n = \frac{1}{L}\int_{-L}^{L} f(x)\cos\frac{n\pi x}{L}dx$$

$$b_n = \frac{1}{L}\int_{-L}^{L} f(x)\operatorname{sen}\frac{n\pi x}{L}dx$$

com $n = 1, 2, 3, \ldots$ (Boyce, 2020).

Vamos ver alguns exemplos, mas antes vamos importar algumas funções e a constante $\pi$ usano a linha de código abaixo.

```
from numpy import sin, cos, pi, exp
```

**Exemplo 8.1.1:** Suponha que existe uma série de Fourier convergindo para a função $f$ definida por

$$f(x) = \begin{cases} -x, & -2 \leq x < 0 \\ x, & 0 \leq x < 2 \end{cases} \qquad \text{com} \qquad f(x+4) = f(x)$$

Vamos ver como são os coeficientes nesta série de Fourier e fazer o gráfico.

```
# definindo a função f
def f(x):
    if x>=2:
        return f(x-4)
```

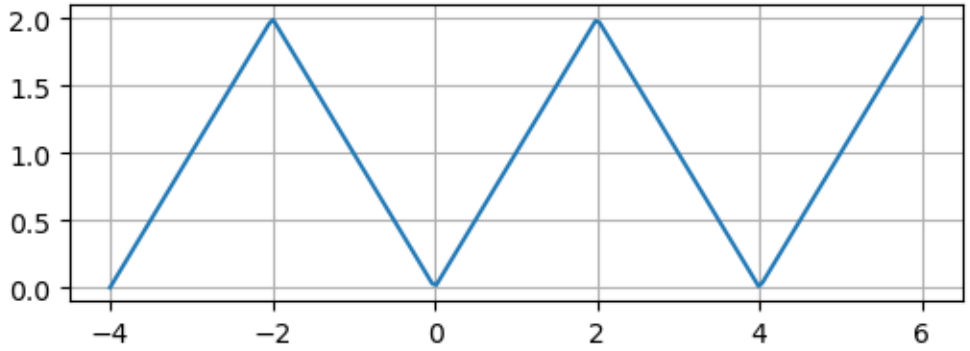
(continua na próxima página)

(continuação da página anterior)

```
    if x<-2:
        return f(x+4)
    if (x>=-2)&(x<0):
        return -x
    if (x>=0)&(x<2):
        return x
```

```
# importando as bibliotecas
import numpy as np
import matplotlib.pyplot as plt
```

```
# plotando a função f
x = np.linspace(-4,6,200)
fig, ax = plt.subplots(figsize=(6,2))
plt.plot(x,[f(xi) for xi in x])
plt.grid()
```




Os coeficientes são dados por

$$a_0 = \frac{1}{2}\int_{-2}^{0}(-x)dx + \frac{1}{2}\int_{0}^{2} x dx = 2$$

$$a_n = \frac{1}{2}\int_{-2}^{0}(-x)\cos\left(\frac{n\pi x}{2}\right)dx + \frac{1}{2}\int_{0}^{2} x\cos\left(\frac{n\pi x}{2}\right)dx = \begin{cases} -\frac{8}{(n\pi)^2}, & n \text{ ímpar} \\ 0, & n \text{ par.} \end{cases}$$

e

$$b_n = \frac{1}{2}\int_{-2}^{0}(-x)\,\text{sen}\left(\frac{n\pi x}{2}\right)dx + \frac{1}{2}\int_{0}^{2} x\,\text{sen}\left(\frac{n\pi x}{2}\right)dx = 0$$

Calculando as integrais, substituindo e simplificando, obtemos $f(x) = 1 - \frac{8}{\pi^2}\sum_{n=1}^{\infty}\frac{1}{(2n-1)^2}\cos\left(\frac{(2n-1)\pi x}{2}\right)$.

O gráfico da função e da série de Fourier com 3 termos é mostrado abaixo:

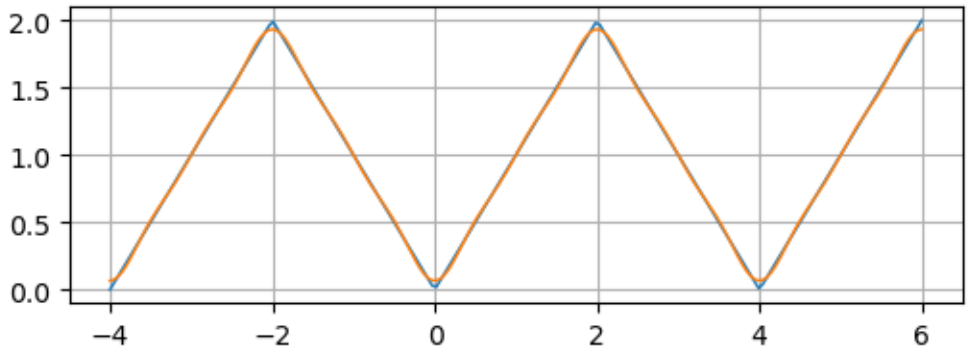
```
# plotando a função
x = np.linspace(-4,6,200)
fig, ax = plt.subplots(figsize=(6,2))
plt.plot(x,[f(xi) for xi in x], lw=1)
```

(continua na próxima página)

(continuação da página anterior)

```
# gerando a série
n_max = 3
soma = np.zeros(len(x))
for n in range(1,n_max+1):
    soma= soma+(1/(2*n-1)**2)*cos((2*n-1)*pi*x/2)
serie = 1-(8/np.pi**2)*soma

# plotando a série
plt.plot(x,serie,lw=1)
plt.grid()
```



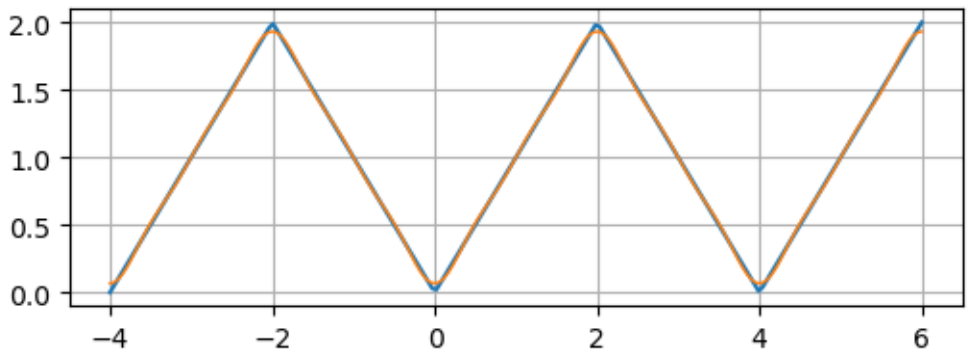

**Exemplo 8.1.2:** A seguir é mostrado como usar **integração numérica** por meio da função `quad` de `scipy.integrate` para obter uma aproximação por série de Fourier da função do primeiro exemplo.

```
from scipy.integrate import quad

soma = 1

for n in range(1,6):
    An = (1/2)*(quad(lambda x: -x*cos(n*pi*x/2),-2,0)[0] +\
                quad(lambda x: x*cos(n*pi*x/2), 0,2)[0])
    Bn = (1/2)*(quad(lambda x: -x*sin(n*pi*x/2),-2,0)[0] +\
                quad(lambda x: x*sin(n*pi*x/2), 0,2)[0])
    soma += An*cos(n*pi*x/2) + Bn*sin(n*pi*x/2)

fig, ax = plt.subplots(figsize=(6,2))
plt.plot(x,[f(xi) for xi in x])
plt.plot(x,soma, lw=1)
plt.grid()
```




**Exemplo 8.1.3:** Os coeficientes de Fourier da onda retangular periódica

$$f(x) = \begin{cases} -2 & \text{se} \quad -\pi < x < 0 \\ 2 & \text{se} \quad 0 < x < \pi \end{cases} \qquad \text{e} \qquad f(x + 2\pi) = f(x).$$

São tais que $a_0 = 0$, $a_n = 0$. Além disso, quando $n$ é par, $b_n = 0$, quando $n$ é impar, $b_n = \frac{8}{n\pi}$. Assim a série de Fourier é dada por

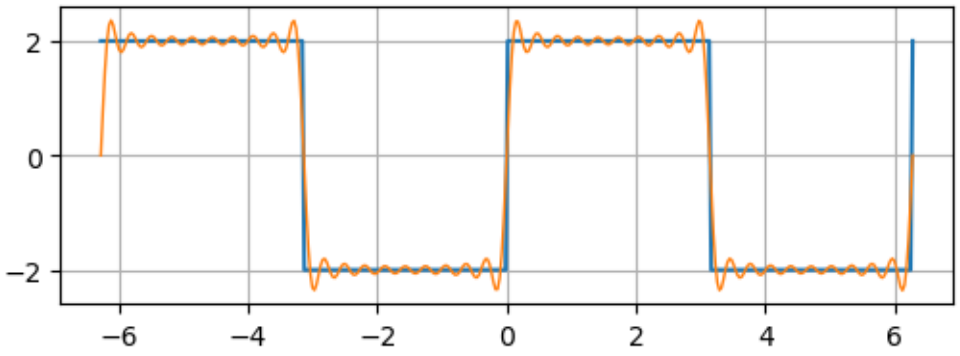
$$\frac{8}{\pi}\left(\operatorname{sen} x + \frac{1}{3}\operatorname{sen} 3x + \frac{1}{5}\operatorname{sen} 5x + \cdots\right).$$

Podemos plotar o gráfico da função e da série como é mostrado a seguir.

```
# definindo a função f
def f(x):
    if x>=pi:
        return f(x-2*pi)
    if x<-pi:
        return f(x+2*pi)
    if (x>=-pi)&(x<0):
        return -2
    if (x>=0)&(x<pi):
        return 2
```

```
xi = np.linspace(-2*pi, 2*pi, 400)
n_max = 20
soma = 0
for n in range(1, n_max, 2):
    soma += (8/np.pi)*sin(n*xi)/n

fig, ax = plt.subplots(figsize=(6,2))
plt.plot(xi,[f(x) for x in xi])
plt.plot(xi, soma, lw = 1)
plt.grid()
plt.show()
```




**Exemplo 8.1.4:** Usando integração numérica para obter uma aproximação por série de Fourier da função $f(x) = sen(e^x)$ no intervalo $(-3, 3)$. Plote o gráfico da função $f$ e da função aproximadora com 10 termos.

```
f = lambda x: sin(exp(x))
```

```
def fourier(f,N):
    n=0
    A0 = 1/np.pi * quad(lambda x: f(x)*np.cos(n*x),-np.pi,np.pi)[0]

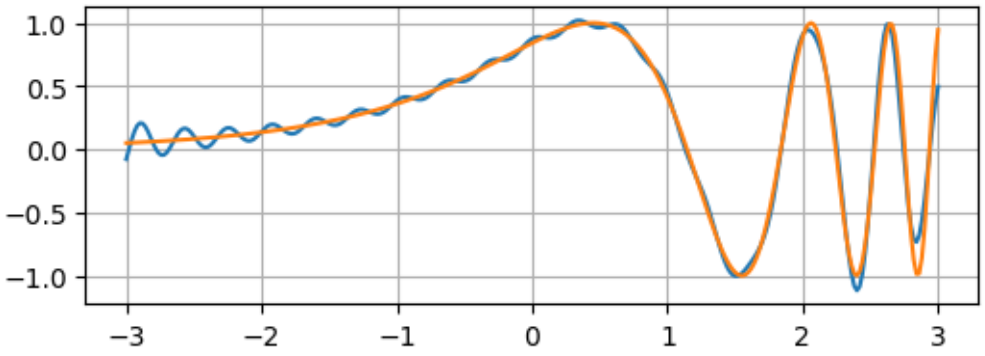
    xi = np.linspace(-3,3,301)
    yi = []

    soma = A0/2

    for x in xi:
        soma = A0/2
        for n in range(1,N):
            An = 1/np.pi * quad(lambda x: f(x)*np.cos(n*x),-np.pi,np.pi)[0]
            Bn = 1/np.pi * quad(lambda x: f(x)*np.sin(n*x),-np.pi,np.pi)[0]
            soma +=  An*np.cos(n*x)+Bn*np.sin(n*x)
        yi.append(soma)

    fig, ax = plt.subplots(figsize=(6,2))
    plt.plot(xi,yi, xi, f(xi))
    plt.grid()
```

```
fourier(f,20)
```

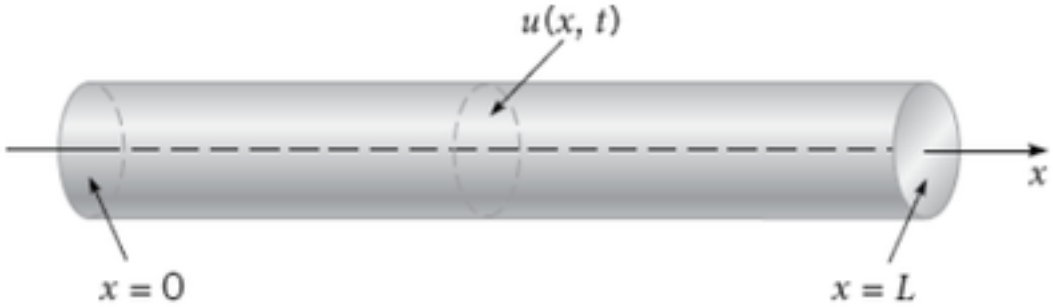



Agora vamos estudar a relação da série de Fourier com a solução da equação do calor unidimensional.

## Condução de calor em uma barra

Considere o problema de condução de calor em uma barra de seção reta uniforme, feita de material homogêneo, com um eixo $x$ coincidindo com o eixo da barra, sendo $x = 0$ e $x = L$ as extremidades.

Suponha que os lados da barra estão perfeitamente isolados e que as dimensões da seção reta são tão pequenas que a temperatura $u$ pode ser considerada constante em qualquer seção. Então, $u$ só depende da coordenada axial $x$ e do instante $t$.




(Fonte: Boyce e Diprima, 2024)

A variação da temperatura na barra é governada pela **equação diferencial parcial** (EDP) conhecida como a **equação do calor**, que pode ser escrita na forma

$$\alpha^2 u_{xx} = u_t, \quad 0 < x < L \quad \text{e} \quad t > 0,$$

em que $\alpha^2$ é difusividade térmica, uma constante que depende do material. Alguns valores típicos são mostrados como exemplo na tabela abaixo

| Material | cm$^2$/s |
|---|---|
| Cobre | 18,5 |
| Aluminio | 0,8418 |
| Ferro | 0,23 |
| Aço carbono | 0,1172 |
| Aço inox | 0,03352 |
| Madeira pinus | 0,00082 |

O problema fundamental de condução de calor é encontrar $u(x,t)$ que satisfaz a equação diferencial para $0 < x < L$ e para $t > 0$, com a condição inicial $u(x,0) = f(x), \quad 0 \leq x \leq L, quando$t = $0 e as condies de contorno$T_1 em$x = $0 e$T_2 em$x = L. Comearemos com as condies de contorno$T_1=T_2=0$,então,

$$u(0,t) = 0, \quad u(L,t) = 0, \quad t > 0.$$

Pelo **método de separação de variáveis**, iniciamos supondo que $u(x,t)$ é um produto de duas outras funções, uma dependendo só de $x$ e a outra dependendo só de $t$. Assim,

$$u(x,t) = X(x)T(t).$$

Substituindo na equação diferencial , obtemos

$$\alpha^2 X''(x)T(t) = X(x)T'(t)$$

ou, simplesmente $\alpha^2 X''T = XT', Dividindo os dois lados por \alpha^2 X T$, obtemos a equação equivalente

$$\frac{X''}{X} = \frac{1}{\alpha^2}\frac{T'}{T},$$

Para que a equação acima seja válida para cada ponto $(x,t)$ em $0 < x < L$, $t > 0$, é preciso que ambos os lados sejam iguais à mesma constante, então escrevemos

$$\frac{X''}{X} = \frac{1}{\alpha^2}\frac{T'}{T} = -\lambda$$

Obtemos, então, as duas equações diferenciais ordinárias a seguir para $X(x)$ e $T(t)$ :

$$X'' + \lambda X = 0 \qquad \text{e} \qquad T' + \alpha^2 \lambda T = 0$$

Substituindo $u(x,t)$ na condição de contorno em $x = 0$, obtemos $u(0,t) = X(0)T(t) = 0$, o que leva a $X(0) = 0$. A condição de contorno em $x = L$ implica que $X(L) = 0$.

As únicas soluções não triviais de $X'' + \lambda X = 0$ sob as condições $X(0) = 0$ e $X(L) = 0$, são as autofunções

$$X_n(x) = \text{sen}\left(\frac{n\pi x}{L}\right), \quad n = 1, 2, 3, ...$$

Associadas aos autovalores $\lambda_n = \frac{n^2\pi^2}{L^2}, \quad n = 1, 2, 3, ...$

Voltando para a equação para $T' + \alpha^2 \lambda T = 0$ e substituindo $\lambda$ por $n^2\pi^2/L^2$, temos

$$T' + \frac{n^2\pi^2\alpha^2}{L^2}T = 0$$

Logo, $T(t)$ é proporcional a $e^{-n^2\pi^2\alpha^2 t/L^2}$. Portanto, multiplicando as soluções, e desprezando as constantes arbitrárias de proporcionalidade, concluímos que as funções

$$u_n(x,t) = e^{-n^2\pi^2\alpha^2 t/L^2}\,\text{sen}\left(\frac{n\pi x}{L}\right), \quad n = 1, 2, 3, ...$$

são as soluções fundamentais do problema da condução do calor $\alpha^2 u_{xx} = u_t, \quad 0 < x < L, \quad t > 0$ com as condições de contorno $u(0,t) = 0, \quad u(L,t) = 0, \quad t > 0$.

Formando combinações lineares do conjunto fundamental de soluções da equação diferencial obtemos uma solução na forma de uma série infinita

$$u(x,t) = \sum_{n=1}^{\infty} c_n u_n(x,t) = \sum_{n=1}^{\infty} c_n e^{-n^2\pi^2\alpha^2 t/L^2} \operatorname{sen}\left(\frac{n\pi x}{L}\right),$$

em que os coeficientes $c_n$ ainda estão indeterminados. Para satisfazer a condição inicial $u(x,0) = f(x), \quad 0 \leq x \leq L$, temos que ter

$$u(x,0) = \sum_{n=1}^{\infty} c_n \operatorname{sen}\left(\frac{n\pi x}{L}\right) = f(x)$$

Assim, precisamos escolher os coeficientes $c_n$ tais que a série em senos na equação acima convirja para a distribuição inicial de temperatura $f(x)$ para $0 \leq x \leq L$. Ou seja, são os coeficientes da série de Fourier em senos de $f$, dados por

$$c_n = \frac{2}{L}\int_0^L f(x) \operatorname{sen}\left(\frac{n\pi x}{L}\right) dx$$

**Exemplo 8.1.5:** Encontre a temperatura $u(x,t)$ em qualquer instante em uma barra de metal com 50 cm de comprimento $\alpha^2 = 1$, isolada nos lados, inicialmente a uma temperatura uniforme de 20 °C em toda a barra e cujas extremidades são mantidas a 0 °C para todo t > 0.

Fazendo $L = 50$ e $f(x) = 20$ para $0 < x < 50$, a solução será dada por

$$u_n(x,t) = \sum_{n=1}^{\infty} c_n e^{-n^2\pi^2 t/2500} \operatorname{sen}\left(\frac{n\pi x}{50}\right)$$

com

$$c_n = \frac{2}{50}\int_0^{50} 20 \operatorname{sen}\left(\frac{n\pi x}{50}\right) dx = \frac{40}{n\pi}(1 - \cos n\pi) = \begin{cases} \frac{80}{n\pi}, & n \ mpar; \\ 0, & n \ par. \end{cases}$$

Substituindo, a solução fica

$$u_n(x,t) = \frac{80}{\pi} \sum_{n=1,3,5,\ldots}^{\infty} \frac{1}{n} e^{-n^2\pi^2 t/2500} \operatorname{sen}\left(\frac{n\pi x}{50}\right)$$

O fator exponencial com potência negativa em cada termo da série faz com que ela convirja rapidamente, exceto para valores pequenos de $t$. Portanto, resultados precisos podem ser obtidos, em geral, usando-se apenas alguns poucos termos da série.

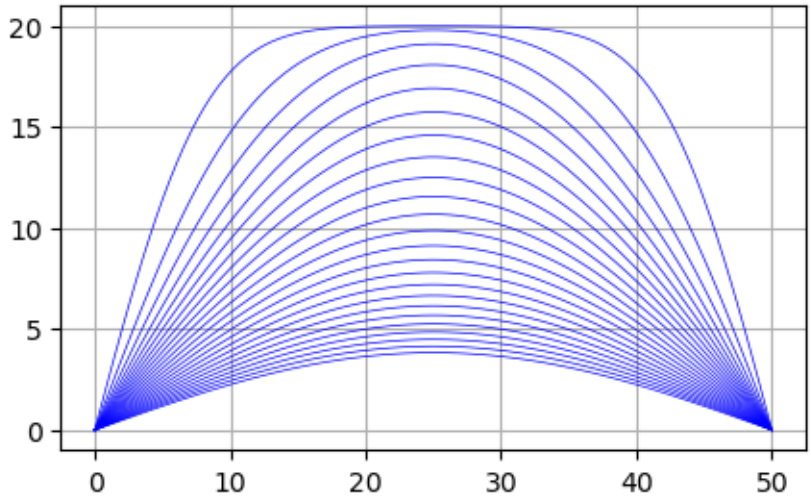
```
def u(x,t,N):
    soma = 0
    for n in range(1,N,2):
        soma += (1/n)*exp((-n**2*pi**2*t)/2500)*sin(n*pi*x/50)
    return (80/pi)*soma

N = 10
x = np.linspace(0,50,100)
fig, ax = plt.subplots(figsize=(5,3))
for t in range(20,500,20):
    plt.plot(x, [u(xi,t,N) for xi in x],
```

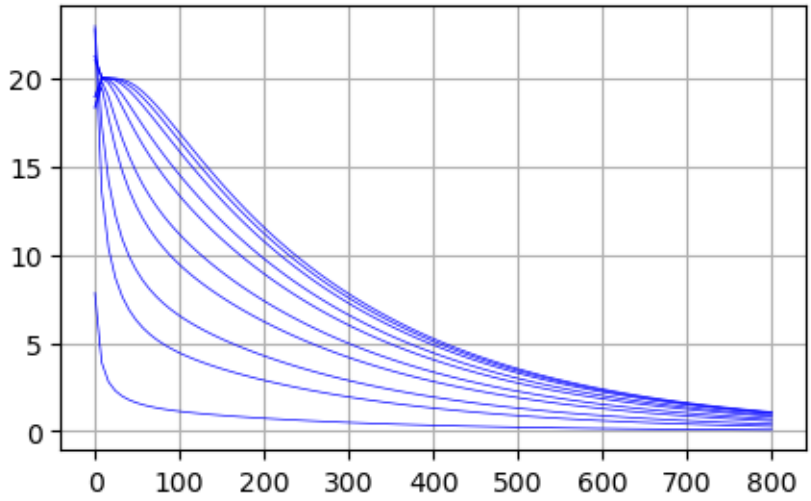
(continua na próxima página)

(continuação da página anterior)

```
            color ='b', linewidth = 0.5)
plt.grid()
```




```
N = 10
t = np.linspace(0,800,100)
fig, ax = plt.subplots(figsize=(5,3))
for x in range(1,50,5):
    plt.plot(t, [u(x,ti,N) for ti in t],
             color ='b', linewidth = 0.5)
plt.grid()
```




```
from matplotlib import cm  # color map
from mpl_toolkits.mplot3d import Axes3D


x, t = np.meshgrid(np.linspace(0, 50, 101),
```

(continua na próxima página)

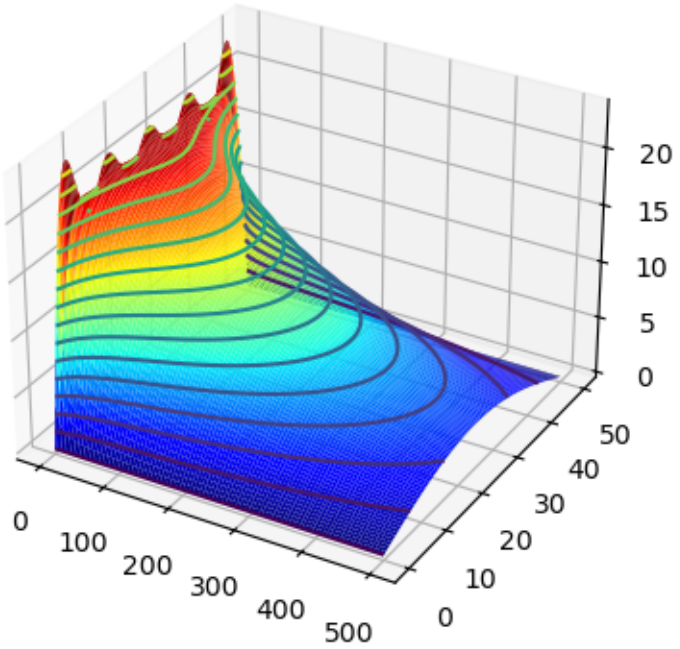
(continuação da página anterior)

```
                    np.linspace(0, 500, 101))

N = 10
ui = x.copy()
for i in range(len(x)):
    for j in range(len(t)):
        ui[i,j] = u(x[i,j],t[i,j],N)


fig = plt.figure(figsize=(10,10))
ax = fig.add_subplot(211,projection="3d")
ax.plot_surface(t, x, ui, rstride=1, cstride=1, cmap=cm.jet)
ax.contour(t, x, ui, 20,linewidths=1.5)

plt.show()
```




## Equação da onda

Dada a EDP

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

sob as condições

$$u(0,t) = 0, \quad u(L,t) = 0 \quad \text{para todo } t \quad \text{e}$$

$$u(x,0) = f(x), \quad u_t(x,0) = g(x) \quad (0 \leqq x \leqq L)$$

Sabe-se que a solução analítica é dada por

$$u(x,t) = \sum_{n=1}^{\infty} u_n(x,t) = \sum_{n=1}^{\infty} (B_n \cos \lambda_n t + B_n^* \operatorname{sen} \lambda_n t) \operatorname{sen} \frac{n\pi}{L} x$$

com $\lambda_n = \frac{cn\pi}{L}$ e

$$B_n = \frac{2}{L} \int_0^L f(x) \operatorname{sen} \frac{n\pi x}{L} dx \quad \text{e} \quad B_n^* = \frac{2}{cn\pi} \int_0^L g(x) \operatorname{sen} \frac{n\pi x}{L} dx$$

**Exemplo 8.1.6:** A solução da equação da onda correspondente à deflexão inicial triangular

$$f(x) = \begin{cases} \frac{2k}{L} x & \text{se} \quad 0 < x < \frac{L}{2} \\ \frac{2k}{L}(L-x) & \text{se} \quad \frac{L}{2} < x < L \end{cases}$$

e à velocidade inicial nula.

A solução será uma série dada por

$$u(x,t) = \frac{8k}{\pi^2} \left[ \frac{1}{1^2} \operatorname{sen} \frac{\pi}{L} x \cos \frac{\pi c}{L} t - \frac{1}{3^2} \operatorname{sen} \frac{3\pi}{L} x \cos \frac{3\pi c}{L} t + \cdots \right]$$

Considerando $c = 1$, $k = 1$ e $L = 2$ e plote gráficos para alguns valores de $t$.

```
import sympy as sp
from sympy import pi
x, t, c, L, k, i = sp.symbols('x t c L k i')
n = sp.symbols('n', integer=True,positive=True)

lamb = c*n*pi/L

f1 = 2*k*x/L
f2 = 2*k*(L-x)/L

Bn = (2/L)*(sp.integrate(f1*sp.sin((n*pi*x)/L),(x,0,L/2)) + \
            sp.integrate(f2*sp.sin((n*pi*x)/L), (x,L/2,L)))
```

```
g = 0

Bna = (2/(c*n*pi))*(sp.integrate(g*sp.sin((n*pi*x)/L),(x,0,L)))

display(Bna)
```

$$0$$

```
un = (Bn*sp.cos(lamb*t) + Bna*sp.sin(lamb*t))*sp.sin(n*pi*x/L)
display(sp.simplify(un))
```

$$\frac{8k \sin\left(\frac{\pi n}{2}\right) \sin\left(\frac{\pi n x}{L}\right) \cos\left(\frac{\pi c n t}{L}\right)}{\pi^2 n^2}$$

```
for i in range(1,7):
    display(un.subs(n,i))
```

$$\frac{8k \sin\left(\frac{\pi x}{L}\right) \cos\left(\frac{\pi ct}{L}\right)}{\pi^2}$$

$$0$$

$$-\frac{8k \sin\left(\frac{3\pi x}{L}\right) \cos\left(\frac{3\pi ct}{L}\right)}{9\pi^2}$$

$$0$$

$$\frac{8k \sin\left(\frac{5\pi x}{L}\right) \cos\left(\frac{5\pi ct}{L}\right)}{25\pi^2}$$

$$0$$

```
un = un.subs({c: 1, k: 1, L: 2})
display(un)
```

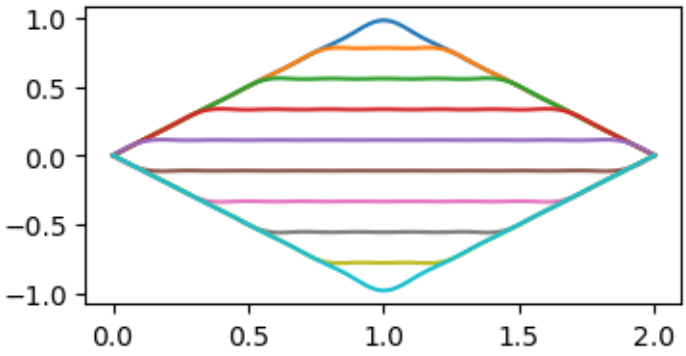
$$\frac{8 \sin\left(\frac{\pi n}{2}\right) \sin\left(\frac{\pi nx}{2}\right) \cos\left(\frac{\pi nt}{2}\right)}{\pi^2 n^2}$$

```
u = sp.lambdify((n,x,t), un)
```

```
xi = np.linspace(0,2,100)
ui = np.zeros(len(xi))

fig, ax = plt.subplots(figsize=(4,2))

for ti in np.linspace(0,2,10):
    ui = np.zeros(len(xi))
    for ni in range(1,20):
        ui += u(ni,xi,ti)
    plt.plot(xi,ui)
#plt.grid()
plt.show()
```




## Exercícios

(Fonte: Boyce e Diprima, 2024)

---

**1.** Encontre a solução do problema de condução de calor

$$
\begin{aligned}
&100u_{xx} = u_t, \quad 0 < x < 1, \quad t > 0\\
&u(0,t) = 0, \quad u(1,t) = 0, \quad t > 0\\
&u(x,0) = \operatorname{sen}(2\pi x) - \operatorname{sen}(5\pi x), \quad 0 \le x \le 1
\end{aligned}
$$

---

**2.** Encontre a solução do problema de condução de calor

$$
\begin{aligned}
&u_{xx} = 4u_t, \quad 0 < x < 2, \quad t > 0\\
&u(0,t) = 0, \quad u(2,t) = 0, \quad t > 0\\
&u(x,0) = 2\operatorname{sen}(\pi x/2) - \operatorname{sen}(\pi x) + 4\operatorname{sen}(2\pi x), \quad 0 \le x \le 2
\end{aligned}
$$

---

Considere a condução de calor em uma barra com 40 cm de comprimento cujas extremidades são mantidas à temperatura $0°\text{C}$ para todo $t > 0$. Em cada um dos Problemas 3 a 6, encontre uma expressão para a temperatura $u(x,t)$ quando a distribuição de temperatura inicial na barra for igual à função dada. Suponha que $\alpha^2 = 1$

**3.** $u(x,0) = 50, \quad 0 < x < 40$

**4.** $u(x,0) = \begin{cases} x, & 0 \le x < 20 \\ 40 - x, & 20 \le x \le 40 \end{cases}$

**5.** $u(x,0) = \begin{cases} 0, & 0 \le x < 10 \\ 50, & 10 \le x \le 30 \\ 0, & 30 < x \le 40 \end{cases}$

**6.** $u(x,0) = x, \quad 0 \le x < 40$

---

**7.** Considere novamente a barra do Problema 3. Para $t = 5$ e $x = 20$, determine quantos termos são necessários para encontrar a solução correta até três casas decimais. Um modo razoável de fazer isso é encontrar $n$ tal que a inclusão de mais um termo não muda as três primeiras casas decimais de $u(20, 5)$. Repita para $t = 20$ e $t = 80$. Chegue a alguma conclusão sobre a velocidade de convergência da série que representa $u(x, t)$.

**8.** Repita o Problema 7 para a barra no Problema 4.

**9.** Repita o Problema 7 para a barra no Problema 5.

**10.** Repita o Problema 7 para a barra no Problema 6.

---

## 8.2 Classificação e discretização de EDPs

Problemas físicos importantes tais como aqueles ligados à condução de calor, à propagação de ondas e à teoria do potencial envolvem duas ou mais variáveis independentes e, portanto, são modelados por meio de equações diferenciais parciais.

As equações diferenciais parciais básicas de condução de calor, propagação de ondas e teoria do potencial, estão associadas a três tipos distintos de fenômenos físicos: processos de difusão, processos oscilatórios e processos independentes do tempo ou estacionários.

Um método importante para resolver equações diferenciais parciais, conhecido como **método de separação de variáveis**, transforma a equação diferencial parcial por um conjunto de equações diferenciais ordinárias. A resolução muitas vezes requer condições iniciais ou de contorno e série em senos e/ou cossenos conhecidas como séries de Fourier (Boyce e Diprima, 2020).

Uma equação diferencial parcial (EDP) de ordem $m$ é uma igualdade envolvendo uma função de $n$ variáveis ( $n \geq 2$ ) e suas respectivas derivadas parciais de até ordem $m(m \geq 1)$, ou seja é uma igualdade do tipo

$$F\left(x_1, \ldots, x_n, u, \frac{\partial u}{\partial x_1}, \ldots, \frac{\partial u}{\partial x_n}, \frac{\partial^2 u}{\partial x_1^2}, \frac{\partial^2 u}{\partial x_1 \partial x_2}, \ldots, \frac{\partial^m u}{\partial x_n^m}\right) = 0$$

onde $u = u\left(x_1, \ldots, x_n\right)$ e $F$ é uma função qualquer.

Podemos classificar as EDP's lineares de $2^a$ ordem com duas variáveis independentes, segundo *arquétipos fundamentais da Física-Matemática*. Essa classificação é originária da classificação das cônicas realizada na geometria analítica. Partimos da equação na forma canônica.

A forma geral canônica das EDP's lineares de 2a ordem com duas variáveis independentes é dada por:

$$A\frac{\partial^2 u}{\partial x^2} + B\frac{\partial^2 u}{\partial x \partial y} + C\frac{\partial^2 u}{\partial y^2} + D\frac{\partial u}{\partial x} + E\frac{\partial u}{\partial y} + Fu = G$$

onde $A, B, C, D, E, F$ e $G$ são funções de das variáveis independentes $x_1 = x, x_2 = y$.

A classificação da equação acima como elíptica, parabólica ou hiperbólica em um ponto $(x_0, y_0)$ é determinada pelos seus coeficientes $A(x_0, y_0)$, $B(x_0, y_0)$ e $C(x_0, y_0)$ de acordo com as seguintes condições gerais:

1. **Equação Elíptica:** $B^2 - 4AC < 0$;
2. **Equação Parabólica:** $B^2 - 4AC = 0$;
3. **Equação Hiperbólica:** $B^2 - 4AC > 0$.

A classificação elíptica, parabólica ou hiperbólica de uma equação diferencial parcial está relacionada com as propriedades físicas, geométricas e matemáticas do problema que a equação representa. Essa classificação ajuda a entender o comportamento das soluções e a escolher técnicas apropriadas para resolvê-las.

- equações elípticas descrevem fenômenos em regime permanente (steady-state).
- equações parabólicas descrevem fenômenos difusivos.
- equações hiperbólicas descrevem fenômenos ondulatórios.

Alguns exemplos de EDPs **elípticas** em duas variáveis são:

- **Equação de Poisson**:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y)$$

- **Equação de Laplace**:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

- **Equação de Helmholtz**:

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} + g(x, y)u(x, y) = f(x, y)$$

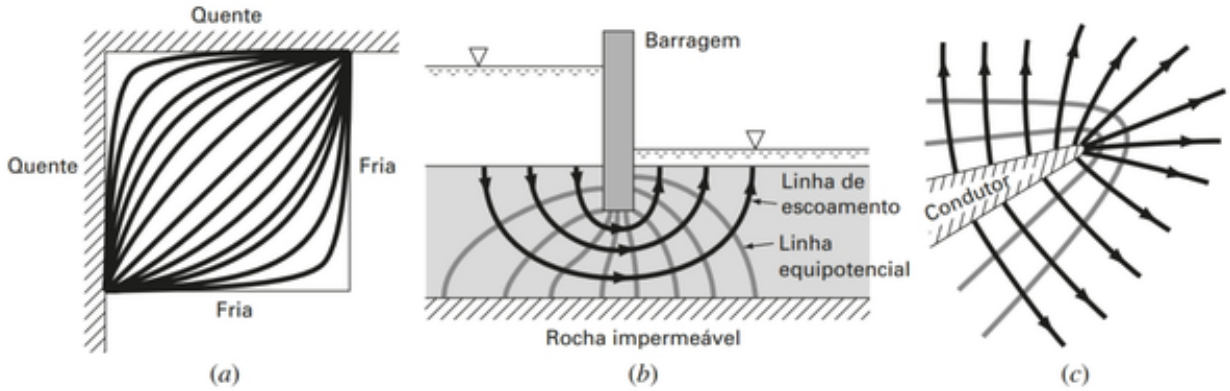



Figura 5.1 - Três problemas de distribuições estacionários que podem ser caracterizados por EDPs elípticas. (a) Distribuição de temperatura em uma placa aquecida, (b) infiltração de água sob uma barragem e (c) o campo elétrico perto da ponta de um condutor.

(Fonte: Chapra, 2016.).

Alguns exemplos de EDPs **parabólicas** em duas variáveis são:

- **Equação do calor**:

$$\frac{\partial u}{\partial t} + \frac{\partial^2 u}{\partial x^2} = 0$$

- **Equação de Fourier**:

$$\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} = 0$$

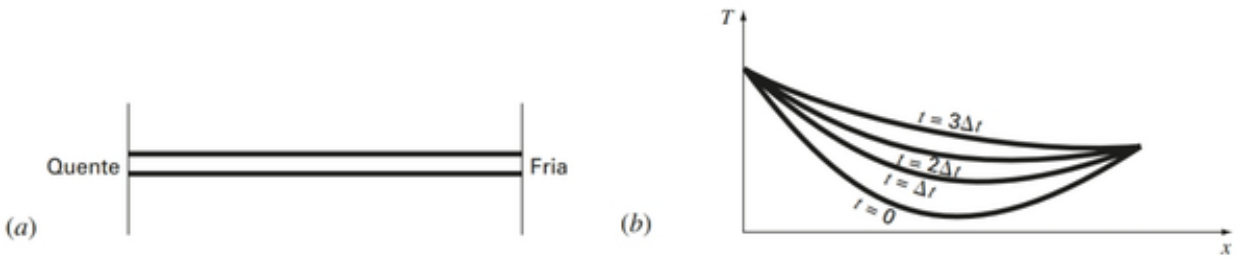



Figura 5.2 - (a) Uma haste longa e fina é isolada em toda parte, exceto nas extremidades. A dinâmica da distribuição unidimensional da temperatura ao longo do comprimento da haste pode ser descrita por uma EDP parabólica. (b) A

solução, que consiste nas distribuições correspondentes ao estado da haste em vários instantes.

Um exemplo de EDP **hiperbólica**:

- **Equação da onda:**

$$\frac{\partial^2 u}{\partial t^2} - \frac{\partial^2 u}{\partial x^2} = 0$$



Figura 5.3 - Uma corda tensionada vibrando com pequena amplitude é um sistema físico simples que pode ser caracterizado por uma EDP hiperbólica.

(Fonte: Chapra, 2016.).

Outros exemplos importantes

- **Equação de Schrödinger** - mecânica quântica
- **Equações de Navier-Stokes** - dinâmica de fluidos viscosos
- **Equação de Black-Scholes** - precificação de opções financeiras
- **Equações de Maxwell** - eletromagnetismo

## Métodos de solução

**Analíticos:** separação de variáveis, transformadas (Fourier, Laplace), séries de potências.

**Numéricos:** diferenças finitas, elementos finitos, volumes finitos, métodos espectrais (necessários para problemas mais complexos).

**Exemplo 8.2.1: (Solução analitica da equação de Laplace)** Considere a equação elíptica de Laplace em 2D

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

no dominio $0 \leq x \leq 4$ e $0 \leq y \leq 4$, e com as condições de contorno

$$u(0,y) = 0 \quad , \quad u(4,y) = 100 \quad \text{para } \ 0 < y < 4, \quad \text{e}$$

$$u(x,0) = 0 \quad , \quad u(x,4) = 0 \quad \text{para } \ 0 < x < 4.$$

O método de separação de variáveis pode ser usado para obter a solução analítica

$$u(x,y) = \sum_{n=1}^{\infty} \frac{200(1-(-1)^n)}{\pi n \sinh(\pi n)} \sinh\left(\frac{n\pi x}{4}\right) \sin\left(\frac{n\pi y}{4}\right)$$

Assim, para cada ponto $(x, y)$ dentro da região $0 \leq x \leq 4$, $0 \leq y \leq 4$, podemos obter uma aproximação para a temperatura $u(x, y)$, expandindo a série de Fourier em um número de termos pré-definidos. Vamos plotar um gráfico para a solução.

```
import numpy as np
import matplotlib.pyplot as plt
```

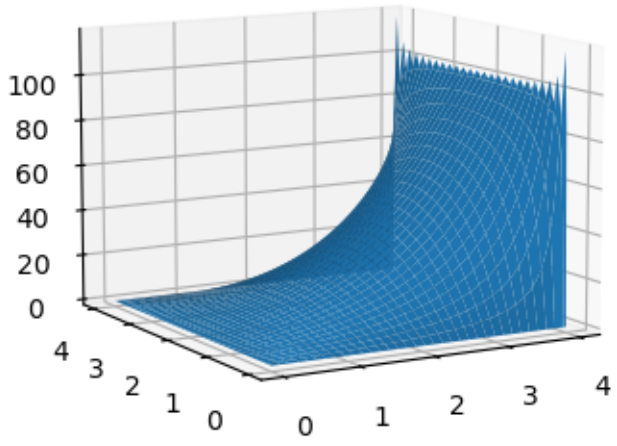
```
def u(x, y, n_max):
  soma = 0
  for n in range(1, n_max):
    soma += (200*(1-(-1)**n)/(np.pi*n*np.sinh(np.pi*n)))*np.sinh(n*np.pi*x/4)*np.
 ↪sin(n*np.pi*y/4)
  return soma
```

Temperatura em $x = 2$ e $y = 1$, usando 20 termos da série:

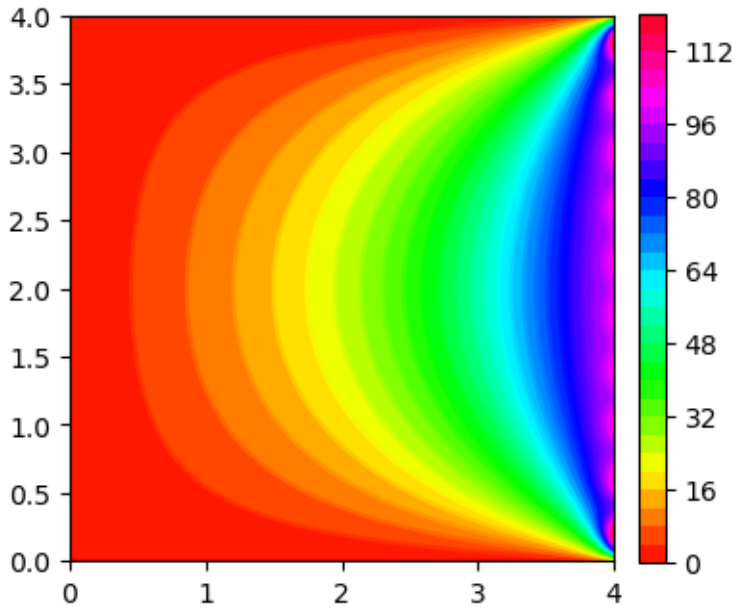
```
x, y = np.meshgrid(np.linspace(0, 4, 101),
                   np.linspace(0, 4, 101))
```

```
# plotando a superfície 3D
ax = plt.figure(figsize=(8, 4)).add_subplot(projection='3d')
ax.plot_surface(x, y, u(x, y, 50))
ax.view_init(10,-120)

plt.show()
```




```
plt.figure(figsize=(4, 4))
plt.contour(x,y,u(x, y, 20),cmap='hsv', levels=30)
plt.contourf(x,y,u(x, y, 20),cmap='hsv', levels=30)
plt.gca().set_aspect('equal')
plt.colorbar(fraction=0.046, pad=0.04)
plt.show()
```




O método da separação de variáveis funciona bem quando:

- A EDP é linear
- O domínio tem geometria simples (retângulo, cilindro, esfera)
- As condições de contorno são homogêneas

Para problemas mais complexos, recorre-se a transformadas integrais ou métodos numéricos. Métodos Numéricos requerem a discretização do domínio do problema.

**Discretização do domínio**

Em geral, a aplicação de métodos numéricos em problemas envolvendo equações diferenciais envolvem a discretização do domínio. Consideremos aqui, como um exemplo, a dicretização de uma área retangular de 30 por 20 (unidades de medida) em pontos igualmente espaçados em $\Delta x = \Delta y = 2,5$ nas direções $x$ e $y$ conforme a figura abaixo.

Nesse exemplo, o domínio é dado por

$$\mathrm{D} = \{(x, y) \mid 0 \leq x \leq 30, \quad 0 \leq y \leq 20\}$$

Dessa forma, o intervalo $[0, 30]$ é discretizado em $N = 12$ subintervalos e o intervalo $[0, 20]$ é discretizado em $M = 8$ subintervalos. Os pontos extremos destes subintervalos passam a definir uma malha bidimensional discreta com $(N + 1) \times (M + 1)$ pontos, ou nós, na qual serão avaliadas as respostas aproximadas. A figura 5.4 mostra a malha e os índices correspondentes a cada nó.

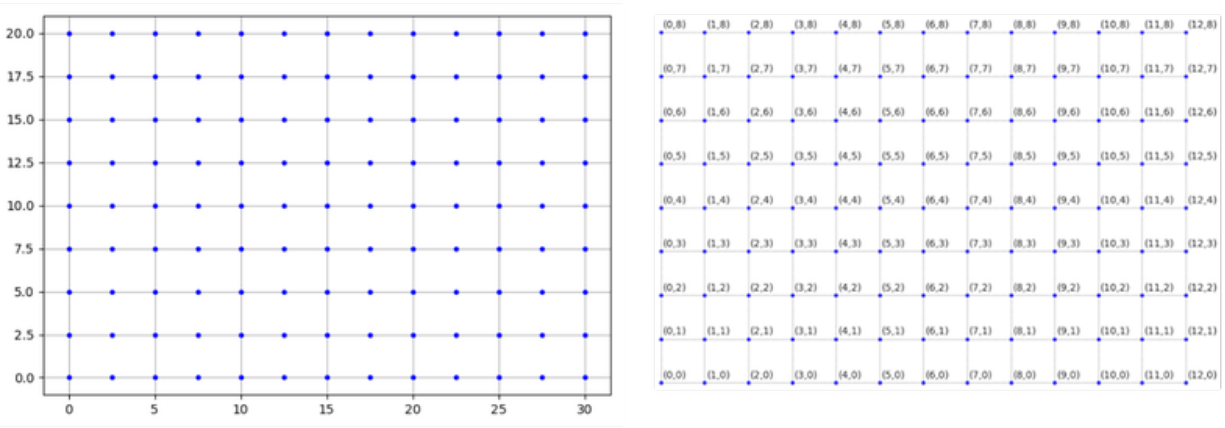


Figura 5.4 - Exemplo de discretização de uma área retangular de 30x20 com 13 pontos na direção $x$ e 9 pontos na direção $y$. O código para gerar essas figuras é mostrado no final do texto.

**Discretização do operador diferencial**

A abordagem numérica também requer a discretização do operador diferencial que consiste em aproximar as derivadas de uma equação diferencial parcial através de diferenças entre os valores discretos das variáveis ao longo de uma malha espacial, transformando a equação contínua em um sistema de equações algébricas solucionáveis.

Uma fórmula de diferenças finitas centrais para aproximar numericamente a derivada de segunda ordem de uma função $u(x, y)$ na direção $x$ é:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(x + \Delta x, y) - 2u(x, y) + u(x - \Delta x, y)}{\Delta x^2}$$

onde $h_x$ é o tamanho do passo na direção $x$ na malha discreta. Essa fórmula estima a derivada de segunda ordem utilizando os valores discretos da função $u$ em três pontos adjacentes na direção $x$. Na direção $y$ a fórmula é análoga.

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u(x, y + \Delta y) - 2u(x, y) + u(x, y - \Delta y)}{\Delta y^2}$$

Uma notação mais compacta para as fórmulas acima, aplicadas a um ponto $(x_i, y_j)$, em que $x_i = x_0 + i\Delta x$ e $y_j = y_0 + j\Delta x$ pode ser escrita como

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x_i, y_j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2} \quad \text{e} \quad \left.\frac{\partial^2 u}{\partial y^2}\right|_{x_j, y_i} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}$$

O conjunto de 5 pontos usados para calcular a derivada aproximada no ponto central é chamado **stencil** e é mostrado na figura abaixo.

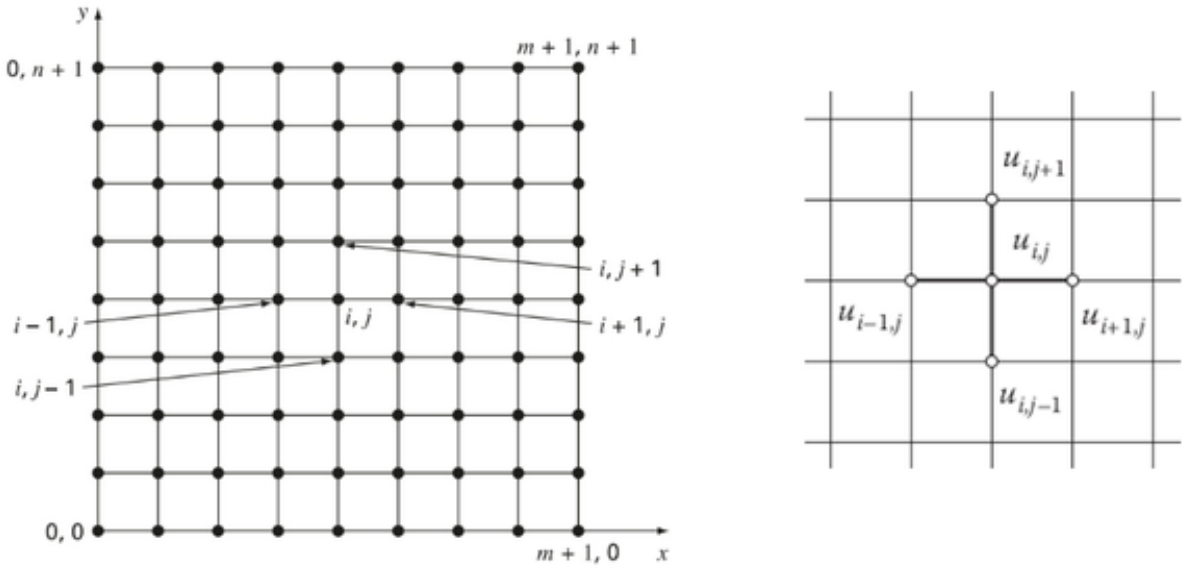



Figura 5.5 - Exemplo de discretização de uma área retangular de 30x20 com 13 pontos na direção $x$ e 9 pontos na direção $y$.

Para a derivada temporal de 1ª ordem, podemos descrever o operador diferencial em nós na malha no instante seguinte utilizando apenas o estado anterior (*Foward Difference Method FDM*). Esse esque dá origem ao chamado *método explícito* a ser resolvido iterativamente.

$$\frac{\partial u}{\partial t} \approx \frac{u\left(x^i, t^{j+1}\right) - u\left(x^i, t^j\right)}{\Delta t}$$

ou utilizando os dados dos nós vizinhos a serem também calculados (*Backward Difference Method - BDM*). Este esquema dá origem ao *método implícito*

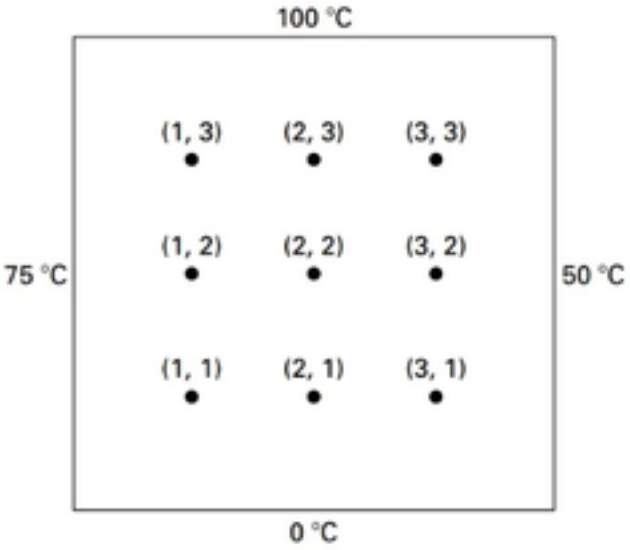
$$\frac{\partial u}{\partial t} \approx \frac{u\left(x^i, t^j\right) - u\left(x^i, t^{j-1}\right)}{\Delta t}$$

Em que o passo no tempo é dado por $\Delta t$ e os subintervalos da malha espacial são de tamanho $\Delta x$.

O esquema conhecido como *método de Crank-Nicolson* utiliza diferenças finitas centrais

$$\frac{\partial u}{\partial t} \approx \frac{u\left(x^i, t^{j+1}\right) - u\left(x^i, t^{j-1}\right)}{2\Delta t}$$

**Exemplo 8.2.2: (A equação de diferença de Laplace)** Considere uma placa aquecida na qual as temperaturas nas bordas são mantidas em níveis constantes (condição de contorno de Dirichlet) como é mostrado na figura abaixo (Fonte: Chapra, 2016).




Vamos obter valores da temperatura nos pontos internos a partir da equação de Laplace

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Usando diferenças centradas baseadas no esquema de grade semelhante ao que aparece na figura 5.4.

Substituindo as fórmulas

$$\frac{\partial^2 T}{\partial x^2} = \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \qquad \text{e} \qquad \frac{\partial^2 T}{\partial y^2} = \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}$$

na equaçao, obtemos

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0$$

Para a grade quadrada, e que $\Delta x = \Delta y$, agrupando os termos podemos reescrever a equação como

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0$$

Para o nó $(1, 1)$, a equação fica

$$-4T_{11} + T_{12} + T_{21} = -75$$

Obtendo as equações para todos os nós interiores chaga-se ao sistema linear com nove equações simultâneas e nove incógnita

$$
\begin{array}{rrrrrrrrrr}
4T_{11} & -T_{21} & & -T_{12} & & & & & & = 75 \\
-T_{11} & +4T_{21} & -T_{31} & & -T_{22} & & & & & = 0 \\
 & -T_{21} & +4T_{31} & & & -T_{32} & & & & = 50 \\
-T_{11} & & & +4T_{12} & -T_{22} & & -T_{13} & & & = 75 \\
 & -T_{21} & & -T_{12} & +4T_{22} & -T_{32} & & -T_{23} & & = 0 \\
 & & -T_{31} & & -T_{22} & +4T_{32} & & & -T_{33} & = 50 \\
 & & & -T_{12} & & & +4T_{13} & -T_{23} & & = 175 \\
 & & & & -T_{22} & & -T_{13} & +4T_{23} & -T_{33} & = 100 \\
 & & & & & -T_{32} & & -T_{23} & +4T_{33} & = 150
\end{array}
$$

```
import numpy as np
```

```
A = np.array([[4,-1,0,-1,0,0,0,0,0],
              [-1,4,-1,0,-1,0,0,0,0],
              [0,-1,4,0,0,-1,0,0,0],
              [-1,0,0,4,-1,0,-1,0,0],
              [0,-1,0,-1,4,-1,0,-1,0],
              [0,0,-1,0,-1,4,0,0,-1],
              [0,0,0,-1,0,0,4,-1,0],
              [0,0,0,0,-1,0,-1,4,-1],
              [0,0,0,0,0,-1,0,-1,4]])
np.linalg.det(A)
```

```
100352.00000000013
```

```
b = np.array([75,0,50,75,0,50,175,100,150])
T = np.linalg.solve(A,b)
print(T)
```

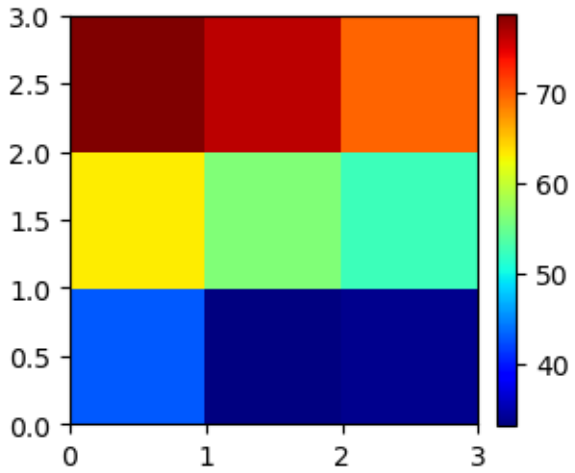
```
[42.85714286 33.25892857 33.92857143 63.16964286 56.25       52.45535714
 78.57142857 76.11607143 69.64285714]
```

```
#Colocando as temperaturas na forma de matriz
T = T.reshape((3,3))
print(T)
```

```
[[42.85714286 33.25892857 33.92857143]
 [63.16964286 56.25       52.45535714]
 [78.57142857 76.11607143 69.64285714]]
```

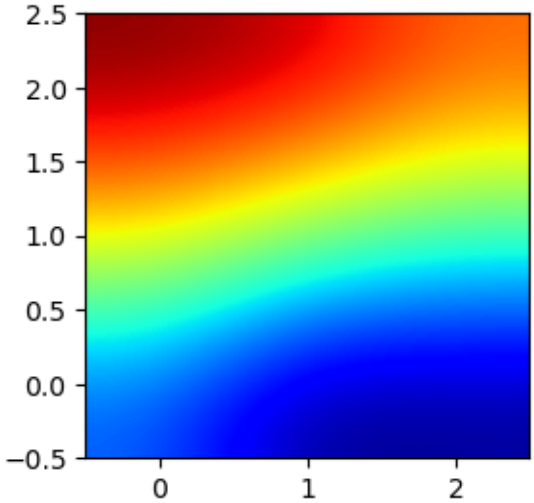
```
#plot
plt.figure(figsize=(3, 3))
plt.pcolor(T,cmap='jet')
plt.gca().set_aspect('equal')

plt.colorbar(fraction=0.046, pad=0.04)
plt.show()
```




```
# usando interpolação para suavisar
fig, ax = plt.subplots(figsize=(3, 3))
ax.imshow(T, interpolation='bicubic', cmap='jet', origin='lower')

plt.show()
```




Tipos de interpolação disponíveis para `imshow`: `nearest`, `bilinear`, `bicubic`, `spline16`,`spline36`, `hanning`, `hamming`, `hermite`, `kaiser`, `quadric`, `catrom`, `gaussian`, `bessel`, `mitchell`, `sinc` e `lanczos`.

**Exemplo 8.2.3:** Considere a equação de Laplace com condições de contorno de Dirichlet, com o domínio $0 < x < 4$,

$0 < y < 4$ e as condições de fronteira

$$T(0, y) = 0, \qquad T(4, y) = 100, \qquad \text{para } 0 < y < 4,$$

$$T(x, 0) = 0, \qquad T(x, 4) = 0, \qquad \text{para } 0 < x < 4.$$

idêntico ao problema do Exemplo 8.2.1.

```
# Parâmetros da malha
nx, ny = 100, 100   # pontos internos
Lx, Ly = 4.0, 4.0 # domínio [0,Lx]x[0,Ly]
h = Lx/(nx+1)      # passo (hx=hy=h)
N = nx*ny        # total de incógnitas
```

```
# Função para mapear (i,j) no índice da equação
def idx(i, j):
    return i+j*nx
```

```
# Aplica as condições de contorno
def contorno(i, j):
    if i==nx-1:
        return 100.0
    else:
        return 0.0
```

```
# Monta o sistema
A = np.zeros((N, N))
b = np.zeros(N)

# construindo o sistema
for j in range(ny):
    for i in range(nx):
        k = idx(i, j)

        # coeficiente central
        A[k, k] = -4.0/(h**2)

        # vizinho esquerdo
        if i > 0:
            A[k, idx(i-1, j)] = 1.0/(h**2)
        else:
            b[k] -= contorno(i, j)/(h**2)

        # vizinho direito
        if i < nx-1:
            A[k, idx(i+1, j)] = 1.0/(h**2)
```

(continua na próxima página)

(continuação da página anterior)

```
        else:
            b[k] -= contorno(i, j) / (h**2)

        # vizinho inferior
        if j > 0:
            A[k, idx(i, j-1)] = 1.0/(h**2)
        else:
            b[k] -= contorno(i, j) / (h**2)

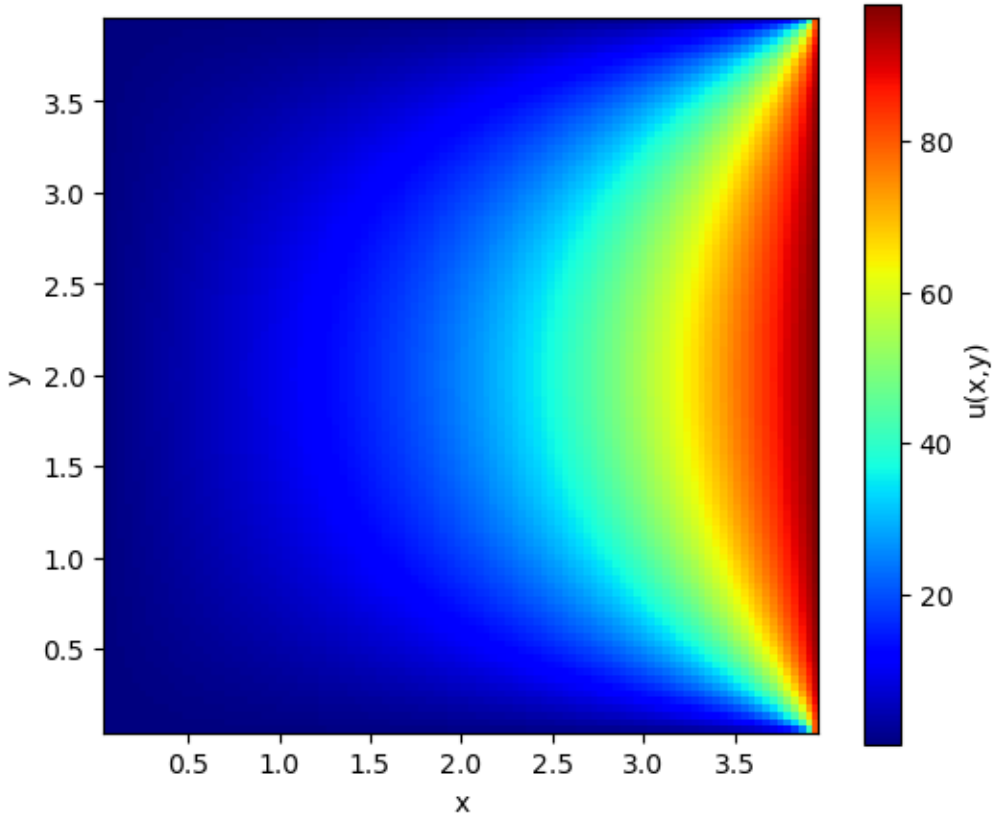
        # vizinho superior
        if j < ny-1:
            A[k, idx(i, j+1)] = 1.0/(h**2)
        else:
            b[k] -= contorno(i, j) / (h**2)
```

```
A
```

```
array([[-2550.25  ,   637.5625,     0.    , ...,     0.    ,     0.    ,
            0.    ],
       [  637.5625, -2550.25  ,   637.5625, ...,     0.    ,     0.    ,
            0.    ],
       [    0.    ,   637.5625, -2550.25  , ...,     0.    ,     0.    ,
            0.    ],
       ...,
       [    0.    ,     0.    ,     0.    , ..., -2550.25  ,   637.5625,
            0.    ],
       [    0.    ,     0.    ,     0.    , ...,   637.5625, -2550.25  ,
          637.5625],
       [    0.    ,     0.    ,     0.    , ...,     0.    ,   637.5625,
        -2550.25  ]])
```

```
# Resolve o sistema linear
u = np.linalg.solve(A, b).reshape(ny,nx)

# Plota o resultado
plt.figure(figsize=(6,5))
plt.imshow(u, extent=[h,Lx-h,h,Ly-h], origin="lower", cmap="jet")
plt.colorbar(label="u(x,y)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```




## Exercícios

**Exercicio 1:** Fonte: Burden(2016) Determine a distribuição de calor estacionária em uma placa de metal fina com dimensões $0,5$ m por $0,5$ m usando $n = m = 4$. Dois lados adjacentes são mantidos a $0°$C e o calor nos outros lados aumenta linearmente de $0°$C em um canto a $100°$C onde os lados se encontram. Coloque os lados com condições de contorno nulas ao longo dos eixos $x$ e $y$.
*Solução:* Nesse caso o problema será expresso como

$$\frac{\partial^2 u}{\partial x^2}(x,y) + \frac{\partial^2 u}{\partial y^2}(x,y) = 0,$$

para $(x,y)$ no conjunto $R = \{(x,y) \mid 0 < x < 0,5, 0 < y < 0,5\}$, com as condições de contorno

$$u(0,y) = 0, u(x,0) = 0, u(x,0,5) = 200x \text{ e } u(0,5,y) = 200y.$$

Se $n = m = 4$ a equação diferença é

$$u_{i-1,j} + u_{i+1,j} - 4u_{i,j} + u_{i,j-1} + u_{i,j+1} = 0$$

para cada $i = 1,2,3$ e $j = 1,2,3$.

**Exercicio 2:** (Burden,2016) Determine uma solução aproximada da equação diferencial parcial elíptica

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 4, \quad 0 < x < 1, \quad 0 < y < 2$$
$$u(x,0) = x^2, \quad u(x,2) = (x-2)^2, \quad 0 \le x \le 1$$
$$u(0,y) = y^2, \quad u(1,y) = (y-1)^2, \quad 0 \le y \le 2$$

Use $h = k = \frac{1}{2}$ e compare os resultados com a solução real $u(x,y) = (x-y)^2$.

---

**Exercicio 3:** (Burden,2016) Determine uma solução aproximada da equação diferencial parcial elíptica

$$\begin{aligned} &\tfrac{\partial^2 u}{\partial x^2} + \tfrac{\partial^2 u}{\partial y^2} = 0, && 1 < x < 2, \quad 0 < y < 1; \\ &u(x,0) = 2\ln x, && u(x,1) = \ln\left(x^2+1\right), \quad 1 \le x \le 2; \\ &u(1,y) = \ln\left(y^2+1\right), && u(2,y) = \ln\left(y^2+4\right), \quad 0 \le y \le 1. \end{aligned}$$

Use $h = k = \frac{1}{3}$ e compare os resultados com a solução real $u(x,y) = \ln\left(x^2+y^2\right)$.

---

**Exercício 4:** Classifique cada uma das EDPs lineares de segunda ordem a seguir como elíptica, parabólica ou hiperbólica, calculando $B^2 - 4AC$ a partir da forma canônica $Au_{xx} + Bu_{xy} + Cu_{yy} + ... = G$.
(a) $u_{xx} + 4u_{xy} + 4u_{yy} = 0$
(b) $3u_{xx} - 2u_{xy} + u_{yy} + u_x = 0$
(c) $u_{xx} - u_{yy} = 0$
(d) $u_{xx} + u_{yy} + 2u = 0$
(e) $u_{xy} = 0$
Para cada item, identifique também a que tipo de fenômeno físico (difusivo, oscilatório ou estacionário) a equação estaria associada.

---

**Exercício 5:** Considere o domínio retangular $D = \{(x,y) \mid 0 \le x \le 6,\ 0 \le y \le 4\}$, discretizado com $\Delta x = \Delta y = 0,5$.
(a) Determine o número de subintervalos $N$ na direção $x$ e $M$ na direção $y$.
(b) Determine o número total de nós da malha, incluindo os nós de fronteira.
(c) Determine o número de nós internos (onde a solução é desconhecida e precisa ser calculada).
(d) Usando `matplotlib`, gere um gráfico de dispersão (`scatter`) mostrando os nós da malha, destacando com cores diferentes os nós de fronteira e os nós internos.

---

**Exercício 6:** A aproximação por diferenças centrais para $u_{xx}$ apresentada nesta seção,

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x_i} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2},$$

pode ser deduzida somando as expansões em série de Taylor de $u(x_i + \Delta x)$ e $u(x_i - \Delta x)$ em torno de $x_i$.
(a) Escreva as duas expansões de Taylor até o termo de quarta ordem.
(b) Some as duas expressões e isole $u''(x_i)$, mostrando que se obtém exatamente a fórmula acima.
(c) Mostre que o erro de truncamento dessa aproximação é $O(\Delta x^2)$, identificando o termo dominante do erro.

**Exercício 7:** Repita o procedimento do **Exemplo 8.2.2** (equação de diferenças de Laplace, malha $3 \times 3$ nós internos) para uma placa quadrada com as seguintes temperaturas constantes nas bordas: esquerda $= 100\,°C$, direita $= 50\,°C$, inferior $= 0\,°C$ e superior $= 200\,°C$.
(a) Escreva o sistema de 9 equações lineares para os nós internos $T_{i,j}$, seguindo o mesmo padrão do sistema mostrado no texto.
(b) Resolva o sistema com `np.linalg.solve` e organize o resultado na forma de uma matriz $3 \times 3$.
(c) Plote a distribuição de temperatura com `pcolor` ou `imshow`, como feito no exemplo.

## 8.3 EDPs Elípticas

Equações elípticas descrevem situações de **equilíbrio**: a distribuição de temperatura em uma placa depois que o regime transiente terminou, o potencial elétrico em uma região sem cargas livres ou a deflexão de uma membrana sob carga estática. Nesses problemas não há variável tempo, e o valor da solução em cada ponto depende do que é imposto em toda a fronteira do domínio. Na Seção 8.2 vimos que uma EDP linear de segunda ordem é elíptica quando $B^2 - 4AC < 0$; o exemplo mais importante é o operador laplaciano.

Nesta seção vamos tratar a **equação de Poisson** em um retângulo $\Omega = [a, b] \times [c, d]$,

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), \qquad (x, y) \in \Omega, \tag{1}$$

com o valor de $u$ prescrito em toda a fronteira $\partial\Omega$ (condição de Dirichlet):

$$u(x, y) = g(x, y), \qquad (x, y) \in \partial\Omega. \tag{2}$$

Quando $f \equiv 0$, a Eq. (1) é a **equação de Laplace**. Na condução de calor em regime permanente, por exemplo, $u$ é a temperatura e $f = -q/k$, em que $q$ é a taxa de geração interna de calor (em W/m$^3$) e $k$ é a condutividade térmica do material.

### A malha e o estêncil de cinco pontos

Dividimos o intervalo $[a, b]$ em $n_x$ subintervalos de comprimento $h_x = (b-a)/n_x$ e o intervalo $[c, d]$ em $n_y$ subintervalos de comprimento $h_y = (d - c)/n_y$, obtendo os nós

$$x_i = a + i\,h_x, \quad i = 0, 1, \ldots, n_x, \qquad\qquad y_j = c + j\,h_y, \quad j = 0, 1, \ldots, n_y.$$

Denotamos por $u_{i,j}$ a aproximação de $u(x_i, y_j)$ e escrevemos $f_{i,j} = f(x_i, y_j)$. Os nós com $i \in \{0, n_x\}$ ou $j \in \{0, n_y\}$ estão na fronteira, onde $u$ é conhecido pela Eq. (2). Os $(n_x - 1)(n_y - 1)$ nós restantes são as incógnitas do problema.

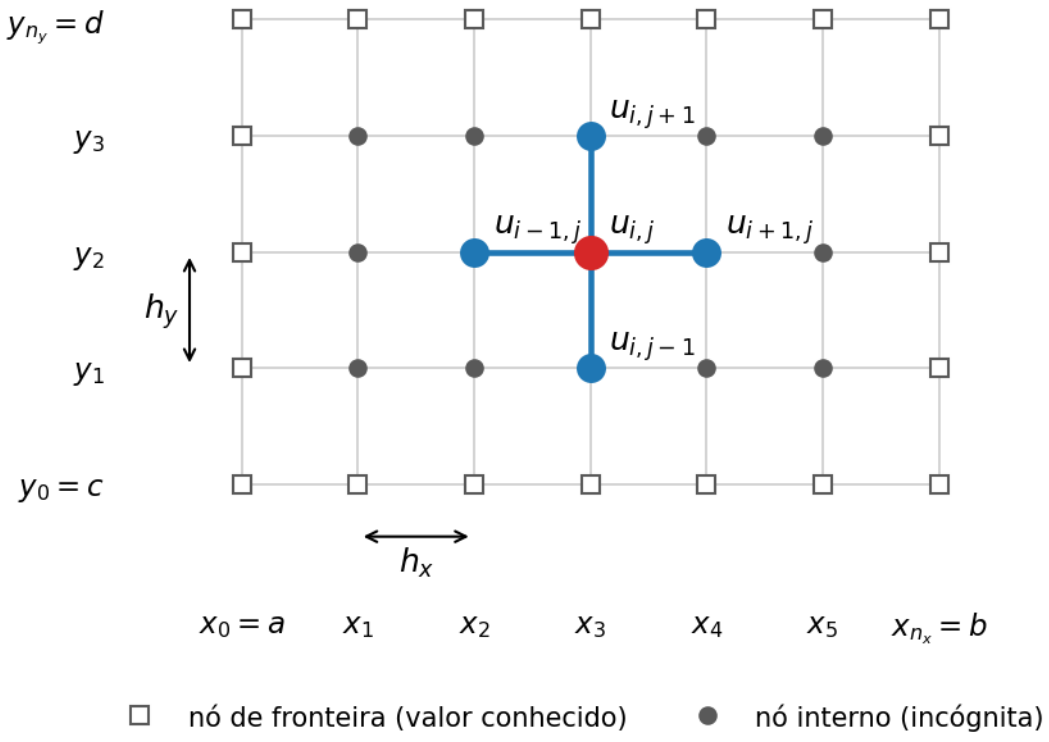



Figura 8.3.1 - Malha retangular e estêncil de cinco pontos em torno do nó $(x_i, y_j)$.

Em cada nó interno, aproximamos as derivadas segundas por diferenças centrais. Somando as expansões de Taylor

$$u(x_i \pm h_x, y_j) = u \pm h_x \frac{\partial u}{\partial x} + \frac{h_x^2}{2}\frac{\partial^2 u}{\partial x^2} \pm \frac{h_x^3}{6}\frac{\partial^3 u}{\partial x^3} + \frac{h_x^4}{24}\frac{\partial^4 u}{\partial x^4} + \cdots,$$

em que $u$ e suas derivadas são calculadas em $(x_i, y_j)$, os termos de ordem ímpar se cancelam e obtemos

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) = \frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j)}{h_x^2} - \frac{h_x^2}{12}\frac{\partial^4 u}{\partial x^4}(\xi, y_j)$$

para algum $\xi \in (x_{i-1}, x_{i+1})$. O mesmo vale na direção $y$, com $h_y$ no lugar de $h_x$. Substituindo essas aproximações na Eq. (1) e desprezando os termos de erro, chegamos às equações de diferenças finitas

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} = f_{i,j}, \qquad i = 1, \ldots, n_x - 1, \quad j = 1, \ldots, n_y - 1. \tag{3}$$

Cada equação envolve apenas o nó $(x_i, y_j)$ e seus quatro vizinhos, que formam o **estêncil de cinco pontos** da Figura 8.3.1. Como os termos desprezados são proporcionais a $h_x^2$ e $h_y^2$, esperamos que o erro da solução numérica diminua como $h^2$: ao dividir o espaçamento por 2, o erro deve cair aproximadamente por 4.

Quando $h_x = h_y = h$, a Eq. (3) assume a forma

$$u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = h^2 f_{i,j}.$$

Para a equação de Laplace ($f = 0$), isso diz que cada $u_{i,j}$ é a média dos valores nos quatro vizinhos. É a versão discreta

da propriedade do valor médio das funções harmônicas, e ajuda a entender por que a solução da equação de Laplace é sempre “suave” e não tem máximos nem mínimos no interior do domínio.

## Montagem do sistema linear

As Eqs. (3) formam um sistema linear $A\mathbf{u} = \mathbf{b}$ com uma incógnita para cada nó interno. Para escrevê-lo, numeramos os nós internos percorrendo primeiro a direção $y$: o nó $(i, j)$ recebe o índice $k = (i-1)(n_y - 1) + (j-1)$. Com essa ordenação, a matriz pode ser escrita de forma compacta com o **produto de Kronecker** $\otimes$:

$$A = \frac{1}{h_x^2}\, T_{n_x-1} \otimes I_{n_y-1} + \frac{1}{h_y^2}\, I_{n_x-1} \otimes T_{n_y-1}, \tag{4}$$

em que $I_m$ é a matriz identidade de ordem $m$ e $T_m$ é a matriz tridiagonal de ordem $m$ com $-2$ na diagonal principal e $1$ nas diagonais vizinhas, que representa a segunda diferença em uma dimensão. Lembre que $P \otimes Q$ é a matriz em blocos cujo bloco $(r, s)$ é $p_{rs}\, Q$. O primeiro termo da Eq. (4) acopla cada nó a seus vizinhos na direção $x$, e o segundo, aos vizinhos na direção $y$.

O vetor $\mathbf{b}$ contém os valores $f_{i,j}$, corrigidos nos nós vizinhos à fronteira: quando um dos quatro vizinhos de $(x_i, y_j)$ está na fronteira, seu valor é conhecido e passa para o lado direito da equação. Por exemplo, para $i = 1$, o termo $u_{0,j}/h_x^2$ é subtraído de $f_{1,j}$.

A matriz $A$ tem ordem $(n_x - 1)(n_y - 1)$, mas no máximo cinco elementos não nulos por linha. Em uma malha com $n_x = n_y = 100$, são cerca de $10^4$ incógnitas e $5 \times 10^4$ elementos não nulos, em uma matriz com quase $10^8$ posições. Por isso, armazenamos $A$ no formato **esparso** do módulo `scipy.sparse`, que guarda apenas os elementos não nulos, e resolvemos o sistema com `scipy.sparse.linalg.spsolve`.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla
```

Para visualizar a estrutura da matriz, vamos montá-la para uma malha pequena, com $n_x = 5$, $n_y = 6$ e $h_x = h_y = 1$, e marcar os elementos não nulos com a função `plt.spy`.

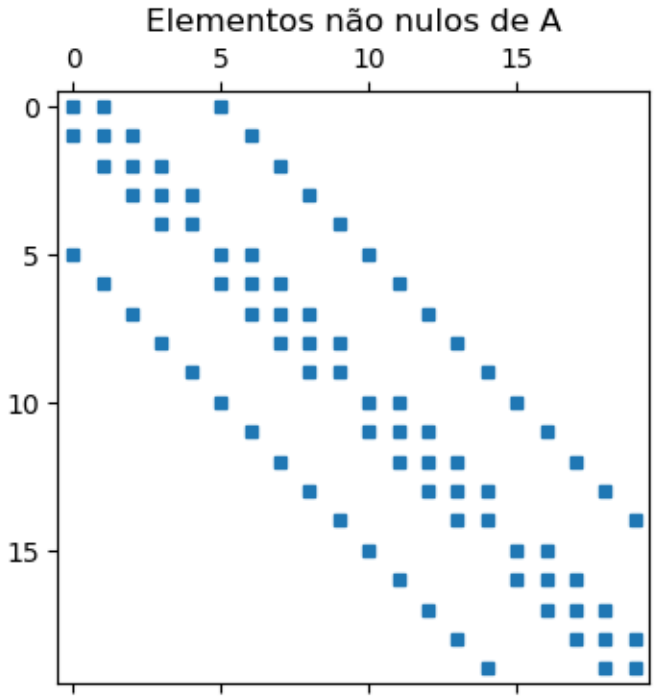
```
def segunda_diferenca(n):
    """Matriz tridiagonal de ordem n com (1, -2, 1) nas diagonais."""
    return sp.diags([1, -2, 1], [-1, 0, 1], shape=(n, n))

nx, ny = 5, 6
A = (sp.kron(segunda_diferenca(nx - 1), sp.identity(ny - 1))
     + sp.kron(sp.identity(nx - 1), segunda_diferenca(ny - 1)))

print(f"ordem da matriz: {A.shape[0]}, elementos não nulos: {A.nnz}")

fig, ax = plt.subplots(figsize=(4, 4))
ax.spy(A, markersize=5)
ax.set_title('Elementos não nulos de A')
plt.show()
```

```
ordem da matriz: 20, elementos não nulos: 82
```




A matriz é **tridiagonal por blocos**: os blocos da diagonal, de ordem $n_y - 1 = 5$, são tridiagonais e ligam cada nó aos vizinhos na direção $y$; os blocos imediatamente acima e abaixo são múltiplos da identidade e ligam os vizinhos na direção $x$. Além disso, $A$ é simétrica e $-A$ é definida positiva, o que garante que o sistema tem solução única.

A rotina a seguir reúne todos esses passos. As funções `f` e `g` devem aceitar arrays do NumPy, e `g` é avaliada apenas nos nós da fronteira. A solução é devolvida em uma matriz `U` em que `U[i, j]` aproxima $u(x_i, y_j)$.

```python
def poisson_dirichlet(f, g, a, b, c, d, nx, ny):
    """
    Resolve u_xx + u_yy = f(x, y) no retângulo [a, b] x [c, d], com
    u = g(x, y) na fronteira, pelo método de diferenças finitas com o
    estêncil de cinco pontos.

    Retorna os vetores x e y dos nós e a matriz U, com U[i, j] ~ u(x[i], y[j]).
    """
    x = np.linspace(a, b, nx + 1)
    y = np.linspace(c, d, ny + 1)
    X, Y = np.meshgrid(x, y, indexing='ij')
    hx2 = (x[1] - x[0])**2
    hy2 = (y[1] - y[0])**2

    # valores prescritos na fronteira
    U = np.zeros((nx + 1, ny + 1))
    U[0, :] = g(x[0], y)
    U[-1, :] = g(x[-1], y)
```

(continua na próxima página)

(continuação da página anterior)

```
    U[:, 0] = g(x, y[0])
    U[:, -1] = g(x, y[-1])

    # matriz do sistema, Eq. (4)
    A = (sp.kron(segunda_diferenca(nx - 1), sp.identity(ny - 1)) / hx2
         + sp.kron(sp.identity(nx - 1), segunda_diferenca(ny - 1)) / hy2)

    # lado direito: f nos nós internos menos a contribuição da fronteira
    B = f(X[1:-1, 1:-1], Y[1:-1, 1:-1]) * np.ones((nx - 1, ny - 1))
    B[0, :] -= U[0, 1:-1] / hx2
    B[-1, :] -= U[-1, 1:-1] / hx2
    B[:, 0] -= U[1:-1, 0] / hy2
    B[:, -1] -= U[1:-1, -1] / hy2

    # resolve e devolve os valores aos nós internos
    U[1:-1, 1:-1] = spla.spsolve(A.tocsr(), B.ravel()).reshape(nx - 1, ny - 1)
    return x, y, U
```

**Exemplo 8.3.1:** Verificação com uma solução conhecida

Uma boa maneira de testar um programa como esse é escolher uma função $u$, calcular o lado direito $f = \nabla^2 u$ correspondente e verificar se a solução numérica reproduz a função escolhida. Tomemos

$$u(x, y) = \sin(\pi x)\sin(\pi y) + xy, \qquad 0 \le x \le 1, \quad 0 \le y \le 1.$$

Como $\nabla^2(xy) = 0$, temos $f(x, y) = -2\pi^2 \sin(\pi x)\sin(\pi y)$. Os valores na fronteira são os da própria $u$: $u = 0$ nas bordas $x = 0$ e $y = 0$, $u = y$ na borda $x = 1$ e $u = x$ na borda $y = 1$. Vamos resolver o problema em malhas cada vez mais finas e medir o erro máximo nos nós.

```
u_exata = lambda x, y: np.sin(np.pi*x)*np.sin(np.pi*y) + x*y
f = lambda x, y: -2*np.pi**2*np.sin(np.pi*x)*np.sin(np.pi*y)

malhas = [10, 20, 40, 80]
erros = []
print(f"{'n':>4} {'h':>8} {'erro máximo':>13} {'razão':>7}")
for n in malhas:
    x, y, U = poisson_dirichlet(f, u_exata, 0, 1, 0, 1, n, n)
    X, Y = np.meshgrid(x, y, indexing='ij')
    erros.append(np.max(np.abs(U - u_exata(X, Y))))
    razao = f"{erros[-2]/erros[-1]:7.2f}" if len(erros) > 1 else ""
    print(f"{n:4d} {1/n:8.4f} {erros[-1]:13.3e} {razao}")
```

```
   n        h   erro máximo   razão
  10   0.1000     8.265e-03
  20   0.0500     2.059e-03    4.01
  40   0.0250     5.142e-04    4.00
```

(continua na próxima página)

(continuação da página anterior)

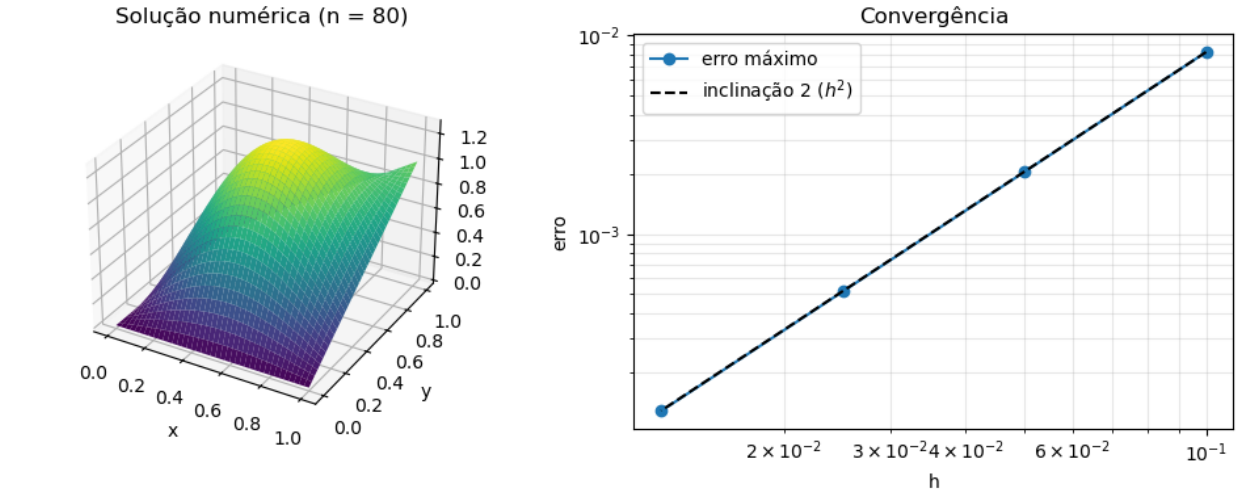
```
  80    0.0125      1.285e-04     4.00
```

```
h = 1 / np.array(malhas)

fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.plot_surface(X, Y, U, cmap='viridis')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_title(f'Solução numérica (n = {malhas[-1]})')

ax2 = fig.add_subplot(1, 2, 2)
ax2.loglog(h, erros, 'o-', label='erro máximo')
ax2.loglog(h, erros[0]*(h/h[0])**2, 'k--', label='inclinação 2 ($h^2$)')
ax2.set_xlabel('h')
ax2.set_ylabel('erro')
ax2.set_title('Convergência')
ax2.legend()
ax2.grid(True, which='both', alpha=0.3)

plt.tight_layout()
plt.show()
```




Cada vez que $h$ é dividido por 2, o erro é dividido por 4, e no gráfico em escala logarítmica os pontos ficam sobre uma reta de inclinação 2. Isso confirma que o método tem **ordem 2**, como previsto pela análise do erro de truncamento.

**Exemplo 8.3.2:** Placa com geração interna de calor

Uma placa de aço inoxidável de $20\,\text{cm} \times 10\,\text{cm}$, com condutividade térmica $k = 15\,\text{W/(m·K)}$, tem uma resistência elétrica embutida que gera calor de modo uniforme, à taxa $q = 5 \times 10^5\,\text{W/m}^3$. A borda inferior está em contato com uma superfície a $60\,°C$, e as outras três bordas são mantidas a $20\,°C$. Supondo que as faces da placa estejam isoladas, de

modo que o calor flua apenas no plano da placa, a temperatura em regime permanente satisfaz

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = -\frac{q}{k}, \qquad 0 \le x \le 0{,}2, \quad 0 \le y \le 0{,}1,$$

com $x$ e $y$ em metros. Qual é a temperatura máxima na placa e onde ela ocorre?

```
k = 15.0       # condutividade térmica, W/(m.K)
q = 5e5        # geração de calor, W/m^3

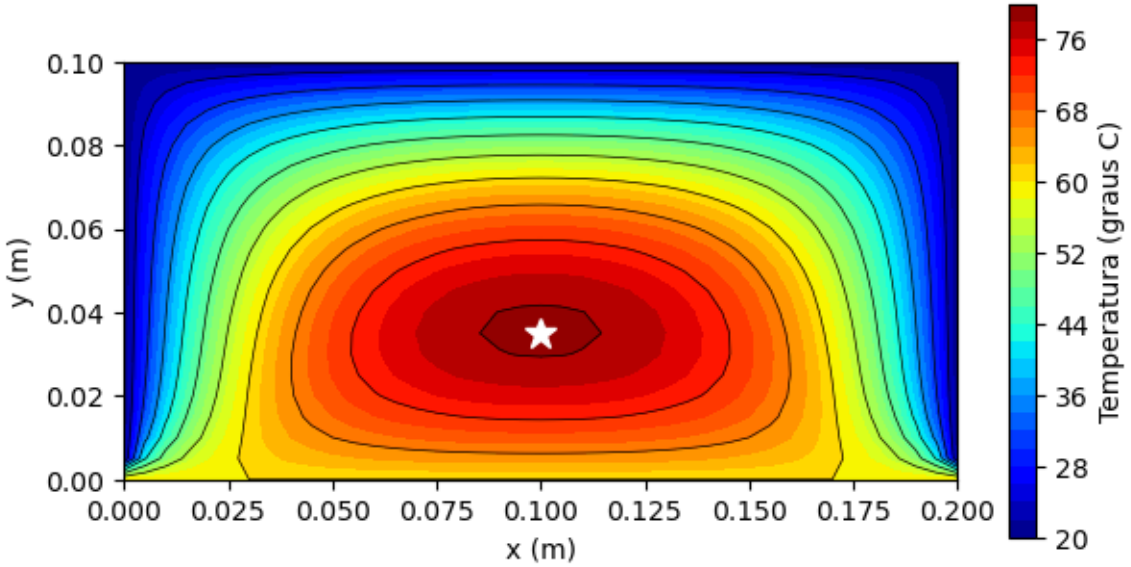
def T_contorno(x, y):
    # 60 graus C na borda inferior (y = 0) e 20 graus C nas demais
    return np.where(y == 0, 60.0, 20.0) * np.ones_like(x + y)

x, y, T = poisson_dirichlet(lambda x, y: -q/k, T_contorno, 0, 0.2, 0, 0.1, 40, 20)

i, j = np.unravel_index(np.argmax(T), T.shape)
print(f"Temperatura máxima: {T[i, j]:.1f} graus C")
print(f"Posição: x = {x[i]:.3f} m, y = {y[j]:.3f} m")

X, Y = np.meshgrid(x, y, indexing='ij')
fig, ax = plt.subplots(figsize=(7, 3.6))
cf = ax.contourf(X, Y, T, levels=30, cmap='jet')
ax.contour(X, Y, T, levels=10, colors='k', linewidths=0.5)
ax.plot(x[i], y[j], 'w*', markersize=12)
ax.set_aspect('equal')
ax.set_xlabel('x (m)')
ax.set_ylabel('y (m)')
plt.colorbar(cf, ax=ax, label='Temperatura (graus C)')
plt.show()
```

```
Temperatura máxima: 78.6 graus C
Posição: x = 0.100 m, y = 0.035 m
```




A temperatura máxima, de cerca de $78{,}6\,°C$, ocorre no eixo de simetria vertical da placa ($x = 0{,}1$ m), mas abaixo do centro geométrico, deslocada na direção da borda inferior, que é a mais quente. Sem a fonte de calor ($q = 0$), a temperatura ficaria entre $20\,°C$ e $60\,°C$ em toda a placa: uma solução da equação de Laplace atinge seus valores máximo e mínimo na fronteira do domínio (**princípio do máximo**), o que está de acordo com a propriedade do valor médio vista acima.

## Método iterativo de Gauss-Seidel

A rotina `poisson_dirichlet()` resolve o sistema (3) por um método direto. Como o número de incógnitas cresce com o quadrado do número de subdivisões, em malhas muito finas, e sobretudo em problemas tridimensionais, a eliminação torna-se cara em tempo e memória, mesmo aproveitando a esparsidade. Além disso, nem sempre é conveniente montar a matriz explicitamente.

Uma alternativa eficiente é o **método iterativo de Gauss-Seidel** (também chamado de método de Liebmann no contexto de EDPs). Para a equação de Laplace, a equação de diferenças finitas

$$T_{i-1,j} + T_{i+1,j} + T_{i,j-1} + T_{i,j+1} - 4T_{i,j} = 0$$

pode ser reescrita na forma de atualização

$$T_{i,j} = \frac{T_{i-1,j} + T_{i+1,j} + T_{i,j-1} + T_{i,j+1}}{4}$$

O algoritmo inicializa o campo com um valor estimado (por exemplo, a média das temperaturas de contorno), aplica essa fórmula repetidamente para cada nó interno — usando sempre os valores mais recentes disponíveis — e itera até que o erro relativo entre iterações consecutivas seja menor que uma tolerância $\varepsilon$:

$$\text{err} = \frac{\| T^{(k+1)} - T^{(k)} \|}{\| T^{(k+1)} \|} < \varepsilon$$

**Exemplo 8.3.3:** Placa aquecida — método de Gauss-Seidel

Considere uma placa quadrada (Chapra, 2016) com temperaturas constantes nas bordas:

| Borda | Temperatura |
|---|---|
| Inferior | 75 °C |
| Superior | 50 °C |
| Esquerda | 0 °C |
| Direita | 100 °C |

Resolver por Gauss-Seidel com tolerância $\varepsilon = 10^{-4}$ e malha $20 \times 20$.

```
import numpy as np
import matplotlib.pyplot as plt

ni, nj = 20, 20

T = np.zeros((ni, nj))
T[0,  :] = 75    # borda inferior
T[-1, :] = 50    # borda superior
T[:,  0] = 0     # borda esquerda
T[:, -1] = 100   # borda direita

T_ant = T.copy()
err_hist = []
err = 1.0

while err > 1e-4:
    for i in range(1, ni - 1):
        for j in range(1, nj - 1):
            T[i, j] = (T[i-1, j] + T[i+1, j] + T[i, j-1] + T[i, j+1]) / 4
    err = np.linalg.norm(T - T_ant) / np.linalg.norm(T)
    err_hist.append(err)
    T_ant = T.copy()

print(f"Convergência em {len(err_hist)} iterações")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))

axes[0].plot(err_hist, '.-', markersize=3)
axes[0].set_yscale('log')
axes[0].set_xlabel('Iteração')
axes[0].set_ylabel('Erro relativo')
axes[0].set_title('Convergência do Gauss-Seidel')
axes[0].grid(True)

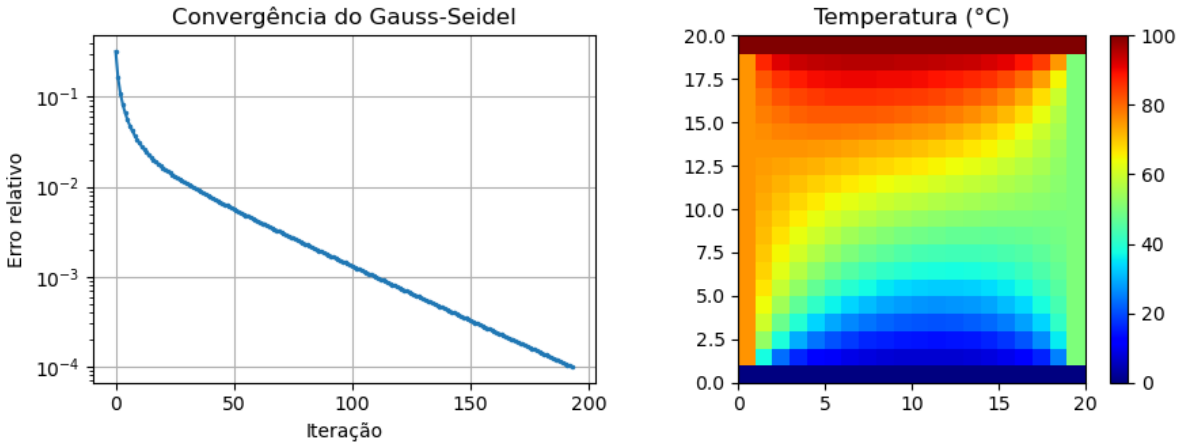
im = axes[1].pcolor(T.T, cmap='jet')
axes[1].set_aspect('equal')
axes[1].set_title('Temperatura (°C)')
```

(continua na próxima página)

(continuação da página anterior)

```
plt.colorbar(im, ax=axes[1])
plt.tight_layout()
plt.show()
```

```
Convergência em 194 iterações
```




## Sobrerelaxação (SOR)

O método de **sobrerelaxação sucessiva** (SOR, do inglês *Successive Over-Relaxation*) acelera a convergência do Gauss-Seidel aplicando um fator de peso $\omega \in (1, 2)$ após cada atualização:

$$T_{i,j}^{\text{novo}} = \omega\, \tilde{T}_{i,j} + (1 - \omega)\, T_{i,j}^{\text{ant}}$$

onde $\tilde{T}_{i,j}$ é o valor calculado pela fórmula de Gauss-Seidel. Para $\omega = 1$ recupera-se o método original; valores $\omega > 1$ induzem uma extrapolação na direção da atualização, acelerando a convergência. O valor ótimo de $\omega$ depende do problema, mas valores entre $1{,}2$ e $1{,}9$ são tipicamente eficazes.

```
T_sor = np.zeros((ni, nj))
T_sor[0,  :] = 75
T_sor[-1, :] = 50
T_sor[:,  0] = 0
T_sor[:, -1] = 100

omega = 1.5
T_ant = T_sor.copy()
err_sor = []
err = 1.0

while err > 1e-4:
    for i in range(1, ni - 1):
        for j in range(1, nj - 1):
            T_gs = (T_sor[i-1, j] + T_sor[i+1, j] + T_sor[i, j-1] + T_sor[i, j+1]) / 4
```

(continua na próxima página)

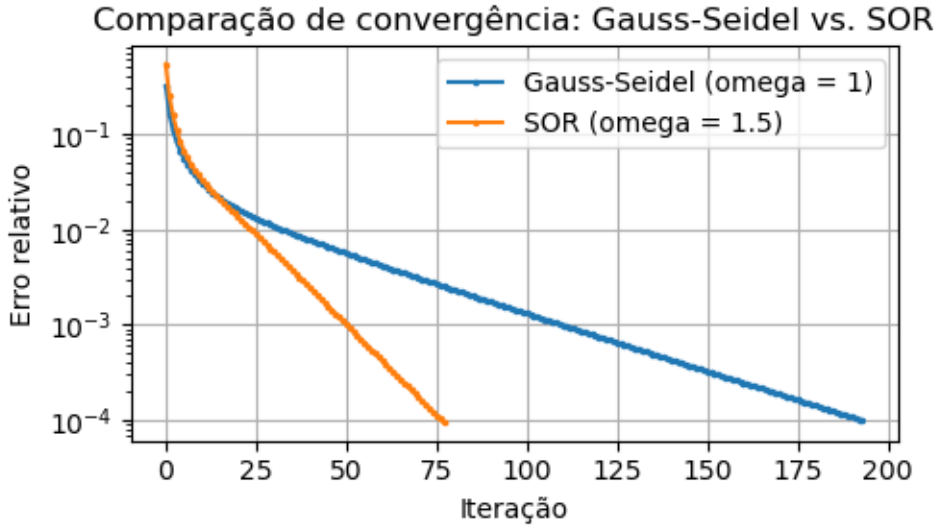
(continuação da página anterior)

```
            T_sor[i, j] = omega * T_gs + (1 - omega) * T_sor[i, j]
    err = np.linalg.norm(T_sor - T_ant) / np.linalg.norm(T_sor)
    err_sor.append(err)
    T_ant = T_sor.copy()

print(f"Gauss-Seidel  : {len(err_hist)} iterações")
print(f"SOR (omega={omega}): {len(err_sor)} iterações")

fig, ax = plt.subplots(figsize=(5, 3))
ax.plot(err_hist, '.-', markersize=3, label='Gauss-Seidel (omega = 1)')
ax.plot(err_sor,  '.-', markersize=3, label=f'SOR (omega = {omega})')
ax.set_yscale('log')
ax.set_xlabel('Iteração')
ax.set_ylabel('Erro relativo')
ax.set_title('Comparação de convergência: Gauss-Seidel vs. SOR')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```

```
Gauss-Seidel  : 194 iterações
SOR (omega=1.5): 78 iterações
```



## Gradiente e fluxo de calor

A distribuição de temperatura $T(x, y)$ determina o **vetor fluxo de calor** em cada ponto da placa. Pela lei de Fourier, o fluxo aponta na direção do gradiente negativo de temperatura:

$$\mathbf{q} = -k' \, \nabla T$$

A função `np.gradient` calcula numericamente as componentes $\partial T/\partial x$ e $\partial T/\partial y$ a partir da solução discreta, e `plt.quiver` exibe o campo vetorial sobreposto à distribuição de temperatura.

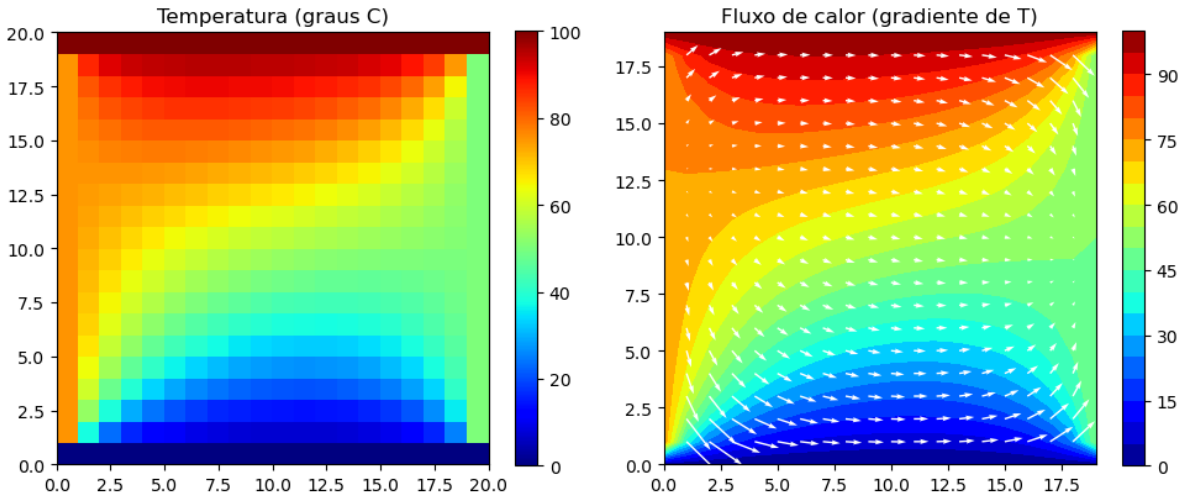
```
dTdy, dTdx = np.gradient(T[1:-1, 1:-1])
ii, jj = np.meshgrid(np.arange(1, nj - 1), np.arange(1, ni - 1))

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

im = axes[0].pcolor(T.T, cmap='jet')
axes[0].set_aspect('equal')
axes[0].set_title('Temperatura (graus C)')
plt.colorbar(im, ax=axes[0])

cf = axes[1].contourf(T.T, levels=20, cmap='jet')
axes[1].quiver(jj, ii, dTdx, dTdy, color='white', scale=200, width=0.004)
axes[1].set_aspect('equal')
axes[1].set_title('Fluxo de calor (gradiente de T)')
plt.colorbar(cf, ax=axes[1])

plt.tight_layout()
plt.show()
```



## Condição de contorno de Neumann

Nos exemplos anteriores, as bordas tinham temperatura prescrita — denominadas **condições de Dirichlet**. Em muitas situações práticas, porém, o que é dado é o **fluxo de calor** na borda, isto é, a derivada normal à superfície. Essas são chamadas **condições de Neumann**.

O caso mais frequente em engenharia é a **borda isolada**, na qual o fluxo é nulo:

$$\left.\frac{\partial T}{\partial n}\right|_{\text{borda}} = 0$$

**Estratégia do ponto fictício.** Para incorporar essa condição no esquema de diferenças finitas, introduz-se um **nó fictício** externo à malha, cujo valor é eliminado algebricamente usando a aproximação por diferenças centrais da derivada.

Para a borda inferior ($j = 0$), a condição de isolamento resulta em:

$$\frac{T_{i,1} - T_{i,-1}}{2\,\Delta y} = 0 \implies T_{i,-1} = T_{i,1}$$

Substituindo na equação do nó $(i, 0)$:

$$T_{i-1,\,0} + T_{i+1,\,0} + 2\,T_{i,\,1} - 4\,T_{i,\,0} = 0$$

O coeficiente **2** no vizinho superior (em vez de 1) é a única modificação em relação à equação interior padrão, refletindo a simetria imposta pelo isolamento.

**Fluxo de calor nas bordas**

A partir da solução numérica, o vetor fluxo de calor pode ser estimado por diferenças finitas centrais e pela lei de Fourier (Chapra e Canale, 2016):

$$q_x = -k' \frac{T_{i+1,j} - T_{i-1,j}}{2\,\Delta x}, \qquad q_y = -k' \frac{T_{i,j+1} - T_{i,j-1}}{2\,\Delta y}$$

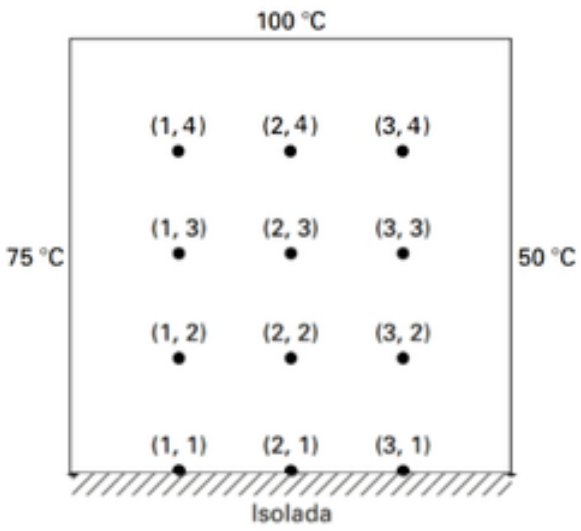
O módulo do fluxo resultante é $q_n = \sqrt{q_x^2 + q_y^2}$ e sua direção é $\theta = \arctan(q_y / q_x)$.

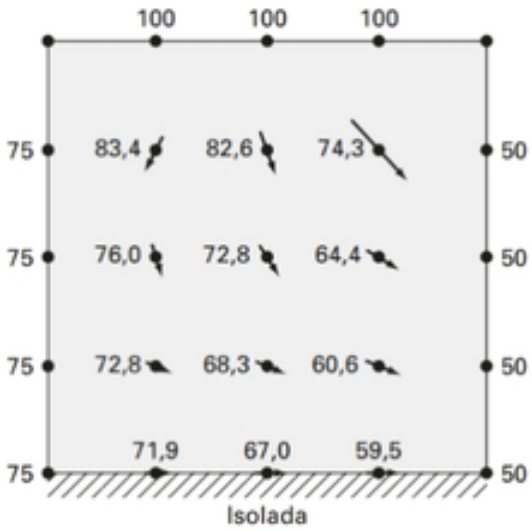
**Exemplo 8.3.4:** Placa aquecida com borda inferior isolada

(Fonte: Chapra e Canale, 2016) Placa de $4\,\text{cm} \times 4\,\text{cm}$ com as seguintes condições de contorno:

| Borda | Condição |
|---|---|
| Esquerda ($x = 0$) | $T = 75\,°C$ (Dirichlet) |
| Direita ($x = 4\,\text{cm}$) | $T = 50\,°C$ (Dirichlet) |
| Superior ($y = 4\,\text{cm}$) | $T = 100\,°C$ (Dirichlet) |
| Inferior ($y = 0$) | $\partial T/\partial y = 0$ (Neumann — isolada) |

Com $\Delta x = \Delta y = 1\,\text{cm}$, a borda inferior passa a ser uma linha de nós ativos (em vez de nós com valor prescrito). O sistema tem $3 \times 4 = 12$ incógnitas.




```
# Sistema 12x12 — nós da borda inferior incluídos (Neumann)
# Linhas: j = 0 (inferior, isolada) até j = 3 (interior)
# Colunas: i = 1, 2, 3 (interior em x)
A_neu = np.array([
    [ 4,-1, 0,-2, 0, 0, 0, 0, 0, 0, 0, 0],
    [-1, 4,-1, 0,-2, 0, 0, 0, 0, 0, 0, 0],
    [ 0,-1, 4, 0, 0,-2, 0, 0, 0, 0, 0, 0],
    [-1, 0, 0, 4,-1, 0,-1, 0, 0, 0, 0, 0],
    [ 0,-1, 0,-1, 4,-1, 0,-1, 0, 0, 0, 0],
    [ 0, 0,-1, 0,-1, 4, 0, 0,-1, 0, 0, 0],
    [ 0, 0, 0,-1, 0, 0, 4,-1, 0,-1, 0, 0],
    [ 0, 0, 0, 0,-1, 0,-1, 4,-1, 0,-1, 0],
    [ 0, 0, 0, 0, 0,-1, 0,-1, 4, 0, 0,-1],
    [ 0, 0, 0, 0, 0, 0,-1, 0, 0, 4,-1, 0],
    [ 0, 0, 0, 0, 0, 0, 0,-1, 0,-1, 4,-1],
    [ 0, 0, 0, 0, 0, 0, 0, 0,-1, 0,-1, 4]
], dtype=float)

b_neu = np.array([75, 0, 50, 75, 0, 50, 75, 0, 50, 175, 100, 150], dtype=float)

T_int = np.linalg.solve(A_neu, b_neu).reshape(4, 3)
print("Temperaturas nos nos internos e na borda inferior (linhas=y crescente,␣
 ↪colunas=x):")
print(np.flipud(np.round(T_int, 2)))
```

```
Temperaturas nos nos internos e na borda inferior (linhas=y crescente, colunas=x):
[[83.41 82.63 74.26]
 [76.02 72.84 64.42]
 [72.81 68.31 60.57]
 [71.91 67.01 59.54]]
```

```
x_mesh, y_mesh = np.meshgrid(np.linspace(0, 4, 6), np.linspace(0, 4, 6))

T_neu = np.zeros((5, 5))
T_neu[-1, :] = 100   # borda superior
T_neu[:,  0] = 75    # borda esquerda
T_neu[:, -1] = 50    # borda direita
T_neu[0:-1, 1:-1] = T_int   # nos internos + borda inferior

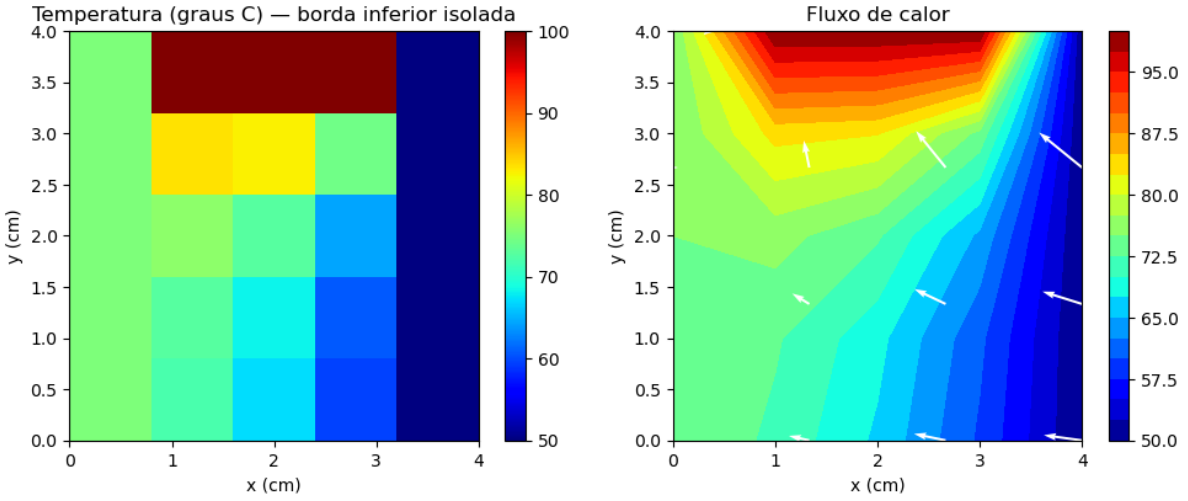
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

im = axes[0].pcolormesh(x_mesh, y_mesh, T_neu, cmap='jet', shading='flat')
axes[0].set_aspect('equal')
axes[0].set_xlabel('x (cm)')
axes[0].set_ylabel('y (cm)')
axes[0].set_title('Temperatura (graus C) — borda inferior isolada')
plt.colorbar(im, ax=axes[0])

dTdy_n, dTdx_n = np.gradient(T_neu[:-1, :-1])
ii_n, jj_n = np.meshgrid(np.linspace(0, 4, 4), np.linspace(0, 4, 4))
xn, yn = np.meshgrid(np.linspace(0,4,5),
                     np.linspace(0,4,5))

cf = axes[1].contourf(xn, yn, T_neu, levels=20, cmap='jet')
axes[1].quiver(ii_n, jj_n, dTdx_n, dTdy_n, color='white', scale=80, width=0.006)
axes[1].set_aspect('equal')
axes[1].set_xlabel('x (cm)')
axes[1].set_ylabel('y (cm)')
axes[1].set_title('Fluxo de calor')
plt.colorbar(cf, ax=axes[1])

plt.tight_layout()
plt.show()
```



## Exercícios

**1.** Use a rotina `poisson_dirichlet()` nos problemas a seguir.
(a) Resolva $\nabla^2 u = 6(x+y)$ em $[0,1]\times[0,1]$, com $u = x^3 + y^3$ na fronteira, para $n_x = n_y = 4$, 8 e 16, e calcule o erro máximo em relação à solução exata $u(x,y) = x^3 + y^3$. Por que o erro fica no nível do erro de arredondamento, mesmo em uma malha tão grosseira? (Sugestão: observe o termo de erro da aproximação das derivadas segundas.)
(b) Resolva a equação de Laplace em $[0,2]\times[0,1]$, com $u = 0$ nas bordas $x = 0$, $y = 0$ e $y = 1$, e $u(2,y) = \operatorname{sin}(\pi y)$. A solução exata é

$$u(x,y) = \frac{\operatorname{sinh}(\pi x)\,\operatorname{sin}(\pi y)}{\operatorname{sinh}(2\pi)}.$$

Use $n_x = 2n_y$ com $n_y = 10$, 20 e 40, e verifique numericamente a ordem de convergência do método.
(c) A deflexão de uma membrana quadrada de lado 1, presa nas bordas e submetida a uma pressão uniforme, é proporcional à solução de $\nabla^2 u = -1$ com $u = 0$ na fronteira. Calcule $u(0{,}5;\ 0{,}5)$ com $n_x = n_y = 20$, 40 e 80 e compare com o valor de referência $0{,}073671$. Faça um gráfico de superfície da solução.

---

**2.** Retome a placa aquecida do **Exemplo 8.3.3** (bordas a $75\,°C$, $50\,°C$, $0\,°C$ e $100\,°C$, malha $20\times20$).
(a) Resolva o problema com o método de SOR para $\omega = 1{,}0$; $1{,}2$; $1{,}5$; $1{,}7$ e $1{,}9$, contando o número de iterações até a convergência ($\varepsilon = 10^{-4}$) em cada caso.
(b) Plote o número de iterações em função de $\omega$.
(c) Com base no gráfico, estime o valor de $\omega$ que minimiza o número de iterações para esse problema.

---

**3.** Considere a mesma placa de $4\,\text{cm}\times4\,\text{cm}$ do **Exemplo 8.3.4**, com $\Delta x = \Delta y = 1\,\text{cm}$, mas agora com a borda **direita** isolada (condição de Neumann, fluxo nulo) em vez da borda inferior. As demais bordas permanecem com condição de Dirichlet: esquerda $= 75\,°C$, superior $= 100\,°C$, inferior $= 50\,°C$.
(a) Adapte a estratégia do ponto fictício (apresentada para a borda inferior) para eliminar o nó fictício na borda direita, obtendo a equação modificada para os nós dessa borda.
(b) Monte o sistema linear $12\times12$ correspondente e resolva com `np.linalg.solve`.
(c) Plote a distribuição de temperatura resultante.

---

**4.** Utilize a solução obtida no **Exemplo 8.3.2** (placa com geração interna de calor).
(a) Calcule o fluxo de calor $\mathbf{q} = -k\nabla T$ a partir da solução discreta, usando `np.gradient` com os espaçamentos da malha.
(b) Plote a distribuição de temperatura e sobreponha o campo vetorial de $\mathbf{q}$ com `plt.quiver`, como foi feito no Exemplo 8.3.3.
(c) Por qual das bordas sai mais calor? Justifique com base no gráfico.

---

**5.** Compare o método direto com o método de Gauss-Seidel na placa do **Exemplo 8.3.3** (bordas a $75\,°C$, $50\,°C$, $0\,°C$ e $100\,°C$).
(a) Escreva uma função `fronteira(x, y)` que reproduza essas condições de contorno e resolva o problema com `poisson_dirichlet()` para $n_x = n_y = 10$, 20, 40 e 80.
(b) Resolva os mesmos problemas com o método de Gauss-Seidel do Exemplo 8.3.3 e meça o tempo gasto por cada método com `time.perf_counter()`.

(c) Como cresce o número de iterações do Gauss-Seidel com o número de subdivisões? Qual dos métodos é mais vantajoso nas malhas finas?

## 8.4 EDPs Parabólicas

*Esta seção é uma tradução adaptada da Seção 9.2 (Parabolic PDE) de YANG, W. Y.; CAO, W.; CHUNG, T.-S.; MORRIS, J.* ***Applied Numerical Methods Using MATLAB****. Hoboken: John Wiley & Sons, 2005. O desenvolvimento do texto, os exemplos e as rotinas computacionais seguem essa obra, assim como parte dos exercícios, com tradução para o português e implementação das rotinas em Python.*

Consideremos uma EDP geral de segunda ordem em duas variáveis independentes $x$ e $y$, que é escrita como

$$A(x,y)\frac{\partial^2 u}{\partial x^2} + B(x,y)\frac{\partial^2 u}{\partial x \partial y} + C(x,y)\frac{\partial^2 u}{\partial y^2} = f\left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right)$$
$$\text{para } x_0 \leq x \leq x_f, y_0 \leq y \leq y_f$$

com as condições de contorno dadas por

$$u\left(x, y_0\right) = b_{y0}(x), \quad u\left(x, y_f\right) = b_{yf}(x),$$
$$u\left(x_0, y\right) = b_{x0}(y), \quad \text{e} \quad u\left(x_f, y\right) = b_{xf}(y)$$

As EDP elípticas são aquelas em que $B^2 - 4AC = 0$ e estão associadas estados de difusão.

Um exemplo de uma EDP (equação diferencial parcial) parabólica é a equação do calor unidimensional, que descreve a distribuição de temperatura $u(x,t)$ ( $x$ é a posição, $t$ é o tempo), dada por:

$$A\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t} \quad \text{para } 0 \leq x \leq x_f, \quad 0 \leq t \leq T \tag{1}$$

Para que essa equação possa ser resolvida, devem ser fornecidas as condições de contorno $u(0,t) = b_0(t)$ e $u\left(x_f, t\right) = b_{xf}(t)$ , bem como a condição inicial $u(x,0) = i_0(x)$

A constante $A$ (ou $\alpha^2$) é a **difusividade térmica** do material, relacionada à condutividade térmica $k'$, à densidade $\rho$ e ao calor específico $C$ por

$$\alpha^2 = \frac{k'}{\rho\, C}$$

A tabela abaixo apresenta valores típicos para materiais de uso comum em engenharia:

| Material | $\alpha^2$ (cm²/s) |
|---|---|
| Cobre | 18,5 |
| Alumínio | 0,8418 |
| Ferro | 0,23 |
| Aço carbono | 0,1172 |
| Aço inox | 0,03352 |
| Madeira pinus | 0,00082 |

A difusividade do cobre é cerca de **22 000 vezes** maior que a da madeira, o que explica por que metais atingem equilíbrio térmico muito mais rapidamente que materiais isolantes.

## 1. O Método Explícito de Euler para Frente

Para aplicar o método das diferenças finitas, dividimos o domínio espacial $\left[0, x_f\right]$ em $M$ partes, cada uma com comprimento $\Delta x = x_f/M$, e dividimos o domínio temporal $[0, T]$ em $N$ intervalos cada um com duração $\Delta t = T/N$.

Em seguida, substituímos a derivada parcial de segunda ordem no lado esquerdo e a derivada parcial de primeira ordem no lado direito da equação (1) pela aproximação de diferenças centrais e pela aproximação de diferenças progressivas, respectivamente, obtendo:

$$A\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} = \frac{u_i^{k+1} - u_i^k}{\Delta t} \tag{2}$$

Isso pode ser reescrito no seguinte algoritmo, chamado de método explícito de Euler para frente, que deve ser resolvido iterativamente:

$$u_i^{k+1} = r\left(u_{i+1}^k + u_{i-1}^k\right) + (1-2r)u_i^k \quad \text{com } r = A\frac{\Delta t}{\Delta x^2} \tag{3}$$

para $i = 1, 2, \ldots, M-1$.

Para encontrar a condição de estabilidade desse algoritmo, substituímos uma solução tentativa

$$u_i^k = \lambda^k e^{ji\pi/P} \quad (P \text{ é qualquer inteiro não nulo }) \tag{4}$$

na Eq. (3), obtendo:

$$\lambda = r\left(e^{j\pi/P} + e^{-j\pi/P}\right) + (1-2r) = 1 - 2r(1-\cos(\pi/P)) \tag{5}$$

Como devemos ter $|\lambda| \leq 1$ para evitar divergência, a condição de estabilidade resulta em:

$$r = A\frac{\Delta t}{\Delta x^2} \leq \frac{1}{2} \tag{6}$$

Isso implica que, à medida que diminuímos o intervalo espacial $\Delta x$ para obter maior precisão, também precisamos diminuir o passo de tempo $\Delta t$, ao custo de mais computações, para não perder a estabilidade.

A rotina em Python `heat_exp( )` foi desenvolvida para implementar esse algoritmo.

### Forma matricial do método explícito

O algoritmo explícito admite uma formulação matricial compacta. Definindo o vetor solução no instante $t_n$:

$$\mathbf{u}^n = \begin{bmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{M-1}^n \\ u_M^n \end{bmatrix}$$

a equação de propagação $u_i^{n+1} = r\left(u_{i+1}^n + u_{i-1}^n\right) + (1-2r)\,u_i^n$ torna-se uma única multiplicação de matrizes:

$$\mathbf{u}^{n+1} = \mathbf{B}\,\mathbf{u}^n, \qquad n = 0, 1, 2, \ldots$$

A **matriz de propagação B** é tridiagonal:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & & & 0 \\ r & 1-2r & r & & \\ & \ddots & \ddots & \ddots & \\ & & r & 1-2r & r \\ 0 & & & 0 & 1 \end{bmatrix}$$

As linhas da fronteira preservam os valores das condições de Dirichlet. Esta perspectiva revela que o critério de estabilidade $r \leq 1/2$ é equivalente a exigir que todos os autovalores de $\mathbf{B}$ tenham módulo $\leq 1$.

**Exemplo 8.4.1:** Equação de difusão com solução exata

$$\frac{\partial u}{\partial t} = D\,\frac{\partial^2 u}{\partial x^2}, \quad u(0,t) = u(L,t) = 0, \quad u(x,0) = \sin(\pi x/L)$$

Solução exata: $u(x,t) = e^{-\pi^2 D t/L^2}\sin(\pi x/L)$

Vamos construir **B** explicitamente e comparar a propagação numérica com a solução de referência.

```
import numpy as np
import matplotlib.pyplot as plt

L = 1.0
D = 0.1
hx = 0.05
ht = 0.0125
r = D * ht / hx**2
print(f"r = {r:.4f}  (criterio de estabilidade: r <= 0.5)")

x = np.linspace(0, L, int(L/hx) + 1)
u0 = np.sin(np.pi * x / L)

# Monta a matriz de propagação B
N = len(x)
B = np.eye(N)
for i in range(1, N - 1):
    B[i, i-1] = r
    B[i, i]   = 1 - 2*r
    B[i, i+1] = r

# Propaga e armazena
n_steps = 480
sol = np.zeros((n_steps + 1, N))
sol[0] = u0
for n in range(n_steps):
    sol[n+1] = B @ sol[n]

# Solução exata
```

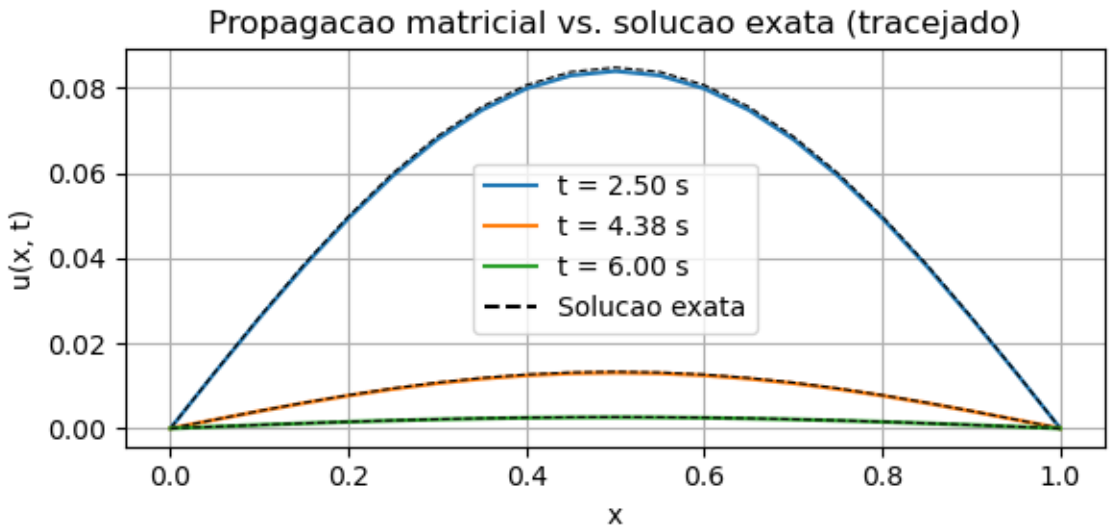
(continua na próxima página)

(continuação da página anterior)

```
sol_ex = lambda x, t: np.exp(-np.pi**2 * D * t / L**2) * np.sin(np.pi * x / L)

fig, ax = plt.subplots(figsize=(6, 3))
for n in [200, 350, 480]:
    t_n = n * ht
    ax.plot(x, sol[n], label=f't = {t_n:.2f} s')
    ax.plot(x, sol_ex(x, t_n), 'k--', lw=0.8)
ax.plot([], [], 'k--', label='Solucao exata')
ax.set_xlabel('x')
ax.set_ylabel('u(x, t)')
ax.set_title('Propagacao matricial vs. solucao exata (tracejado)')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```

```
r = 0.5000  (criterio de estabilidade: r <= 0.5)
```




**Experimento de instabilidade:** aumente `ht` para `0.013` ($r \approx 0{,}52 > 1/2$) e execute novamente para observar a divergência da solução.

```
import numpy as np

def heat_exp(a, xf, T, it0, bx0, bxf, M, N):
    """
    Resolve a * u_xx = u_t para 0 <= x <= xf, 0 <= t <= T

    it0(x): condição inicial
    bx0(t): condição de contorno em x = 0
    bxf(t): condição de contorno em x = xf
```

(continua na próxima página)

(continuação da página anterior)

```
    """

    dx = xf / M
    x = np.linspace(0, xf, M + 1)

    dt = T / N
    t = np.linspace(0, T, N + 1)

    # matriz solução (linhas: espaço, colunas: tempo)
    u = np.zeros((M + 1, N + 1))

    # condição inicial
    for i in range(M + 1):
        u[i, 0] = it0(x[i])

    # condições de contorno
    for n in range(N + 1):
        u[0, n] = bx0(t[n])
        u[M, n] = bxf(t[n])

    r = a * dt / dx**2
    r1 = 1 - 2 * r

    # esquema explícito de Euler
    for k in range(N):
        for i in range(1, M):
            u[i, k + 1] = r * (u[i + 1, k] + u[i - 1, k]) + r1 * u[i, k]

    return u, x, t
```

## 2. Método de Euler Implícito Regressivo

Nesta seção, consideramos outro algoritmo chamado método implícito de Euler para trás, que surge ao substituir a aproximação por diferenças regressivas (backward difference) para a derivada parcial de primeira ordem no lado direito da Eq. (1), resultando em:

$$A\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} = \frac{u_i^k - u_i^{k-1}}{\Delta t} \tag{7}$$

ou, equivalentemente,

$$-ru_{i-1}^k + (1+2r)u_i^k - ru_{i+1}^k = u_i^{k-1} \quad \text{com } r = A\frac{\Delta t}{\Delta x^2} \tag{8}$$

para $i = 1, 2, \ldots, M-1$.

Se os valores de $u_0^k$ e $u_M^k$ nas extremidades são dados por condições de contorno do tipo Dirichlet, então a equação

acima pode ser escrita como um sistema de equações lineares simultâneas:

$$\begin{bmatrix} 1+2r & -r & 0 & \cdots & 0 & 0 \\ -r & 1+2r & -r & \cdots & 0 & 0 \\ 0 & -r & 1+2r & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1+2r & -r \\ 0 & 0 & 0 & \cdots & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_1^k \\ u_2^k \\ u_3^k \\ \vdots \\ u_{M-2}^k \\ u_{M-1}^k \end{bmatrix} = \begin{bmatrix} u_1^{k-1} + ru_0^k \\ u_2^{k-1} \\ u_3^{k-1} \\ \vdots \\ u_{M-2}^{k-1} \\ u_{M-1}^{k-1} + ru_M^k \end{bmatrix} \tag{9}$$

A matriz dos coeficientes é tridiagonal e, portanto, o sistema linear pode ser resolvido eficientemente por métodos especializados para matrizes tridiagonais, como o algoritmo de Thomas.

E quanto ao caso em que são fornecidos os valores de $\frac{\partial u}{\partial x}\big|_{x=0} = b_0'(t)$ em uma das extremidades?

Nesse caso, aproximamos essa condição de contorno do tipo Neumann por

$$\frac{u_1^k - u_{-1}^k}{2\Delta x} = b_0'(k), \tag{10}$$

e a combinamos com uma equação adicional associada à variável desconhecida $u_0^k$,

$$-ru_{-1}^k + (1+2r)u_0^k - ru_1^k = u_0^{k-1}, \tag{11}$$

para obter

$$(1+2r)u_0^k - 2ru_1^k = u_0^{k-1} - 2rb_0'(k)\Delta x. \tag{12}$$

Acrescentamos essa equação ao sistema dado pela Eq. (9), obtendo

$$\begin{bmatrix} 1+2r & -2r & 0 & 0 & \cdots & 0 & 0 \\ -r & 1+2r & -r & 0 & \cdots & 0 & 0 \\ 0 & -r & 1+2r & -r & \cdots & 0 & 0 \\ 0 & 0 & -r & 1+2r & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \cdots & 1+2r & -r \\ 0 & 0 & 0 & \cdots & \cdots & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_0^k \\ u_1^k \\ u_2^k \\ u_3^k \\ \vdots \\ u_{M-2}^k \\ u_{M-1}^k \end{bmatrix} = \begin{bmatrix} u_0^{k-1} - 2rb_0'(k)\Delta x \\ u_1^{k-1} \\ u_2^{k-1} \\ u_3^{k-1} \\ \vdots \\ u_{M-2}^{k-1} \\ u_{M-1}^{k-1} + ru_M^k \end{bmatrix}. \tag{13}$$

Observe que a condição de Neumann é incorporada eliminando-se o ponto fictício $u_{-1}^k$. Como consequência, a primeira linha da matriz tridiagonal é modificada: o coeficiente $-r$ que apareceria em frente a $u_1^k$ torna-se $-2r$, enquanto o termo independente passa a conter a contribuição da derivada prescrita na fronteira, $-2rb_0'(k)\Delta x$. Dessa forma, a condição de fluxo (ou gradiente) na extremidade é incorporada diretamente ao sistema linear implícito.

Equações como a Eq. (9) ou a Eq. (13) são particularmente interessantes porque podem ser resolvidas de forma muito eficiente, explorando sua estrutura tridiagonal, e têm estabilidade garantida devido à sua dominância diagonal. A estabilidade incondicional da Eq. (9) pode ser demonstrada substituindo a Eq. (4) na Eq. (8):

$$-re^{-j\pi/P} + (1+2r) - re^{j\pi/P} = \frac{1}{\lambda},$$

ou, equivalentemente,

$$\lambda = \frac{1}{1 + 2r(1 - \cos(\pi/P))}, \quad |\lambda| \leq 1. \tag{14}$$

Como $r \geq 0$ e $1 - \cos(\pi/P) \geq 0$, o denominador é sempre maior ou igual a 1 . Consequentemente,

$$0 < \lambda \leq 1,$$

o que implica que $|\lambda| \leq 1$ para qualquer escolha de $\Delta t$ e $\Delta x$. Portanto, o método é incondicionalmente estável, isto é, não exige nenhuma restrição entre o passo temporal e o espaçamento da malha para manter a estabilidade numérica.

A rotina a seguir, denominada `heat_imp( )`, implementa esse algoritmo para resolver a EDP (1) com condições de contorno usuais (do tipo Dirichlet), utilizando o sistema linear dado pela Eq. (9).

```
import numpy as np

def heat_imp(a, xf, T, it0, bx0, bxf, M, N):
    """
    Resolve a*u_xx = u_t para 0 <= x <= xf e 0 <= t <= T
    usando o método implícito de Euler.

    Parâmetros
    ----------
    a   : coeficiente de difusão
    xf  : comprimento do domínio espacial
    T   : tempo final
    it0 : função da condição inicial u(x,0)
    bx0 : função da condição de contorno em x=0
    bxf : função da condição de contorno em x=xf
    M   : número de subintervalos espaciais
    N   : número de subintervalos temporais

    Retorna
    -------
    u : matriz (M+1) x (N+1)
    x : vetor espacial
    t : vetor temporal
    """

    dx = xf / M
    x = np.linspace(0, xf, M + 1)

    dt = T / N
    t = np.linspace(0, T, N + 1)

    # solução
    u = np.zeros((M + 1, N + 1))

    # condição inicial
    for i in range(M + 1):
        u[i, 0] = it0(x[i])
```

(continua na próxima página)

(continuação da página anterior)

```
    # condições de contorno
    for n in range(N + 1):
        u[0, n] = bx0(t[n])
        u[M, n] = bxf(t[n])

    r = a * dt / dx**2
    r2 = 1 + 2 * r

    # matriz tridiagonal A
    A = np.zeros((M - 1, M - 1))

    for i in range(M - 1):
        A[i, i] = r2

        if i > 0:
            A[i - 1, i] = -r
            A[i, i - 1] = -r

    # avanço temporal
    for k in range(1, N + 1):

        b = u[1:M, k - 1].copy()

        # contribuição das fronteiras
        b[0] += r * u[0, k]
        b[-1] += r * u[M, k]

        u[1:M, k] = np.linalg.solve(A, b)

    return u, x, t
```

## 3. O Método de Crank-Nicholson

Voltemos agora à Eq. (7) e tentemos aprimorar o método implícito de Euler regressivo. A aproximação por diferenças no lado esquerdo é tomada no instante de tempo $k$, enquanto a aproximação por diferenças no lado direito é tomada no ponto médio entre os instantes $k$ e $k-1$, se a interpretarmos como uma aproximação por diferença central com passo temporal $\Delta t/2$. Isso não parece inconsistente?

Que tal tomar as aproximações por diferenças de ambos os lados no mesmo instante de tempo — por exemplo, no ponto médio entre $k+1$ e $k$ - para manter um equilíbrio? Para isso, tomamos a média das aproximações por diferenças centrais do lado esquerdo nos instantes $k+1$ e $k$, obtendo

$$\frac{A}{2}\left(\frac{u_{i+1}^{k+1}-2u_i^{k+1}+u_{i-1}^{k+1}}{\Delta x^2}+\frac{u_{i+1}^{k}-2u_i^{k}+u_{i-1}^{k}}{\Delta x^2}\right)=\frac{u_i^{k+1}-u_i^k}{\Delta t}. \tag{15}$$

Essa expressão conduz ao chamado método de Crank-Nicholson:

$$-ru_{i+1}^{k+1} + 2(1+r)u_i^{k+1} - ru_{i-1}^{k+1} = ru_{i+1}^{k} + 2(1-r)u_i^{k} + ru_{i-1}^{k}, \tag{16}$$

com

$$r = A\frac{\Delta t}{\Delta x^2}.$$

O método de Crank-Nicholson pode ser visto como uma combinação dos métodos explícito e implícito, utilizando a média entre os operadores espaciais avaliados nos tempos $k$ e $k+1$. Como resultado, ele mantém a estabilidade incondicional do método implícito e alcança segunda ordem de precisão no tempo, enquanto os métodos de Euler direto e regressivo possuem apenas primeira ordem de precisão temporal. Por essa razão, o método de Crank-Nicholson é amplamente utilizado na resolução numérica da equação do calor e de outras equações parabólicas.

Com condições de contorno de Dirichlet/Neumann em $x_0$ e $x_M$, respectivamente A Eq. (16) pode ser escrita no seguinte sistema tridiagonal de equações:

$$\begin{bmatrix} 2(1+r) & -r & 0 & \cdot & 0 & 0 \\ -r & 2(1+r) & -r & \cdot & 0 & 0 \\ 0 & -r & 2(1+r) & \cdot & 0 & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & 0 & 0 & \cdot & 2(1+r) & -r \\ 0 & 0 & 0 & \cdot & -2r & 2(1+r) \end{bmatrix} \begin{bmatrix} u_1^{k+1} \\ u_2^{k+1} \\ u_3^{k+1} \\ \cdot \\ u_{M-1}^{k+1} \\ u_M^{k+1} \end{bmatrix}$$

$$= \begin{bmatrix} 2(1-r) & r & 0 & \cdot & 0 & 0 \\ r & 2(1-r) & r & \cdot & 0 & 0 \\ 0 & r & 2(1-r) & \cdot & 0 & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & 0 & 0 & \cdot & 2(1-r) & r \\ 0 & 0 & 0 & \cdot & 2r & 2(1-r) \end{bmatrix} \begin{bmatrix} u_1^{k} \\ u_2^{k} \\ u_3^{k} \\ \cdot \\ u_{M-1}^{k} \\ u_M^{k} \end{bmatrix} + \begin{bmatrix} r\left(u_0^{k+1} + u_0^{k}\right) \\ 0 \\ 0 \\ \cdot \\ 0 \\ 2r\left(b'_M(k+1) + b'_M(k)\right) \end{bmatrix} \tag{17}$$

Esse sistema também pode ser resolvido de maneira muito eficiente devido à sua estrutura tridiagonal. Além disso, sua estabilidade incondicional pode ser demonstrada substituindo a Eq. (4) na Eq. (16), obtendo-se

$$2\lambda(1 + r(1 - \cos(\pi/P))) = 2(1 - r(1 - \cos(\pi/P))),$$

ou seja,

$$\lambda = \frac{1 - r(1 - \cos(\pi/P))}{1 + r(1 - \cos(\pi/P))}, \quad |\lambda| \leq 1. \tag{18}$$

O algoritmo `heat_CN()` implementa o método de Crank-Nicholson.

```
def heat_CN(a, xf, T, it0, bx0, bxf, M, N):
    """
    Método de Crank-Nicolson para a equação do calor

        u_t = a u_xx

    em 0 <= x <= xf, 0 <= t <= T.
```

(continua na próxima página)

(continuação da página anterior)

```
    """

    dx = xf / M
    x = np.linspace(0.0, xf, M + 1)

    dt = T / N
    t = np.linspace(0.0, T, N + 1)

    u = np.zeros((M + 1, N + 1))

    # condição inicial
    for i in range(M + 1):
        u[i, 0] = it0(x[i])

    # condições de contorno
    for n in range(N + 1):
        u[0, n] = bx0(t[n])
        u[M, n] = bxf(t[n])

    r = a * dt / dx**2

    r1 = 2 * (1 + r)
    r2 = 2 * (1 - r)

    # matriz tridiagonal
    A = np.zeros((M - 1, M - 1))

    for i in range(M - 1):
        A[i, i] = r1

        if i > 0:
            A[i - 1, i] = -r
            A[i, i - 1] = -r

    # avanço temporal
    for k in range(1, N + 1):

        b = (
            np.concatenate((
                [r * u[0, k]],
                np.zeros(M - 3),
                [r * u[M, k]]
            ))
            + r * (u[0:M-1, k-1] + u[2:M+1, k-1])
            + r2 * u[1:M, k-1]
```

(continua na próxima página)

(continuação da página anterior)

```
        )

        u[1:M, k] = np.linalg.solve(A, b)

    return u, x, t
```

**Exemplo 8.4.2:** EDP Parabólica Unidimensional: Equação do Fluxo de Calor

Considere a EDP parabólica

$$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t}, \quad 0 \leq x \leq 1, \quad 0 \leq t \leq 0.1. \tag{E.1}$$

com a condição inicial e as condições de contorno

$$u(x,0) = \sin(\pi x), \quad u(0,t) = 0, \quad u(1,t) = 0. \tag{E.2}$$

Construímos o programa `solve_heat` para utilizar as rotinas `heat_exp()` , `heat_imp()` e `heat_CN()` na resolução dessa equação.

Observe que, com o espaçamento espacial

$$\Delta x = \frac{x_f}{M} = \frac{1}{20}$$

e o passo temporal

$$\Delta t = \frac{T}{N} = \frac{0.1}{100} = 0.001$$

temos

$$r = A\frac{\Delta t}{\Delta x^2} = 1 \cdot \frac{0.001}{(1/20)^2} = 0.4. \tag{E.3}$$

Como esse valor satisfaz a condição de estabilidade

$$r \leq \frac{1}{2},$$

dada pela Eq. (6), os três métodos produzem resultados satisfatórios, apresentando um erro relativo de aproximadamente 0.013 .

Entretanto, se diminuirmos o espaçamento espacial para

$$\Delta x = \frac{1}{25},$$

a fim de obter uma resolução espacial mais refinada, obtemos

$$r = 0.625,$$

o que viola a condição de estabilidade do método explícito. Nessa situação, o método de Euler direto ( `heat_exp( )` ) torna-se instável e a solução diverge. Em contraste, o método implícito de Euler regressivo ( `heat_imp( )` ) e o método de Crank-Nicholson ( `heat_CN( )` ) continuam funcionando adequadamente, conforme pode ser vistos nos gráficos.

Por outro lado, se mantivermos o espaçamento espacial em

$$\Delta x = \frac{1}{25}$$

e reduzirmos o passo temporal para

$$\Delta t = \frac{0.1}{120},$$

temos

$$r = \frac{0.1/120}{(1/25)^2} \approx 0.521.$$

Esse valor ainda está muito próximo do limite de estabilidade do método explícito. Ao utilizar um passo temporal suficientemente pequeno (de forma que a condição $r \leq 1/2$ seja efetivamente satisfeita), o método explícito volta a apresentar comportamento estável. Nessas condições, tanto o método explícito quanto os métodos implícitos produzem soluções bastante precisas, com erro relativo inferior a 0.001.

Isso significa que, com cerca de 30% a mais de esforço computacional, obtém-se uma solução mais precisa, apesar de o valor

$$r = 0.5208$$

não satisfazer rigorosamente a condição de estabilidade. Esse resultado sugere que a condição

$$r \leq \frac{1}{2}$$

para a estabilidade do método explícito de Euler direto não é uma condição necessária, mas apenas uma condição suficiente.

Além disso, quando o método explícito converge, sua precisão pode ser superior à do método implícito de Euler regressivo. Entretanto, de modo geral, ela não supera a precisão obtida pelo método de CrankNicholson.

```
import matplotlib.pyplot as plt
```

```
#solve_heat

# parâmetro da equação u_t = a*u_xx
a = 1.0

# condição inicial
it0 = lambda x: np.sin(np.pi * x)

# condições de contorno
bx0 = lambda t: 0.0
bxf = lambda t: 0.0

# parâmetros da malha
xf = 1.0
M = 25

T = 0.1
```

(continua na próxima página)

(continuação da página anterior)

```
N = 100

# solução analítica
uo = lambda x, t: np.sin(np.pi * x) * np.exp(-np.pi**2 * t)

# Método explícito
u1, x, t = heat_exp(a, xf, T, it0, bx0, bxf, M, N)

fig = plt.figure(1)
ax = fig.add_subplot(111, projection='3d')
Tgrid, Xgrid = np.meshgrid(t, x)
ax.plot_surface(Tgrid, Xgrid, u1)
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_title('Método Explícito')

# Método implícito
u2, x, t = heat_imp(a, xf, T, it0, bx0, bxf, M, N)

fig = plt.figure(2)
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(Tgrid, Xgrid, u2)
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_title('Método Implícito')

# Método de Crank-Nicolson
u3, x, t = heat_CN(a, xf, T, it0, bx0, bxf, M, N)

fig = plt.figure(3)
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(Tgrid, Xgrid, u3)
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_title('Método de Crank-Nicolson')

# solução analítica na malha
X, Tm = np.meshgrid(x, t, indexing='ij')
Uo = uo(X, Tm)

# evitar divisão por zero
aUo = np.abs(Uo) + np.finfo(float).eps

MN = M * N
```

(continua na próxima página)

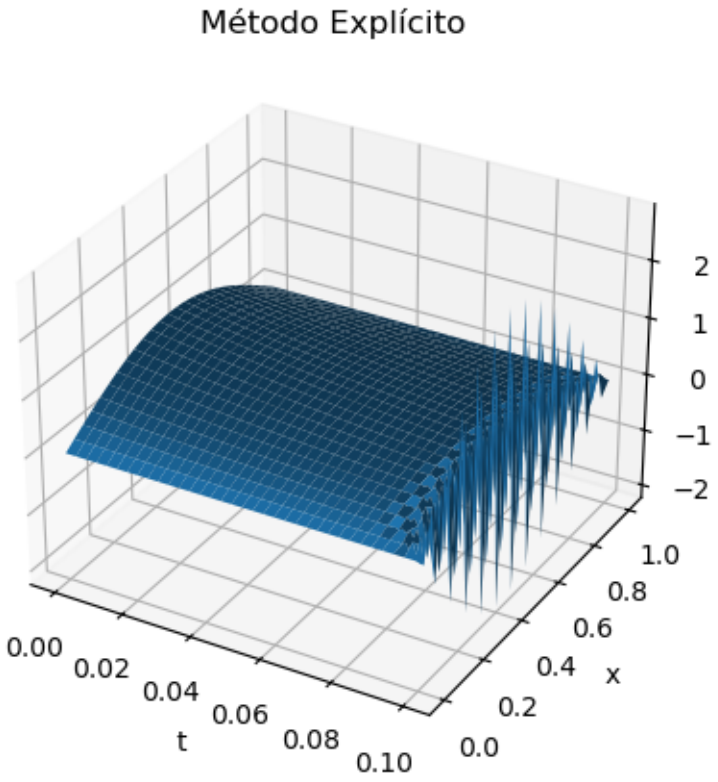
(continuação da página anterior)

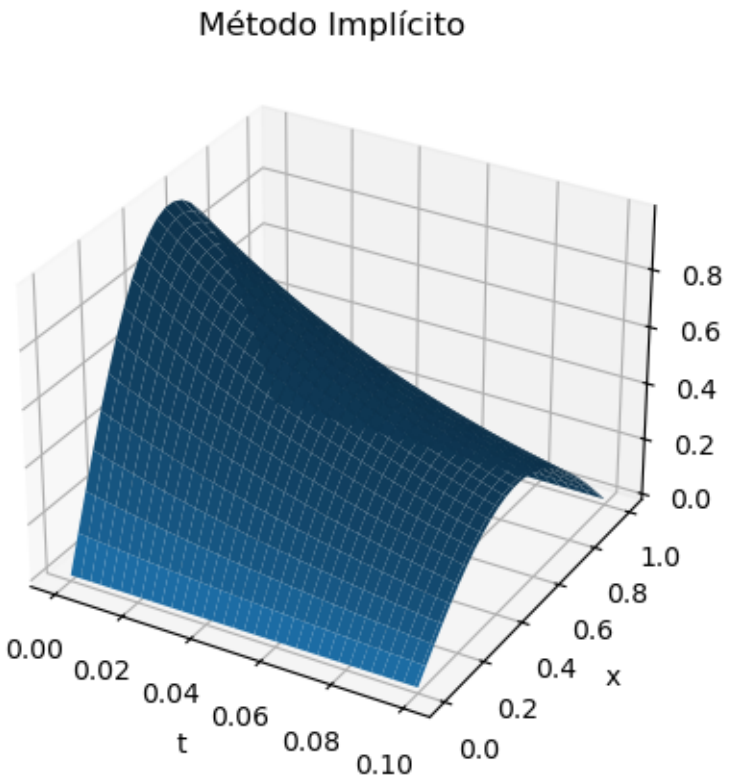
```
# erros relativos normalizados
err1 = np.linalg.norm((u1 - Uo) / aUo) / MN
err2 = np.linalg.norm((u2 - Uo) / aUo) / MN
err3 = np.linalg.norm((u3 - Uo) / aUo) / MN

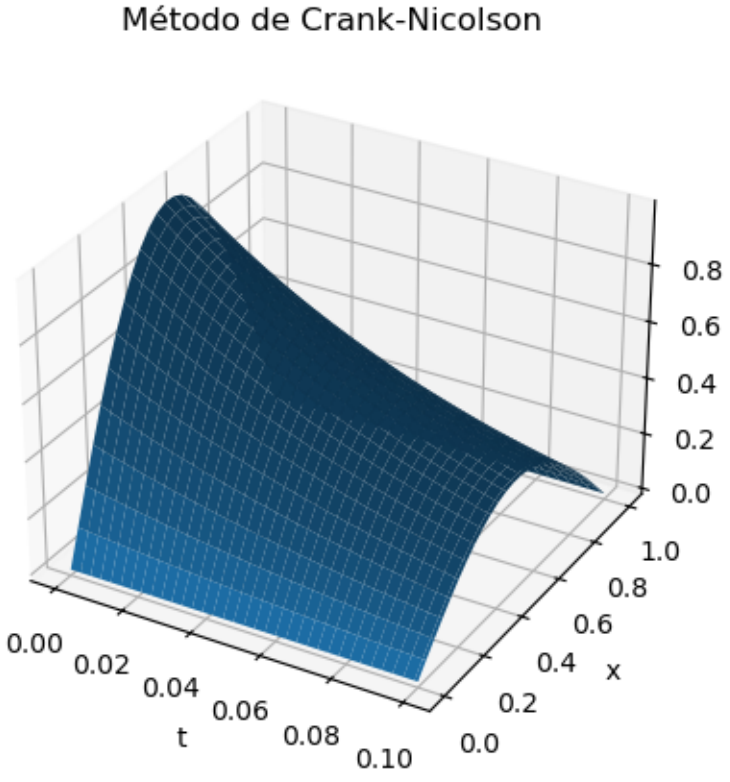
print(f"Erro método explícito      = {err1:e}")
print(f"Erro método implícito      = {err2:e}")
print(f"Erro Crank-Nicolson        = {err3:e}")

plt.show()
```

```
Erro método explícito      = 1.762968e-02
Erro método implícito      = 1.053045e-03
Erro Crank-Nicolson        = 1.050821e-03
```






**Comentário:**

Este exemplo evidencia a principal diferença entre os três métodos:

- **Euler explícito (Forward Euler):**
  - Simples de implementar;
  - Exige a condição de estabilidade: $r = A\frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}$
  - Pode divergir quando a malha espacial é refinada sem uma redução correspondente do passo temporal.
- **Euler implícito (Backward Euler):**
  - Incondicionalmente estável;
  - Requer a solução de um sistema linear a cada passo de tempo;
  - Possui precisão temporal de primeira ordem.
- **Crank-Nicholson:**
  - Também é incondicionalmente estável;
  - Requer a solução de um sistema linear tridiagonal;
  - Possui precisão temporal de segunda ordem;
  - Geralmente oferece a melhor combinação entre estabilidade e precisão.

Para este problema específico, a solução analítica é

$$u(x,t) = \sin(\pi x)e^{-\pi^2 t}$$

que é a expressão utilizada para calcular os erros relativos das aproximações numéricas.

## Equação do calor 2D (transiente)

A extensão da equação do calor para domínios bidimensionais é

$$\frac{\partial u}{\partial t} = \alpha\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \quad (x,y) \in \Omega, \quad t > 0$$

Usando diferenças finitas centrais no espaço e progressivas no tempo, com $\Delta x = \Delta y = h$, a equação de diferenças é

$$u_{i,j}^{k+1} = \gamma\left(u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4\,u_{i,j}^k\right) + u_{i,j}^k, \qquad \gamma = \alpha\frac{\Delta t}{h^2}$$

A condição de estabilidade para o caso 2D é mais restritiva que no caso 1D: $\gamma \leq 1/4$.

**Exemplo 8.4.3:** Placa retangular com aquecimento nas bordas

Placa de $50 \times 30$ unidades, temperatura inicial nula em toda a região interior. Condições de contorno constantes: borda superior a $80\,°C$, borda direita a $50\,°C$, bordas esquerda e inferior a $0\,°C$.

Parâmetros: $\alpha = 2$, $h = 0{,}5$, $\Delta t = 0{,}01$ s $\Rightarrow \gamma = 0{,}08 < 0{,}25$ (estável).

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

xmax, ymax = 50, 30
kmax = 500
alpha = 2.0
h     = 0.5
ht    = 0.01
gamma = alpha * ht / h**2
```

(continua na próxima página)

(continuação da página anterior)

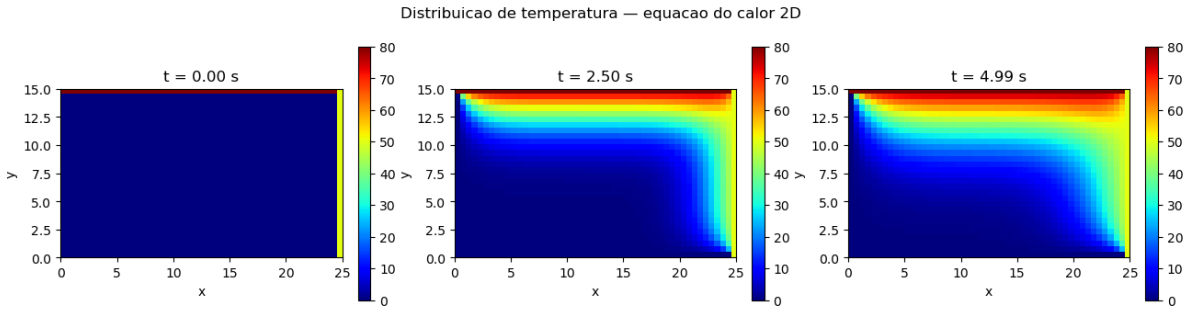
```
print(f"gamma = {gamma:.4f}  (criterio 2D: gamma <= 0.25)")

u = np.zeros((kmax, ymax, xmax))
u[:, -1, :] = 80.0
u[:, :, -1] = 50.0

for k in range(kmax - 1):
    u[k+1, 1:-1, 1:-1] = (
        gamma * (
            u[k, 2:,   1:-1] + u[k, :-2,  1:-1]
          + u[k, 1:-1, 2:]   + u[k, 1:-1, :-2]
          - 4 * u[k, 1:-1, 1:-1]
        )
        + u[k, 1:-1, 1:-1]
    )
    u[k+1, -1, :] = 80.0
    u[k+1, :, -1] = 50.0

fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
for ax, k in zip(axes, [0, kmax//2, kmax-1]):
    im = ax.imshow(u[k], cmap='jet', vmin=0, vmax=80, origin='lower',
                   extent=[0, xmax*h, 0, ymax*h])
    ax.set_title(f't = {k*ht:.2f} s')
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    plt.colorbar(im, ax=ax, fraction=0.046)
plt.suptitle('Distribuicao de temperatura — equacao do calor 2D')
plt.tight_layout()
plt.show()
```

```
gamma = 0.0800  (criterio 2D: gamma <= 0.25)
```




```
# Animação da evolução temporal
fig, ax = plt.subplots(figsize=(6, 4))
im = ax.imshow(u[0], cmap='jet', vmin=0, vmax=80, origin='lower',
```

(continua na próxima página)

(continuação da página anterior)

```
                extent=[0, xmax*h, 0, ymax*h])
plt.colorbar(im, ax=ax, label='Temperatura (graus C)')
ax.set_xlabel('x')
ax.set_ylabel('y')

def _animate(k):
    im.set_array(u[k])
    ax.set_title(f't = {k*ht:.3f} s')
    return [im]

anim = animation.FuncAnimation(fig, _animate, frames=kmax,
                               interval=30, blit=True, repeat=False)
from IPython.display import HTML
HTML(anim.to_jshtml())
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[8], line 17
     14 anim = animation.FuncAnimation(fig, _animate, frames=kmax,
     15                                interval=30, blit=True, repeat=False)
     16 from IPython.display import HTML
---> 17 HTML(anim.to_jshtml())

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/animation.py:1353, in␣
 ↪Animation.to_jshtml(self, fps, embed_frames, default_mode)
   1349         path = Path(tmpdir, "temp.html")
   1350         writer = HTMLWriter(fps=fps,
   1351                             embed_frames=embed_frames,
   1352                             default_mode=default_mode)
-> 1353         self.save(str(path), writer=writer)
   1354         self._html_representation = path.read_text()
   1356 return self._html_representation

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/animation.py:1109, in␣
 ↪Animation.save(self, filename, writer, fps, dpi, codec, bitrate, extra_args,␣
 ↪metadata, extra_anim, savefig_kwargs, progress_callback)
   1107         progress_callback(frame_number, total_frames)
   1108         frame_number += 1
-> 1109 writer.grab_frame(**savefig_kwargs)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/animation.py:775, in␣
 ↪HTMLWriter.grab_frame(self, **savefig_kwargs)
    773     return
    774 f = BytesIO()
--> 775 self.fig.savefig(f, format=self.frame_format,
```

(continua na próxima página)

(continuação da página anterior)

```
    776                 dpi=self.dpi, **savefig_kwargs)
    777 imgdata64 = base64.encodebytes(f.getvalue()).decode('ascii')
    778 self._total_bytes += len(imgdata64)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/figure.py:3390, in Figure.
 ↪savefig(self, fname, transparent, **kwargs)
   3388     for ax in self.axes:
   3389         _recursively_make_axes_transparent(stack, ax)
-> 3390 self.canvas.print_figure(fname, **kwargs)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/backend_bases.py:2193, in␣
 ↪FigureCanvasBase.print_figure(self, filename, dpi, facecolor, edgecolor,␣
 ↪orientation, format, bbox_inches, pad_inches, bbox_extra_artists, backend,␣
 ↪**kwargs)
   2189 try:
   2190     # _get_renderer may change the figure dpi (as vector formats
   2191     # force the figure dpi to 72), so we need to set it again here.
   2192     with cbook._setattr_cm(self.figure, dpi=dpi):
-> 2193         result = print_method(
   2194             filename,
   2195             facecolor=facecolor,
   2196             edgecolor=edgecolor,
   2197             orientation=orientation,
   2198             bbox_inches_restore=_bbox_inches_restore,
   2199             **kwargs)
   2200 finally:
   2201     if bbox_inches and restore_bbox:

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/backend_bases.py:2043, in␣
 ↪FigureCanvasBase._switch_canvas_and_return_print_method.<locals>.<lambda>(*args,␣
 ↪**kwargs)
   2039     optional_kws = {  # Passed by print_figure for other renderers.
   2040         "dpi", "facecolor", "edgecolor", "orientation",
   2041         "bbox_inches_restore"}
   2042     skip = optional_kws - {*inspect.signature(meth).parameters}
-> 2043     print_method = functools.wraps(meth)(lambda *args, **kwargs: meth(
   2044         *args, **{k: v for k, v in kwargs.items() if k not in skip}))
   2045 else:  # Let third-parties do as they see fit.
   2046     print_method = meth

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/backends/backend_agg.
 ↪py:497, in FigureCanvasAgg.print_png(self, filename_or_obj, metadata, pil_kwargs)
    450 def print_png(self, filename_or_obj, *, metadata=None, pil_kwargs=None):
    451     """
    452     Write the figure to a PNG file.
```

(continua na próxima página)

(continuação da página anterior)

```
    453
   (...)
    495         *metadata*, including the default 'Software' key.
    496     """
--> 497     self._print_pil(filename_or_obj, "png", pil_kwargs, metadata)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/backends/backend_agg.
↪py:445, in FigureCanvasAgg._print_pil(self, filename_or_obj, fmt, pil_kwargs,␣
↪metadata)
    440 def _print_pil(self, filename_or_obj, fmt, pil_kwargs, metadata=None):
    441     """
    442     Draw the canvas, then save it using `.image.imsave` (to which
    443     *pil_kwargs* and *metadata* are forwarded).
    444     """
--> 445     FigureCanvasAgg.draw(self)
    446     mpl.image.imsave(
    447         filename_or_obj, self.buffer_rgba(), format=fmt, origin="upper",
    448         dpi=self.figure.dpi, metadata=metadata, pil_kwargs=pil_kwargs)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/backends/backend_agg.
↪py:388, in FigureCanvasAgg.draw(self)
    385 # Acquire a lock on the shared font cache.
    386 with (self.toolbar._wait_cursor_for_draw_cm() if self.toolbar
    387       else nullcontext()):
--> 388     self.figure.draw(self.renderer)
    389     # A GUI class may be need to update a window using this draw, so
    390     # don't forget to call the superclass.
    391     super().draw()

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/artist.py:95, in _
↪finalize_rasterization.<locals>.draw_wrapper(artist, renderer, *args, **kwargs)
     93 @wraps(draw)
     94 def draw_wrapper(artist, renderer, *args, **kwargs):
---> 95     result = draw(artist, renderer, *args, **kwargs)
     96     if renderer._rasterizing:
     97         renderer.stop_rasterizing()

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/artist.py:72, in allow_
↪rasterization.<locals>.draw_wrapper(artist, renderer)
     69     if artist.get_agg_filter() is not None:
     70         renderer.start_filter()
---> 72     return draw(artist, renderer)
     73 finally:
     74     if artist.get_agg_filter() is not None:
```

(continua na próxima página)

(continuação da página anterior)

```
File ~/anaconda3/lib/python3.12/site-packages/matplotlib/figure.py:3154, in Figure.
 ↪draw(self, renderer)
   3151         # ValueError can occur when resizing a window.
   3153 self.patch.draw(renderer)
-> 3154 mimage._draw_list_compositing_images(
   3155     renderer, self, artists, self.suppressComposite)
   3157 for sfig in self.subfigs:
   3158     sfig.draw(renderer)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/image.py:132, in _draw_
 ↪list_compositing_images(renderer, parent, artists, suppress_composite)
    130 if not_composite or not has_images:
    131     for a in artists:
--> 132         a.draw(renderer)
    133 else:
    134     # Composite any adjacent images together
    135     image_group = []

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/artist.py:72, in allow_
 ↪rasterization.<locals>.draw_wrapper(artist, renderer)
     69     if artist.get_agg_filter() is not None:
     70         renderer.start_filter()
---> 72     return draw(artist, renderer)
     73 finally:
     74     if artist.get_agg_filter() is not None:

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axes/_base.py:3034, in _
 ↪AxesBase.draw(self, renderer)
   3031     for spine in self.spines.values():
   3032         artists.remove(spine)
-> 3034 self._update_title_position(renderer)
   3036 if not self.axison:
   3037     for _axis in self._axis_map.values():

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axes/_base.py:2978, in _
 ↪AxesBase._update_title_position(self, renderer)
   2976 top = max(top, bb.ymax)
   2977 if title.get_text():
-> 2978     ax.yaxis.get_tightbbox(renderer)  # update offsetText
   2979     if ax.yaxis.offsetText.get_text():
   2980         bb = ax.yaxis.offsetText.get_tightbbox(renderer)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axis.py:1336, in Axis.get_
 ↪tightbbox(self, renderer, for_layout_only)
   1333     renderer = self.figure._get_renderer()
```

(continua na próxima página)

(continuação da página anterior)

```
   1334 ticks_to_draw = self._update_ticks()
-> 1336 self._update_label_position(renderer)
   1338 # go back to just this axis's tick labels
   1339 tlb1, tlb2 = self._get_ticklabel_bboxes(ticks_to_draw, renderer)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axis.py:2609, in YAxis._
 ↪update_label_position(self, renderer)
   2605     return
   2607 # get bounding boxes for this axis and any siblings
   2608 # that have been set by `fig.align_ylabels()`
-> 2609 bboxes, bboxes2 = self._get_tick_boxes_siblings(renderer=renderer)
   2610 x, y = self.label.get_position()
   2611 if self.label_position == 'left':

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axis.py:2161, in Axis._
 ↪get_tick_boxes_siblings(self, renderer)
   2159 axis = ax._axis_map[name]
   2160 ticks_to_draw = axis._update_ticks()
-> 2161 tlb, tlb2 = axis._get_ticklabel_bboxes(ticks_to_draw, renderer)
   2162 bboxes.extend(tlb)
   2163 bboxes2.extend(tlb2)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/axis.py:1315, in Axis._
 ↪get_ticklabel_bboxes(self, ticks, renderer)
   1313 if renderer is None:
   1314     renderer = self.figure._get_renderer()
-> 1315 return ([tick.label1.get_window_extent(renderer)
   1316          for tick in ticks if tick.label1.get_visible()],
   1317         [tick.label2.get_window_extent(renderer)
   1318          for tick in ticks if tick.label2.get_visible()])

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/text.py:956, in Text.get_
 ↪window_extent(self, renderer, dpi)
    951     raise RuntimeError(
    952         "Cannot get window extent of text w/o renderer. You likely "
    953         "want to call 'figure.draw_without_rendering()' first.")
    955 with cbook._setattr_cm(self.figure, dpi=dpi):
--> 956     bbox, info, descent = self._get_layout(self._renderer)
    957     x, y = self.get_unitless_position()
    958     x, y = self.get_transform().transform((x, y))

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/text.py:381, in Text._get_
 ↪layout(self, renderer)
    379 clean_line, ismath = self._preprocess_math(line)
    380 if clean_line:
```

(continua na próxima página)

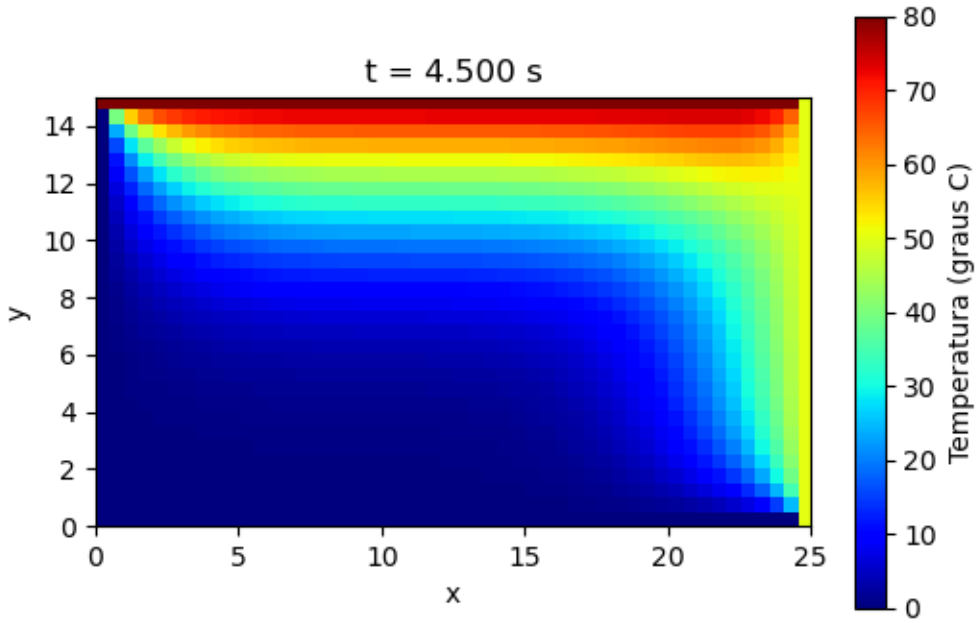
(continuação da página anterior)

```
--> 381     w, h, d = _get_text_metrics_with_cache(
    382         renderer, clean_line, self._fontproperties,
    383         ismath=ismath, dpi=self.figure.dpi)
    384 else:
    385     w = h = d = 0

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/text.py:70, in _get_text_
↪metrics_with_cache(renderer, text, fontprop, ismath, dpi)
     66 """Call ``renderer.get_text_width_height_descent``, caching the results."""
     67 # Cached based on a copy of fontprop so that later in-place mutations of
     68 # the passed-in argument do not mess up the cache.
     69 return _get_text_metrics_with_cache_impl(
---> 70     weakref.ref(renderer), text, fontprop.copy(), ismath, dpi)

File ~/anaconda3/lib/python3.12/site-packages/matplotlib/font_manager.py:899, in␣
↪FontProperties.copy(self)
    896         _api.check_in_list(valid_fonts, math_fontfamily=fontfamily)
    897     self._math_fontfamily = fontfamily
--> 899 def copy(self):
    900     """Return a copy of self."""
    901     return copy.copy(self)

KeyboardInterrupt:
```



## Exercícios

Modifique o programa `solve_heat` para que ele possa resolver as seguintes EDPs utilizando o método explícito de Euler direto, o método implícito de Euler regressivo e o método de Crank-Nicholson.

**(a)** $\$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t}, \quad 0 \le x \le 1, \quad 0 \le t \le 0.1\$$

com as condições inicial e de contorno

$$u(x,0) = x^4, \quad u(0,t) = 0, \quad u(1,t) = 1.$$

**(i)** Com a região de solução dividida em

$$M \times N = 10 \times 20,$$

o método explícito de Euler direto converge? Qual é o valor de

$$r = A\frac{\Delta t}{(\Delta x)^2}?$$

**(ii)** Se aumentarmos $M$ e $N$ para

$$M \times N = 20 \times 40,$$

visando obter maior precisão, o método explícito de Euler direto ainda converge? Qual é o valor de

$$r = A\frac{\Delta t}{(\Delta x)^2}?$$

**(iii)** Qual deve ser o número $N$ de subintervalos ao longo do eixo $t$ para que o valor de $r$ permaneça o mesmo quando $M = 20$ ?

Com esse valor de $r$, o método explícito de Euler direto converge?

**(b)** Resolva o problema de condução de calor

$$u_{xx} = 4\,u_t, \quad 0 < x < 2, \quad t > 0$$

$$u(0,t) = 0, \quad u(2,t) = 0, \quad u(x,0) = 2\,\mathrm{sen}\Big(\frac{\pi x}{2}\Big) - \mathrm{sen}(\pi x) + 4\,\mathrm{sen}(2\pi x)$$

Compare com a solução exata $u(x,t) = 2\,e^{-\pi^2 t/16}\,\mathrm{sen}(\pi x/2) - e^{-\pi^2 t/4}\,\mathrm{sen}(\pi x) + 4\,e^{-\pi^2 t}\,\mathrm{sen}(2\pi x)$.

**Problemas adicionais — Barra de 40 cm** (Fonte: Boyce e DiPrima, 2024)

Extremidades mantidas a $0\,°C$, $\alpha^2 = 1$. Para cada condição inicial abaixo, resolva numericamente com os três métodos e construa o gráfico de $u(x,t)$ para alguns instantes de tempo.

**(c)** $u(x,0) = 50, \quad 0 < x < 40$

**(d)** $u(x,0) = \begin{cases} x, & 0 \le x < 20 \\ 40 - x, & 20 \le x \le 40 \end{cases}$

**(e)** $u(x,0) = \begin{cases} 0, & 0 \le x < 10 \\ 50, & 10 \le x \le 30 \\ 0, & 30 < x \le 40 \end{cases}$

**(f)** $u(x,0) = x, \quad 0 \le x < 40$

**Análise de convergência**

**(g)** Considere a barra do Problema **(c)**. Para $t = 5$ e $x = 20$, determine quantos termos da série de Fourier são

necessários para que a inclusão de mais um termo não altere as três primeiras casas decimais de $u(20, 5)$. Repita para $t = 20$ e $t = 80$. O que se conclui sobre a velocidade de convergência?

**(h)** Repita o Problema **(g)** para a barra do Problema **(d)**.

**(i)** Repita o Problema **(g)** para a barra do Problema **(e)**.

**(j)** Repita o Problema **(g)** para a barra do Problema **(f)**.

## 8.5 EDPs Hiperbólicas

*Esta seção é uma tradução adaptada da Seção 9.3 (Hyperbolic PDE) de YANG, W. Y.; CAO, W.; CHUNG, T.-S.; MORRIS, J.* ***Applied Numerical Methods Using MATLAB****. Hoboken: John Wiley & Sons, 2005. O desenvolvimento do texto, os exemplos e as rotinas computacionais seguem essa obra, assim como parte dos exercícios, com tradução para o português e implementação das rotinas em Python.*

Consideremos uma EDP geral de segunda ordem em duas variáveis independentes $x$ e $y$, que é escrita como

$$A(x,y)\frac{\partial^2 u}{\partial x^2} + B(x,y)\frac{\partial^2 u}{\partial x \partial y} + C(x,y)\frac{\partial^2 u}{\partial y^2} = f\left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right)$$
$$\text{para } x_0 \leq x \leq x_f, y_0 \leq y \leq y_f$$

com as condições de contorno dadas por

$$u\left(x, y_0\right) = b_{y0}(x), \quad u\left(x, y_f\right) = b_{yf}(x),$$
$$u\left(x_0, y\right) = b_{x0}(y), \quad \text{e} \quad u\left(x_f, y\right) = b_{xf}(y)$$

As EDP elípticas são aquelas em que $B^2 - 4AC > 0$ e estão associadas a fenômenos ondulatórios.

Um exemplo de EDP hiperbólica é a equação da onda unidimensional para a função amplitude $u(x,t)$, em que $x$ representa a posição e $t$ representa o tempo:

$$A\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial^2 u(x,t)}{\partial t^2}, \qquad 0 \leq x \leq x_f, \quad 0 \leq t \leq T \tag{1}$$

Para que essa equação seja solucionável, devem ser fornecidas as condições de contorno $u(0,t) = b_0(t) \quad \text{e} \quad u(x_f,t) = b_{xf}(t)$, bem como as condições iniciais $u(x,0) = i_0(x) \quad \text{e} \quad \frac{\partial u}{\partial t}\big|_{t=0} = i_0'(x)$.

**Método Explícito das Diferenças Centrais**

Da mesma forma que nas EDPs parabólicas, substituímos as derivadas de segunda ordem em ambos os lados da Equação (1) por suas aproximações de diferenças centrais de três pontos:

$$A\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} = \frac{u_i^{k+1} - 2u_i^k + u_i^{k-1}}{\Delta t^2}, \qquad \Delta x = \frac{x_f}{M}, \quad \Delta t = \frac{T}{N} \tag{2}$$

o que conduz ao método explícito das diferenças centrais:

$$u_i^{k+1} = r\left(u_{i+1}^k + u_{i-1}^k\right) + 2(1-r)u_i^k - u_i^{k-1}, \qquad r = A\frac{\Delta t^2}{\Delta x^2} \tag{3}$$

Como $u_i^{-1} = u(x_i, -\Delta t)$ não é conhecido, não podemos obter $u_i^1$ diretamente da Equação (3) com $k = 0$:

$$u_i^1 = r\left(u_{i+1}^0 + u_{i-1}^0\right) + 2(1-r)u_i^0 - u_i^{-1} \tag{4}$$

Portanto, aproximamos a condição inicial para a derivada temporal por uma diferença central:

$$\frac{u_i^1 - u_i^{-1}}{2\Delta t} = i_0'(x_i) \tag{5}$$

e utilizamos essa relação para eliminar $u_i^{-1}$ da Equação (3):

$$u_i^1 = r\left(u_{i+1}^0 + u_{i-1}^0\right) + 2(1-r)u_i^0 - \left(u_i^1 - 2i_0'(x_i)\Delta t\right)$$

resultando em

$$u_i^1 = \frac{1}{2}r\left(u_{i+1}^0 + u_{i-1}^0\right) + (1-r)u_i^0 + i_0'(x_i)\Delta t \tag{6}$$

Utilizamos a Equação (6), juntamente com as condições iniciais, para obter $u_i^1$. Em seguida, aplicamos a Equação (3) para $k = 1, 2, ....$

Observe os seguintes fatos:

- Deve-se ter $r \leq 1$ para garantir a estabilidade do método.
- A precisão da solução melhora à medida que $r$ aumenta, o que corresponde à diminuição de $\Delta x$.

Portanto, é razoável escolher $r = 1$.

A condição de estabilidade pode ser obtida substituindo a Equação (4) da Seção 8.4 na Equação (3) e aplicando o teste de Jury:

$$\lambda = 2r\cos\left(\frac{\pi}{P}\right) + 2(1-r) - \lambda^{-1},$$

ou, equivalentemente,

$$\lambda^2 + 2\left(r\left(1 - \cos\left(\frac{\pi}{P}\right)\right) - 1\right)\lambda + 1 = 0.$$

Para que o método seja estável, as soluções dessa equação devem permanecer dentro do círculo unitário, o que exige

$$r \leq \frac{1}{1-\cos(\pi/P)}, \qquad r = A\frac{\Delta t^2}{\Delta x^2} \leq 1. \tag{7}$$

O objetivo da rotina `wave()` apresentada a seguir é implementar esse algoritmo para resolver a equação da onda unidimensional.

**Exemplo 8.5.1:** Uma EDP Hiperbólica: Onda Unidimensional (Vibração)

Considere a seguinte EDP hiperbólica unidimensional:

$$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial^2 u(x,t)}{\partial t^2}, \qquad 0 \leq x \leq 1, \quad 0 \leq t \leq 2 \tag{E.1}$$

com as condições iniciais

$$u(x,0) = x(1-x), \quad \frac{\partial u}{\partial t}(x,0) = 0 \tag{E.2a}$$

e as condições de contorno

$$u(0,t) = 0, \qquad u(1,t) = 0. \tag{E.2b}$$

Foi desenvolvido o programa `solve_wave` para utilizar a rotina `wave()` na resolução dessa equação. A execução do programa produz os resultados mostrados nos gráficos a seguir, além de uma visualização dinâmica da vibração da corda.

```
import numpy as np
```

(continua na próxima página)

(continuação da página anterior)

```
def wave(a, xf, T, it0, i1t0, bx0, bxf, M, N):
    """
    Resolve a EDP hiperbólica

        a * u_xx = u_tt

    para:
        0 <= x <= xf
        0 <= t <= T

    Condições iniciais:
        u(x,0) = it0(x)
        u_t(x,0) = i1t0(x)

    Condições de contorno:
        u(0,t)  = bx0(t)
        u(xf,t) = bxf(t)

    Parâmetros
    ----------
    a : float
        Constante da equação.
    xf : float
        Comprimento do domínio espacial.
    T : float
        Tempo final.
    it0 : função
        Condição inicial u(x,0).
    i1t0 : função
        Condição inicial u_t(x,0).
    bx0 : função
        Condição de contorno em x=0.
    bxf : função
        Condição de contorno em x=xf.
    M : int
        Número de subintervalos espaciais.
    N : int
        Número de subintervalos temporais.

    Retorna
    --------
    u : ndarray, shape (M+1, N+1)
        Solução numérica.
    x : ndarray
        Malha espacial.
```

(continua na próxima página)

(continuação da página anterior)

```
    t : ndarray
        Malha temporal.
    """

    dx = xf / M
    dt = T / N

    x = np.linspace(0, xf, M + 1)
    t = np.linspace(0, T, N + 1)

    u = np.zeros((M + 1, N + 1))

    # condição inicial
    u[:, 0] = it0(x)

    # condições de contorno
    u[0, :] = bx0(t)
    u[M, :] = bxf(t)

    r = a * (dt / dx) ** 2
    r1 = r / 2
    r2 = 2 * (1 - r)

    # primeiro passo temporal (Eq. 6)
    u[1:M, 1] = (
        r1 * u[0:M-1, 0]
        + (1 - r) * u[1:M, 0]
        + r1 * u[2:M+1, 0]
        + dt * i1t0(x[1:M])
    )

    # passos seguintes (Eq. 3)
    for k in range(2, N + 1):
        u[1:M, k] = (
            r * u[0:M-1, k-1]
            + r2 * u[1:M, k-1]
            + r * u[2:M+1, k-1]
            - u[1:M, k-2]
        )

    return u, x, t
```

```
import numpy as np
import matplotlib.pyplot as plt
```

(continua na próxima página)

(continuação da página anterior)

```
# Parâmetro da equação
a = 1

# Condições iniciais (Eq. E.2a)
it0 = lambda x: x * (1 - x)
i1t0 = lambda x: np.zeros_like(x)

# Condições de contorno (Eq. E.2b)
bx0t = lambda t: np.zeros_like(t)
bxft = lambda t: np.zeros_like(t)

# Parâmetros da malha
xf = 1
M = 20

T = 2
N = 50

# Resolve a EDP
u, x, t = wave(
    a=a,
    xf=xf,
    T=T,
    it0=it0,
    i1t0=i1t0,
    bx0=bx0t,
    bxf=bxft,
    M=M,
    N=N
)
```

```
from mpl_toolkits.mplot3d import Axes3D

T_grid, X_grid = np.meshgrid(t, x)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

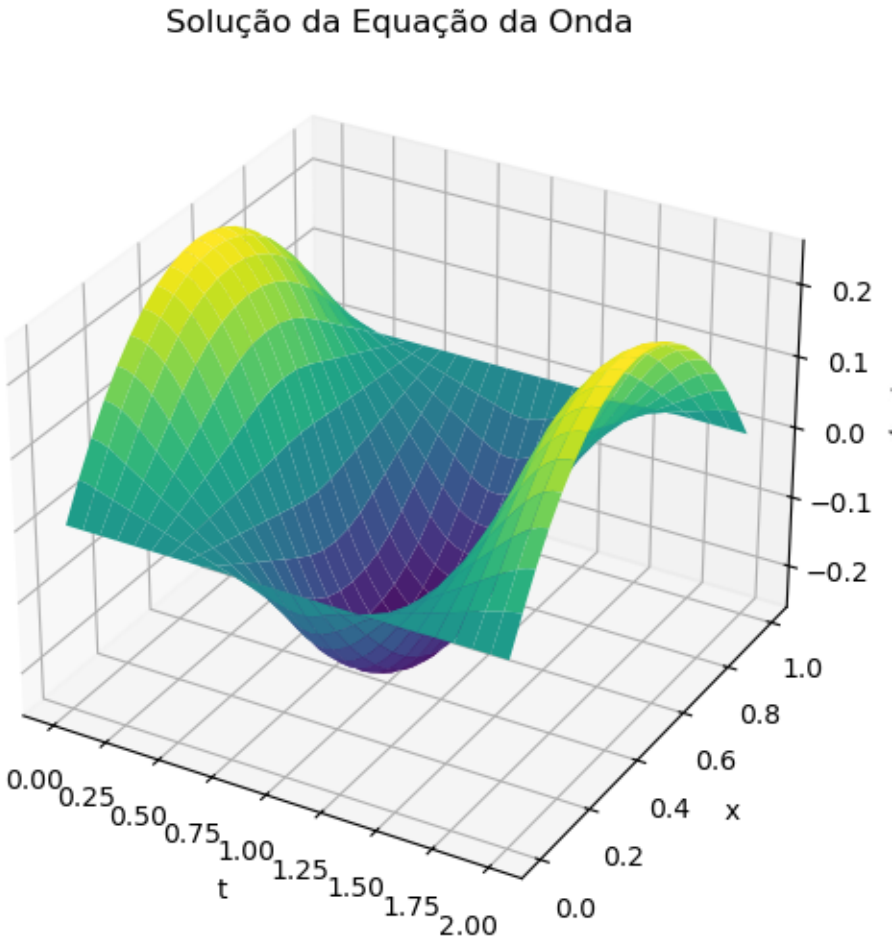
ax.plot_surface(
    T_grid,
    X_grid,
    u,
    cmap='viridis'
)
```

(continua na próxima página)

(continuação da página anterior)

```
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_zlabel('u(x,t)')
ax.set_title('Solução da Equação da Onda')

plt.show()
```




```
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

fig, ax = plt.subplots(figsize=(8,4))

line, = ax.plot([], [], lw=2)

ax.set_xlim(0, xf)
ax.set_ylim(-0.3, 0.3)

ax.set_xlabel('x')
```

(continua na próxima página)

(continuação da página anterior)

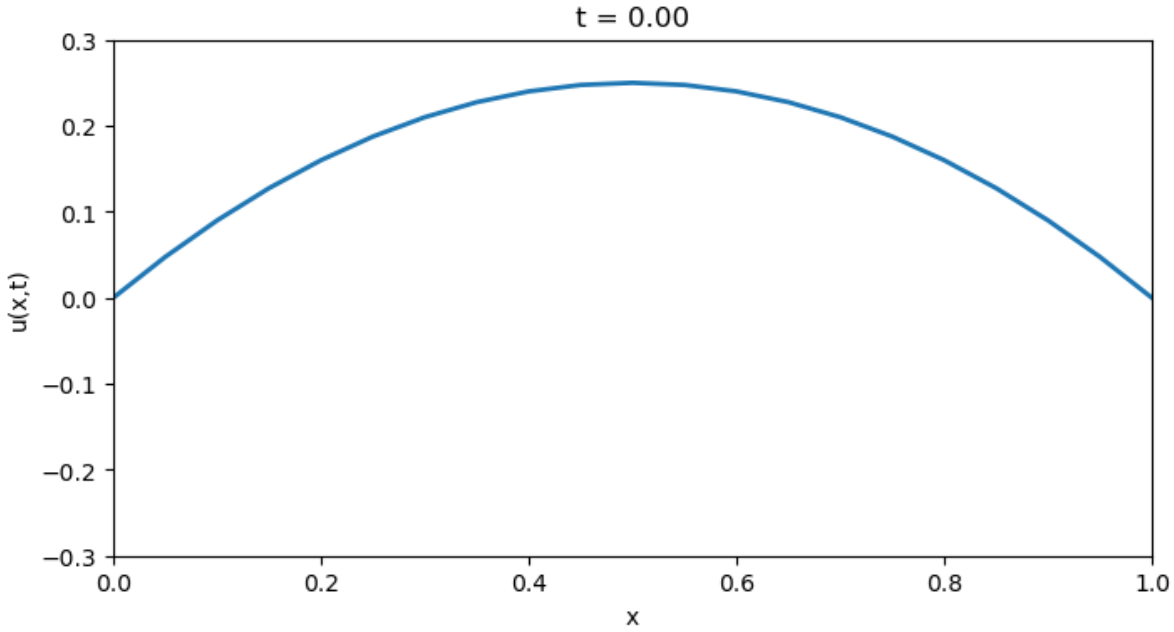
```
ax.set_ylabel('u(x,t)')

def update(frame):
    line.set_data(x, u[:, frame])
    ax.set_title(f't = {t[frame]:.2f}')
    return line,

ani = FuncAnimation(
    fig,
    update,
    frames=N+1,
    interval=100,
    blit=False
)

HTML(ani.to_jshtml())
```

```
<IPython.core.display.HTML object>
```




**Exemplo 8.5.2:** Equação da Onda 1D com Solução Exata — Implementação Vetorizada

Determine uma aproximação para a solução do problema hiperbólico

$$\frac{\partial^2 u}{\partial t^2}(x,t) - 4\,\frac{\partial^2 u}{\partial x^2}(x,t) = 0, \quad 0 < x < 1, \quad 0 < t,$$

com condições de contorno $u(0,t) = u(1,t) = 0$ e condições iniciais

$$u(x,0) = \sin(\pi x), \qquad \frac{\partial u}{\partial t}(x,0) = 0,$$

usando $h = 0{,}1$ ($m = 10$ subintervalos) e $k = 0{,}05$ ($N = 20$ passos de tempo). A solução exata é $u(x,t) = \sin(\pi x)\cos(2\pi t)$.

Utilizamos as fórmulas (6) e (3) — derivadas na seção anterior — com $r = \alpha^2 k^2/h^2$, onde $\alpha^2 = 4$:

```
import numpy as np
import matplotlib.pyplot as plt

l, T_max = 1.0, 1.0
m, N = 10, 20
alpha = 2.0         # a tal que u_tt = a² u_xx, aqui a² = 4

h = l / m
k = T_max / N
r = (alpha * k / h)**2   # r = a²k²/h²  (condição de estabilidade: r ≤ 1)
print(f'h = {h:.3f}, k = {k:.3f}, r = {r:.4f}')

x = np.linspace(0, l, m + 1)
t = np.linspace(0, T_max, N + 1)

f0  = np.sin(np.pi * x)    # u(x, 0) = f(x)
df0 = np.zeros_like(x)     # du/dt(x, 0) = g(x) = 0

w = np.zeros((N + 1, m + 1))
w[0] = f0

# Passo de partida n = 1  [Eq. (6)]
w[1, 1:-1] = (0.5*r*(f0[2:] + f0[:-2])
              + (1 - r)*f0[1:-1]
              + k*df0[1:-1])

# Passos subsequentes n >= 2  [Eq. (3)]
for n in range(2, N + 1):
    w[n, 1:-1] = (r*(w[n-1, 2:] + w[n-1, :-2])
                  + 2*(1 - r)*w[n-1, 1:-1]
                  - w[n-2, 1:-1])
```

```
h = 0.100, k = 0.050, r = 1.0000
```

```
# Comparação com a solução exata em t = T_max
u_exact_T = np.sin(np.pi * x) * np.cos(2 * np.pi * T_max)

# Erro máximo global
T_g, X_g = np.meshgrid(t, x)
u_exact_all = np.sin(np.pi * X_g) * np.cos(2 * np.pi * T_g)  # (m+1, N+1)
err_max = np.max(np.abs(w.T - u_exact_all))
print(f'Erro máximo (‖w - u_exata‖∞) = {err_max:.2e}\n')
```

(continua na próxima página)

(continuação da página anterior)

```
print(f'{"x":>6}  {"w(x,1.0)":>12}  {"u_exata(x,1.0)":>16}  {"|erro|":>10}')
for i in range(m + 1):
    print(f'{x[i]:6.2f}  {w[N,i]:12.6f}  {u_exact_T[i]:16.6f}  '
          f'{abs(w[N,i]-u_exact_T[i]):10.2e}')
```

```
Erro máximo (‖w - u_exata‖∞) = 9.99e-16

     x      w(x,1.0)    u_exata(x,1.0)      |erro|
  0.00      0.000000          0.000000    0.00e+00
  0.10      0.309017          0.309017    5.55e-17
  0.20      0.587785          0.587785    2.22e-16
  0.30      0.809017          0.809017    0.00e+00
  0.40      0.951057          0.951057    3.33e-16
  0.50      1.000000          1.000000    0.00e+00
  0.60      0.951057          0.951057    2.22e-16
  0.70      0.809017          0.809017    0.00e+00
  0.80      0.587785          0.587785    0.00e+00
  0.90      0.309017          0.309017    5.55e-17
  1.00      0.000000          0.000000    1.22e-16
```

```
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

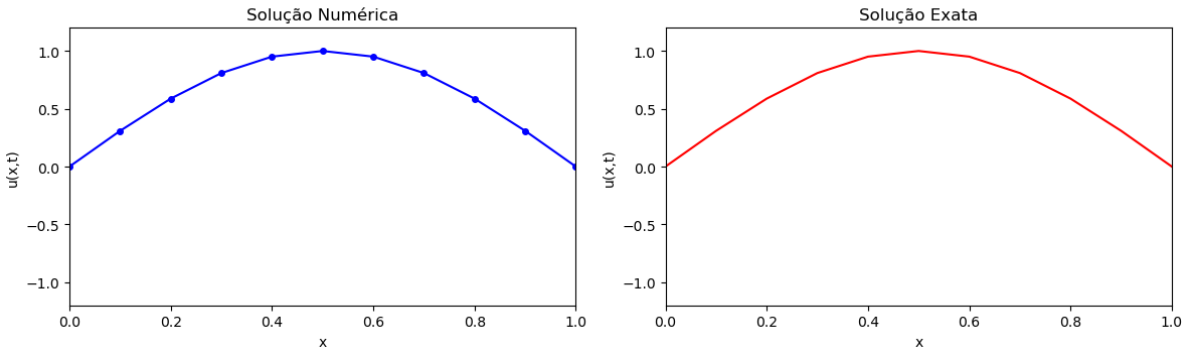
u_ex_t = np.sin(np.pi * x[:, None]) * np.cos(2 * np.pi * t[None, :])  # (m+1, N+1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
line_num, = ax1.plot([], [], 'b-o', ms=4)
ax1.set(xlim=(0, 1), ylim=(-1.2, 1.2), xlabel='x', ylabel='u(x,t)',
        title='Solução Numérica')
line_ex,  = ax2.plot([], [], 'r-')
ax2.set(xlim=(0, 1), ylim=(-1.2, 1.2), xlabel='x', ylabel='u(x,t)',
        title='Solução Exata')
sup = fig.suptitle('t = 0.00')

def _update(frame):
    line_num.set_data(x, w[frame])
    line_ex.set_data(x, u_ex_t[:, frame])
    sup.set_text(f't = {t[frame]:.2f}')
    return line_num, line_ex, sup

ani = FuncAnimation(fig, _update, frames=N + 1, interval=100, blit=True)
plt.tight_layout()
HTML(ani.to_jshtml())
```

```
<IPython.core.display.HTML object>
```




**EDP Hiperbólica Bidimensional**

Nesta seção, consideramos a equação da onda bidimensional para a função amplitude $u(x, y, t)$, em que $(x, y)$ representa a posição e $t$ representa o tempo:

$$A\left(\frac{\partial^2 u(x,y,t)}{\partial x^2} + \frac{\partial^2 u(x,y,t)}{\partial y^2}\right) = \frac{\partial^2 u(x,y,t)}{\partial t^2}, \tag{8}$$

para

$$0 \leq x \leq x_f, \qquad 0 \leq y \leq y_f, \qquad 0 \leq t \leq T.$$

Para que essa equação possa ser resolvida, devem ser fornecidas as condições de contorno

$$u(0, y, t) = b_{x0}(y, t), \qquad u(x_f, y, t) = b_{xf}(y, t),$$

$$u(x, 0, t) = b_{y0}(x, t), \qquad u(x, y_f, t) = b_{yf}(x, t)$$

bem como a condição inicial

$$u(x, y, 0) = i_0(x, y)$$

e a condição para a derivada temporal inicial

$$\left.\frac{\partial u}{\partial t}\right|_{t=0} = i_0'(x, y).$$

Da mesma forma que no caso unidimensional, substituímos as derivadas de segunda ordem em ambos os lados da equação por suas aproximações por diferenças centrais de três pontos:

$$A\left(\frac{u^k_{i,j+1} - 2u^k_{i,j} + u^k_{i,j-1}}{\Delta x^2} + \frac{u^k_{i+1,j} - 2u^k_{i,j} + u^k_{i-1,j}}{\Delta y^2}\right) = \frac{u^{k+1}_{i,j} - 2u^k_{i,j} + u^{k-1}_{i,j}}{\Delta t^2},$$

com

$$\Delta x = \frac{x_f}{M_x}, \qquad \Delta y = \frac{y_f}{M_y}, \qquad \Delta t = \frac{T}{N}. \tag{9}$$

Essa discretização conduz ao método explícito das diferenças centrais:

$$u^{k+1}_{i,j} = r_x\left(u^k_{i,j+1} + u^k_{i,j-1}\right) + 2\left(1 - r_x - r_y\right)u^k_{i,j} + r_y\left(u^k_{i+1,j} + u^k_{i-1,j}\right) - u^{k-1}_{i,j}, \tag{10}$$

onde

$$r_x = A\frac{\Delta t^2}{\Delta x^2}, \qquad r_y = A\frac{\Delta t^2}{\Delta y^2}.$$

Como $u_{i,j}^{-1} = u(x_j, y_i, -\Delta t)$ não é conhecido, não podemos obter $u_{i,j}^1$ diretamente a partir da Fórmula (10) com $k = 0$:

$$u_{i,j}^1 = r_x\left(u_{i,j+1}^0 + u_{i,j-1}^0\right) + 2\left(1 - r_x - r_y\right)u_{i,j}^0 + r_y\left(u_{i+1,j}^0 + u_{i-1,j}^0\right) - u_{i,j}^{-1}. \tag{11}$$

Portanto, aproximamos a condição inicial para a derivada temporal utilizando uma diferença central:

$$\frac{u_{i,j}^1 - u_{i,j}^{-1}}{2\Delta t} = i_0'(x_j, y_i). \tag{12}$$

Utilizamos então essa relação para eliminar $u_{i,j}^{-1}$ da Equação (11), obtendo

$$\begin{aligned} u_{i,j}^1 = &\frac{1}{2}\Big[r_x\left(u_{i,j+1}^0 + u_{i,j-1}^0\right) + r_y\left(u_{i+1,j}^0 + u_{i-1,j}^0\right)\Big] \\ &+ \left(1 - r_x - r_y\right)u_{i,j}^0 + i_0'(x_j, y_i)\,\Delta t. \end{aligned} \tag{13}$$

Utilizamos a Equação (13) juntamente com as condições iniciais para calcular $u_{i,j}^1$. Em seguida, aplicamos a Equação (10) para $k = 1, 2, ....$

Uma condição suficiente para a estabilidade do método é

$$r = \frac{4A\Delta t^2}{\Delta x^2 + \Delta y^2} \leq 1. \tag{14}$$

O objetivo da rotina `wave2()` é implementar esse algoritmo para resolver numericamente a equação da onda bidimensional.

**Exemplo 8.5.3:** Uma EDP Hiperbólica: Onda Bidimensional (Vibração) em uma Membrana Quadrada

Considere a seguinte EDP hiperbólica bidimensional:

$$\frac{1}{4}\left(\frac{\partial^2 u(x,y,t)}{\partial x^2} + \frac{\partial^2 u(x,y,t)}{\partial y^2}\right) = \frac{\partial^2 u(x,y,t)}{\partial t^2}, \tag{E.1}$$

para

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 2, \qquad 0 \leq t \leq 2.$$

com condições de contorno homogêneas

$$u(0,y,t) = 0, \quad u(2,y,t) = 0, \quad u(x,0,t) = 0, \qquad u(x,2,t) = 0, \tag{E.2}$$

e condições iniciais

$$u(x,y,0) = 0.1\sin(\pi x)\sin\left(\frac{\pi y}{2}\right), \qquad \frac{\partial u}{\partial t}(x,y,0) = 0. \tag{E.3}$$

Foi desenvolvido o programa `solve_wave2` para utilizar a rotina `wave2()` na resolução dessa equação. A execução do programa produz os resultados mostrados, além de uma visualização dinâmica da vibração da membrana.

Observe que a condição de estabilidade é satisfeita, pois

$$r = \frac{4A\Delta t^2}{\Delta x^2 + \Delta y^2} = \frac{4(1/4)(2/20)^2}{(2/20)^2 + (2/20)^2} = \frac{1}{2} \leq 1. \tag{E.4}$$

Portanto, a discretização escolhida garante a estabilidade do método explícito utilizado.

```
def wave2(a, D, T, it0, i1t0, bxyt, Mx, My, N):
    """
    Resolve

        a(u_xx + u_yy) = u_tt

    em

        D[0] <= x <= D[1]
        D[2] <= y <= D[3]
        0 <= t <= T

    usando diferenças finitas explícitas.
    """

    # malha
    dx = (D[1] - D[0]) / Mx
    dy = (D[3] - D[2]) / My
    dt = T / N

    x = np.linspace(D[0], D[1], Mx + 1)
    y = np.linspace(D[2], D[3], My + 1)
    t = np.linspace(0, T, N + 1)

    # condição inicial
    u = np.zeros((My + 1, Mx + 1))
    ut = np.zeros((My + 1, Mx + 1))

    for i in range(1, My):
        for j in range(1, Mx):
            u[i, j] = it0(x[j], y[i])
            ut[i, j] = i1t0(x[j], y[i])

    adt2 = a * dt**2

    rx = adt2 / dx**2
    ry = adt2 / dy**2

    rxy1 = 1 - rx - ry
    rxy2 = 2 * rxy1

    u_1 = u.copy()
    u_2 = np.zeros_like(u)

    # histórico temporal opcional
```

(continua na próxima página)

(continuação da página anterior)

```
    U = np.zeros((N + 1, My + 1, Mx + 1))

    for k in range(N + 1):

        tk = k * dt

        # condições de contorno
        for i in range(My + 1):
            u[i, 0] = bxyt(x[0], y[i], tk)
            u[i, Mx] = bxyt(x[Mx], y[i], tk)

        for j in range(Mx + 1):
            u[0, j] = bxyt(x[j], y[0], tk)
            u[My, j] = bxyt(x[j], y[My], tk)

        if k == 0:

            # Eq. (13)
            for i in range(1, My):
                for j in range(1, Mx):

                    u[i, j] = (
                        0.5 * (
                            rx * (u_1[i, j-1] + u_1[i, j+1])
                            + ry * (u_1[i-1, j] + u_1[i+1, j])
                        )
                        + rxy1 * u_1[i, j]
                        + dt * ut[i, j]
                    )

        else:

            # Eq. (10)
            for i in range(1, My):
                for j in range(1, Mx):

                    u[i, j] = (
                        rx * (u_1[i, j-1] + u_1[i, j+1])
                        + ry * (u_1[i-1, j] + u_1[i+1, j])
                        + rxy2 * u_1[i, j]
                        - u_2[i, j]
                    )

        U[k] = u
```

(continua na próxima página)

(continuação da página anterior)

```
        # atualização dos buffers
        u_2 = u_1.copy()
        u_1 = u.copy()

    return U, x, y, t
```

```
import numpy as np

# Condição inicial (Eq. E.3)
def it0(x, y):
    return 0.1 * np.sin(np.pi * x) * np.sin(np.pi * y / 2)

# Velocidade inicial
def i1t0(x, y):
    return np.zeros_like(x)

# Condições de contorno homogêneas
def bxyt(x, y, t):
    return np.zeros_like(np.asarray(x))

# Parâmetros do problema
a = 0.25

# Domínio [xmin, xmax, ymin, ymax]
D = [0, 2, 0, 2]

T = 2

Mx = 40
My = 40
N = 40

# Resolução da EDP
U, x, y, t = wave2(
    a=a,
    D=D,
    T=T,
    it0=it0,
    i1t0=i1t0,
    bxyt=bxyt,
    Mx=Mx,
    My=My,
    N=N
)
```

```
import matplotlib.pyplot as plt

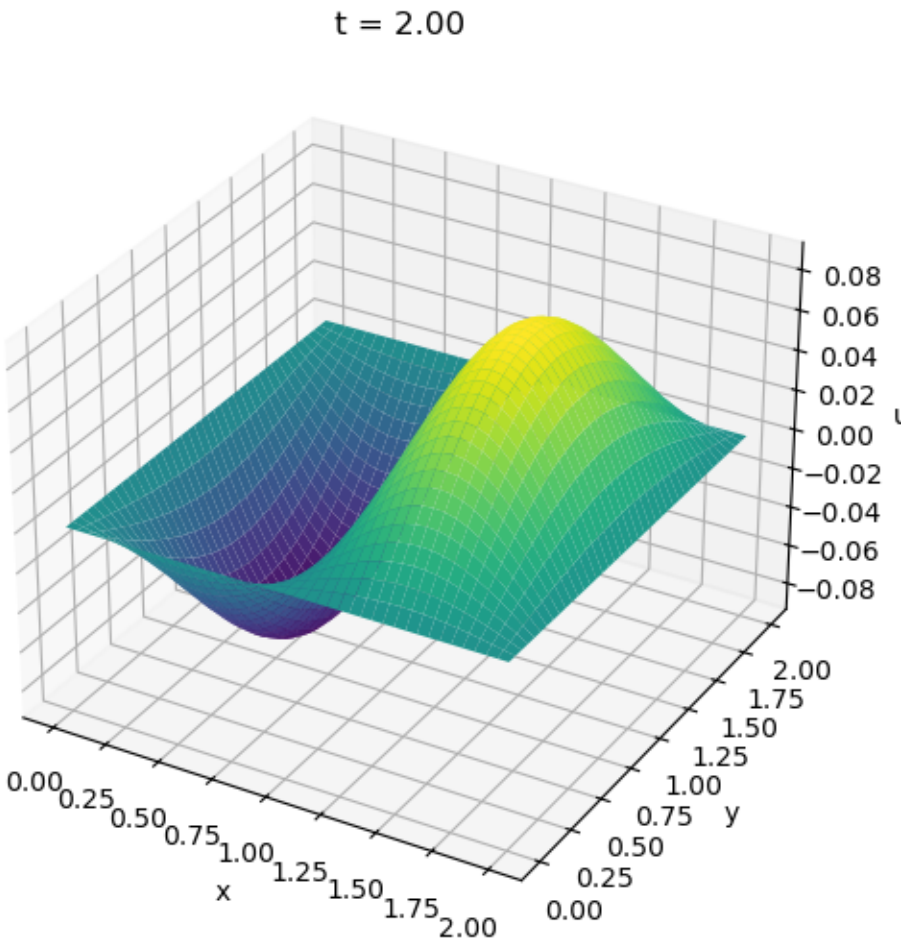
X, Y = np.meshgrid(x, y)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, Y, U[-1], cmap='viridis')

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('u')
ax.set_title(f't = {t[-1]:.2f}')

plt.show()
```




```
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
```

(continua na próxima página)

(continuação da página anterior)

```
X, Y = np.meshgrid(x, y)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

def update(k):
    ax.clear()

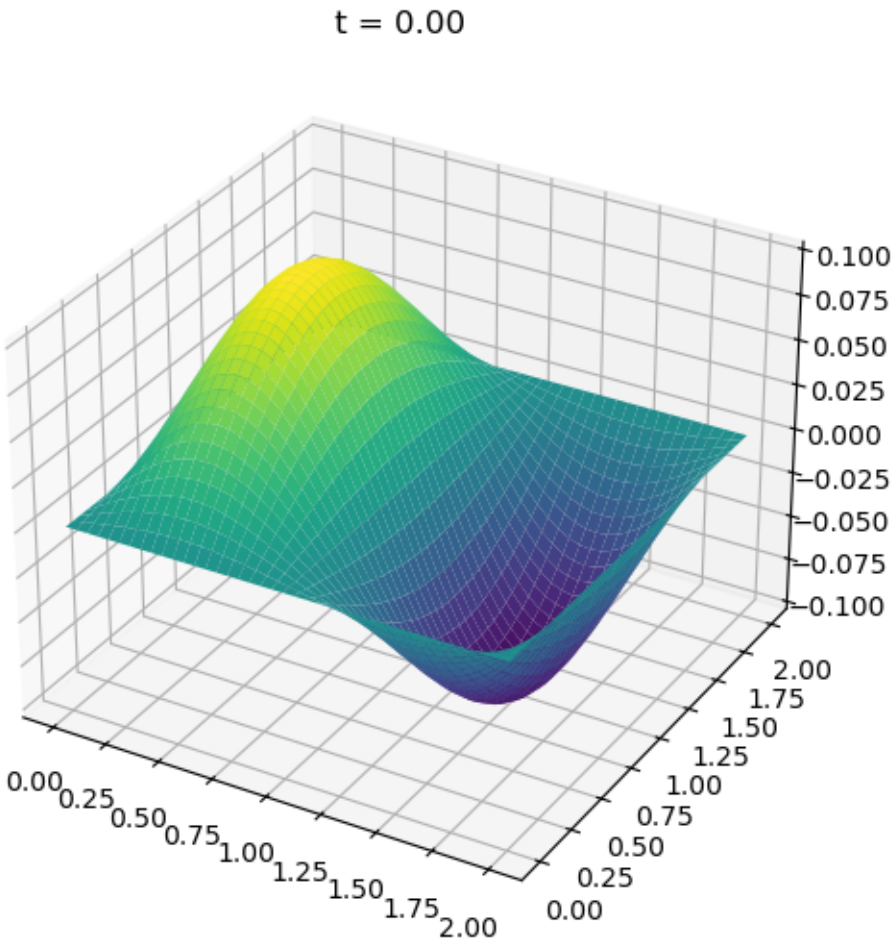
    ax.plot_surface(
        X, Y, U[k],
        cmap='viridis'
    )

    ax.set_zlim(-0.1, 0.1)
    ax.set_title(f't = {t[k]:.2f}')

ani = FuncAnimation(
    fig,
    update,
    frames=len(t),
    interval=100
)

HTML(ani.to_jshtml())
```

```
<IPython.core.display.HTML object>
```




## Exercícios

Utilize como modelo a implementação vetorizada do **Exemplo 8.5.2** (ou a rotina `wave()`) para resolver as equações da onda a seguir e comparar os resultados com a solução exata indicada. Escolha valores adequados de $m$, $N$ e $T$ que garantam a estabilidade ($r \leq 1$).

---

**1.** Resolva a equação da onda

$$\frac{\partial^2 u}{\partial t^2} - \frac{\partial^2 u}{\partial x^2} = 0, \quad 0 < x < 1, \quad 0 < t,$$

com $u(0,t) = u(1,t) = 0$, $u(x,0) = \text{sen}(\pi x)$ e $\dfrac{\partial u}{\partial t}(x,0) = 0$. Solução exata: $u(x,t) = \cos(\pi t)\,\text{sen}(\pi x)$.

---

**2.** Resolva a equação da onda

$$\frac{\partial^2 u}{\partial t^2} - \frac{1}{16\pi^2}\frac{\partial^2 u}{\partial x^2} = 0, \quad 0 < x < 0{,}5, \quad 0 < t,$$

com $u(0,t) = u(0{,}5,t) = 0$, $u(x,0) = 0$ e $\dfrac{\partial u}{\partial t}(x,0) = \text{sen}(4\pi x)$. Solução exata: $u(x,t) = \text{sen}(t)\,\text{sen}(4\pi x)$.

**3.** Resolva a equação da onda

$$\frac{\partial^2 u}{\partial t^2} - \frac{\partial^2 u}{\partial x^2} = 0, \quad 0 < x < 1, \quad 0 < t,$$

com $u(0,t) = u(1,t) = 0$, $u(x,0) = \text{sen}(2\pi x)$ e $\dfrac{\partial u}{\partial t}(x,0) = 2\pi\,\text{sen}(2\pi x)$. Solução exata: $u(x,t) = \text{sen}(2\pi x)\,(\cos(2\pi t) + \text{sen}(2\pi t))$.

---

**4.** Retome a corda triangular do **Exemplo 8.1.6** (Seção 8.1), com $c = 1$, $k = 1$ e $L = 2$, cuja deflexão inicial é

$$f(x) = \begin{cases} x, & 0 < x < 1 \\ 2 - x, & 1 < x < 2 \end{cases}$$

e velocidade inicial nula. Resolva o problema numericamente com a rotina `wave()` (usando `it0 = f`, `i1t0 = 0` e $a = c^2 = 1$) e compare, no mesmo gráfico, a solução numérica com a série de Fourier obtida analiticamente em 8.1 (trunque a série em 20 termos).

---

**5.** Considere novamente o problema do **Exemplo 8.5.2** ($u_{tt} - 4u_{xx} = 0$, $u(x,0) = \text{sen}(\pi x)$, $h = 0{,}1$). Mantendo $h = 0{,}1$, aumente o passo de tempo $k$ até violar a condição de estabilidade $r = \alpha^2 k^2/h^2 \leq 1$ (por exemplo, $k = 0{,}06$). Execute a simulação e plote a solução numérica em função de $x$ para os últimos instantes de tempo. Descreva o que acontece com a amplitude da solução e explique, em termos da Eq. (7), por que isso ocorre.

---

**6.** Para o modo de vibração $u(x,y,0) = A_0\,\text{sen}\left(\dfrac{n\pi x}{L_x}\right)\text{sen}\left(\dfrac{m\pi y}{L_y}\right)$ com velocidade inicial nula, mostre (substituindo na EDP $A(u_{xx} + u_{yy}) = u_{tt}$) que a solução oscila no tempo com frequência angular

$$\omega_{n,m} = \sqrt{A\left[\left(\frac{n\pi}{L_x}\right)^2 + \left(\frac{m\pi}{L_y}\right)^2\right]}.$$

Utilizando a rotina `wave2()` do **Exemplo 8.5.3** (domínio $[0,2] \times [0,2]$, $A = 0{,}25$), simule o modo $n = 1$, $m = 2$ (isto é, $u(x,y,0) = 0{,}1\,\text{sen}(\pi x/2)\,\text{sen}(\pi y)$) e meça o período de oscilação observado no centro da membrana. Compare com o período teórico $T_{n,m} = 2\pi/\omega_{n,m}$.

---

**7.** Uma corda de comprimento 1, presa nas extremidades ($u(0,t) = u(1,t) = 0$), é dedilhada fora do centro, no ponto $x = 0{,}3$, resultando na deflexão inicial triangular

$$f(x) = \begin{cases} \dfrac{0{,}05}{0{,}3}\,x, & 0 \leq x \leq 0{,}3 \\ \dfrac{0{,}05}{0{,}7}\,(1 - x), & 0{,}3 < x \leq 1 \end{cases}$$

com velocidade inicial nula e $a = 1$. Resolva com a rotina `wave()` e plote $u(x,t)$ para vários instantes $t$ em um mesmo gráfico. Observe e descreva como o pulso inicial se decompõe em duas ondas viajando em sentidos opostos e como cada uma se reflete ao atingir as extremidades fixas.

**8.** Compare o comportamento qualitativo de uma EDP parabólica e de uma EDP hiperbólica submetidas à mesma condição inicial. Considere, em $0 \leq x \leq 1$, com contorno $u(0,t) = u(1,t) = 0$ e condição inicial $u(x,0) = \text{sen}(\pi x)$:
(a) a equação do calor $u_t = u_{xx}$, resolvida com a função `heat_exp` da Seção 8.4;
(b) a equação da onda $u_{tt} = u_{xx}$, com velocidade inicial nula, resolvida com a função `wave` desta seção.
Plote, em dois painéis lado a lado, o perfil de $u(x,t)$ para os mesmos instantes de tempo em cada caso. Discuta a diferença qualitativa entre os dois comportamentos (decaimento monótono *versus* oscilação persistente) e relacione isso à ordem da derivada temporal em cada equação.

# Referências


- ARENALES, Selma; DAREZZO, Adilson. **Cálculo Numérico: Aprendizagem com Apoio de Software**. 2. ed. São Paulo: Cengage Learning, 2016.
- BOYCE, William E.; DIPRIMA, Richard C.; MEADE, Douglas B. **Equações Diferenciais Elementares e Problemas de Valores de Contorno**. 12. ed. Rio de Janeiro: LTC, 2024.
- BURDEN, Richard L.; FAIRES, J. Douglas; BURDEN, Annette M. **Análise Numérica**. 10. ed. São Paulo: Cengage Learning, 2016.
- CHAPRA, Steven C.; CANALE, Raymond P. **Métodos Numéricos para Engenharia**. 7. ed. Porto Alegre: McGraw-Hill, 2016.
- FRANCO, Neide M. B. **Cálculo Numérico**. São Paulo: Pearson Prentice Hall, 2006.
- HARRIS, Charles R. et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi:10.1038/s41586-020-2649-2.
- JUSTO, Dagoberto Adriano Rizzotto et al., editors. **Cálculo Numérico: Um Livro Colaborativo – Versão Python**. Porto Alegre: REAMAT/UFRGS, 2020.
- LARSON, Ron. **Elementos de Álgebra Linear**. 8. ed. Porto Alegre: +A Educação – Cengage Learning Brasil, 2017.
- PRESS, William H. et al. **Numerical Recipes: The Art of Scientific Computing**. 3. ed. Cambridge: Cambridge University Press, 2007.
- RUGGIERO, Márcia A. G.; LOPES, Vera Lúcia da R. **Cálculo Numérico: Aspectos Teóricos e Computacionais**. 2. ed. São Paulo: Pearson, 2016.
- VIRTANEN, Pauli et al. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi:10.1038/s41592-019-0686-2.
- YANG, Won Y. et al. **Applied Numerical Methods Using MATLAB**. Hoboken, NJ: John Wiley & Sons, 2005.
- SciPy Developers. Scipy api reference. 2026. URL: https://docs.scipy.org/doc/scipy/reference/ (visited on 2026-07-30).